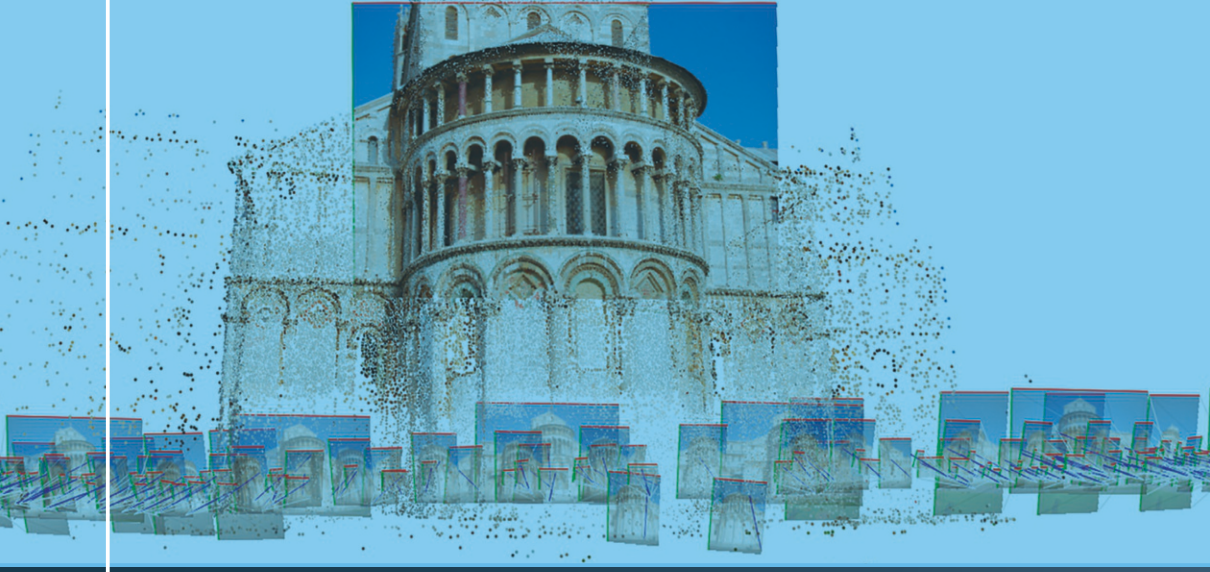


Andrea Fusiello

Computer Vision: Three-dimensional Reconstruction Techniques

Springer

Computer Vision: Three-dimensional Reconstruction Techniques

Andrea Fusiello

Computer Vision: Three-dimensional Reconstruction Techniques

Springer

Andrea Fusiello
University of Udine
Udine, Italy

ISBN 978-3-031-34506-7 ISBN 978-3-031-34507-4 (eBook)
https://doi.org/10.1007/978-3-031-34507-4

Translation from the Italian language edition: "Visione Computazionale" by Andrea Fusiello, © Edizioni Franco Angeli 2022. Published by Edizioni Franco Angeli. All Rights Reserved.

© Edizioni Franco Angeli under exclusive license to Springer Nature Switzerland AG 2024
This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors, and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Switzerland AG
The registered company address is: Gewerbestrasse 11, 6330 Cham, Switzerland

Paper in this product is recyclable.

To my mother

Foreword

Deep learning has brought undeniable successes and some breakthroughs in image recognition and scene description. It is nevertheless true that geometric computer vision remains a fundamental field. Given the impressive state-of-the-art and the rapid pace of progress in deep learning, it would be of course risky to rule out the possibility that the solution to many geometric vision problems, for instance reconstructing 3D structure from multiple images, can be learned from millions of examples. Yet we believe that a principled, approach that obtains the geometric structure of what we see through applied mathematics provides more insight. We would also go as far as suggesting that, in the end, such an approach can be even more fun to study and implement.

The book on geometric computer vision that we published in 1998 was received so well that it became a standard textbook for the field. As it happens inevitably to any advanced textbook, the relevance of its contents has tarnished as the state of the art of research has moved on.

Today, Andrea Fusiello's book offers an admirable contemporary view on geometric computer vision. The Matlab code of the algorithms offers a hands-on counterpart to the algebraic derivations, making the material truly complete and useful. Ultimately, the book makes the theoretical basis on which 3D computer vision can be built and developed accessible. With this book, geometry is back into the picture!

Andrea's book fills also a gap in the current panorama of computer vision textbooks. It gives students and practitioners a compact yet exhaustive overview of geometric computer vision. It tells a story that is logical and easy to follow, yet rigorous in its mathematical presentation. We have no doubt that it will be well received by teachers, students and practitioners alike. We are also sure that it will contribute to remind the community of the beauty of building mathematical models for computer vision , and that more than a superficial understanding of algebra and geometry has a lot to offer, both in terms of results and intellectual pleasure.

As d'Alembert reportedly wrote, algebra is generous: she often gives more than is asked of her.

University of Dundee Emanuele Trucco
Università degli Studi di Genova Alessandro Verri

Preface

People are usually more convinced by reasons they discovered themselves than by those found by others.

—B. Pascal

Computer vision is a rapidly advancing field that has had a profound impact on the way we interact with technology. From facial recognition to self-driving cars, the applications of computer vision are vast and ever-expanding. Geometry plays a fundamental role in this discipline, providing the necessary mathematical framework to understand the underlying principles of how we perceive and interpret visual information in the world around us.

This text delves into the theories and computational techniques used for determining the geometric properties of solid objects through images. It covers the fundamental concepts and provides the necessary mathematical background for more advanced studies. The book is divided into clear and concise chapters that cover a wide range of topics, including image formation, camera models, feature detection, and 3D reconstruction. Each chapter includes detailed explanations of the theory, as well as practical examples to help readers understand and apply the concepts presented.

With a focus on teaching, the book aims to find a balance between complexity of the theory and its practical applicability in terms of implementation. Instead of providing an all-encompassing overview of the current state of the field, it offers a selection of specific methods with enough detail for readers to implement them. To aid the reader in implementation, most of the methods discussed in the book are accompanied by a MATLAB listing and the sources are available on Github at https://github.com/fusiello/Computer_Vision_Toolkit.

This approach results in leaving out several valuable topics and algorithms, but this does not mean that they are any less important than the ones that have been included; it is simply a personal choice.

The book has been written with the intention of being used as a primary resource for students of university courses on computer vision, specifically final-year undergraduates or postgraduates of computer science or engineering degrees. It is also useful for self-study and for those who are using computer vision for practical applications outside of academia.

Basic knowledge of linear algebra is necessary, while other mathematical concepts can be introduced as needed through included appendices. The modular structure allows instructors to adapt the material to fit their course syllabus, but it is recommended to cover at least the chapters on fundamental geometric concepts, namely Chaps. 3, 4, 5, 6, 7, 8.

This edition has been updated to ensure that it is accessible to a global audience, while also ensuring that the material is current and up-to-date with the latest developments in the field. To accomplish this, the book has been translated to English and has undergone extensive revision from its previous version, which was published by Franco Angeli, Milano. The majority of the chapters have undergone changes, which include the inclusion of new material, as well as the reorganisation of existing content.

I hope that you will find this book to be a valuable resource as you explore the exciting world of computer vision.

Udine, Italy Andrea Fusiello
December, 2022

Acknowledgements

This text is derived from the handouts I have prepared for academic courses or seminar presentations over the past 20 years. The first chapters were born, in embryonic form, in 1997 and then evolved and expanded to the current version. I would like to thank the students of the University of Udine and the University of Verona who, during these years, have pointed out errors, lacks and unclear parts. Homeopatic traces of my PhD thesis can also be found here and there.

The text benefited from the timely corrections suggested by Federica Arrigoni, Guido Maria Cortelazzo, Fabio Crosilla, Riccardo Gherardi, Luca Magri, Francesco Malapelle, Samuele Martelli, Eleonora Maset, Roberto Rinaldo, and Roberto Toldo, whom I sincerely thank. The residual errors are solely my responsibility.

Credits for figures are recognised in the respective captions.

Contents

1 Introduction 1
1.1 The Prodigy of Vision 1
1.2 Low-Level Computer Vision 1
1.3 Overview of the Book 3
1.4 Notation 3
References 4

2 Fundamentals of Imaging 5
2.1 Introduction 5
2.2 Perspective 5
2.3 Digital Images 7
2.4 Thin Lenses 8
2.4.1 Telecentric Optics 10
2.5 Radiometry 11
References 13

3 The Pinhole Camera Model 15
3.1 Introduction 15
3.2 Pinhole Camera 15
3.3 Simplified Pinhole Model 16
3.4 General Pinhole Model 18
3.4.1 Intrinsic Parameters 19
3.4.2 Extrinsic Parameters 22
3.5 Dissection of the Perspective Projection Matrix 25
3.5.1 Collinearity Equations 28
3.6 Radial Distortion 29
Problems 32
References 33

4 Camera Calibration 35
4.1 Introduction 35
4.2 The Direct Linear Transform Method 35

4.3 Factorisation of the Perspective Projection Matrix 37
4.4 Calibrating Radial Distortion 38
4.5 The Sturm-Maybank-Zhang Calibration Algorithm 39
Problems 43
References 44

5 Absolute and Exterior Orientation 45
5.1 Introduction 45
5.2 Absolute Orientation 45
5.2.1 Orthogonal Procrustes Analysis 46
5.3 Exterior Orientation 49
5.3.1 Fiore's Algorithm 50
5.3.2 Procrustean Method 52
5.3.3 Direct Method 54
Problems 55
References 56

6 Two-View Geometry 57
6.1 Introduction 57
6.2 Epipolar Geometry 57
6.3 Fundamental Matrix 60
6.4 Computing the Fundamental Matrix 61
6.4.1 The Seven-Point Algorithm 62
6.4.2 Preconditioning 63
6.5 Planar Homography 64
6.5.1 Computing the Homography 67
6.6 Planar Parallax 69
Problems 70
References 71

7 Relative Orientation 73
7.1 Introduction 73
7.2 The Essential Matrix 73
7.2.1 Geometric Interpretation 75
7.2.2 Computing the Essential Matrix 76
7.3 Relative Orientation from the Essential Matrix 77
7.3.1 Closed Form Factorisation of the Essential Matrix 80
7.4 Relative Orientation from the Calibrated Homography 81
Problems 84
References 85

8 Reconstruction from Two Images 87
8.1 Introduction 87
8.2 Triangulation 87
8.3 Ambiguity of Reconstruction 89
8.4 Euclidean Reconstruction 90
8.5 Projective Reconstruction 92

8.6 Euclidean Upgrade from Known Intrinsic Parameters 93
8.7 Stratification 94
Problems 96
References 97

9 Non-linear Regression 99
9.1 Introduction 99
9.2 Algebraic Versus Geometric Distance 99
9.3 Non-linear Regression of the PPM 102
9.3.1 Residual 102
9.3.2 Parameterisation 103
9.3.3 Derivatives 103
9.3.4 General Remarks 106
9.4 Non-linear Regression of Exterior Orientation 106
9.5 Non-linear Regression of a Point in Space 107
9.5.1 Residual 107
9.5.2 Derivatives 108
9.5.3 Radial Distortion 109
9.6 Regression in the Joint Image Space 110
9.7 Non-linear Regression of the Homography 111
9.7.1 Residual 111
9.7.2 Parameterisation 112
9.7.3 Derivatives 112
9.8 Non-linear Regression of the Fundamental Matrix 115
9.8.1 Residual 115
9.8.2 Parameterisation 116
9.8.3 Derivatives 116
9.9 Non-linear Regression of Relative Orientation 118
9.9.1 Parameterisation 119
9.9.2 Derivatives 119
9.10 Robust Regression 121
Problems 123
References 124

10 Stereopsis: Geometry 125
10.1 Introduction 125
10.2 Triangulation in the Normal Case 126
10.3 Epipolar Rectification 129
10.3.1 Calibrated Rectification 129
10.3.2 Uncalibrated Rectification 133
Problems 137
References 138

11 Features Points 139
11.1 Introduction 139
11.2 Filtering Images 140

11.2.1 Smoothing 141
11.2.2 Derivation 144
11.3 LoG Filtering 146
11.4 Harris-Stephens Operator 149
11.4.1 Matching and Tracking 153
11.4.2 Kanade-Lucas-Tomasi Algorithm 153
11.4.3 Predictive Tracking 154
11.5 Scale Invariant Feature Transform 155
11.5.1 Scale-Space 155
11.5.2 SIFT Detector 157
11.5.3 SIFT Descriptor 160
11.5.4 Matching 161
References 163

12 Stereopsis: Matching 165
12.1 Introduction 165
12.2 Constraints and Ambiguities 165
12.3 Local Methods 168
12.3.1 Matching Cost 169
12.3.2 Census Transform 170
12.4 Adaptive Support 173
12.4.1 Multiresolution Stereo Matching 174
12.4.2 Adaptive Windows 175
12.5 Global Matching 176
12.6 Post-Processing 180
12.6.1 Reliability Indicators 180
12.6.2 Occlusion Detection 180
References 182

13 Range Sensors 185
13.1 Introduction 185
13.2 Structured Lighting 186
13.2.1 Active Stereopsis 187
13.2.2 Active Triangulation 187
13.2.3 Ray-Plane Triangulation 189
13.2.4 Scanning Methods 190
13.2.5 Coded-Light Methods 191
13.3 Time-of-Flight Sensors 192
13.4 Photometric Stereo 194
13.4.1 From Normals to Coordinates 196
13.5 Practical Considerations 198
References 200

14 Multi-View Euclidean Reconstruction 201
14.1 Introduction 201
14.1.1 Epipolar Graph 202

14.1.2 The Case of Three Images 204
14.1.3 Taxonomy 206
14.2 Point-Based Approaches 207
14.2.1 Adjustment of Independent Models 207
14.2.2 Incremental Reconstruction 208
14.2.3 Hierarchical Reconstruction 209
14.3 Frame-Based Approach 210
14.3.1 Synchronisation of Rotations 212
14.3.2 Synchronisation of Translations 214
14.3.3 Localisation from Bearings 216
14.4 Bundle Adjustment 217
14.4.1 Jacobian of Bundle Adjustment 219
14.4.2 Reduced System 221
References 224

15 3D Registration 225
15.1 Introduction 225
15.1.1 Generalised Procrustes Analysis 226
15.2 Correspondence-Less Methods 228
15.2.1 Registration of Two Point Clouds 228
15.2.2 Iterative Closest Point 228
15.2.3 Registration of Many Point Clouds 231
References 232

16 Multi-view Projective Reconstruction and Autocalibration 233
16.1 Introduction 233
16.1.1 Sturm-Triggs Factorisation Method 233
16.2 Autocalibration 236
16.2.1 Absolute Quadric Constraint 237
16.2.2 Mendonça-Cipolla Method 240
16.3 Autocalibration via H_∞ 243
16.4 Tomasi-Kanade's Factorisation 244
16.4.1 Affine Camera 245
16.4.2 The Factorisation Method for Affine Camera 245
Problems 247
References 248

17 Multi-view Stereo Reconstruction 249
17.1 Introduction 249
17.2 Volumetric Stereo in Object-Space 251
17.2.1 Shape from Silhouette 252
17.2.2 Szeliski's Algorithm 254
17.2.3 Voxel Colouring 255
17.2.4 Space Carving 257
17.3 Volumetric Stereo in Image-Space 258
17.4 Marching Cubes 261
References 263

18 Image-Based Rendering 265
18.1 Introduction 265
18.2 Parametric Transformations 265
18.2.1 Mosaics 266
18.2.2 Image Stabilisation 270
18.2.3 Perspective Rectification 272
18.3 Non-parametric Transformations 274
18.3.1 Transfer with Depth 274
18.3.2 Transfer with Disparity 274
18.3.3 Epipolar Transfer 275
18.3.4 Transfer with Parallax 276
18.3.5 Ortho-Projection 277
18.4 Geometric Image Transformation 279
Problems 281
References 282

A Notions of Linear Algebra 283
A.1 Introduction 283
A.2 Scalar Product 283
A.3 Matrix Norm 284
A.4 Inverse Matrix 285
A.5 Determinant 285
A.6 Orthogonal Matrices 287
A.7 Linear and Quadratic Forms 288
A.8 Rank 288
A.9 QR Decomposition 290
A.10 Eigenvalues and Eigenvectors 290
A.11 Singular Value Decomposition 292
A.12 Pseudoinverse 296
A.13 Cross Product 297
A.14 Kronecker's Product 299
A.15 Rotations 301
A.16 Matrices Associated with Graphs 302
References 303

B Matrix Differential Calculation 305
B.1 Derivatives of Vector and Matrix Functions 305
B.2 Derivative of Rotations 309
B.2.1 Axis/Angle Representation 309
B.2.2 Euler Representation 310
References 311

C Regression 313
C.1 Introduction 313
C.2 Least-Squares 313
C.2.1 Linear Least-Squares 314
C.2.2 Non-linear Least-Squares 315
C.2.3 The Levenberg-Marquardt Method 317
C.3 Robust Regression 318
C.3.1 Outliers and Robustness 318
C.3.2 M-Estimators 319
C.3.3 Least Median of Squares 321
C.3.4 RANSAC 323
C.4 Propagation of Uncertainty 324
C.4.1 Covariance Propagation in Least-Squares 325
References 326

D Notions of Projective Geometry 327
D.1 Introduction 327
D.2 Perspective Projection 327
D.3 Homogeneous Coordinates 329
D.4 Equation of the Line 330
D.5 Transformations 331
Reference 332

E MATLAB Code 333

Index 335

Acronyms

PPM	Perspective Projection Matrix
COP	Centre of Projection
SVD	Singular Value Decomposition
OPA	Orthogonal Procrustes analysis
GPA	Generalised Procrustes analysis
BA	Bundle Adjustment
AIM	Adjustment of Independent Models
DLT	Direct Linear Transform
ICP	Iterative Closest Point
GSD	Ground Sampling Distance
IRLS	Iteratively Reweighted Least-Squares
LMS	Least Median of Squares
RANSAC	Random Sample Consensus
SFM	Structure from Motion
SGM	Semi Global Matching
SO	Scanline Optimisation
WTA	Winner Takes All
DSI	Disparity Space Image
LS	Least-Squares
LM	Levenberg-Marquardt
SIFT	Scale Invariant Feature Transform
TOF	Time of Flight
SSD	Sum of Squared Difference
SAD	Sum of Absolute Difference
NCC	Normalised Cross Correlation

Listings

3.1 Projective transformation 18
3.2 Parameterisation of K (Jacobian omitted for space reasons) 23
3.3 Construction of the PPM 27
3.4 Radial distortion: direct model 31
3.5 Radial distortion: inverse model 32
4.1 DLT method 37
4.2 Resection with preconditioning 37
4.3 Factorisation of the PPM 38
4.4 Regression of radial distortion 39
4.5 Calibration of radial distortion with resection 40
4.6 SMZ calibration 42
5.1 Weighted OPA 48
5.2 Fiore's linear exterior orientation 52
5.3 Iterative exterior orientation 53
6.1 Eight-point algorithm 62
6.2 Linear computation of F 64
6.3 Preconditioning 64
6.4 Linear computation of H 68
7.1 Linear computation of E 77
7.2 Relative orientation from the factorisation of E 79
7.3 Relative orientation from E in closed form 81
7.4 Relative orientation from calibrated H 84
8.1 Triangulation 88
8.2 Reconstruction from two images 92
8.3 Euclidean upgrade of a projective reconstruction 95
9.1 Non-linear calibration 105
9.2 Computation of the residual of reprojection and its derivatives 105
9.3 Non-linear regression of exterior orientation 107
9.4 Non-linear regression of a point (triangulation) 109
9.5 Non-linear regression of H 114
9.6 Parameterisation of H 114

9.7 Sampson distance for H 114
9.8 Non-linear regression of F 118
9.9 Parameterisation of F 118
9.10 Sampson distance for F 119
9.11 Non-linear regression of relative orientation 120
9.12 Parameterisation of E 121
9.13 Robust estimate of F 122
9.14 Robust estimate of H 123
10.1 Triangulation from disparity 128
10.2 Calibrated epipolar rectification 132
10.3 Uncalibrated epipolar rectification 137
11.1 Harris-Stephens operator 152
12.1 Stereo with NCC 169
12.2 Stereo with SSD 170
12.3 Stereo with SCH 172
13.1 Photometric stereo 195
14.1 Rotation synchronisation 214
14.2 Translation synchronisation 215
14.3 Localisation from bearings 217
15.1 Generalised Procrustean Analysis 227
15.2 Iterative closest point 230
16.1 Sturm-Triggs projective reconstruction 236
16.2 Mendonça-Cipolla self-calibration 243
16.3 LVH calibration 244
17.1 Reconstruction from silhouettes 253
17.2 General scheme (stub) of space carving 257
18.1 Homography synchronisation 268
18.2 Image transform with backward mapping 281
C.1 Gauss-Newton method 316
C.2 IRLS 321
C.3 Least median of squares 322
C.4 MSAC 324

Chapter 1
Introduction



Airplanes do not flap their wings

—Frederick Jelinek

1.1 The Prodigy of Vision

If we pause to reflect detachedly on vision as a sensory ability, we must agree with Ullman that it is prodigious:

> As seeing agents, we are so used to the benefits of vision, and so unaware of how we actually use it, that it took a long time to appreciate the almost miraculous achievements of our visual system. If one tries to adopt a more objective and detached attitude, by considering the visual system as a device that records a band of electromagnetic radiation as an input, and then uses it to gain knowledge about surrounding objects that emit and reflect it, one cannot help but be struck by the richness of information this system provides. (Ullman 1996)

Computer vision was born as a branch of artificial intelligence in the 70s of the last century, essentially as a confluence of pattern recognition, signal and image processing, photogrammetry, perceptual psychology and neurophysiology/brain research, but has since evolved into an autonomous discipline with its own methods, paradigms and problems.

In the modern approach it does not endeavour to replicate human vision. In fact, this attempt would probably be doomed to failure because of the inherent difference between the two hardware. The comparison that is often evoked is that of flight: the efforts to replicate animal flight in history have all failed, while airplanes, with a completely different approach, have solved the problem more than satisfactorily, surpassing in some aspects the birds themselves.

1.2 Low-Level Computer Vision

Computer vision is the discipline of computer science concerned with the extraction of information from images. The information can be numerical in nature (e.g.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_1

spatial coordinates) or symbolic (e.g. identities and relationships between objects). Simplifying, we could say that it is about finding out *what* is present in the scene and *where*.

Following Ullman (1996) we use to distinguish between *low-level* and *high-level* vision. The former is concerned with extracting certain physical properties of the visible environment, such as depth, three-dimensional shape and object contours.

Conversely, *high-level* vision is concerned with the extraction of shape properties, spatial relations, object recognition and classification.

The traditional distinction between low-level and high-level, reflected in the architectures of traditional vision systems, is now very much blurred by deep learning. This book focuses on reconstructing a precise and accurate geometric model of the scene, a task that can hardly be delegated to a neural network, however deep it may be. We will study computational methods (algorithms) that aim to obtain a representation of the solid (*sterèos*) structure of the three-dimensional world sensed through two-dimensional projections of it, the images.

This approach can be easily framed within the theory of *reconstructionism* developed by Marr in the late 1970s:

> In the theory of visual processes, the underlying task is to reliably derive properties of the world from images of it; the business of isolating constraints that are both powerful enough to allow a process to be defined and generally true of the world is a central theme of our inquiry. (Marr 1982)

This, of course, is not the only possible paradigm in vision. Aloimonos and Shulman (1989), for example, argue that the description of the world should not be generic, but dependent on the goal.

Low-level computer vision can be effectively described as the inverse of computer graphics (Fig. 1.1), in which, given:

- the geometric description of the scene,
- the radiometric description of the scene (light sources and surface properties),
- the description of the acquisition device (camera),

the computer produces the synthetic image as seen by the camera.

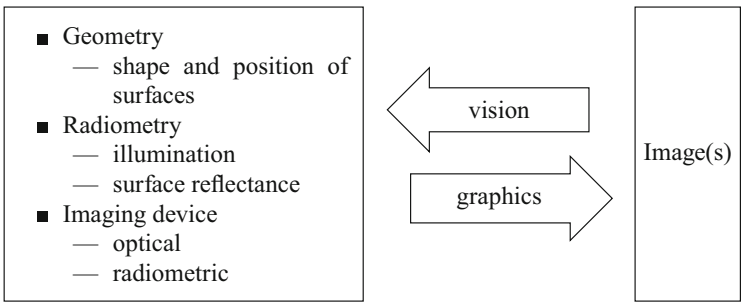


Fig. 1.1 Relation between vision and graphics

The dimensional reduction operated by the projection and the multiplicity of the causes that concur to determine the brightness and the colour make the inverse problem *ill-posed* and non-trivial. The human visual system exploits multiple visual cues (Palmer 1999) to solve this problem. Several computational techniques, collectively referred to as *shape from X* algorithms, exploit the same range of visual cues and other optical phenomena to recover the shape of objects from their images. Shape from silhouette, shape from stereo, shape from motion, shape from shading, shape from texture and shape from focus/defocus are the most commonly studied. Some of them will be discussed in this book.

As for the others, let us briefly mention them here. Shape from shading uses the gradients of the light intensity in an image to infer the 3D shape of an object (similarly to photometric stereo, but with a single image), while shape from texture uses the texture patterns present in an image to infer the 3D shape of an object. Shape from focus/defocus takes advantage of the fact that blur of an object in an image is related to its depth. More detailed descriptions of these algorithms can be found, for example, in Jain et al. (1995); Trucco and Verri (1998).

1.3 Overview of the Book

The first eight chapters of the book cover the basics of the discipline, and Chaps. 3–8 in particular should be studied sequentially as a block. The next ten can be selected to form a tailor-made course; the dependencies are shown in Fig. 1.2. Some (Chaps. 11 and 12) focus on imaging techniques to determine dense or sparse correspondences, while others deal more with geometric aspects such as multi-view reconstruction and self-calibration (Chaps. 14 and 16). The last chapter represents a departure from the main strand of reconstruction in that it deals with the synthesis of images from other images. The appendices report some mathematical facts that make the book self-contained.

1.4 Notation

The notation largely follows that of Faugeras (1993), with 3D points called M (from the French "monde") and camera matrices called P (from "projection" and "perspective"), although P for points and M for matrices would have sounded more intuitive. Vectors are in bold, the 2D ones in lower case, the 3D ones in upper case. Matrices are capitalised. The ~ above a vector is used to distinguish its Cartesian

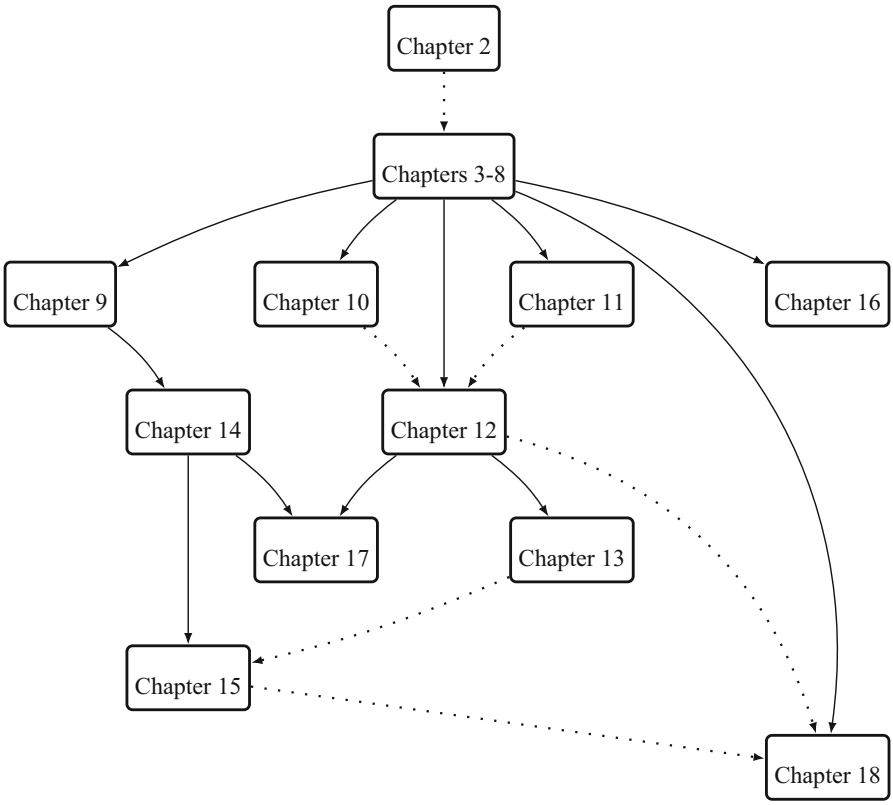


Fig. 1.2 Structure of the book with dependencies. Solid lines indicate a requirement, while dashed ones specify a recommendation

representation (with ~) from the homogeneous one (without ~). This choice is the opposite of Faugeras (1993) and is motivated by the fact that homogeneous coordinates are the default in this book, while Cartesian coordinates are rarely used.

References

J. Aloimonos and D. Shulman. *Integration of Visual Modules. An Extension to the Marr Paradigm*. Academic Press, Waltham, MA, 1989.

O. Faugeras. *Three-Dimensional Computer Vision: A Geometric Viewpoint*. The MIT Press, Cambridge, MA, 1993.

R. Jain, R. Kasturi, and B.G. Schunk. *Machine Vision*. Computer Science Series. McGraw-Hill International Editions, 1995.

D. Marr. *Vision*. Freeman, San Francisco, CA, 1982.

Stephen E. Palmer. *Vision science: photons to phenomenology*. MIT Press, Cambridge, Mass., 1999.

E. Trucco and A. Verri. *Introductory Techniques for 3-D Computer Vision*. Prentice-Hall, Upper Saddle River, NJ, 1998.

D. Ullman. *High-level Vision*. The MIT Press, Cambridge, MA, 1996.

Chapter 2
Fundamentals of Imaging



Sarà adunque pittura non altro che intersegazione della pirramide visiva, sicondo data distanza, posto il centro e constituiti i lumi, in una certa superficie con linee e colori artificiose representata.

—L. B. Alberti

2.1 Introduction

An imaging device works by collecting light reflected from objects in the scene and creating a two-dimensional image. If we want to use the image to gain information about the scene, we need to be familiar with the nature of this process that we would like to be able to reverse.

The word "camera" is the Latin word for "chamber". It was originally used in reference to the *camera obscura* (dark chamber), a dark, enclosed room used by artists and scientists in the sixteenth century to observe the projected image of an external object through a small hole. Over time, the term was adapted to refer to the devices used to capture and record images, such as cameras.

2.2 Perspective

The simplest geometric model of image formation is the *pinhole camera*, represented in Fig. 2.1; it is based on the same principle as the *camera obscura*. The tiny hole (*pinhole*) in one wall of the room lets in a ray of light for each point in the scene so that an image of the outside world is drawn on the opposite wall (*image plane*).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_2

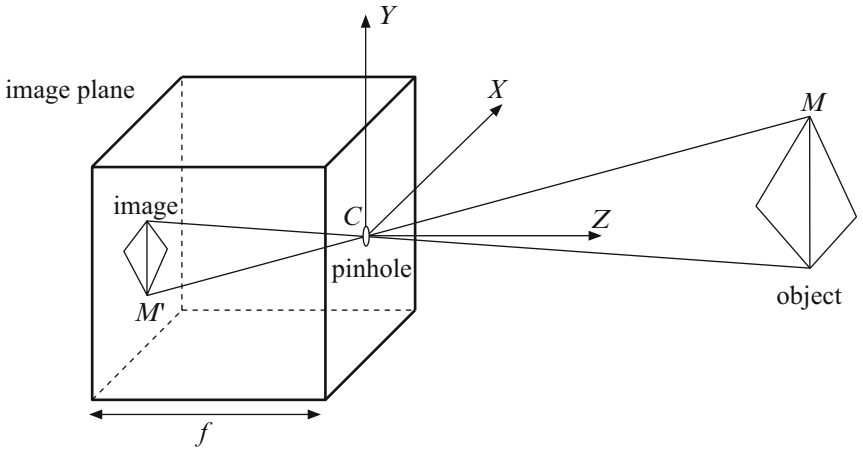


Fig. 2.1 Image formation in the camera obscura/pinhole camera

Let M be a point in the scene, of coordinates (X, Y, Z), and let M', of coordinates (X', Y', Z') be its projection onto the image plane through the pinhole C. If f is the distance of C from the image plane (*focal length*), then from the similitude of the triangles we get:

$$\frac{-X'}{f} = \frac{X}{Z} \qquad \frac{-Y'}{f} = \frac{Y}{Z} \tag{2.1}$$

and therefore

$$X' = \frac{-fX}{Z}, \quad Y' = \frac{-fY}{Z}, \quad Z' = -f. \tag{2.2}$$

Note that the image is inverted with respect to the scene, both left-right and top-bottom, as indicated by the minus sign. These equations define the image formation process, which is called *perspective projection*.

The parameter f determines the magnification factor of the image: if the image plane is close to the pinhole, the image is reduced in size, while it will become larger and larger as f increases. Since the working area of the image plane is framed, moving the image plane away also reduces the *field of view*, that is the solid angle that contains the portion of the scene that appears in the image.

The division by Z is responsible for the *foreshortening effect*, whereby the apparent size of an object in the image decreases according to its distance from the observer, such as the sleepers of the tracks of Fig. 2.2, which, although being of the same length in reality, appear shortened to different extents in the image. In the



Fig. 2.2 The projection on the left is definitely perspective—note the converging lines—while the aerial image on the right approximates an orthographic projection—the distance to the object is certainly very large compared to its depth. Photos by Markus Winkler (left) and Max Böttinger (right) from Unsplash

words of Leonardo da Vinci: "Infra le cose d'egual grandezza quella che sarà più distante dall'ochio si dimostrerà di minore figura".[1]

If the object being framed is relatively thin, compared to its average distance from the observer, one can approximate the perspective projection with the *orthographic projection*. The idea is as follows: if the depth Z of the object points varies over an interval $Z_0 \pm \Delta Z$, with $\dfrac{\Delta Z}{Z_0} \ll 1$, then the perspective scale factor f/Z can be approximated by a constant f/Z_0. The projection equations then become:

$$X' = \frac{-f}{Z_0} X, \quad Y' = \frac{-f}{Z_0} Y, \tag{2.3}$$

which is an orthographic projection composed with a scaling by a factor of f/Z_0.

Leonardo da Vinci recommended using this approximation whenever $\dfrac{\Delta Z}{Z_0} < \dfrac{1}{10}$.

2.3 Digital Images

In a digital camera, the image plane consists of a silicon image sensor, which can be a CCD (*Charge-Coupled Device*) or CMOS (*Complementary Metal-Oxide Semiconductor*) array. An image sensor measures approximately $50\,\text{mm}^2$ and contains an order of 10^6 photosensitive elements, arranged in an $n \times m$ array of

[1] "Of things of equal size, that which is most distant from the eye will appear of lesser figure".



Fig. 2.3 A digital (greyscale) image is a matrix of values between 0 and 255. The elements (pixels) become visible if the image is magnified

rectangular cells, each of which converts the energy of incident light radiation into an electrical potential.

This array is converted into a digital image (Fig. 2.3), that is, into a $N \times M$ matrix (e.g. 1024×768) of integer values (e.g. in $[0, 255]$). The elements of the matrix are named *pixels* (picture elements).

We will denote by $I(u, v)$ the brightness value of the image in the pixel identified by row v and column u. The size of the sensor matrix ($n \times m$) is not necessarily the same as the pixel matrix, or image ($N \times M$); for this reason, one pixel in the image corresponds to a rectangular area on the sensor, called *footprint* of the pixel, not necessarily equal to a cell. Its size is called *effective pixel size* (order of 10^{-6} m).

2.4 Thin Lenses

Vertebrate eyes and cameras use lenses. A lens, being larger than a pinhole, can gather more light. The downside is that not the whole scene can be in focus at the same time. The approximation we make for the optics of the imaging system—which in general is very complex, being made up of multiple lenses—is that of the *thin lens*. Thin lenses are ideal devices that enjoy the following properties:

1. The rays parallel to the optical axis incident on the lens are refracted so that they pass through a point on the optical axis called *focus* F.
2. The rays passing through the *centre* C of the lens are unaffected.

The distance of the focus F from the centre of the lens C is called *focal length* D (Fig. 2.4). It depends on the radii of curvature of the two lens surfaces and the refractive index of the constituent material.

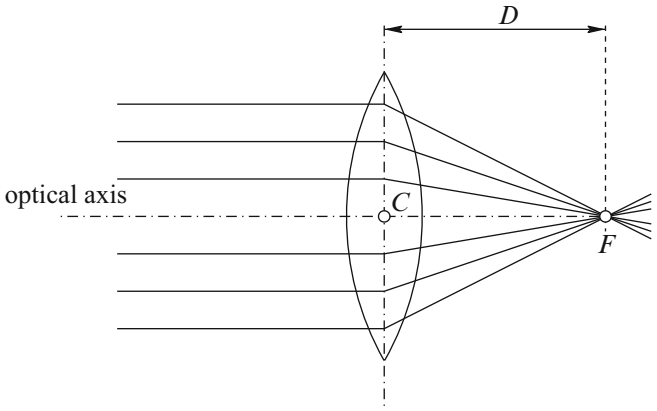


Fig. 2.4 Thin lens

For non-thin lenses there is no "center" of the lens, but two *nodal points* (front and rear) are defined to characterise its behaviour. By definition, an input ray directed at the front nodal point leads to an output ray which appears to come from the back nodal point, forming the same angle with respect to the optical axis. For a thin lens, the two nodal points coincide in the centre of the lens.

Given a point of the scene M it is possible to construct graphically the image (or *conjugate point*) M' using two special rays that depart from M: the ray parallel to the optical axis, which after refraction, passes through F and the ray that passes unchanged through C (Fig. 2.5).

Due to this construction and the similitude of the triangles, we obtain the *conjugate points formula* (or *equation of the thin lens*):

$$\frac{1}{Z} + \frac{1}{Z'} = \frac{1}{D}. \tag{2.4}$$

The image of a point in the scene at distance Z from the lens is produced (in focus) at a distance Z' from the lens, which depends on the depth Z of the point and the focal length D of the lens.

The image of the point M, when in focus, is formed on the image plane at the same point predicted by the pinhole model with the pinhole coincident with the centre of the lens C: in fact, the ray passing through C is common to the two geometric constructions. The other light rays that leave M and are collected by the lens increase the light reaching M'.

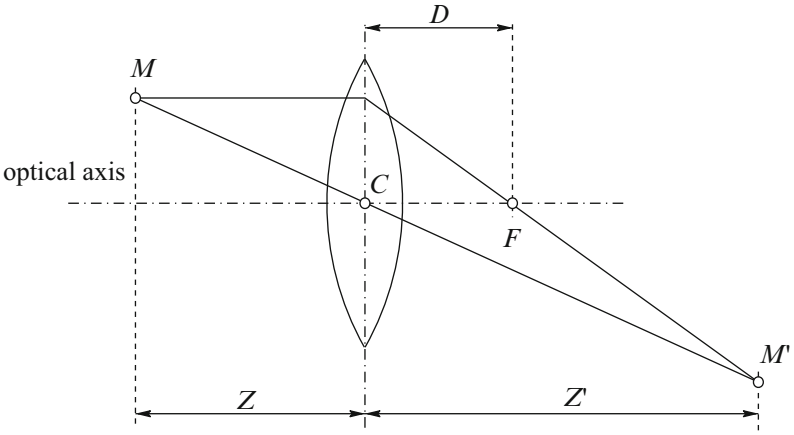


Fig. 2.5 Construction of the conjugate point

If (2.4) is not verified one gets a blurred image of the point, that is, a circle which is named *circle of confusion*. The photosensitive elements have a small but finite size. As long as the circle of confusion does not exceed the size of the photosensitive element, the image is in focus. Therefore, there is an interval of depths for which the points are in focus. This range is called *depth of field*. It is inversely proportional to the diameter of the lens; in fact, the pinhole camera has an infinite depth of field. The light that is collected, on the other hand, is directly proportional to the diameter of the lens.

The focal length of the lens is a different notion from the focal length of the pinhole camera. Two different notations, f and D, were used on purpose, although, unfortunately, the two quantities have the same name.

2.4.1 Telecentric Optics

It is interesting to note that by placing a diaphragm with a hole in correspondence of the focus F, as in Fig. 2.6, rays leaving M in a direction parallel to the optical axis are selected, while other rays are blocked. This results in a device that produces an image according to a pattern that is obviously different from the pinhole/perspective model. The image of the point M, in fact, does not depend on its depth: this is indeed a *telecentric camera*, which realises an orthographic projection.

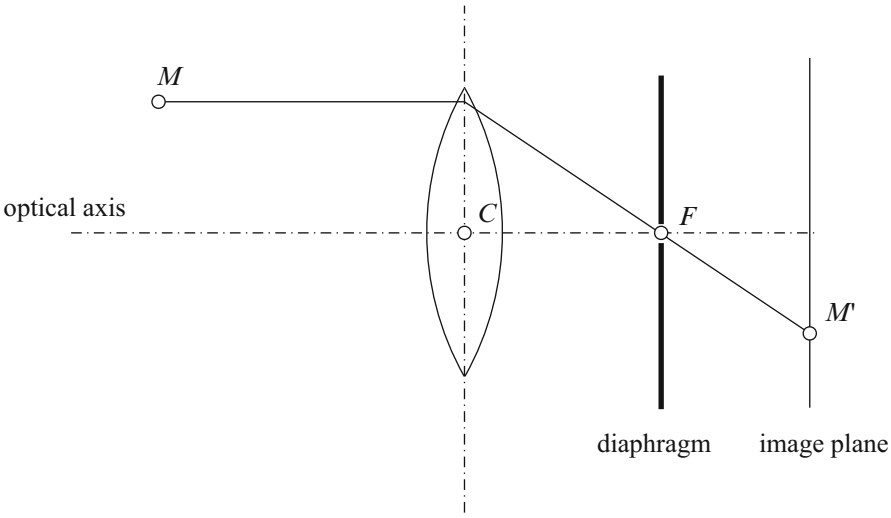


Fig. 2.6 Principle of the telecentric camera

2.5 Radiometry

Light is emitted by the sun or other sources and interacts with objects in the scene; part is absorbed, and part is scattered and propagates in new directions. Among these directions, some points towards the camera sensor and contributes to the image formation.

The brightness $I(M')$ of a pixel M' in the image I is proportional to the "amount of light" that the surface S_M—centred at a point M—scatters in the direction of the camera, where S_M is the area of the surface that projects into the pixel M'. This in turn depends on the surface reflectance and the spatial distribution of the light sources.

A material is characterised from the radiometric point of view by its *reflectance*, defined as is the ratio of energy reflected to the total energy incident on a surface. Reflectance is largely determined by the physical properties of the surface in relation to the wavelength and incident/viewing angle.

The "amount of light" travelling along a ray is formally measured by the *radiance*: radiance $L(M, \mathbf{v})$ is defined as the power of light radiation per unit area per unit solid angle emitted (or received) by point M along the direction $\mathbf{v}$.

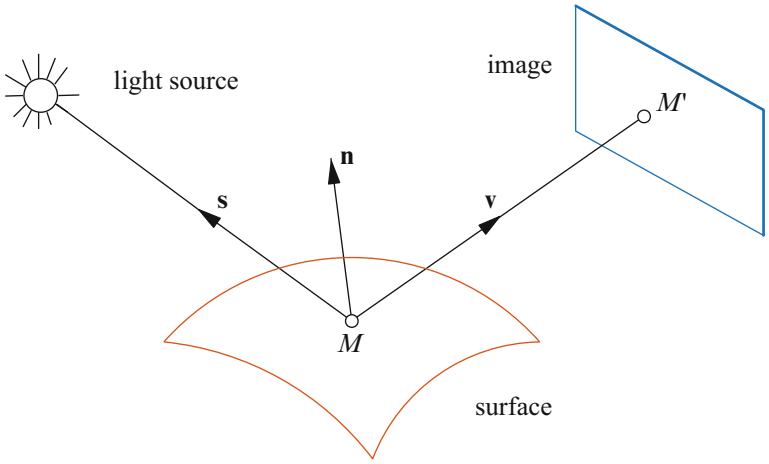


Fig. 2.7 Elements involved in the reflectance equation

Neglecting attenuation due to the atmosphere, the radiance L_o leaving a point M of the surface in the direction of M' coincides with the radiance reaching M' from the same direction, therefore:

$$I(M') = L_o(M, \mathbf{v}). \tag{2.5}$$

The reflectance of a material is encoded by its *bidirectional reflectance distribution function* (BRDF). The BRDF $\rho(M, \mathbf{v}, , \mathbf{s})$ depends on the surface point M, the lighting direction $\mathbf{s}$ and the viewing direction $\mathbf{v}$ (refer to Fig. 2.7). In general, given a point M, ρ is a 4D function parameterised by the angular coordinates of $\mathbf{v}$ and $\mathbf{s}$. For many materials, however, it depends only on the mutual angles between the surface normal $\mathbf{n}$, $\mathbf{v}$ and $\mathbf{s}$ (Woodham 1980), so it reduces to 3D.

The radiance $L_o(M, \mathbf{v})$ leaving M in the direction $\mathbf{v}$ is given by the *reflectance equation*:

$$L_o(M, \mathbf{v}) = \int_{\mathbf{s}\in\Omega} \rho(M, \mathbf{v}, \mathbf{s}) L_i(M, \mathbf{s})(\mathbf{n}^\mathsf{T}\mathbf{s})\ \mathrm{d}\mathbf{s} \tag{2.6}$$

where the integral computes the radiance incident at point M from all directions in a hemisphere above the surface centred on the surface normal $\mathbf{n}$.

The scalar product $(\mathbf{n}^\mathsf{T}\mathbf{s})$ takes into account the fact that the radiance is measured on a unit area that is perpendicular to $\mathbf{s}$, while the surface is oriented as $\mathbf{n}$. In order to accurately calculate the reflectance equation, all direct and indirect light sources should be considered in the integral, and the radiance reaching M would then depend on every other surface patch visible from it. However, in a first approximation, only the primary sources are taken into account and, thanks to the principle of superposition of effects, only one source can be considered at a time. If the light

source is assumed to be a distant point, the integral reduces to a single evaluation of the BRDF, and the reflectance equation can be written as follows:

$$L_o(M, \mathbf{v}) = \rho(M, \mathbf{v}, \mathbf{s}) L_i(M, \mathbf{s}) \mathbf{n}^\mathsf{T}\mathbf{s} \, [\![\mathbf{n}^\mathsf{T}\mathbf{s} \geq 0]\!]. \tag{2.7}$$

where $L_i(M, \mathbf{s})$ is the emitted radiance, $\mathbf{s}$ is the direction under which M sees the light source and $\mathbf{n}$ is the versor of the normal in M (refer to Fig. 2.7). The Iverson brackets $[\![\cdot]\!]$ are introduced to account only for light contributions that form an acute angle with $\mathbf{n}$.

The *Iverson brackets* $[\![\cdot]\!]$ construct a binary function from a predicate $P(x)$ and are defined as follows:

$$[\![P(x)]\!] = \begin{cases} 1, & \text{if } P(x) \\ 0, & \text{otherwise .} \end{cases} \tag{2.8}$$

In MATLAB this is implicitly done by identifying logical values with 1/0.

The simplest BRDF is the constant one, which describes materials that scatter the light in all directions uniformly. This phenomenon is called *diffuse scattering* and follows the so-called *Lambert's law*:

$$L_o(M) = \rho(M) L_i(M, \mathbf{s}) \mathbf{n}^\mathsf{T}\mathbf{s} \, [\![\mathbf{n}^\mathsf{T}\mathbf{s} \geq 0]\!]. \tag{2.9}$$

The radiance $L_o(M)$ emitted by M does not depend on the viewing direction, so diffusive surfaces appear equally bright from every angle. $\rho(M)$ is the albedo of M, which takes values between 0 (the light is not reflected by the object and the material appears black) and 1 (all light is reflected and the material appears of the colour of the light). At the opposite side of an ideal spectrum of behaviours lies the specular reflection, where scattering is concentrated along one direction only, the one for which the reflected and incident rays lie in the same plane and the angle of reflection is equal to the angle of incidence: this is the behaviour of a perfect mirror.

This topic is extensively discussed in computer graphics textbooks (Akenine-Möller et al. 2008); for our purposes we will assume that the surfaces of the scene are diffusive, and treat deviations from this model as disturbances.

References

Tomas Akenine-Möller, Eric Haines, and Naty Hoffman. *Real-Time Rendering 3rd Edition*. A. K. Peters, Ltd., Natick, MA, USA, 2008. ISBN 987-15-68814-24-7.

Robert J. Woodham. Photometric method for determining surface orientation from multiple images. *Optical Engineering*, 19(1):191139, 1980.

Chapter 3
The Pinhole Camera Model



3.1 Introduction

Referring back to the geometric aspect of image formation, covered in Chap. 2, we shall now introduce a geometric model for the camera, that is, we will study how the position of a point in the scene and the position of the corresponding point in the image are related. We make the simplifying assumption that lenses can be neglected as far as the geometry of projection is concerned.

3.2 Pinhole Camera

The geometric model of the camera that we will adopt is the so-called *pinhole* or perspective model, which consists (refer to Fig. 3.1) of the *image plane* $\mathcal{Q}$ and the *centre of projection* (Centre of Projection (COP)) C, located at a distance f (*focal length*) from the image plane. The bundle of lines with centre in C is also called the *projective bundle*.

The line through C orthogonal to $\mathcal{Q}$ is the *principal axis* of the camera and its intersection with $\mathcal{Q}$ is called the *principal point*. The plane $\mathcal{F}$ parallel to $\mathcal{Q}$ and containing C is called the *focal plane*. The points in $\mathcal{F}$ are projected to infinity on $\mathcal{Q}$.

To analytically describe the perspective projection operated by the camera, we need to introduce appropriate Cartesian coordinate systems in which to represent the points in the three-dimensional (3D) space and the points in the image plane.

Let us consider, initially, a *very special case*, in which the coordinate systems are chosen so as to obtain particularly simple equations. In Sect. 3.4 we will enhance the model by studying the more general case.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_3

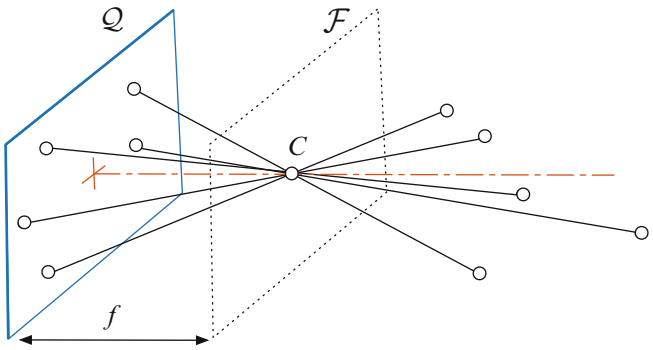


Fig. 3.1 Pinhole camera. C is the COP, $\mathcal{Q}$ is the image plane and f is the focal length

3.3 Simplified Pinhole Model

The coordinate system in which the points of the three-dimensional space are represented is called the *world* (or *object*) coordinate system.

Let us introduce the *camera reference frame*, a right-handed triplet (X, Y, Z) of axes centred in C and with the Z axis coincident with the principal axis. In this simplified model, the world reference frame is chosen coincident with the camera reference frame.

We also introduce a reference frame (u, v) for the plane $\mathcal{Q}$ centred at the principal point and with the axes u and v oriented as X and Y, respectively, as shown in Fig. 3.2.

Now consider a point M of coordinates $\widetilde{\mathbf{M}} = [X, Y, Z]^\mathsf{T}$ in 3D space and let m of coordinates $\widetilde{\mathbf{m}} = [u, v]^\mathsf{T}$ be its projection onto $\mathcal{Q}$ through C. In the following, unless otherwise noted, we will always use ~ to denote Cartesian coordinates and distinguish them from homogeneous coordinates.

By simple considerations on the similitude of triangles (Fig. 3.3), we arrive at the following relation:

$$\frac{f}{Z} = \frac{-u}{X} = \frac{-v}{Y} \tag{3.1}$$

that is

$$\begin{cases} u = \dfrac{-f}{Z}\, X \\ v = \dfrac{-f}{Z}\, Y. \end{cases} \tag{3.2}$$

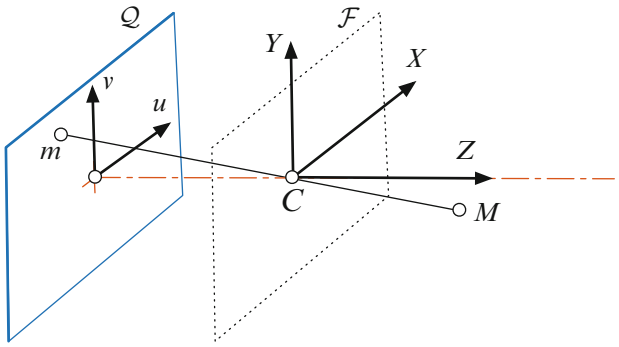


Fig. 3.2 Simplified model of the camera with reference frames. The point M is projected onto the point m where the line CM intersects the image plane $\mathcal{Q}$

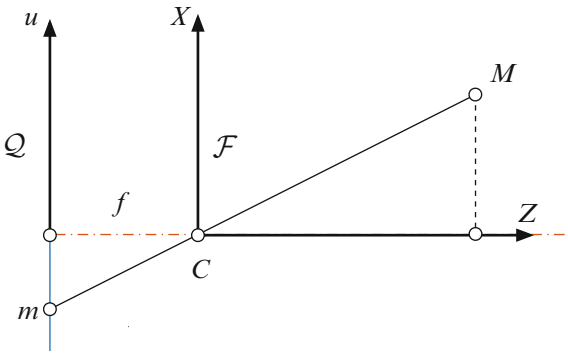


Fig. 3.3 Nadiral view of the camera model. The two triangles that have $\overline{CM}$ and $\overline{Cm}$ for hypotenuse are similar to each other

This is the *perspective projection*. The transformation in Cartesian coordinates is clearly non-linear because of the division by Z. Using homogeneous coordinates[1] instead it becomes *linear*.

Let therefore be

$$\mathbf{m} = \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} \quad \text{and} \quad \mathbf{M} = \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{3.3}$$

the homogeneous coordinates of m and M, respectively. Note that by setting the last coordinate to 1, we have excluded the points at infinity (to include them we would

[1] The homogeneous coordinates of a 2D point in the image are the (x_1, x_2, x_3), where $(x_1/x_3, x_2/x_3)$ are the corresponding Cartesian coordinates. Thus, there is a one-to-many correspondence between Cartesian and homogeneous coordinates. Homogeneous coordinates can represent any point in the Euclidean plane and also points at infinity, which have the third component equal to zero and therefore do not have a corresponding Cartesian representation. See Appendix D.

have used a generic last component). So the equation for perspective projection, in this simplified case, rewrites:

$$Z\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = Z\begin{bmatrix} -fX/Z \\ -fY/Z \\ 1 \end{bmatrix} = \begin{bmatrix} -fX \\ -fY \\ Z \end{bmatrix} = \underbrace{\begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{P} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}. \tag{3.4}$$

Switching to matrix notation:

$$\mathbf{m} = \frac{1}{Z} P\mathbf{M} \tag{3.5}$$

or, also:

$$\mathbf{m} \simeq P\mathbf{M} \tag{3.6}$$

where $\simeq$ means "equal up to a scaling factor".

The matrix P represents the geometric model of the camera and is called Perspective Projection Matrix (PPM) or *camera matrix*.

Listing 3.1 reports the implementation of a generic projective transformation, which includes perspective projection as a special case.

Listing 3.1 Projective transformation

```
function Y = htx( T, X )
%HTX Apply homogeneous transform
%   Apply transformation T in homogeneous coordinates
%   T can be any dimension
%   T 3x4 is a projection
%   T 3x3 is a transformation of the projective plane
%   T 4x4 is a transformation of the projective space

Y = T * [X; ones(1, size(X,2))]; % apply tranform

nr = size(Y,1);

Y = Y ./ repmat(Y(nr,:),nr,1); % perspective division

Y(nr,:) = []; % remove trailing ones

end
```

3.4 General Pinhole Model

A realistic model of a camera, describing the transformation from 3D coordinates to pixel coordinates, as well as the perspective transformation (Fig. 3.4), must take into account:

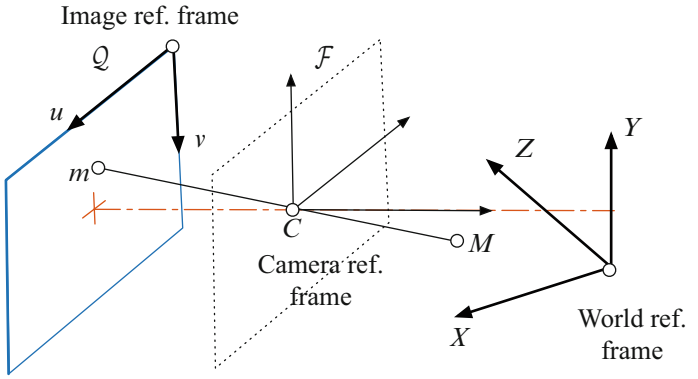


Fig. 3.4 General model of the camera with reference frames. Note that the image plane should be viewed from the half-space opposite to C, so that the origin of the image reference is in the upper left corner of the image

- The change of coordinates from metres in the sensor plane to pixels in the image.
- The change of coordinates between the camera reference frame and the world reference frame.

3.4.1 Intrinsic Parameters

The transformation from the sensor coordinates to pixels is modelled by an affine transformation A that takes into account the translation of the origin at the upper left corner, the independent rescaling of the u and v axes for changing the unit of measurement from metres to pixels and the reversal of the direction of the axes:

$$A = \begin{bmatrix} -k_u & 0 & u_0 \\ 0 & -k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3.7}$$

where (u_0, v_0) are the coordinates in pixels of the principal point and k_u (k_v) is the inverse of the *effective pixel size* (see Sect. 2.3) along the direction u (v), measured in m^{-1}. The matrix A transforms the coordinates from metres to pixels; hence, it pre-multiplies the PPM, which becomes:

$$P = \begin{bmatrix} -k_u & 0 & u_0 \\ 0 & -k_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} -f & 0 & 0 & 0 \\ 0 & -f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} fk_u & 0 & u_0 & 0 \\ 0 & fk_v & v_0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = K[I_3, \mathbf{0}] \tag{3.8}$$

where

$$K = \begin{bmatrix} fk_u & 0 & u_0 \\ 0 & fk_v & v_0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{and} \quad [I_3, \mathbf{0}] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}. \tag{3.9}$$

K is the *matrix of intrinsic parameters*, while $[I_3, \mathbf{0}]$ encodes the essence of the perspective transformation, without any parameters. It is obtained when the world reference frame coincides with the camera reference frame and the image points are expressed in *normalised image coordinates*, that is,

$$\mathbf{q} = K^{-1}\mathbf{m}. \tag{3.10}$$

Note that whereas pixel coordinates are always measurable in the image, to step to normalised image coordinates the matrix of intrinsic parameters is needed.

The most general K includes also the *skew* angle θ that corresponds to the angle between the u and v axes, that normally is $\pi/2$:

$$K = \begin{bmatrix} fk_u & -fk_u/\tan\theta & u_0 \\ 0 & fk_v/\sin\theta & v_0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{3.11}$$

Let us define the focal length expressed in (horizontal) pixels $\alpha_u = fk_u$, and the pixel aspect ratio $\tau = k_v/k_u$, then the most general matrix of intrinsic parameters writes:

$$K = \begin{bmatrix} \alpha_u & -\alpha_u/\tan\theta & u_0 \\ 0 & \tau\alpha_u/\sin\theta & v_0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{3.12}$$

It has five degrees of freedom, for it depends on five *intrinsic parameters*: $\alpha_u, u_0, v_0, \tau, \theta$. Reduced parameterisation from five to one parameter can be obtained by making educated guesses on some of them. Normally, it is safe to assume $\theta = \pi/2$, as we implicitly did in the beginning. Also, very often the footprint of the pixel is square, that is, $\tau = 1$. Pushing this further, one could reduce the intrinsic parameters to the sole focal length α_u by fixing the principal point (u_0, v_0) in the centre of the image.

Some authors parameterise K with its entries by introducing the focal length expressed in vertical pixels α_v and the skew parameter s:

$$K = \begin{bmatrix} \alpha_u & s & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{3.13}$$

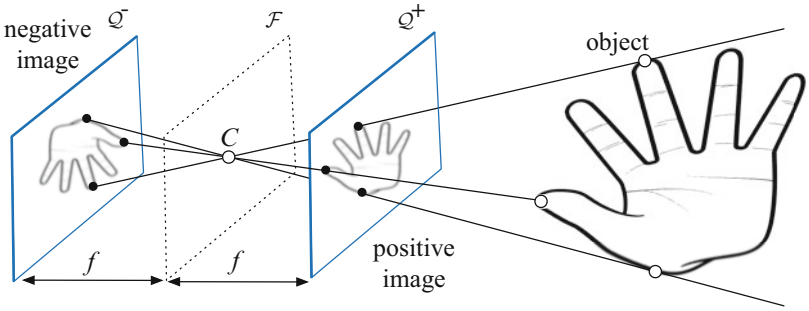


Fig. 3.5 Positive and negative images

Although from the physical point of view the image plane can only be behind the pinhole, ideally it can also be placed in front, at the same distance f, obtaining the so-called *positive image*, which has the same size of the *negative image* (refer to Fig. 3.5), but which is not inverted. This trick was first recommended in the Renaissance by Leon Battista Alberti. The choice of whether to have positive or negative image is only a convention: what changes is the sign of f.

So far we have modelled a real camera, with negative focal length $(-f)$, in which the image plane is placed behind the pinhole and the image is the inverted (if we entered a real *camera obscura*, this is how we would see the image). However, the images we receive from electronic devices are not inverted, so the positive focal $(+f)$ model is the one most consistent with the digital image formation process. We will therefore choose this model from now on. Beware that positive focal implies $K_{11} < 0$ and $K_{22} < 0$.

3.4.1.1 Field of View

The *field of view* is the apex angle of the *pyramid of vision*, that is the pyramid with apex in the COP that extends along the optical axes to infinity and encompasses the extent of the world that is imaged by the camera. The same apex angle is shared by the pyramid that has the image as its base and apex in the COP, whose height is the focal length (Fig. 3.6).[2]

Since the base of the pyramid is—in general—a rectangle, there are two apex angles, one horizontal and one vertical. The horizontal field of view ν, for example, is related to the focal length α_u through the image width w by:

[2] Humans instead have a circular sensor; hence, we speak of the *cone of vision*. People have an approximate 60° angle of undistorted vision that extends as a fictitious cone from their eyes forward.

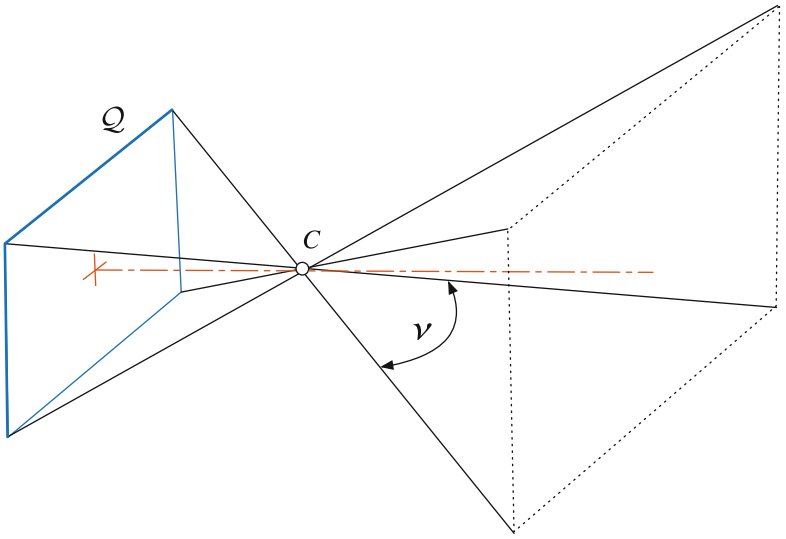


Fig. 3.6 The pyramid of vision (extending to the right of C) and the field of view ν

$$\nu = 2 \arctan\left(\frac{w}{2\alpha_u}\right) \tag{3.14}$$

And, conversely:

$$\alpha_u = \frac{w}{2\tan(\nu/2)} \tag{3.15}$$

Focal length and field of view are related, but the latter is a more intelligible parameter than the focal length in pixels: an educated guess for ν lies in the range from 15° to 135°. For this reason, in our MATLAB toolkit we choose to parameterise α_u as in (3.15), obtaining the following five intrinsic parameters: $\nu, u_0, v_0, \tau, \theta$. Thanks to the modular approach of the toolkit, however, this aspect remains encapsulated in the function `par2K` (Listing 3.2) that computes the parameterisation and its derivative. We will henceforth use α_u as the first parameter in the text, but it is understood that the implementation uses ν instead.

3.4.2 Extrinsic Parameters

To account for the fact that—in general—the world reference frame does not coincide with the camera reference frame, we must introduce a coordinate change consisting of a direct *isometry*, that is, a rotation $R \in SO(3)$ followed by a translation $\mathbf{t}$. Let us denote by $\mathbf{M}_c$ the homogeneous coordinates of a point in the

Listing 3.2 Parameterisation of K (Jacobian omitted for space reasons)

```
function [K,J] = par2K( p_in , sz)
%PAR2K Return a K matrix and its jacobian from parameters

n = length(p_in); % number of input parameters
std_width = 2000; % standard image width

if nargin < 2
    sz = [std_width std_width*3/4]; % default size
end
p =  [-1.2, sz(1)/2, sz(2)/2, 1, pi/2]; % default values

p(1:n) = p_in;   % overwrite default

a =  0.5*sz(1)/tan(0.5*p(1));

K = [ a, -a/tan(p(5)),       p(2)
      0,  a*p(4)/sin(p(5)),  p(3)
      0   0                  1   ];
```

camera coordinate system and $\mathbf{M}$ the homogeneous coordinates of the same point in the world coordinate system. We can therefore write:

$$\mathbf{M}_c = G\mathbf{M} \tag{3.16}$$

where

$$G = \begin{bmatrix} R & \mathbf{t} \\ 0 & 1 \end{bmatrix}. \tag{3.17}$$

The matrix G encodes the *exterior orientation* (position and angular attitude[3]) of the camera with respect to the world reference frame. Specifically, the rows of R are the X, Y and Z axes of the camera reference frame expressed in the world coordinate system.

By appropriately parameterising the rotation R (see Sect. A.15), the exterior orientation of the camera is encoded by six *extrinsic parameters*: three Euler angles ω, φ, κ that specify the rotation and three components t_1, t_2, t_3 for the translation.

From (3.8) we obtain:

$$\mathbf{m} \simeq K[I_3, \mathbf{0}]\mathbf{M}_c, \tag{3.18}$$

and substituting (3.16) into the equation above gives:

$$\mathbf{m} \simeq K[I_3, \mathbf{0}]G\mathbf{M}, \tag{3.19}$$

and therefore

[3] In navigation orientation is the determination of the east cardinal point (*oriens* in latin); by extension, in topography and photogrammetry, it indicates the determination of position and direction with respect to cardinal points or other reference frames.

$$P = K[I_3, \mathbf{0}]G. \tag{3.20}$$

This is the most general form of the perspective projection matrix: G encodes the *exterior orientation* of the camera, K contains its intrinsic parameters (by analogy, we say that it encodes the *interior orientation*), while the matrix $[I, \mathbf{0}]$ represents a perspective transformation from camera coordinates to normalised image coordinates.

Depending on convenience, we may also consider the following expression for the PPM:

$$P = K[R, \mathbf{t}] \tag{3.21}$$

which is obtained by substituting into (3.20) the block form of G and developing the product with the central matrix.

By letting

$$\mathbf{t} = \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix} \quad \text{e} \quad R = \begin{bmatrix} \mathbf{r}_1^\mathsf{T} \\ \mathbf{r}_2^\mathsf{T} \\ \mathbf{r}_3^\mathsf{T} \end{bmatrix} \tag{3.22}$$

we get the following expression for P as a function of the elements of the matrices K and G:

$$P = \begin{bmatrix} \alpha_u \mathbf{r}_1^\mathsf{T} + u_0 \mathbf{r}_3^\mathsf{T} & \alpha_u t_1 + u_0 t_3 \\ \alpha_v \mathbf{r}_2^\mathsf{T} + v_0 \mathbf{r}_3^\mathsf{T} & \alpha_v t_2 + v_0 t_3 \\ \mathbf{r}_3^\mathsf{T} & t_3 \end{bmatrix}. \tag{3.23}$$

The rows of R could be further exploded into sines and cosines of the angles ω, φ and κ (see Sect. A.15).

A PPM is defined up to an arbitrary scale factor. In fact, if one replaces P with λP in (3.6), one obtains the same projection, for every non-zero real λ. Thus, if one wants to bring a generic PPM into the form of (3.23), it is necessary to rescale it so that the vector corresponding to $\mathbf{r}_3$ has norm one.

P has 12 entries but has 11 degrees of freedom since it loses one due to the scaling factor: this implies that it must depend on 11 independent parameters, that are 5 intrinsic parameters plus 6 extrinsic parameters.

Let us summarise, with the help of Fig. 3.7, the coordinate systems we introduced and their relationship.

World coordinates: are the generic coordinates that represent points in 3D space, denoted by $\widetilde{\mathbf{M}} = (X, Y, Z)$.

(continued)

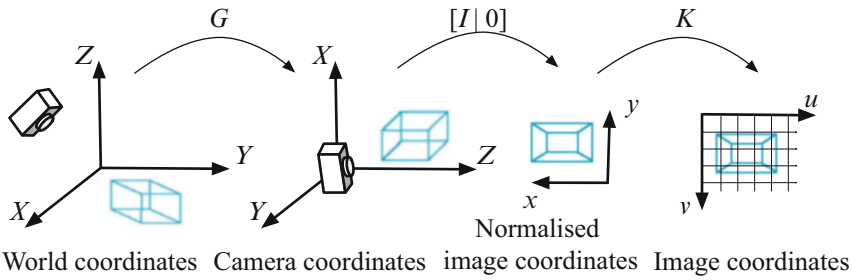


Fig. 3.7 Coordinate systems involved in camera modelling

Camera coordinates: are the coordinates of the 3D points in a reference frame with origin in the COP and integral with the camera. They will often be referred to with the same notation as the world coordinates $\widetilde{\mathbf{M}} = (X, Y, Z)$ and only in special cases, where they need to be distinguished, will a subscript be used ($\widetilde{\mathbf{M}}_c$).

Normalised image coordinates: are measured in metres with origin at the principal point and denoted by $\widetilde{\mathbf{q}} = (x, y)$.

Image coordinates: are measured in pixels with origin in the upper left corner of the image and denoted by $\widetilde{\mathbf{m}} = (u, v)$.

3.5 Dissection of the Perspective Projection Matrix

In this section we shall see how to derive some properties of the perspective projection directly from the PPM.

Let us write the PPM according to its rows

$$P = \begin{bmatrix} \mathbf{p}_1^\top \\ \mathbf{p}_2^\top \\ \mathbf{p}_3^\top \end{bmatrix} \tag{3.24}$$

and insert this into (3.6):

$$\mathbf{m} \simeq \begin{bmatrix} \mathbf{p}_1^\top \mathbf{M} \\ \mathbf{p}_2^\top \mathbf{M} \\ \mathbf{p}_3^\top \mathbf{M} \end{bmatrix} = \begin{bmatrix} \mathbf{p}_1^\top \\ \mathbf{p}_2^\top \\ \mathbf{p}_3^\top \end{bmatrix} \mathbf{M} \tag{3.25}$$

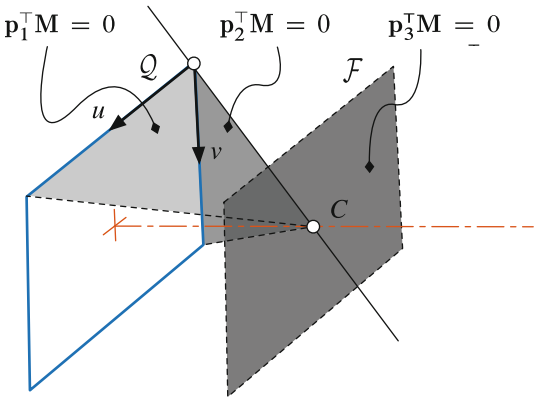


Fig. 3.8 Geometric determination of the centre of projection

so that the perspective projection equation becomes (this is the generalisation of (3.2)):

$$\begin{cases} u = \dfrac{\mathbf{p}_1^\top \mathbf{M}}{\mathbf{p}_3^\top \mathbf{M}} \\ v = \dfrac{\mathbf{p}_2^\top \mathbf{M}}{\mathbf{p}_3^\top \mathbf{M}}. \end{cases} \tag{3.26}$$

The focal plane $\mathcal{F}$ has equation $\mathbf{p}_3^\top \mathbf{M} = 0$ and contains the points that project to infinity (excluding C). The planes of equation $\mathbf{p}_1^\top \mathbf{M} = 0$ and $\mathbf{p}_2^\top \mathbf{M} = 0$ contain the points that project into the image axes $u = 0$ and $v = 0$, respectively (Fig. 3.8).

The COP is defined by the intersection of these three planes. The intersection of $\mathbf{p}_1^\top \mathbf{M} = 0$ and $\mathbf{p}_2^\top \mathbf{M} = 0$ is the line projecting into $(0, 0)$, which must necessarily pass through C, which belongs to $\mathcal{F}$. Therefore, C must belong to all three of these planes:

$$\begin{cases} \mathbf{p}_1^\top \mathbf{C} = 0 \\ \mathbf{p}_2^\top \mathbf{C} = 0 \\ \mathbf{p}_3^\top \mathbf{C} = 0 \end{cases} \tag{3.27}$$

that is, $P\mathbf{C} = \mathbf{0}$. In other words, C is the null space of P, which, by the rank-nullity theorem (Theorem A.2) has dimension one. Note that the COP is the only point in space for which the projection is not defined, and, in fact, $\mathbf{0}$ is the only triplet that does not represent any point in the projective plane.

If we partition P as $P = [\widetilde{P}, \mathbf{p}_4]$ and $\mathbf{C} = [\widetilde{\mathbf{C}}, 1]^\top$ the system (3.27) rewrites:

$$\widetilde{P}\widetilde{\mathbf{C}} + \mathbf{p}_4 = \mathbf{0}. \tag{3.28}$$

The coordinates of the COP are:

$$\widetilde{\mathbf{C}} = -\widetilde{P}^{-1}\mathbf{p}_4. \tag{3.29}$$

Setting $P = K[R, \mathbf{t}]$ in the above equation yields the relation between the COP and the translation component of the exterior orientation:

$$\widetilde{\mathbf{C}} = -R^{\mathsf{T}}\mathbf{t}. \tag{3.30}$$

The parameterisation of the exterior orientation of a PPM through the Euler angles ω, φ, κ and the translation $\mathbf{t}$ although useful and appropriate from the mathematical point of view is not very intuitive to the human user. The main graphics libraries, for example, allow to specify a camera by: position ("where the eye is"), a point on the optical axis ("where it is looking") and a point "above" to fix the vertical direction. Listing 3.3 shows a MATLAB implementation of this idea; it can be used in simulated experiments to get a realistic camera placement. Note that the function returns only the exterior orientation of the camera, that is, $P = [R, \mathbf{t}]$.

Listing 3.3 Construction of the PPM

```
function G = camera( cop, look, up )
%CAMERA Camera exterior orientation given COP,LOOK and UP

    R(3,:) = (look(:) - cop(:))';                % Z axis
    R(3,:) = R(3,:)/norm(R(3,:));
    R(1,:) = cross((up(:) - cop(:))',R(3,:))';   % X axis
    R(1,:) = R(1,:)/norm(R(1,:));
    R(2,:) = cross(R(3,:),R(1,:));               % Y axis

    G = [R, -R*cop(:) ];
end
```

The *optical ray* of point m is the straight line through the COP C and m itself, that is, the geometric locus of points $\mathbf{M}$ such that $\mathbf{m} \simeq P\mathbf{M}$. On the optical ray of m lie all points in space of which the point m is the projection.

One point that belongs to this line is C, by definition. Another is the point (at infinity) that has coordinates:

$$\begin{bmatrix} \widetilde{P}^{-1}\mathbf{m} \\ 0 \end{bmatrix},$$

in fact, it suffices to project this point to verify that:

$$P \begin{bmatrix} \widetilde{P}^{-1}\mathbf{m} \\ 0 \end{bmatrix} = \widetilde{P}\widetilde{P}^{-1}\mathbf{m} = \mathbf{m}. \tag{3.31}$$

The parametric equation of the optical ray of $\mathbf{m}$ is therefore the following:

$$\mathbf{M} = \mathbf{C} + \lambda \begin{bmatrix} \widetilde{P}^{-1}\mathbf{m} \\ 0 \end{bmatrix}, \quad \lambda \in \mathbb{R} \cup \{\infty\}. \tag{3.32}$$

3.5.1 Collinearity Equations

Let us rewrite the perspective projection in normalised image coordinates, with the points M in camera coordinates $\widetilde{\mathbf{M}}_c = [X_c, Y_c, Z_c]^\mathsf{T}$:

$$\begin{cases} x = \dfrac{X_c}{Z_c} \\ y = \dfrac{Y_c}{Z_c}. \end{cases} \tag{3.33}$$

Using (3.30) we get that $\widetilde{\mathbf{M}}_c = R\widetilde{\mathbf{M}} + \mathbf{t} = R(\widetilde{\mathbf{M}} + R^\mathsf{T}\mathbf{t}) = R(\mathbf{M} - \widetilde{\mathbf{C}})$ then

$$\begin{cases} x = \dfrac{\mathbf{r}_1^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}{\mathbf{r}_3^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})} \\ y = \dfrac{\mathbf{r}_2^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}{\mathbf{r}_3^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}. \end{cases} \tag{3.34}$$

Furthermore, being:

$$\begin{cases} u = u_0 + \alpha_u x \\ v = v_0 + \alpha_v y \end{cases} \tag{3.35}$$

one finally has:

$$\begin{cases} u = u_0 + \alpha_u \dfrac{\mathbf{r}_1^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}{\mathbf{r}_3^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})} \\ v = v_0 + \alpha_v \dfrac{\mathbf{r}_2^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}{\mathbf{r}_3^\mathsf{T}(\widetilde{\mathbf{M}} - \widetilde{\mathbf{C}})}. \end{cases} \tag{3.36}$$

We obtained what in photogrammetry are known as *collinearity equations* (Kraus 2007). They express the fact that the COP C, the object point M and the image point m are aligned on the same optical ray. The rows of R can be further expanded into

their components, which are combinations of trigonometric functions of the angles ω, φ, κ.

3.6 Radial Distortion

Real photographic objectives are complex optical system made of non-thin lenses. In such devices one can identify several *nodal points*, placed at a certain distance from each other.

If these lie along the optical axis of the system and other alignment conditions hold (*orthoscopy* condition), we can simplify the system and consider only two nodal points: one for the outer air-glass interface and one for the inner glass-air interface. In this condition the ray passing through the outer nodal point and the ray passing through the inner nodal point are parallel, and therefore, we can consider null the distance between the two nodal points, thus obtaining the *thin lens* model.

If the orthoscopy condition is violated, the thin lens model is not applicable and this in turn results in a deviation from the pinhole model, which causes a distortion of the image, called *radial distortion*, whose effect is illustrated in Fig. 3.9.

The standard model in computer vision[4] is a transformation defined on the normalised image coordinates that brings the ideal (undistorted) ones (x, y) onto the observable (distorted) ones $(\check{x}, \check{y})$:

$$\begin{cases} \check{x} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \ldots) \\ \check{y} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6 \ldots) \end{cases} \tag{3.37}$$

where $r^2 = x^2 + y^2$. The degree of the polynomial in r^2 determines the accuracy with which one wants to model the phenomenon; however, be careful that too high a degree may lead to modelling the noise that inevitably plagues the measurements, instead of the phenomenon itself. Very often we stop at the first coefficient and in any case rarely exceed the fourth.

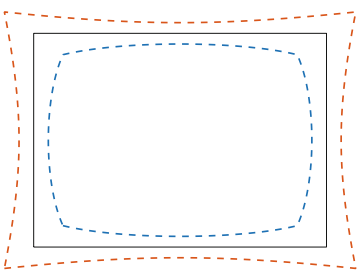


Fig. 3.9 Cushion (outer dashed red line) and barrel (inner dashed blue line) radial distortion, obtained with $k_1 = 0.4$ and $k_1 = -0.4$, respectively

[4] Beware that other models are in use in the field of photogrammetry.

We call $\widetilde{\rho} : \mathbb{R}^2 \to \mathbb{R}^2$ the function defined by (3.37), such that

$$\widetilde{\rho}\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) = \begin{bmatrix} \breve{x} \\ \breve{y} \end{bmatrix} \tag{3.38}$$

where the $\widetilde{\ }$ indicates that the function works in Cartesian coordinates.

Listing 3.4 reports the implementation of the radial distortion, including the computation of the Jacobian matrix of the map, both with respect to the ideal coordinates and with respect to the coefficients.

In order to apply radial distortion in pixel coordinates, one has to switch to normalised image coordinates, apply the distortion and switch back to pixel coordinates, that is:

$$\begin{bmatrix} \breve{u} \\ \breve{v} \\ 1 \end{bmatrix} = K\rho\left(K^{-1}\begin{bmatrix} u \\ v \\ 1 \end{bmatrix}\right) \tag{3.39}$$

where ρ is the homogeneous counterpart of $\widetilde{\rho}$.

The inverse map of $\widetilde{\rho}$, the one that brings the distorted coordinates into ideal ones, that is, compensates for the radial distortion, is not easy to write in closed form. Although intuition tells us that a barrel distortion can be compensated for by applying a cushion one and vice versa, the polynomial derived from the analytic inversion (Drap and Lefèvre 2016) is of higher degree than the original (e.g. 10 coefficients are needed to invert a 4-coefficient model), and the coefficients are not always decreasing: in some situations they have increasing modulus and alternate signs, producing "unstable" behaviour. For these reasons we prefer to rely on the numerical inversion, which entails the solution of a system of polynomial equations (for each point) in the unknowns x and y:

$$\widetilde{\rho}\left(\begin{bmatrix} x \\ y \end{bmatrix}\right) - \begin{bmatrix} \breve{x} \\ \breve{y} \end{bmatrix} = \mathbf{0} \tag{3.40}$$

If (e.g.) the Gauss-Newton method is applied, the Jacobian matrix of $\widetilde{\rho}$ must be computed:

$$\mathsf{D}\widetilde{\rho} = \begin{bmatrix} \dfrac{\partial \widetilde{\rho}_1}{\partial x} & \dfrac{\partial \widetilde{\rho}_1}{\partial y} \\ \dfrac{\partial \widetilde{\rho}_2}{\partial x} & \dfrac{\partial \widetilde{\rho}_2}{\partial y} \end{bmatrix}. \tag{3.41}$$

Listing 3.4 Radial distortion: direct model

```
function [m_d,Jm,Jk] = rdx(k_in,m_i,K)
    %RDX Apply radial distortion

    n = length(k_in); % # of parameters
    k = zeros(4,1);  % default values (up to 4)
    k(1:n) = k_in;    % overwrite default

    % convert to normalizaed image coordinates
    m_i = htx(inv(K),m_i);
    x = m_i(1,1); y = m_i(2,1);
    r2 = ( m_i(1,:).^2 + m_i(2,:).^2 ); % square radius

    m_d = m_i.*(k(1)*r2 + k(2)*r2.^2 + k(3)*r2.^3 + k(4)*r2.^4 + 1);

    % convert back to pixels
    m_d = htx(K,m_d);

    % Jacobian matrices for a **single** point
    Jm = K(1:2,1:2) * ...
        [x*(2*k(1)*x + 4*k(2)*x*r2(1) + 6*k(3)*x*r2(1)^2 + 8*k(4)*x*r2(1)^3)...
         + k(2)*r2(1)^2 + k(3)*r2(1)^3 + k(4)*r2(1)^4 + k(1)*r2(1) + 1,...
         x*(2*k(1)*y + 4*k(2)*y*r2(1) + 6*k(3)*y*r2(1)^2 + 8*k(4)*y*r2(1)^3);
         y*(2*k(1)*x + 4*k(2)*x*r2(1) + 6*k(3)*x*r2(1)^2 + 8*k(4)*x*r2(1)^3),...
         y*(2*k(1)*y + 4*k(2)*y*r2(1) + 6*k(3)*y*r2(1)^2 + 8*k(4)*y*r2(1)^3)...
         + k(2)*r2(1)^2 + k(3)*r2(1)^3 + k(4)*r2(1)^4 + k(1)*r2(1) + 1]...
         * inv(K(1:2,1:2)); %#ok<MINV>

    Jk = double.empty(2,0);
    for i = 1:n
        Jk = [Jk, K(1:2,1:2) * [x;y].* r2(1).^i];
    end

end
```

In the case of a single coefficient k_1, one must solve the system of two equations of degree three:

$$\begin{cases} x + k_1 x^3 + k_1 x y^2 - \check{x} = 0 \\ y + k_1 y^3 + k_1 x^2 y - \check{y} = 0, \end{cases} \tag{3.42}$$

whose Jacobian matrix is:

$$\mathsf{D}\widetilde{\rho} = \begin{bmatrix} 2\,k_1\,x^2 + k_1\,\left(x^2 + y^2\right) + 1 & 2\,k_1, x\,y \\ 2\,k_1, x\,y & 2\,k_1\,y^2 + k_1\,\left(x^2 + y^2\right) + 1 \end{bmatrix}. \tag{3.43}$$

The development of the Jacobian up to the fourth coefficient can be found in Listing 3.4, while Listing 3.5 reports the implementation of the inverse radial distortion model. The function first normalises the coordinates, then computes the inverse model and finally transforms back into pixels.

Listing 3.5 Radial distortion: inverse model

```
function m_i = irdx(k_in,m_d,K)
    %IRDX Undo radial distortion

    n = length(k_in);   % # of parameters
    kappa = zeros(4,1); % default values (up to 4)
    kappa(1:n) = k_in;  % overwrite default

    % convert to normalizaed image coordinates
    m_d = htx(inv(K),m_d);

    % for each point independently
    for i=1:size(m_d,2)
        m_i(:,i) = lsq_nonlin(@(x)fobj(x,m_d(:,i),kappa),m_d(:,i));
    end

    % convert back to pixels
    m_i = htx(K,m_i);
end

function [res,J]  = fobj(m_ideal,m_dist,k)

    [m,J] = rdx(k,m_ideal,eye(3)); % in NIC
    res = m - m_dist;
end
```

The inverse model is only used to correct sparse points. If, on the other hand, we intend to transform the whole image, the direct transformation is sufficient because backward mapping is used in that case (see Sect. 18.4).

Problems

3.1 Prove that if M is a point at infinity, its projection is given by $\mathbf{m} \simeq \widetilde{P}\widetilde{\mathbf{M}}$ where $P = [\widetilde{P}, \mathbf{p}_4]$.

3.2 Let us consider a circle in space, centred on the optical axis and orthogonal to it: $\widetilde{\mathbf{M}} = (r\cos\phi, r\sin\phi, Z_0)$ (in the camera frame). Its projection onto the image is: $\widetilde{\mathbf{m}} = (\frac{r\cos\phi}{Z_0}\frac{r\sin\phi}{Z_0})$, that is, still a circle. If the centre of the circle moves off the optical axis, does the projection remain a circle?

3.3 Consider the situation, typical in aerial photogrammetry, when the scene consists of a plane parallel to the image plane (called ground, in this context). In this case the image is related to the ground plane by a similitude; the *photographic scale* of the image is defined as the ratio between a distance measured on the image (in pixels) and the corresponding distance on the ground (in metres). Derive the formula for the photographic scale given the focal length in pixels and the distance to the ground. Note that if the framed object is not planar or not parallel to the image plane, the scale is not constant across the image.

3.4 With reference to Problem 3.3 observe that the inverse of the photographic scale coincides with the pixel footprint on the ground, or Ground Sampling Distance (GSD).

3.5 Equation (3.6) with the explicit scaling factor writes:

$$\zeta \mathbf{m} = P\mathbf{M}. \tag{3.44}$$

Prove that if P is normalised as in (3.23), then ζ is equal to the distance of point M from the focal plane, a.k.a. *depth* of the point.

3.6 Prove that if P is normalised as in (3.23), the parameter λ in the equation of the optical ray (3.32) is equal to the depth ζ of the point, as defined in Problem 3.5

3.7 Prove that in a pinhole camera the translation of the COP along the optical axis and the variation of the focal length (a.k.a. zoom) do not produce the same effect.

3.8 The perspective projection of a bundle of parallel lines in the object space is a bundle of lines in the image plane passing through a common point, called *vanishing point*. Find the coordinates of the vanishing point for a given direction $\widehat{\mathbf{n}}$ in the object space.

3.9 Prove that if the intrinsic parameters are known, the angle between two optical rays can be computed and derive an expression fort it.

References

Pierre Drap and Julien Lefèvre. An Exact Formula for Calculating Inverse Radial Lens Distortions. *Sensors*, 16(6):807, 2016.

Karl Kraus. *Photogrammetry - Geometry from Images and Laser Scans - 2nd edition*. Walter de Gruyter, Berlin, 2007.

Chapter 4
Camera Calibration



4.1 Introduction

In the previous chapter we introduced a geometric model for the camera and we shall now address the problem of precisely and accurately determining the parameters of that model, a process that goes by the name of *calibration* of the camera.

The idea of the *resection* methods for calibration is that by knowing the image projections of 3D points of known coordinates (*control points*), it should be possible to obtain the unknown parameters by solving the perspective projection equation. The specific algorithm we will illustrate, called Direct Linear Transform (DLT), directly estimates the PPM, from which the intrinsic and extrinsic parameters can subsequently be derived.

Resection is not the only way to perform calibration (e.g. Caprile and Torre 1990 uses vanishing points). In Sect. 4.5 we will also see a non-resection calibration method.

4.2 The Direct Linear Transform Method

Given the correspondences between n *non-coplanar* control points $\mathbf{M}_i$ and their projections in the image $\mathbf{m}_i$, it is required to compute a matrix P such that:

$$\mathbf{m}_i \simeq P\mathbf{M}_i \quad i = 1 \ldots n. \tag{4.1}$$

For example, the control points can be the vertices of the black squares on the two visible facets of the trihedron depicted in Fig. 4.1 and their coordinates are known by construction (the world reference frame is fixed integral with the edges of the trihedron). The image coordinates of the control points can be obtained manually or

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_4

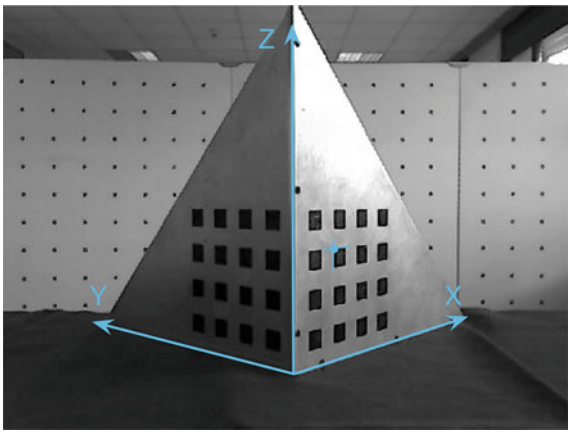


Fig. 4.1 Calibration object with the world reference frame superimposed. One of the control points is highlighted

by classical image processing methods (e.g. using template matching or the Harris-Stephens operator which we will see in Sect. 11.4).

To eliminate the scaling factor from (4.1) we exploit the cross product, rewriting the previous equation as:

$$\mathbf{m}_i \times P\mathbf{M}_i = \mathbf{0}. \tag{4.2}$$

Then, using the properties of the Kronecker product (Sect. A.14), and in particular the "vec trick" (A.59), we derive that:

$$\begin{aligned} \mathbf{m}_i \times P\mathbf{M}_i = \mathbf{0} &\iff [\mathbf{m}_i]_\times P\mathbf{M}_i = \mathbf{0} \iff \\ \operatorname{vec}([\mathbf{m}_i]_\times P\mathbf{M}_i) = \mathbf{0} &\iff (\mathbf{M}_i^\mathsf{T} \otimes [\mathbf{m}_i]_\times) \operatorname{vec} P = \mathbf{0}. \end{aligned} \tag{4.3}$$

These are three equations in 12 unknowns, but only 2 of them are independent; in fact, the rank of $(\mathbf{M}_i^\mathsf{T} \otimes [\mathbf{m}_i]_\times)$ is two since it is the Kronecker product of a matrix of rank one with a matrix of rank two.

With n points we obtain a system of $2n$ homogeneous linear equations, which we can write in matrix form as

$$A \operatorname{vec} P = \mathbf{0} \tag{4.4}$$

where A is the $2n \times 12$ matrix of coefficients and depends on the coordinates of the control points, while the vector of unknowns vec P contains the 12 elements of P read by columns. In theory, then, six non-coplanar points are sufficient for the computation of P; if any four of them are coplanar, the rank of A drops and the DLT algorithm fails (Faugeras 1993).

In practice, it is advisable to use all the available points (the more, the better) to compensate for the inevitable measurement errors. System (4.4) is therefore solved with the least-squares method: according to Proposition A.13, the solution is the least right singular vector of A.

Listing 4.1 implements this method.

Listing 4.1 DLT method

```
function P = dlt( x, X, w)
%DLT Direct Linear Transform

   if nargin <3, w = ones(size(X,2),1); end % weights

    X = [X; ones(1, size(X,2))];
    x = [x; ones(1, size(x,2))];

    L = [];
    for i = 1: size(X,2)
        L = [L; w(i) * kron( X(:,i)', skew(x(:,i)) )];
    end

    [~,~,V] = svd(L);
    P = reshape(V(:,end),size(x,1),[]);
end
```

To obtain more precise and accurate results, before the DLT it is appropriate to apply a coordinate transformation, which is called *preconditioning*. We report here the Listing 4.2 of the function that does DLT resection with preconditioning, referring to Chap. 6 for the theoretical discussion and the implementation of the `precond` function.

Be aware that, while preconditioning of image points is always appropriate, as regards the 3D points it is recommended by Hartley and Zisserman (2003) only if they are "shallow", that is, their depth variation is small compared to the distance from the camera.

Listing 4.2 Resection with preconditioning

```
function P = resect_lin(m,M)
%RESECT_LIN  Camera matrix (resection)
% use DLT algorithm

    % pre-conditioning
    [T1,m] = precond(m);
    [T2,M] = precond(M); % !!! only if M is "shallow" !!!

    P = dlt(m, M);

    % apply the inverse scaling
    P = T1 \ P * T2;
end
```

4.3 Factorisation of the Perspective Projection Matrix

Given a 3×4 matrix P of full rank, we are required to decompose it as

$$P \simeq K[R, \mathbf{t}]. \tag{4.5}$$

Let us focus on the submatrix $\widetilde{P}$ in $P = [\widetilde{P}, \mathbf{p}_4]$: by comparison with $P = [KR, K\mathbf{t}]$, we have that $\widetilde{P} = KR$ with K upper triangular and R orthogonal (with positive determinant). Let

$$\widetilde{P}^{-1} = QU \tag{4.6}$$

be the QR factorisation of $\widetilde{P}^{-1}$, with Q orthogonal and U upper triangular. Since $\widetilde{P}^{-1} = R^{-1}K^{-1}$, it suffices to set

$$R = \det(Q)Q^{\mathsf{T}} \quad \text{and} \quad K = U^{-1} \tag{4.7}$$

where the product by $\det(Q)$ is used to make the determinant of R positive, as required for a rotation matrix. We are allowed to change sign to R since this results in changing sign to P, which can absorb an arbitrary scaling factor. For the same reason, we can rescale K by imposing $K(3,3) = 1$.

The translation $\mathbf{t}$ is eventually calculated with

$$\mathbf{t} = \det(Q)K^{-1}\mathbf{p}_4 = \det(Q)U\mathbf{p}_4, \tag{4.8}$$

where we take into account the possible change of sign of P.

Listing 4.3 shows the MATLAB implementation of the method.

Listing 4.3 Factorisation of the PPM

```
1   function [K,R,t] = krt (P )
2   %KRT Intrinsic and extrinsic parameters from P
3
4   [Q,U] = qr(inv(P(1:3, 1:3)));
5   % enforce negative focal
6   D = diag(sign(diag(U)).*[-1;-1;1]);
7   Q = Q*D;  U = D*U;
8   % fix sign of R
9   s = det(Q);
10  R = s*Q';
11  t = s*U*P(1:3,4);
12  K =  inv(U./U(3,3));
```

Note: by the Theorem A.3, the factorisation QR is unique up to the sign of the diagonal elements of R. This gives us the freedom to impose that the focal is positive or negative, depending on the convention adopted (line 6 of Listing 4.3).

4.4 Calibrating Radial Distortion

Let us now see how to calibrate the radial distortion: the goal is to derive the coefficients $k_1, k_2, \ldots$. Suppose initially that we know the PPM with its parameters and the 3D coordinates of a number of control points, let $\mathbf{M}$ be one of them and

let $\mathbf{m} = [u, v, 1]^\mathsf{T}$ be the corresponding ideal image point, according to the pinhole model:

$$\mathbf{m} \simeq P\mathbf{M}. \tag{4.9}$$

Due to radial distortion the point $\breve{\mathbf{m}} = [\breve{u}, \breve{v}, 1]^\mathsf{T}$ observed in the image does not coincide with the ideal point, according to (3.37). Each point gives two linear equations (in normalised image coordinates) in the unknown coefficients $k_1, k_2, \ldots$:

$$\begin{cases} \breve{x} - x = k_1 x r^2 + k_2 x r^4 + k_3 x r^6 \ldots \\ \breve{y} - y = k_1 y r^2 + k_2 y r^4 + k_3 y r^6 \ldots \end{cases} \tag{4.10}$$

Since there are usually more points than coefficients, we obtain an overdetermined system to be solved with least-squares, as we do in Listing 4.4.

Listing 4.4 Regression of radial distortion

```
function k = radial(m_dist, m_ideal, K,n)
    %RADIAL Regress radial distortion parameters

    % convert to normalizaed image coordinates
    m_dist = htx(inv(K),m_dist);
    m_ideal = htx(inv(K),m_ideal);

    L=[];
    r2 = ( m_ideal(1,:).^2 + m_ideal(2,:).^2 ); % square radius
    for i = 1:size(m_ideal,2)
        L = [L; kron(ones(1,n), m_ideal(:,i)) .* (r2(i).^(1:n)) ];
    end
    % linear regression
    k = L \ (m_dist(:)-m_ideal(:));
end
```

The assumption that the PPM is known, however, is unrealistic, since the radial distortion must be estimated at the same time as the calibration, thus the ideal points are not accessible. It is therefore necessary to proceed iteratively, alternating between resection and radial distortion estimation, by which the distorted coordinates are corrected, until convergence is achieved, as in Listing 4.5. Note that in the iterative step we use the inverse model of radial distortion to correct the observed coordinates given the current estimate of the radial distortion coefficients (thus approximating the ideal undistorted coordinates).

4.5 The Sturm-Maybank-Zhang Calibration Algorithm

In calibration by resection (which we described in the Sect. 4.1), one needs a non-planar object on which the coordinates of the control points have to be measured with great accuracy. In practice, such an object is difficult to construct without a

Listing 4.5 Calibration of radial distortion with resection

```
function [k,P_est] = calib_radial( x_dist, X, n)
    %CALIB_RADIAL Calibrate radial distortion with resection
    %  n is the number od coefficient

    % constants
    StepTol = 1e-6;
    MaxIterations = 100;

    res = +Inf;
    x_u = x_dist; % x_u are the undistorted coordinates
    k = 0;
    for iter = 1: MaxIterations

        % resection with current undistorted
        P_est = resect_lin(x_u, X);
        [K,~,~]  =  krt(P_est);

        % project with curent PPM
        x_est = htx(P_est,X);

        % compute radial distortion
        k = radial(x_dist, x_est, K, n);

        % update undistorted with current estimate of distortion
        x_u = irdx(k,x_dist,K);

        % convergence test
        prev_res = res;
        res =  rmse(x_u(:)-x_est(:));
        if abs(res-prev_res)<StepTol
            break;
        end
    end
end
```

machine shop at hand; much easier is to get hold of a planar object. The Sturm-Maybank-Zhang (SMZ) calibration method (Sturm and Maybank 1999; Zhang 2000) relies precisely on many (at least three) images of one plane, instead of one image of many (at least two) planes that is required by resection.

We begin by establishing that the map between a plane Π in object space and its perspective image is a homography (see Appendix D), as can be easily verified by choosing the world coordinate system so that the plane Π has equation $Z = 0$ (see Fig. 4.2). Writing the perspective projection equation for a point belonging to the plane, we get

$$\mathbf{m} \simeq \underbrace{\left[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4\right]}_{P} \begin{bmatrix} X \\ Y \\ 0 \\ 1 \end{bmatrix} = \underbrace{\left[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_4\right]}_{H} \begin{bmatrix} X \\ Y \\ 1 \end{bmatrix} \tag{4.11}$$

where we have represented P with its columns: $P = \left[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3, \mathbf{p}_4\right]$. Thus, the map from Π to the image is represented by a non-singular 3×3 matrix $H = \left[\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_4\right]$, that is, a *homography*.

We assume that there is a plane in the scene of which we are able to calculate the homography that maps it to the image. It is customary to prepare for this purpose a planar calibration object, on which a grid or a chessboard is drawn, as

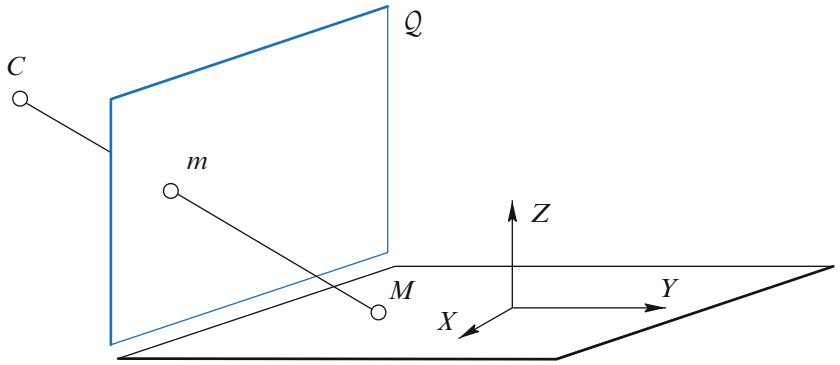


Fig. 4.2 The map bringing any plane Π of $\mathbb{P}^3$ on the image plane is a homography



Fig. 4.3 Some images of the calibration chessboard

in Fig. 4.3. The matching between the points in the image m and the corresponding control points M in the plane should be easy to obtain. The homography H is computed from these correspondences using the DLT algorithm. As a matter of fact, the function `dlt` given in Listing 4.1 serves to both resection and homography calculation (see Sect. 6.5.1).

Given that $P = K[R, \mathbf{t}]$, and $R = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3]$, we have:

$$H = \lambda K[\mathbf{r}_1, \mathbf{r}_2, \mathbf{t}] \tag{4.12}$$

where λ is an unknown scalar that accounts for the fact that the homography H computed with DLT is given up to a scaling factor.

By writing $H = [\mathbf{h}_1, \mathbf{h}_2, \mathbf{h}_3]$, we therefore get:

$$\mathbf{r}_1 = \lambda K^{-1}\mathbf{h}_1 \quad \text{and} \quad \mathbf{r}_2 = \lambda K^{-1}\mathbf{h}_2. \tag{4.13}$$

Due to the fact that the columns of R are orthonormal, one can constraint the intrinsic parameters. In particular, the orthogonality $\mathbf{r}_1^\top \mathbf{r}_2 = 0$ yields $\mathbf{h}_1^\top (K K^\top)^{-1}\mathbf{h}_2 = 0$ or, equivalently

$$\mathbf{h}_1^\top B \mathbf{h}_2 = 0 \tag{4.14}$$

Listing 4.6 SMZ calibration

```
function [P,K] = calibSMZ( H )
%CALIBSMZ Camera calibration from object-image homographies
% Sturm-Maybank-Zhang

numV = length(H);
S = duplication(3);

L = [];                 % build linear system
for i =1:numV
    L=[L;(kron(H{i}(:,2)',H{i}(:,1)'))*S;
        (kron(H{i}(:,1)',H{i}(:,1)')-kron(H{i}(:,2)',H{i}(:,2)'))*S];
end

[~,~,V] = svd(L);       % solve
B = reshape(S*V(:,end),3,[]);

if trace(B)<0
    B=-B; % either B or -B is p.d.
end
iK = diag([-1,-1,1])*chol(B); % force negative focal

K = inv(iK); K = K./K(3,3);

% compute extrinsic parameters
for i = 1:numV
    A = iK * H{i};
    A = A/norm(A(:,1)); % remove scale

    R = [A(:,1:2) , cross(A(:,1), A(:,2))];
    t = A(:,3);

    %  project onto SO(3)
    [U,~,V] = svd(R);
    R = U*diag([1,1,det(V'*U)])*V';

    if   [0,0,1] * (-R'*t)  < 0
        % change sign if cop is in the negative halfspace
        R(:,1:2) =  - R(:,1:2);
        t = - t;
    end

    P{i} =  K * [R,t];
end
end
```

where $B = (K K^\top)^{-1}$. Similarly, the condition on the norm: $\mathbf{r}_1^\top \mathbf{r}_1 = \mathbf{r}_2^\top \mathbf{r}_2$ translates into:

$$\mathbf{h}_1^\top B \mathbf{h}_1 = \mathbf{h}_2^\top B \mathbf{h}_2. \tag{4.15}$$

Thanks to the "vec trick", the last two equations are rewritten as:

$$(\mathbf{h}_2^\top \otimes \mathbf{h}_1^\top) \operatorname{vec} B = 0 \tag{4.16}$$

$$\big((\mathbf{h}_1^\top \otimes \mathbf{h}_1^\top) - (\mathbf{h}_2^\top \otimes \mathbf{h}_2^\top)\big) \operatorname{vec} B = 0. \tag{4.17}$$

B is a symmetric 3×3 matrix; therefore, its unique elements are only six. This can be formally taken into account by introducing the vech operator and the duplication matrix D_3 (see Sect. A.14):

$$(\mathbf{h}_2^\top \otimes \mathbf{h}_1^\top) D_3 \operatorname{vech} B = 0 \tag{4.18}$$

$$\left((\mathbf{h}_1^\top \otimes \mathbf{h}_1^\top) - (\mathbf{h}_2^\top \otimes \mathbf{h}_2^\top)\right) D_3 \operatorname{vech} B = 0. \tag{4.19}$$

In summary, one image provides two equations in six unknowns. If one takes n images of the plane (with different orientations, as in Fig. 4.3), one can stack the resulting $2n$ equations into a linear system of equations

$$A \operatorname{vech} B = \mathbf{0} \tag{4.20}$$

where A is a $2n \times 6$ matrix. If $n \geq 3$ there is a solution, determined up to a scaling factor. From $\operatorname{vech} B$ we derive B and then K (by Cholesky factorisation), from which we then go back to R and $\mathbf{t}$.

The implementation is given in Listing 4.6. Some caveats are in order, which are related to the fact that homographies are known up to a scale, including a sign. Therefore, B is not necessarily positive definite as one would expect from its definition, but either B or $-B$ is positive definite. To test which of these two cases occur, we exploit the fact that the trace of a positive definite matrix is positive, being the sum of positive eigenvalues. Finally, a sign on the extrinsic parameters (derived from H) is arbitrarily fixed by imposing that the cameras lie in the positive half-space of the $Z = 0$ plane.

The SMZ method does not minimise a geometric residual and as such produces suboptimal results, so it is advisable to refine the obtained parameters with a bundle adjustment (BA) (Sect. 14.4) in which both intrinsic and extrinsic parameters are unknown.

Problems

4.1 Use (3.23) to extract R in closed form from P and then also K and $\mathbf{t}$.

4.2 Prove that the principal point is the barycentre of the triangle identified by the vanishing points of three orthogonal directions in space. This observation is part of the calibration method of Caprile and Torre (1990).

4.3 If some interior orientation parameters are known (or can be assumed to be so), it becomes possible to estimate the remaining ones from a single homography, exploiting the same equations as in the SMZ method. In fact, knowledge of some of the entries of K results in constraints on the entries of B, which add up to the two equations provided by the homography. In particular, if $\theta = \pi/2$ and u_0, v_0 are

known (or assumed to be in the centre of the image), it is possible to estimate α_u, α_v directly from a single homography. Work out the four additional equations.

4.4 Experiment with the camera calibration of your smartphone using the SMZ algorithm implemented in the MATLAB *Calibration Toolkit* https://github.com/fusiello/Calibration_Toolkit.

References

B. Caprile and V. Torre. Using vanishing points for camera calibration. *International Journal of Computer Vision*, 4:127–140, 1990.

O. Faugeras. *Three-Dimensional Computer Vision: A Geometric Viewpoint*. The MIT Press, Cambridge, MA, 1993.

R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2nd edition, 2003.

Peter F. Sturm and Stephen J. Maybank. On plane-based camera calibration: A general algorithm, singularities, applications. In *CVPR*, pages 1432–1437. IEEE Computer Society, 1999.

Z. Zhang. A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(11):1330–1334, 2000.

Chapter 5
Absolute and Exterior Orientation



5.1 Introduction

In computer vision and photogrammetry several *orientation* problems are defined, that require the computation of a 3D direct isometry (rotation + translation) from correspondences between points. According to the space where the corresponding points belong, the following problems are established:

- *Absolute orientation* (3D-3D): The coordinates of some 3D points with respect to two different 3D reference frames are given; we are required to determine the transformation between the two reference frames.
- *Exterior orientation* (3D-2D): The position of some 3D points and their projection in the image are given; we are required to determine the transformation between the camera reference frame and the world reference frame. The intrinsic parameters are assumed to be known.
- *Relative orientation* (2D-2D): The projections of some 3D points and two distinct images are given; we are required to determine the transformation between the two camera reference frames. The intrinsic parameters are assumed to be known.

Absolute orientation will be addressed in Sect. 5.2 where a method based on the Orthogonal Procrustes analysis (OPA) (Arun et al. 1987), which ultimately uses the Singular Value Decomposition (SVD), is described. Exterior orientation will be reduced to the former in Sect. 5.3.1, while relative orientation will be dealt with in Chap. 7.

5.2 Absolute Orientation

Suppose we have two sets of n 3D point coordinates, which correspond to a single object, but which are expressed in two different reference frames, linked by a direct

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_5

isometry. We will call one of these sets $\mathcal{D}$ and the other $\mathcal{M}$. We assume that for every point in $\mathcal{M}$ the corresponding point in $\mathcal{D}$ is known. The *absolute orientation* (or registration) problem is to find the direct isometry to apply to $\mathcal{M}$ that brings it to coincide with $\mathcal{D}$ or, in the presence of noise, that minimise the distance between the corresponding points of two sets.

Consider the more general absolute orientation problem with scale, in which the relation between the two sets is a similitude:

$$\mathbf{D}_i = s(R\mathbf{M}_i + \mathbf{t}) \quad \text{for each } i = 1 \dots n \geq 3 \tag{5.1}$$

where $R \in SO(3)$, $\mathbf{t}$ is a 3×1 vector representing a translation, s is a scalar and $\mathbf{D}_i, \mathbf{M}_i$ are corresponding points (in Cartesian coordinates[1]) in the sets $\mathcal{D}$ and $\mathcal{M}$. The goal of registration is to solve:

$$\min_{s,R,\mathbf{t}} \sum_{i=1}^{n} \|\mathbf{D}_i - s(R\mathbf{M}_i + \mathbf{t})\|^2. \tag{5.2}$$

5.2.1 Orthogonal Procrustes Analysis

Let D be the $3 \times n$ matrix obtained by juxtaposing next to each other the vectors $\mathbf{D}_i$ and M the matrix obtained in the same way with the vectors $\mathbf{M}_i$. This allows us to rewrite (5.1) in matrix form:

$$D = s(RM + \mathbf{t}\mathbf{1}^\top), \tag{5.3}$$

and the objective function as well:

$$\chi = \sum_{i=1}^{n} \|\mathbf{D}_i - sR(\mathbf{M}_i + \mathbf{t})\|^2 = \|D - s(RM + \mathbf{t}\mathbf{1}^\top)\|_F^2. \tag{5.4}$$

Note that if $\mathbf{t}$ is a vector of d elements and $\mathbf{1}$ is a vector of n elements all equal to 1, then $\mathbf{t}\mathbf{1}^\top$ replicates the vector n times, thus producing a $d \times n$ matrix.

If there were no scale and translation, this would be an orthogonal Procrustes problem, which we could solve by applying Proposition A.14; we will now see how to fall back in this case anyway.

Observe that if A is a $n \times n$ matrix, $A\mathbf{1}/n$ is the vector containing the average computed over the rows of A. Thus, averaging over the rows of both members of (5.3) produces:

[1] In this section, in order to enlighten the notation, the Cartesian coordinates of the points are denoted without the $\tilde{}$.

$$D\mathbf{1}/n = s(RM + \mathbf{t}\mathbf{1}^\mathsf{T})\mathbf{1}/n = sRM\mathbf{1}/n + s\mathbf{t}\mathbf{1}^\mathsf{T}\mathbf{1}/n \tag{5.5}$$

which immediately gives rise to an expression for the translation:

$$\mathbf{t} = \left(\frac{1}{s}D - RM\right)\mathbf{1}/n. \tag{5.6}$$

To eliminate the translation, let us replace this expression in (5.3):

$$D = s\left(RM + \left(\frac{1}{s}D - RM\right)\mathbf{1}\mathbf{1}^\mathsf{T}/n\right) = s\left(RM + \frac{1}{s}D\mathbf{1}\mathbf{1}^\mathsf{T}/n - RM\mathbf{1}\mathbf{1}^\mathsf{T}/n\right) \tag{5.7}$$

thus obtaining

$$D\underbrace{(I - \mathbf{1}\mathbf{1}^\mathsf{T}/n)}_{J_n} = sRM\underbrace{(I - \mathbf{1}\tau/n)}_{J_n}. \tag{5.8}$$

Note that the matrix $J_n = I - \mathbf{1}^\mathsf{T}/n$, which is symmetric and idempotent, has the effect of subtracting the column average from each column of the matrix to which it is applied:

$$D - (D\mathbf{1}/n)\mathbf{1}^\mathsf{T} = D(I - \mathbf{1}^\mathsf{T}/n). \tag{5.9}$$

So, if the columns are points, as in our case, it subtracts from the coordinates of each point those of the centroid of the set.

This problem is equivalent to the original one, but the translation has been removed; it will be computed at the end after recovering R and s. If it were not for the scale, one could solve (5.8) by applying the OPA (Proposition A.14) between the two matrices DJ_n and MJ_n. Let us note, however, that the scale is irrelevant in determining the rotation, since it is absorbed by the diagonal matrix in the SVD, which does not contribute to determine R. Thus, the rotation is given by the SVD of $DJ_nJ_n^\mathsf{T}M^\mathsf{T} = DJ_nM^\mathsf{T}$, being J_n symmetric and idempotent:

$$R = U\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \det(V^\mathsf{T}U) \end{bmatrix} V^\mathsf{T} \tag{5.10}$$

where $USV^\mathsf{T} = DJ_nM^\mathsf{T}$.

The scale is finally computed as the solution of a one-dimensional least-squares problem. Given two matrices A and B, the problem $A = sB$ is rewritten: $\operatorname{vec} A = s \operatorname{vec} B$, whose least-squares solution is:

$$s = \frac{(\operatorname{vec} B)^\mathsf{T} \operatorname{vec} A}{(\operatorname{vec} B)^\mathsf{T} \operatorname{vec} B} = \frac{\operatorname{tr}(B^\mathsf{T}A)}{\operatorname{tr}(B^\mathsf{T}B)} \tag{5.11}$$

thanks to the properties of the trace. In our case we obtain:

$$s = \frac{\mathrm{tr}(D J_n M^\mathsf{T} R^\mathsf{T})}{\mathrm{tr}(M J_n M^\mathsf{T})}. \tag{5.12}$$

Referring to the proof of Proposition A.14, we observe that the numerator of (5.12) coincides with $\mathrm{tr}(S)$, and thus, in the implementation we can save some computation by exploiting the matrix S computed in the previous SVD.

It is easily shown that the solution we derived minimises the residual (5.4), which expresses the sum of the squares of the distances between corresponding points.

Both with scale (similitude) and without scale (isometry), a minimum of three points is required.

The function `opa` given in Listing 5.1 implements an extension of the method just described, in which the matches are weighted by a diagonal $n \times n$ matrix W. With $W = I$ we obtain the unweighted method described in the text.

Listing 5.1 Weighted OPA

```
function [R,t,s] = opa(D,M,w)
    %OPA Orthogonal Procrustes analysis
    % Solves the othogonal procrustes problem D=s*(R*M+t)

    if nargin==3  % Weights  provided
        w=w(:); % make sure it is a column
        % same as  D = D*W;  M = M*W;  but faster
        D = bsxfun(@times, w', D);
        M = bsxfun(@times, w', M);

    else  % otherwise no weights
        w = ones(size(M,2),1);
    end

    J = eye(size(M,2)) - (w*w')/(w'*w);  % centering matrix

    [U,S,V] = svd(D*J*M'); % solve for rotation
    R = U*diag([1,1,det(V'*U)])*V';

    if nargout==2  % no scale
        s = 1;
    else   % solve for scale
        s = trace(S) / trace(M*J*M');
        % trace(D*J*M'*R') == trace(S)
    end

    t =(D/s -R*M)*w/(w'*w); % solve for translation
end
```

Absolute orientation can also be formulated as the solution of a system in which each point provides three equations. This formulation has the advantage of being able to exploit partial information about the coordinates of the

(continued)

control points; in fact, each coordinate provides one equation. In the case of aerial photogrammetry (Kraus 2007) one can have, for example, planimetric and/or altimetric control points. A similitude is determined with seven known coordinates (except in degenerate cases).

5.3 Exterior Orientation

Suppose we know the coordinates of some 3D *control points* together with their projections in the image and the camera intrinsic parameters. The *exterior orientation* problem (a.k.a *perspective pose* or *Perspective-n-Point* (PnP)) consists in determining the direct isometry between the camera reference frame and the world reference frame (in which the control points are given).

In essence, this is a problem similar to the camera calibration, which consists in recovering interior and exterior orientation of the camera. If the intrinsic parameters are already known, one is left with the problem of computing the extrinsic parameters only, which encodes the exterior orientation of the camera.

Exterior orientation can be recovered with a *resection* procedure, if we assume that enough 3D *control points* together with their projections in the image are given (Fig. 5.1). We already encountered the resection algorithm in the context of calibration. In fact, it generically denotes the estimation of the orientation (only exterior or interior as well) of the camera from 3D control points and their images.

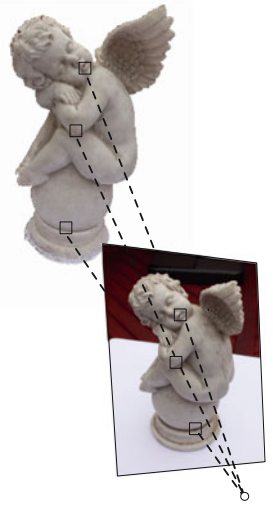


Fig. 5.1 Three control points and their projections determine the exterior orientation of the camera

Let $\mathbf{M}_1 \dots \mathbf{M}_n$ be the n control points, whose coordinates (in the world coordinate system) can be measured, and let $\mathbf{m}_1 \dots \mathbf{m}_n$ be the respective projections onto the image. Since the intrinsic parameters are known, one can express the image points in *normalised image coordinates* $\mathbf{q}_i = [x_i, y_i, 1]^\mathsf{T} = K^{-1}\mathbf{m}_i$ and write the perspective projection equation as:

$$\mathbf{q}_i \simeq [R, \mathbf{t}]\mathbf{M}_i . \tag{5.13}$$

If $n \geq 6$ the DLT method (Sect. 4.1) can be applied, obtaining a perspective projection matrix that is identified with $[R, \mathbf{t}]$.

This approach is not satisfactory because: (i) the matrix R thus obtained is not necessarily orthogonal (due to the noise in the data), and therefore, this property must be forced a posteriori (Proposition A.14); (ii) more control points (six) than the bare minimum (three) are used.

To see that three control points are sufficient to compute the exterior orientation, let us expand (5.13) into:

$$\begin{cases} x_i = \dfrac{\mathbf{r}_1^\mathsf{T}\widetilde{\mathbf{M}}_i + t_1}{\mathbf{r}_3^\mathsf{T}\widetilde{\mathbf{M}}_i + t_3} \\ y_i = \dfrac{\mathbf{r}_2^{tr}\widetilde{\mathbf{M}}_i + t_2}{\mathbf{r}_3^\mathsf{T}\widetilde{\mathbf{M}}_i + t_3} \end{cases} \tag{5.14}$$

where $R = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3]^\mathsf{T}$ and $\mathbf{t} = [t_1, t_2, t_3]^\mathsf{T}$. If the rotation matrix R is expressed as a function of three parameters (e.g. with three Euler angles), we will eventually end up with a *non-linear* system in six unknowns, which can be determined if at least three control points are given, for each control point generates two equations. As a matter of fact, exterior orientation can be recovered *in closed form* with three points.

5.3.1 Fiore's Algorithm

We introduce here a method proposed by Fiore (2001) that reduces the problem to that of *absolute orientation*, thus producing a rotation matrix by construction.

Let us rewrite (5.13) with explicit scaling factors ζ_i:

$$\zeta_i \mathbf{q}_i = [R, \mathbf{t}]\mathbf{M}_i = R\widetilde{\mathbf{M}}_i + \mathbf{t} \quad i = 1 \dots n. \tag{5.15}$$

We can clearly see that if the factors ζ_i were known, we would be faced with an absolute orientation problem. Let us therefore focus on the computation of ζ_i. To this end, we rewrite (5.15) in matrix form:

$$\underbrace{[\mathbf{q}_1, \mathbf{q}_2, \dots \mathbf{q}_n]}_{Q} \operatorname{diag}(\boldsymbol{\zeta}) = [R, \mathbf{t}] \underbrace{[\mathbf{M}_1, \mathbf{M}_2, \dots \mathbf{M}_n]}_{M} \tag{5.16}$$

where $\boldsymbol{\zeta} = [\zeta_1, \zeta_2, \dots \zeta_n]^\mathsf{T}$ and diag constructs the diagonal matrix given a vector. Given the SVD of M: $M = U D V^\mathsf{T}$, let V_r be the matrix consisting of the last $n - r$ columns of V, with $r = \mathrm{r}(M)$, which generate the null space of M. Multiplying both members of (5.16) by V_r has the effect of cancelling out the right-hand member:

$$Q \operatorname{diag}(\boldsymbol{\zeta}) V_r = [R, \mathbf{t}] M V_r = 0_{4 \times (n-r)}. \tag{5.17}$$

Taking the vec of both members and exploiting the "vec trick" (A.59) yields:

$$\operatorname{vec}(Q \operatorname{diag}(\boldsymbol{\zeta}) V_r) = \mathbf{0} \iff \left(V_r^\mathsf{T} \otimes Q\right) \operatorname{vec}(\operatorname{diag}(\boldsymbol{\zeta})) = \mathbf{0}. \tag{5.18}$$

The vector $\operatorname{vec}(\operatorname{diag}(\boldsymbol{\zeta}))$ does not only contain the unknowns $\boldsymbol{\zeta}$ but also zeros from the off-diagonal elements of $\operatorname{diag}(\boldsymbol{\zeta})$. To fix this, similarly to what is done with the vectorisation of symmetric matrices, we introduce an appropriate matrix D such that $\operatorname{vec}(\operatorname{diag}(\boldsymbol{\zeta})) = D\boldsymbol{\zeta}$ and then substitute in the previous equation, obtaining:

$$\left(V_r^\mathsf{T} \otimes Q\right) D\boldsymbol{\zeta} = \mathbf{0}. \tag{5.19}$$

By stacking enough equations like this one can derive $\boldsymbol{\zeta}$—up to a multiplicative constant—as the generator of the null space of the coefficient matrix. How many control points are needed? The size of the coefficient matrix in (5.19) is $3(n-r) \times n$, and to determine a one-dimensional family of solutions, it must have rank $n - 1$, so it must be: $3(n - r) \geq n - 1$. It follows that $n \geq (3r - 1)/2$ control points are needed. For example, six points in general positions ($r = 4$) are needed, but they drop to four it they are coplanar ($r = 3$).

Now that the right-hand member of (5.15) is known up to a scaling factor, all that remains is to solve the absolute orientation problem (with scaling) represented by (5.15) with the OPA method described in Sect. 5.2.1

Listing 5.2 implements the procedure just described. To limit the size of the intermediate matrices that are used in the computation of the coefficient matrix, instead of using (5.19), the following property (Proposition A.20) is used in place of the "vec-trick":

$$\operatorname{vec}(Q \operatorname{diag}(\boldsymbol{\zeta}) V_r) = \left(\left(V_r^\mathsf{T} \otimes \mathbf{1}_3\right) \circ (\mathbf{1}_{(n-r)} \otimes Q)\right) \boldsymbol{\zeta} \tag{5.20}$$

where $\circ$ denotes the Hadamard (element by element) product.

Listing 5.2 Fiore's linear exterior orientation

```
function [R,t] = exterior_lin(m,M,K)
%EXTERIOR_LIN Exterior orientation with Fiore's algorithm

    Q = K\ensure_homogeneous(m);   % homogeneous NIC
    M = [M; ones(1,size(M,2))  ];  % homogeneous

    [~, D, V] = svd (M);
    r = sum(diag(D)>max(size(M))*eps(max(D(:)))); % = rank(M)
    Vr = V(:,r+1:end);


    % solve null space
    [~, ~, V] = svd(kron(Vr',ones(3,1)) .* kron(ones(size(Vr,2),1), Q));
    z = V(:,end);

    %  fix sign of scale
    z = z * sign(z(1));

    [R,t,~] = opa(Q * diag(z), M(1:3,:)); % with scale

end
```

Fiore's method is linear and fast, but in general, it needs more than three control points. In contrast, the non-linear method that we will describe in Sect. 9.4 only needs the minimum number of points, but it is iterative, being based on the Gauss-Newton algorithm.

5.3.2 *Procrustean Method*

The method proposed by Garro et al. (2012) formulates the exterior orientation as a Procrustean problem. Let us rewrite (5.16) as:

$$Q\,\mathrm{diag}(\boldsymbol{\zeta}) = R\widetilde{\mathbf{M}} + \mathbf{t}\mathbf{1}^{\mathsf{T}}. \tag{5.21}$$

As already noted, if $\boldsymbol{\zeta}$ is known, this reduces to an absolute orientation problem, which is solved with OPA (see Sect. 5.2.1). If, on the other hand, R and $\mathbf{t}$ are known, one is left with the linear problem of solving for $\boldsymbol{\zeta}$. The algorithm (Listing 5.3) proceeds by alternating between these two steps until convergence.

Solving for $\boldsymbol{\zeta}$ in (5.21) follows a path similar to the solution of (5.19). Instead of the "vec-trick" we exploit Proposition A.20, to write

$$\mathrm{vec}(Q\,\mathrm{diag}(\boldsymbol{\zeta})) = ((I_n \otimes \mathbf{1}_m) \circ (\mathbf{1}_n \otimes Q))\,\boldsymbol{\zeta} \tag{5.22}$$

and the fact that $((I_n \otimes \mathbf{1}_m) \circ (\mathbf{1}_n \otimes Q))$ is the diagonal block matrix which has the columns of Q as blocks, implemented by the MATLAB function `diagc`.

Listing 5.3 Iterative exterior orientation

```
function [R,t] = exterior_iter(m,M,K)
 %EXTERIOR_ITER Exterior orientation with anisotropic Procrustes analysis

    Q = K\ensure_homogeneous(m);   % homogeneous NIC

    MaxIterations = 500;
    FunctionTol = 1e-6;
    StepTol = 1e-6;

    obj = +Inf;
    z = ones(1,size(Q,2));
    for  i = 1:MaxIterations

        % solve for R,t
        [R,t] = opa(Q*diag(z),M);
        B = R*M + t;

        % update objective and test convergence
        prevobj= obj;
        obj = norm(B - Q*diag(z),'fro');
        if norm(obj-prevobj)<StepTol || obj<FunctionTol
            break; end

        % solve for z
        z = diagc(Q)\B(:);
        %  fix sign of scale
        z = z * sign(z(1));
        z(z<0) = 0;
    end
```

The geometrical interpretation is that $\boldsymbol{\zeta}$ are the lengths of the segments of optical rays from the COP to the control points, which we will call "legs" (Fig. 5.2). The method therefore alternates between estimating the length of the legs and calculating the orientation of the camera by aligning the control points with the endpoints of the legs.

In their seminal paper, Fischler and Bolles (1981) gave the following definition of the PnP problem:

> Given the relative spatial locations of n control points, and given the angle to every pair of control points from an additional point called the Center of Perspective (CP), find the lengths of the line segments ("legs") joining the CP to each of the control points. We call this the "perspective-n-point" problem (PnP).

With $n = 3$ control points, the legs can be computed in closed form (and the problem is then called "P3P").

Consider the tetrahedron with the COP and the control points as vertices (Fig. 5.2). The edges of the triangle defined by the control points are known, the COP and the legs are unknown, but one can compute the angles between the legs (Problem 3.9). From these inputs, the triangles can be solved by applying the law of cosines and finding the roots of a fourth-degree polynomial, as described by Grunert (1841). An additional point is needed to discriminate between the four solutions. For a discussion on these ambiguities and a more accessible account of the original

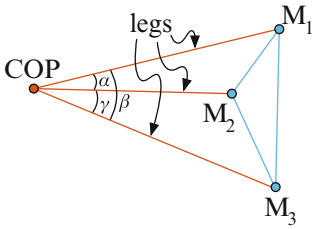


Fig. 5.2 The blue edges connecting the control points are known, and the angles α, β, γ as well. The length of the legs can be recovered (up to some ambiguities) without knowing the COP



Fig. 5.3 Example application of the direct method. The spray bottle is the object whose model is known. On the right is the gradient map on which the camera orientation retrieval is based

method, the reader is referred to (Wang et al. 2020). An excellent review can be found in (Haralick et al. 1994). For an example of recent methods, see (Gao et al. 2003; Kneip et al. 2011).

5.3.3 Direct Method

A weakness of all the methods that attack the problem with an analytical approach is the fact that they require the identification of the projections of the control points in the images, a trivial operation if done manually, but subject to errors if done by the computer, which can negatively affect the final solution.

Marchand et al. (1999) describe an alternative technique which, assuming we have a digital model of an object (instead of a set of control points), avoids matching at all. The idea is simple: the correct exterior orientation is the one for which the projection of the model coincides with its image. The "coincidence" is measured along edges, by summing the modulus of the image gradient at the projected model contour (see Fig. 5.3).

The computation of the exterior orientation is therefore reduced to the maximisation of an appropriate objective function that indicates how much the contours of the model correspond with those visible in the image, represented by maxima of the gradient:

$$E(\mathbf{g}) = \frac{1}{|\mathcal{C}_\mathbf{g}|} \sum_{\mathbf{x} \in \mathcal{C}_\mathbf{g}} \|\nabla I(\mathbf{x})\| = \frac{1}{|\mathcal{C}_\mathbf{g}|} \sum_{\mathbf{x}} \|\nabla I(\mathbf{x})\| [\![\mathbf{x} \in \mathcal{C}(\mathbf{g})]\!] \tag{5.23}$$

where $\nabla I(\mathbf{x})$ is the gradient of the image (grey scale), $\mathcal{C}(\mathbf{g})$ is the (visible) projection of the model contour according to the extrinsic parameters $\mathbf{g} = [\omega, \varphi, \kappa, t_1, t_2, t_3]^\mathsf{T}$ and $[\![\mathbf{x} \in \mathcal{C}(\mathbf{g})]\!]$ is its characteristic function. Every point belonging to $\mathcal{C}(\mathbf{g})$ and with non-zero gradient makes a contribution to the objective function.

In order to be more selective on the gradients that contribute to the objective function and reduce the effect of noise or fine textures, it is advisable to also take into account the direction of the gradient by projecting it onto the normal of the projected model contour $\mathbf{n}(\mathbf{x})$:

$$E(\mathbf{g}) = \frac{1}{|\mathcal{C}_\mathbf{g}|} \sum_{\mathbf{x}} |\nabla I(\mathbf{x})^\mathsf{T} \mathbf{n}(\mathbf{x})| [\![\mathbf{x} \in \mathcal{C}(\mathbf{g})]\!]. \tag{5.24}$$

To have a larger and smoother convergence basin, one can also consider a smoothed characteristic function that decreases with the distance from the contour according to a Gaussian law.

The objective function is then iteratively maximised by employing direct techniques (using only the value of the function, not its derivative), such as Hooke and Jeeves (1961). As in any iterative optimisation, an initial solution close to the global minimum is needed. Therefore, this method can be effectively employed in a video *tracking* scheme, where the exterior orientation obtained for the current frame provides the starting solution for computing the orientation for the next frame (Valinetti et al. 2001).

The same idea of the direct method of Sect. 5.3.3 was employed in the calibration technique introduced by Robert (1996). The author uses the DLT method with six points to obtain an initial estimate of the PPM, which then refines by minimising an objective function similar to (5.23).

Problems

5.1 Given two vectors $\mathbf{a}$ and $\mathbf{b}$ prove that the least-squares solution of $\mathbf{a} = s\mathbf{b}$ is

$$s = \frac{\mathbf{b}^\mathsf{T}\mathbf{a}}{\mathbf{b}^\mathsf{T}\mathbf{b}} \tag{5.25}$$

Convince yourself that this is different from $s = \|\mathbf{a}\|/\|\mathbf{b}\|$ or $s = \text{mean}(a_i/b_i)$ when $\mathbf{a} \neq s\mathbf{b}$.

References

K. S. Arun, T. S. Huang, and S. D. Blostein. Least-squares fitting of two 3-D point sets. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 9(5):698–700, September 1987.

Paul D. Fiore. Efficient linear solution of exterior orientation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(2):140–148, 2001.

M. A. Fischler and R. C. Bolles. Random Sample Consensus: A paradigm model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24 (6):381–395, June 1981.

Xiao-Shan Gao, Xiao-Rong Hou, Jianliang Tang, and Hang-Fei Cheng. Complete solution classification for the perspective-three-point problem. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 25:930–943, August 2003.

V. Garro, F. Crosilla, and A. Fusiello. Solving the PnP problem with anisotropic orthogonal procrustes analysis. In *Second Joint 3DIM/3DPVT Conference: 3D Imaging, Modeling, Processing, Visualization and Transmission (3DIMPVT)*, 2012.

Johann August Grunert. Das pothenotische problem in erweiterter gestalt nebst bber seine anwendungen in der geodasie. *Grunerts Archiv fur Mathematik und Physik*, pages 238–248, 1841.

Robert M. Haralick, Chung-Nan Lee, Karsten Ottenberg, and Michael Nölle. Review and analysis of solutions of the three point perspective pose estimation problem. *International Journal of Computer Vision*, 13:331–356, December 1994.

R. Hooke and T.A. Jeeves. Direct search solution of numerical and statistical problems. *Journal of the Association for Computing Machinery (ACM)*, pages 212–229, 1961.

L Kneip, D Scaramuzza, and R Siegwart. A novel parametrization of the perspective-three-point problem for a direct computation of absolute camera position and orientation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2011.

Karl Kraus. *Photogrammetry - Geometry from Images and Laser Scans - 2nd edition*. Walter de Gruyter, Berlin, 2007.

Éric Marchand, Patrick Bouthemy, Francois Chaumette, and Valérie Moreau. Robust real-time visual tracking using a 2-D-3-D model-based approach. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1999.

L Robert. Camera calibration without feature extraction. *Computer Vision, Graphics, and Image Processing*, 63(2):314–325, March 1996.

A. Valinetti, A. Fusiello, and V. Murino. Model tracking for video-based virtual reality. In *Proceedings of the 11th International Conference on Image Analysis and Processing (ICIAP)*, pages 372–377, Palermo, IT, 2001. IAPR, IEEE Computer Society Press. doi:10.1109/ICIAP.2001.957038.

Bo Wang, Hao Hu, and Caixia Zhang. Geometric interpretation of the multi-solution phenomenon in the P3P problem. *J. Math. Imaging Vis.*, 62(9):1214–1226, nov 2020. ISSN 0924-9907. https://doi.org/10.1007/s10851-020-00982-5.

Chapter 6
Two-View Geometry



6.1 Introduction

We shall now study the relationship that links two images framing the same scene from two different points of view. In particular, we will be concerned with the relationship that exists between the *homologous points*, which are defined as the points in the two images that are projections of the same point in objects space (Fig. 6.1).

Epipolar geometry describes the relationship between two images of the same scene, so it is fundamental to any computer vision technique based on more than one image.

6.2 Epipolar Geometry

Given a point m in the first image, we shall investigate what constraints exist on the position of its homologous m' in the second image. Some simple geometric considerations indicate that the homologous point of m must lie on a straight line in the second image, called the *epipolar line* of m.

Consider the case illustrated in Fig. 6.2. Given a point m in the first image, its homologous m' in the second image is constrained to lie on the intersection of the image plane with the plane determined by m, C and C', called the *epipolar plane*. This is because the point m' can be the projection of any point in space lying on the optical ray of m. Furthermore, it is observed that all epipolar lines of an image pass through the same point, called *epipole*, and that the epipolar planes constitute a bundle of planes having in common the *baseline*, that is, the line passing through the COP C and C'.

Fixed a world coordinate system, given two PPMs $P = [\widetilde{P}, \mathbf{p}_4]$ and $P' = [\widetilde{P}', \mathbf{p}'_4]$, we know that:

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_6



Fig. 6.1 Stereo pair. Two homologous (image) points are the projection of the same object point

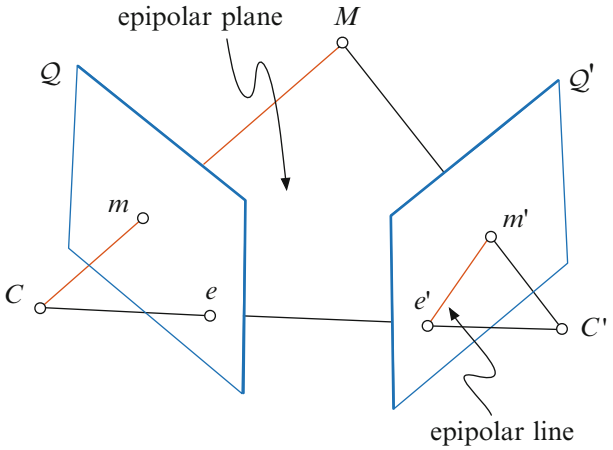


Fig. 6.2 Epipolar geometry

$$\begin{cases} \mathbf{m} \simeq P\mathbf{M} \\ \mathbf{m}' \simeq P'\mathbf{M}. \end{cases} \tag{6.1}$$

The epipolar line corresponding to m is the projection according to P' of the optical ray of m, which has equation:

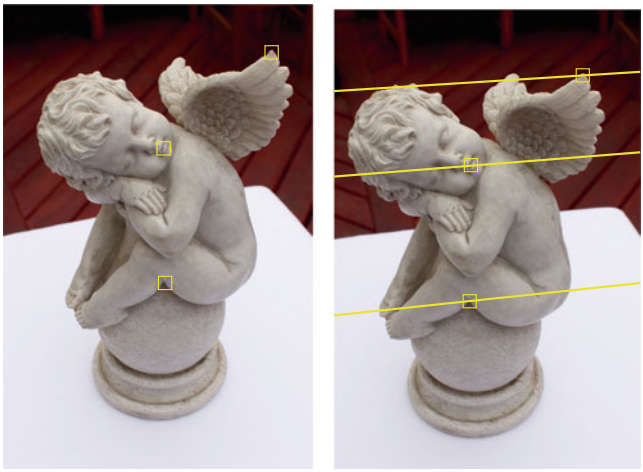


Fig. 6.3 The epipolar lines corresponding to the points marked with a square in the left image are drawn on the right

$$\mathbf{M} = \mathbf{C} + \lambda \begin{bmatrix} \widetilde{P}^{-1}\mathbf{m} \\ 0 \end{bmatrix}. \tag{6.2}$$

Since:

$$P'\mathbf{C} = P' \begin{bmatrix} -\widetilde{P}^{-1}\mathbf{p}_4 \\ 1 \end{bmatrix} = \mathbf{p}'_4 - \widetilde{P}'\widetilde{P}^{-1}\mathbf{p}_4 = \mathbf{e}' \tag{6.3}$$

and

$$P' \begin{bmatrix} \widetilde{P}^{-1}\mathbf{m} \\ 0 \end{bmatrix} = \widetilde{P}'\widetilde{P}^{-1}\mathbf{m}, \tag{6.4}$$

the epipolar line of m has equation:

$$\mathbf{m}' \simeq \lambda \widetilde{P}'\widetilde{P}^{-1}\mathbf{m} + \mathbf{e}'. \tag{6.5}$$

This is the equation (in homogeneous coordinates) of the line through the points $\mathbf{e}'$ (epipole) and the projection of the point at infinity $\widetilde{P}'\widetilde{P}^{-1}\mathbf{m}$.

Figure 6.3 shows an example where some epipolar lines are plotted in the right image.

When the COP is in the focal plane of the conjugate camera, the epipole $\mathbf{e}'$ lies at infinity and the epipolar lines form a bundle of parallel lines. A very special case is when both epipoles lie at infinity, which happens when the baseline CC' is contained in both focal planes, that is, the two image planes are parallel to the baseline. In this

case, which is called *normal* configuration, the epipolar lines form bundle of parallel lines in both images.

6.3 Fundamental Matrix

To prove that there is a *bilinear relationship* between the homologous points, we need to further manipulate (6.5). We multiply to the left and to the right by $[\mathbf{e}']_\times$, the antisymmetric matrix that acts as the cross product with $\mathbf{e}'$, obtaining (recall that $[\mathbf{a}]_\times \mathbf{a} = \mathbf{0}$):

$$[\mathbf{e}']_\times \mathbf{m}' \simeq \lambda [\mathbf{e}']_\times \widetilde{P}' \widetilde{P}^{-1} \mathbf{m}. \tag{6.6}$$

The left-hand side is a vector orthogonal to $\mathbf{m}'$, so if we multiply both sides by $\mathbf{m}'^\mathsf{T}$, we get:

$$0 = \mathbf{m}'^\mathsf{T} [\mathbf{e}']_\times \widetilde{P}' \widetilde{P}^{-1} \mathbf{m}. \tag{6.7}$$

This is the *Longuet-Higgins equation* and represents a bilinear form in $\mathbf{m}$ and $\mathbf{m}'$.

Geometrically, it is consistent with the fact that, as is explained in Appendix D, the line through two points is represented by the cross product of the two points, so the epipolar line of $\mathbf{m}$ is represented (in homogeneous coordinates) by the vector:

$$\mathbf{e}' \times (\widetilde{P}' \widetilde{P}^{-1} \mathbf{m}). \tag{6.8}$$

The matrix

$$F = [\mathbf{e}']_\times \widetilde{P}' \widetilde{P}^{-1} \tag{6.9}$$

which contains the coefficients of the bilinear form is called the *fundamental matrix*. The Longuet-Higgins equation then rewrites:

$$\mathbf{m}'^\mathsf{T} F \mathbf{m} = 0. \tag{6.10}$$

It states that the corresponding point of $\mathbf{m}$ in the second image must lie on its epipolar line $F\mathbf{m}$; in fact, the line defined by $F\mathbf{m}$ is the epipolar line of m.

Since $\det([\mathbf{e}']_\times) = 0$, we have that $\det(F) = 0$. In particular, since $\widetilde{P}' \widetilde{P}^{-1}$ is non-singular, F has the same rank as $[\mathbf{e}']_\times$ which is two. Furthermore, the matrix F is defined up to a scaling factor, since if it is multiplied by an arbitrary scalar, (6.10) is still satisfied.

In general, a 3×3 matrix has nine degrees of freedom; each one of the above constraints removes one degree of freedom, so a fundamental matrix has only seven degrees of freedom.

Observe that by transposing (6.10), we obtain the symmetric relation from the second image to the first image:

$$\mathbf{m}^\mathsf{T} F^\mathsf{T} \mathbf{m}' = 0. \tag{6.11}$$

The epipole $\mathbf{e}'$ is easily derived from F as a generator of the null space of F^T. In fact, since the epipole belongs to all epipolar lines, it must be $\mathbf{e}'^\mathsf{T} F\mathbf{m} = 0$ for every $\mathbf{m}$, and therefore, $\mathbf{e}'^\mathsf{T} F = \mathbf{0} = F^\mathsf{T}\mathbf{e}'$. Similarly $\mathbf{e}^\mathsf{T} F^\mathsf{T} = \mathbf{0}$.

6.4 Computing the Fundamental Matrix

The fundamental matrix can be computed from homologous points, using the method described in the following section. Excellent references for the computation of F are (Zhang 1998) and (Luong and Faugeras 1996).

Given a set of point correspondences (at least eight, as we shall see) $\{(\mathbf{m}_i, \mathbf{m}'_i) \mid i = 1, \ldots, n\}$, we want to determine the fundamental matrix F that connects the points in the bilinear relation:

$$\mathbf{m}_i{}'^\mathsf{T} F \mathbf{m}_i = 0. \tag{6.12}$$

The unknown matrix can be easily derived through the use of the Kronecker product and the "vec trick" (Sect. A.14). In fact:

$$F\mathbf{m}_i = 0 \iff \operatorname{vec}\left(\mathbf{m}_i{}'^\mathsf{T} F \mathbf{m}_i\right) = 0 \iff \left(\mathbf{m}_i^\mathsf{T} \otimes \mathbf{m}_i'^\mathsf{T}\right) \operatorname{vec} F = 0.$$

So each point correspondence generates a linear homogeneous equation in the nine unknown elements of the matrix F (read by columns). From n corresponding points, we obtain a linear system of n equations:

$$\underbrace{\begin{bmatrix} \mathbf{m}_1^\mathsf{T} \otimes \mathbf{m}_1'^\mathsf{T} \\ \mathbf{m}_2^\mathsf{T} \otimes \mathbf{m}_2'^\mathsf{T} \\ \vdots \\ \mathbf{m}_n^\mathsf{T} \otimes \mathbf{m}_n'^\mathsf{T} \end{bmatrix}}_{A_n} \operatorname{vec} F = 0. \tag{6.13}$$

The solution to the homogeneous linear system is the null space of A_n. With at least $n = 8$ points in general position, this null space reaches dimension one, so the solution is determined up to a multiplicative constant. Therefore, this method is called the *eight-point algorithm*.

Not always the eight-point algorithm succeeds in uniquely determining a fundamental matrix: a degenerate case occurs whenever the null space of A_n has dimension greater than one. This happens if the eight points lie on a plane or on the intersection of three quadrics, like, for example, the eight vertices of a cube. The study of the rank of A_n is described in (Agarwal et al. 2017), while the *critical loci*, which also include the position of the cameras, are studied in (Hartley and Kahl 2007). A degeneracy also occur when the two COPs are the same (no translation).

When more than eight-point correspondences are available, the linear system is overdetermined and can be solved with the least-squares method. According to Proposition A.13, the solution is the least right singular vector of A_n. The MATLAB implementation is given in Listing 6.1. It is understood that despite its name, the function accepts as input any number of points ≥ 8.

Listing 6.1 Eight-point algorithm

```
function E = eight_points(m2, m1, w)
%EIGHT_POINTS 8-points algorithm for F/E

    if nargin <3, w = ones(size(m1,2),1); end % weights

    m1 = ensure_homogeneous(m1);
    m2 = ensure_homogeneous(m2);

    L = [];
    for i = 1: size(m1,2)
        L = [L; w(i) * kron( m1(:,i)' , m2(:,i)') ];
    end

    [~,~,V] = svd(L);
    E = reshape(V(:,end),3,3);

end
```

6.4.1 The Seven-Point Algorithm

The matrix F obtained as a solution of the system (6.13) will not, in general, have rank two. This can be forced a posteriori by replacing F with $\widehat{F}$, the matrix of rank two closest to F in Frobenius norm, following the Eckart-Young theorem (A.6). Alternatively, it is possible to easily derive a "seven-point algorithm" that yields a F of rank two by construction. Indeed, since F possesses only seven degrees of freedom, seven correspondences must be sufficient to compute it, with the addition of the rank constraint.

The matrix of coefficients A_7 that is generated from seven matches has dimension 7×9 and a null space of dimension two, so the generic solution is written as a linear combination of two particular solutions:

$$F = F_8 + \mu F_9 \tag{6.14}$$

where F_8 and F_9 are the two matrices corresponding to the last two singular vectors in the Singular Value Decomposition (SVD) of A_7. Only one coefficient μ is needed because of the scale ambiguity of F. The specific μ that makes F singular is the solution of:

$$\det(F_8 + \mu F_9) = 0 \tag{6.15}$$

which is a third-degree polynomial in μ and therefore can be solved analytically, yielding one or three real solutions.

6.4.2 Preconditioning

The eight-point algorithm (but the same can be said about the seven-point algorithm) has been criticised for being too sensitive to noise and thus not very useful in practical applications. However, Hartley (1995) showed that the instability of the method is mainly due to a malconditioning problem rather than to its linear nature. He observed, in fact, that homogeneous pixel coordinates most likely yield an ill-conditioned system of linear equations. In fact, typical image coordinates are $[\approx 10^3, \approx 10^3, 1]$, so the entries of A_n ranges from 1 to $\approx 10^6$, and consequently, A_n has a high conditioning number, which makes the solution of the system unstable.

By applying a simple preconditioning technique—which consists of transforming the points so that all the three coordinates have the same orders of magnitude—the conditioning number is reduced and the results are more stable. The points are translated so that their centroid coincides with the origin, and then they are scaled so that the average distance from the origin is $\sqrt{2}$. The latter choice comes from the observation that the best conditioning occurs when the mean coordinates are $[\approx 1, \approx 1, 1]$, and this point has distance $\sqrt{2}$ from the origin.

Let T and T' be the resulting transformations in the two images and $\bar{\mathbf{m}} = T\mathbf{m}$, $\bar{\mathbf{m}}' = T'\mathbf{m}'$ the transformed points. Using $\bar{\mathbf{m}}$ and $\bar{\mathbf{m}}'$ in the eight-point algorithm, we obtain a fundamental matrix $\bar{F}$ that can be related to the original one by $F = T'^{\mathsf{T}} \bar{F} T$, as can be easily shown.

The MATLAB implementation of the computation of F with preconditioning is given in Listings 6.2 and 6.3.

Listing 6.2 Linear computation of F

```
function F = fund_lin(m2,m1,w)
%FUND_LIN  Fundamental matrix with 8-points algorithm

if nargin < 3 || isempty(w)
    w = ones(size(m1,2),1);  % weights
end

% pre-conditioning
[T1,m1] = precond(m1);
[T2,m2] = precond(m2);

F = eight_points(m2, m1, w);

% enforce singularity of  F
[U,D,V] = svd(F);
D(3,3) = 0; F = U*D*V';

% apply the inverse scaling
F = T2' * F * T1;

end
```

Listing 6.3 Preconditioning

```
function [T,m] = precond(m)
%PRECOND Normalize the coordinates of kD points

    avg = mean(m,2) ;
    m = m - avg;
    scale = mean(sqrt(sum(m.^2,1)))/sqrt(size(m,1));
    m = m./scale;

    T = [eye(size(m,1)), -avg]./scale;
    T(end+1,end)=1;

end
```

Preconditioning is useful not only in the computation of F but also in all those cases where the coefficient matrix is obtained from homogeneous image coordinates, so, for example, also in the case of resection (Sect. 4.2) or in the computation of a homography (Sect. 6.5.1).

6.5 Planar Homography

While in the most general case homologous points in two images are related by the fundamental matrix, there are instances of practical interest in which point correspondences between two images are encoded by a projectivity of $\mathbb{P}^2$, or *homography* or collineation. This occurs when the observed points lie on a plane in object space.

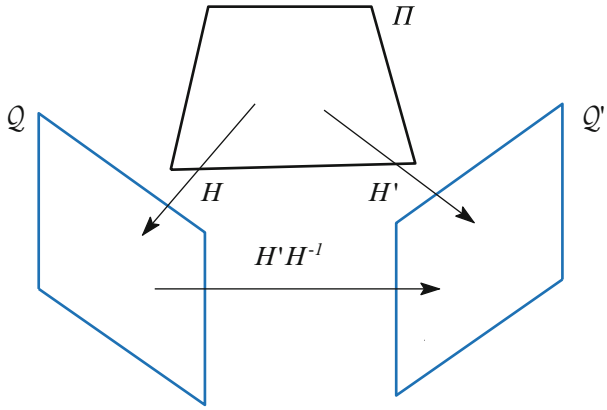


Fig. 6.4 The plane Π induces a homography of $\mathbb{P}^2$ between the two images

We already established in Sect. 4.5 that the map between a plane Π in object space and its perspective image is a homography. Here we observe that the projections of the points of Π in two images are related by a homography as well. In fact, we have a homography between Π and the left image, and likewise a homography between Π and the right image. By composing the inverse of the former with the latter (see Fig. 6.4), we obtain a homography from the left image to the right image.

We will then say that Π *induces* a homography between the two images, represented by a matrix H_Π, in the sense that corresponding points in the two images are associated via H_Π. In formulae:

$$\mathbf{m}' \simeq H_\Pi \mathbf{m} \quad \text{if } \mathbf{m}' \simeq P'\mathbf{M},\ \mathbf{m} \simeq P\mathbf{M}, \text{ and } \mathbf{M} \in \Pi. \tag{6.16}$$

The matrix H_Π is 3×3 non-singular, and being defined up to a scale factor has eight degrees of freedom. Three of these are due to the choice of the plane Π, the other five arise from the *compatibility* with the epipolar geometry, represented by F. In fact, all points related via H_Π must also satisfy the epipolar constraint, that is:

$$\begin{cases} \mathbf{m}'^{\mathsf{T}} F \mathbf{m} = 0 \\ \mathbf{m}' = H_\Pi \mathbf{m} \end{cases} \tag{6.17}$$

which implies:

$$(H_\Pi \mathbf{m})^{\mathsf{T}} F \mathbf{m} = 0. \tag{6.18}$$

This is equivalent to saying that $H_\Pi^{\mathsf{T}} F$ must be antisymmetric, that is, $H_\Pi^{\mathsf{T}} F + F^{\mathsf{T}} H_\Pi = 0_{3\times3}$. Due to the homogeneity of the equation and the symmetry of the matrix on the left-hand side, this results in five constraints.

To derive an expression for H_Π we need to start again from the epipolar geometry. If we take the reference frame of the first camera as the world reference frame, we can write the following two PPMs:

$$P = K[I, \mathbf{0}] = [K, \mathbf{0}] \quad \text{and} \quad P' = K'[R, \mathbf{t}]. \tag{6.19}$$

Substituting these PPMs into the equation of the epipolar line of $\mathbf{m}$ (6.5), we obtain

$$\mathbf{m}' \simeq \lambda K' R K^{-1} \mathbf{m} + K' \mathbf{t} \tag{6.20}$$

with

$$\mathbf{e}' = K'[R, \mathbf{t}] \begin{bmatrix} \mathbf{0} \\ 1 \end{bmatrix} = K' \mathbf{t}. \tag{6.21}$$

In general, two homologous points $\mathbf{m}$ and $\mathbf{m}'$, which are the projection of the 3D point $\mathbf{M}$ onto the first and second cameras, respectively, are related by (6.20). However, if the 3D point $\mathbf{M}$ lies on a plane Π, then (6.20) can be specialised. Using the equation of the plane $\mathbf{n}^\top \widetilde{\mathbf{M}} = d$, where d is the distance of the plane from the origin and $\mathbf{n}$ is its normal, after some rewritings (see Problem 6.4) one gets:

$$\mathbf{m}' \simeq K' \left(R + \frac{\mathbf{t}\, \mathbf{n}^\top}{d} \right) K^{-1} \mathbf{m}. \tag{6.22}$$

The latter equation states that there is a homography induced by the plane Π between the two images, defined by the matrix:

$$H_\Pi = K' \left(R + \frac{\mathbf{t}\, \mathbf{n}^\top}{d} \right) K^{-1}. \tag{6.23}$$

If Π approaches the plane at infinity, that is, $d \to \infty$, in the limit we get the matrix of the homography induced by the plane at infinity $H_\infty = K' R K^{-1}$. This homography relates the projections of points lying the plane at infinity, that is, it maps vanishing points of one image into vanishing points of the other image.

Formula (6.22) applies to any pair of cameras framing a plane Π, with a totally general relative orientation. There is, however, another special case when two images are linked by a homography, which depends on the motion of the camera and not on the structure of the scene. This occurs when the baseline (i.e. the relative translation) is zero, In fact, plugging $\mathbf{t} = \mathbf{0}$ into (6.20) one gets:

$$\mathbf{m}' \simeq K' R K^{-1} \mathbf{m} = H_\infty \mathbf{m}. \tag{6.24}$$

Thus, H_∞ plays a dual role:

- Like all homographies induced by a plane, it relates the projections of points on the plane at infinity in the two images.
- In addition, H_∞ associates the projections of *all* the points in the scene (no matter what plane they belong to) between the two images when the camera makes a purely rotational motion.

6.5.1 Computing the Homography

Given n homologous points $(\mathbf{m}_i; \mathbf{m}'_i)$, we are required to determine the homography matrix H such that:

$$\mathbf{m}'_i \simeq H\mathbf{m}_i \quad i = 1 \ldots n. \tag{6.25}$$

Taking advantage of the cross product to eliminate the multiplicative factor, the equation is rewritten:

$$\mathbf{m}'_i \times H\mathbf{m}_i = \mathbf{0}. \tag{6.26}$$

Along the same lines as the derivation made in Sect. 4.1, we get:

$$\mathbf{m}'_i \times H\mathbf{m}_i = \mathbf{0} \iff [\mathbf{m}'_i]_\times H\mathbf{m}_i = \mathbf{0} \iff$$
$$\text{vec}([\mathbf{m}'_i]_\times H\mathbf{m}_i) = \mathbf{0} \iff (\mathbf{m}_i^\mathsf{T} \otimes [\mathbf{m}'_i]_\times)\,\text{vec}\,H = \mathbf{0}$$

and similarly, we conclude that the matrix $(\mathbf{m}_i^\mathsf{T} \otimes [\mathbf{m}_i{}']_\times)$ has rank two, so only two out of three equations are linearly independent. For n points we obtain a system of $2n$ homogeneous linear equations, which we can write as

$$A\,\text{vec}\,H = \mathbf{0} \tag{6.27}$$

where A is the $2n \times 9$ matrix of coefficients obtained by stacking two equations for each correspondence, while the vector of unknowns vec H contains the nine elements of H read by columns. So, four points in general position[1] determine a matrix of coefficients A of rank eight whose one-dimensional null space is the sought solution up to a scale factor. Thus, a homography is determined by its action on four points, as illustrated in Fig. 6.5.

[1] There must not be three aligned.

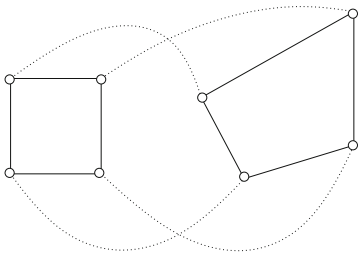


Fig. 6.5 Four corresponding points determine a homography

For $n > 4$ points, the least-squares solution is the least right singular vector of A (see Proposition A.13).

As seen in Sect. 6.3 for the fundamental matrix, also in the computation of H, it is useful to transform the points so that the problem becomes better conditioned. Let $\bar{\mathbf{m}} = T\mathbf{m}$ and $\bar{\mathbf{m}}' = T'\mathbf{m}'$. Using the correspondences between the transformed points $\bar{\mathbf{m}}$ and $\bar{\mathbf{m}}'$, one finds a homography of the plane $\bar{H}$ that is related to the sought one by $\bar{H} = T'HT^{-1}$, as can easily be seen.

The MATLAB implementation of the homography computation with preconditioning is given in Listing 6.4 and is based on the `dlt` function given in Listing 4.1.

Listing 6.4 Linear computation of H

```
function H = hom_lin(m2,m1,w)
%HOM_LIN  Homography with DLT algorithm

if nargin < 3 || isempty(w)
    w = ones(size(m1,2),1); % weights
end

% pre-conditioning
[T1,m1] = precond(m1);
[T2,m2] = precond(m2);

H = dlt(m2, m1, w);

% post-conditioning
H = T2\H * T1;
end
```

Not all homographies are induced by a plane. For example, the homography determined by four random pairs of points almost certainly will not be compatible with the epipolar geometry; hence, it cannot be generated by a plane. But also four random homologous pairs, unless the corresponding tie points are coplanar in space, do not determine a plane-induced homography. In contrast, homographies computed with three homologous pairs and the epipole are induced by a plane, by construction.

6.6 Planar Parallax

Let us consider (6.20) which we report here for the convenience of the reader:

$$\mathbf{m}' \simeq \lambda H_\infty \mathbf{m} + K'\mathbf{t}. \tag{6.28}$$

One way to interpret it is that a point m is associated with its homologous in two steps: first H_∞ is applied and then a correction, called *parallax*, is added along the epipolar line.

This observation is not only true for H_∞ but generalises to any plane (Shashua and Navab 1996). In fact, after a few steps along the lines of the procedure used to derive (6.22), we obtain:

$$\mathbf{m}' \simeq H_\Pi \mathbf{m} + \left(\frac{a}{d\zeta}\right)\mathbf{e}' \tag{6.29}$$

where a is the distance of the point $\mathbf{M}$ (of which $\mathbf{m}$ and $\mathbf{m}'$ are the projections) from the plane Π and ζ is its depth with respect to the first camera.

When $\mathbf{M}$ belongs to the plane Π, then $\mathbf{m}' \simeq H_\Pi \mathbf{m}$; otherwise, there is a residual term called *parallax* $\gamma = \dfrac{a}{d\,\zeta}$. The parallax is the projection onto the plane of the second camera of the segment of the optical ray between M and the intersection with Π (see Fig. 6.6).

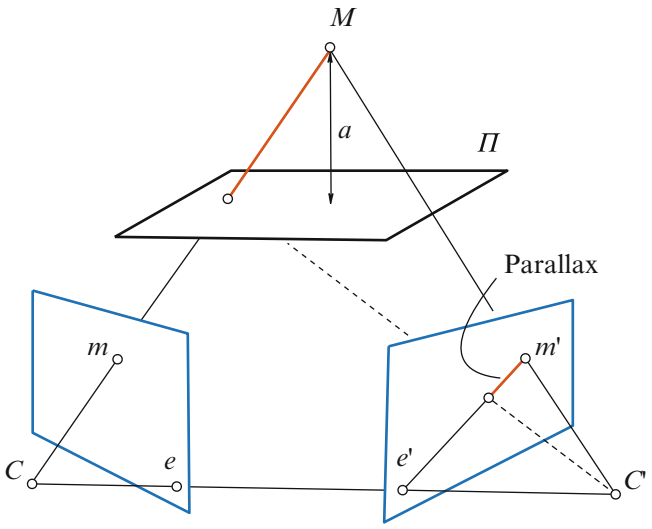


Fig. 6.6 Epipolar geometry and planar parallax. Parallax with respect to plane Π is the projection of the segment joining the point M and its projection onto Π along the optical ray



Fig. 6.7 The first two images are the original ones. The third one is the superimposition of the second one with the first one transformed according to the homography of the building facade (the points used to calculate the homography are highlighted)

Equation (6.29) is an alternative representation of epipolar geometry, taking a particular plane as a reference. Note that:

- γ is independent of the choice of the second image.
- γ is proportional to the inverse of depth ζ.
- When the reference plane is the plane at infinity $\gamma = \dfrac{1}{\zeta}$.
- the parallax field is radial with centre in the epipole.

A simple experiment to visualise parallax is shown in Fig. 6.7. Take two images of a scene containing at least one plane (the facade of the building); apply the homography induced by that plane to the whole first image. Observe that the points in the plane coincide when the two images are superimposed while those off the plane (the statue of Dante) do not. The difference in position between corresponding points is the parallax.

Problems

6.1 If the PPMs are normalised as in (3.23), we can rewrite the equation of the epipolar line with the point depths (ζ and ζ') made explicit:

$$\zeta' \mathbf{m}' = \zeta \widetilde{P}' \widetilde{P}^{-1} \mathbf{m} + \mathbf{e}'. \tag{6.30}$$

6.2 Prove that the epipole $\mathbf{e}'$ is in the null space of $F^\top$ by algebraic manipulations using the fact that $\left[\mathbf{e}'\right]_\times \mathbf{e}' = \mathbf{0}$.

6.3 In the step that led to (6.7), we could arrive at the same result by observing that after multiplication by $\mathbf{m}'^\top$ the left member is a quadratic form and $[\mathbf{e}']_\times$ is antisymmetric.

6.4 Derive (6.22).

6.5 Show that, given two images whose PPMs are $P = [\widetilde{P}, \mathbf{p}_4]$ and $P' = [\widetilde{P}', \mathbf{p}'_4]$, the homography of the plane at infinity between the two is $H_\infty = \widetilde{P}'\widetilde{P}^{-1}$.

6.6 Show that if the homography H transfers points between two images, then the homography $H^{-\mathsf{T}}$ transfers straight lines between the same two images.

6.7 Prove that, however fixed a plane Π, it always holds: $H_\Pi \mathbf{e} \simeq \mathbf{e}'$. It means that to compute the homography, three points belonging to the plane and the epipoles are enough, to have a total of four homologous points.

6.8 Using the result of Problem 6.7, show that, given two homographies H_Π and H_Σ induced by two different planes in the same image pair, then the epipole $\mathbf{e}'$ is the eigenvector corresponding to the distinct eigenvalue of $H_\Pi H_\Sigma^{-1}$. What do the other two eigenvectors represent?

References

Sameer Agarwal, Hon-Leung Lee, Bernd Sturmfels, and Rekha R. Thomas. On the existence of epipolar matrices. *International Journal of Computer Vision*, 121(3):403–415, Feb 2017.

R. I. Hartley. In defence of the 8-point algorithm. In *Proceedings of the International Conference on Computer Vision*, pages 1064–1071, Washington, DC, USA, 1995. IEEE Computer Society.

Richard Hartley and Fredrik Kahl. Critical configurations for projective reconstruction from multiple views. *International Journal of Computer Vision*, 71(1):5–47, Jan 2007.

Q.-T. Luong and O. D. Faugeras. The fundamental matrix: Theory, algorithms, and stability analysis. *International Journal of Computer Vision*, 17:43–75, 1996.

A. Shashua and N. Navab. Relative affine structure: Canonical model for 3D from 2D geometry and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(9): 873–883, September 1996.

Z. Zhang. Determining the epipolar geometry and its uncertainty: A review. *International Journal of Computer Vision*, 27(2):161–195, March/April 1998.

Chapter 7
Relative Orientation



7.1 Introduction

Consider the situation where some points of the 3D space—called in this context *tie points*—are projected into two different cameras, and for each point in one camera the homologous point in the other camera is given. The *relative orientation* problem consists in determining the orientation (which includes position and angular attitude) of one camera with respect to the other, assuming known intrinsic parameters.

It is customary to refer also to *motion* recovery, implicitly assuming the equivalent scenario to the previous one in which a static scene is framed by a moving camera with known intrinsic parameters. In each case, orientation or rigid motion is represented by a direct isometry.

The approach that we will describe in Sect. 7.2 follows the one proposed by Longuet-Higgins (1981): it is based on the *essential matrix*, which describes the epipolar geometry of two perspective images with known intrinsic parameters and from which the rigid motion of the camera can be derived with a factorisation described in Sect. 7.3.

Note that the translational component of the displacement can only be calculated up to a scaling factor, because it is impossible to determine (without additional information) whether the motion observed in the image is caused by a nearby object with the camera moving slowly or by a distant object with the camera moving faster. This fact is known as the *depth-speed ambiguity*.

7.2 The Essential Matrix

Suppose we have a camera, with known intrinsic parameters, that is moving in a static environment following an unknown trajectory. Consider two images taken by

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_7

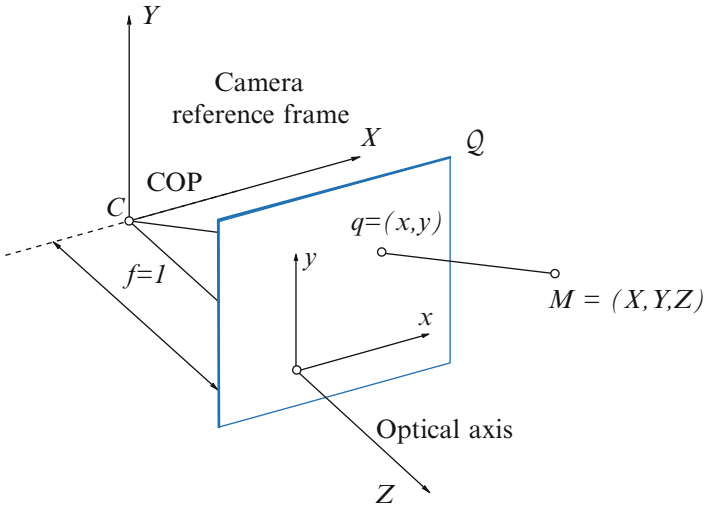


Fig. 7.1 Reference frames and normalised image coordinates system

the camera at two different time instants and assume that a number of homologous points, in *normalised image coordinates*, are given.

Let P and P' be the PPM of the cameras corresponding to the two time instants and $\mathbf{q} = K^{-1}\mathbf{m}$, $\mathbf{q}' = K'^{-1}\mathbf{m}'$ the normalised coordinates of the homologous points in the images, which are projections of the same tie point M (Fig. 7.1).

Without loss of generality (see Problem 7.1) we represent the 3D points in the reference frame of the first camera and thus write the following two PPMs:

$$P = [I, \mathbf{0}] \quad \text{and} \quad P' = [I, \mathbf{0}]G = [R, \mathbf{t}]. \tag{7.1}$$

By retracing the derivation of the Longuet-Higgins equation (6.7) with these two particular PPMs, we obtain the bilinear form that links homologous points in normalised image coordinates (also called the "Longuet-Higgins equation"):

$$\mathbf{q}'^{\mathsf{T}}[\mathbf{t}]_\times R\mathbf{q} = 0. \tag{7.2}$$

The matrix

$$E = [\mathbf{t}]_\times R \tag{7.3}$$

which contains the coefficients of the form is called the *essential matrix*. It depends on three parameters for rotation and on *two* parameters for translation. In fact, (7.2) is homogeneous with respect to $\mathbf{t}$, that is, the modulus of the vector does not matter. This reflects the depth-speed ambiguity, that is, the fact that we cannot derive the absolute scale of the scene without an additional parameter, for example knowledge

of the distance between two tie points. Therefore, an essential matrix has only five degrees of freedom, accounting for rotation (three parameters) and translation up to a scale factor (two parameters).

In terms of constraints, we can observe that the essential matrix is defined up to a scaling factor and is singular, since $\det[\mathbf{t}]_\times = 0$. This brings the degrees of freedom to seven; in order to match the five degrees of freedom of the parameterisation, we need to be able to exhibit other two constraints. The theorem in the next section proves that these two constraints are the equality of the two non-zero singular values of E (a polynomial in the elements of E, which yields two independent constraints).

The essential matrix and the fundamental matrix are related, since both encode rigid motion between two cameras. The former relates the *normalised image coordinates* of the homologous points, while the latter relates the *pixel* coordinates of the same points. It is easy to verify that:

$$F = K'^{-\mathsf{T}} E K^{-1} \text{ and vice versa } E = K'^{\mathsf{T}} F K. \tag{7.4}$$

7.2.1 Geometric Interpretation

The Longuet-Higgins equation (7.2):

$$\mathbf{q}_i'^{\mathsf{T}} \mathbf{t} \times R\mathbf{q}_i = 0. \tag{7.5}$$

holds for any pair of homologous points $(\mathbf{q}_i, \mathbf{q}_i')$, in normalised image coordinates. We can visualise $\mathbf{q}$ and $\mathbf{q}'$ (Fig. 7.2) as the vectors of $\mathbb{R}^3$ that connect the COP (origin of the camera reference frame) and the corresponding point on the image (which has equation $Z = 1$, in front of the camera). Then the Longuet-Higgins equation can be interpreted as the coplanarity of the three vectors $\mathbf{q}'$, $\mathbf{t}$, $R\mathbf{q}$ ($\mathbf{q}$ is rotated to bring it into the reference frame of the left camera), since the left member is nothing but the *triple product* of these vectors, which cancels if and only if they are coplanar (see Appendix A).

> Equation (7.2) is at the basis of the method proposed by Horn (1990b) for the computation of relative orientation. He observed that by dividing (7.2) by $\|\mathbf{q}_i' \times R\mathbf{q}_i\|$, one obtains a geometrically meaningful quantity that can be iteratively minimised.

This geometric interpretation also allows the position of M to be calculated very easily, a process called *triangulation* (which will be discussed in Chap. 8). Since the three vectors $\mathbf{q}'$, $\mathbf{t}$, $R\mathbf{q}$ are coplanar when we scale $\mathbf{q}'$ and $R\mathbf{q}$ so that they meet in M, their difference must be equal to t. In other words:

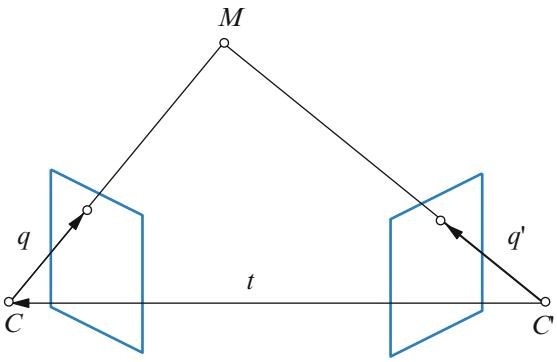


Fig. 7.2 The Longuet-Higgins equation as the coplanarity of three vectors

$$\mathbf{t} = \zeta' \mathbf{q} - \zeta R \mathbf{q} \tag{7.6}$$

where ζ and ζ' are the unknown scaling factors. This is a system of three equations in two unknowns that can be solved for ζ and ζ'. It is easy to see that they are the depth of point M in the right and left cameras, respectively.

7.2.2 Computing the Essential Matrix

The essential matrix can be computed from the correspondences of at least eight points, using the method described for the fundamental matrix in Sect. 6.4. All it takes is replacing the pixel coordinates with the normalised image coordinates.

Note that, due to the inevitable errors that plague measurements, in general, the matrix E found by solving the system of linear equations will not satisfy the requirements of Theorem 7.1, that is, it will not have two equal singular values and one singular value equal to zero. However, this can be forced a posteriori via SVD.

Let E be a 3×3 matrix and $E = UDV^\mathsf{T}$ its SVD with $D = \text{diag}(\sigma_1, \sigma_2, \sigma_3)$ and $\sigma_1 \geq \sigma_2 \geq \sigma_3$. It can be shown that (similarly to the Eckart-Young's Theorem)

$$\widehat{E} = U \,\text{diag}\left(\frac{\sigma_1 + \sigma_2}{2}, \frac{\sigma_1 + \sigma_2}{2}, 0\right) V^\mathsf{T} \tag{7.7}$$

is the closest matrix to E, in Frobenius norm, that satisfies the requirements of Theorem 7.1.

Since E depends on five parameters, it is possible, in principle, to compute it with five correspondences plus the polynomial constraints provided by Theorem 7.1. Faugeras and Maybank (1990) showed that this is indeed the case and that ten distinct solutions exist. Subsequently, algorithms for computing E with five points were proposed by Nister (2003) and Li and Hartley (2006).

Listing 7.1 Linear computation of E

```
function E = essential_lin(q2,q1,K1,K2)
%ESSENTIAL_LIN  Essential matrix with 8-points algorithm

    q1 = K1\ensure_homogeneous(q1);
    q2 = K2\ensure_homogeneous(q2);
    % now q1 and q2 are homogeneous NIC

    E = eight_points(q2(1:2,:), q1(1:2,:));

    % enforce constraints on E
    [U,D,V] = svd(E);
    s = (D(1,1)+D(2,2))/2;
    E = U * diag([s,s,0]) * V';

end
```

The MATLAB function that computes the essential matrix from point correspondences (Listing 7.1) makes use of the function `eight_points` that we introduced for computing the fundamental matrix. What changes is the input—pixel coordinates for F versus normalised image coordinates for E—and the type of constraints that are forced a posteriori.

7.3 Relative Orientation from the Essential Matrix

The following theorem due to Huang and Faugeras (1989) characterises the essential matrix and allows us to factorise it in rotation and translation.

Theorem 7.1 (Essential Matrix Factorisation) *A real* 3×3 *matrix* E *can be factorised as a product of a non-zero antisymmetric matrix and a rotation matrix if and only if* E *has two equal non-zero singular values and one singular value equal to zero.*

Proof Let $E = UDV^\top$ be the SVD of E, with $D = \text{diag}(1, 1, 0)$ (with no loss of generality, since E is defined up to a scale factor) and U and V orthogonal. The key observation is that:

$$D = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}}_{S'} \underbrace{\begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}}_{R'} \tag{7.8}$$

where S' is antisymmetric and R' is a rotation matrix (of $-\pi/2$ on the z-axis). Thus,

$$E = US'R'V^\top = (US'U^\top)(UR'V^\top) \simeq \underbrace{(US'U^\top)}_{S} \underbrace{\det(UV^\top)(UR'V^\top)}_{R}. \tag{7.9}$$

Taking $S = US'U^\mathsf{T}$ and $R = \det(UV^\mathsf{T})UR'V^\mathsf{T}$, the sought factorisation is: $E = SR$. In fact, the matrix $US'U^\mathsf{T}$ is antisymmetric (Problem 7.2) and the matrix $\det(UV^\mathsf{T})UR'V^\mathsf{T}$ is a rotation (Problem 7.3). This proves one implication; let us now see the other direction.

Let $E = SR$ where R is a rotation matrix and S is antisymmetric. By virtue of Proposition A.3, we have $S \simeq US'U^\mathsf{T}$. Also from (7.8) we get: $S' = \text{diag}(1, 1, 0)R'^\mathsf{T}$. Therefore:

$$E \simeq U\,\text{diag}(1, 1, 0)(R'^\mathsf{T}U^\mathsf{T}R). \tag{7.10}$$

Since $(R'^\mathsf{T}U^\mathsf{T}R)$ is orthogonal, this is a singular value decomposition of E with two equal singular values and one null, as was to be proved.

Observe that $\mathbf{t}$ spans to the null space of S; hence, it belongs to null space of E^T, which means that $\mathbf{t}$ is equal to the third column of U.

The factorisation of E constructed in the first part of the proof is not unique. In fact, it is possible to transpose both S' and R' obtaining in all four combinations, of which two change the sign of E (irrelevant, since it is defined up to a multiplication by a scalar). In fact, we see that $S'R'^\mathsf{T} = -D$, while by the definition of antisymmetric: $S'^\mathsf{T} = -S'$.

In summary, then, the four possible factorisations are given by:

$$S = \pm US'U^\mathsf{T} \quad \text{and} \quad R = \det(UV^\mathsf{T})U\left\{R'\ R'^\mathsf{T}\right\}V^\mathsf{T}. \tag{7.11}$$

The geometric interpretation of the four solutions is as follows: changing the sign of S swaps right with left, while choosing R or R' results in a rotation of the camera by 180° around the baseline. In all cases it is possible to triangulate the tie points, obtaining a three-dimensional model, but only in one case do the tie points lie in front of both cameras (Fig. 7.3), and this requirement identifies the only physically feasible solution. A single tie point is sufficient to discriminate.

The property that specifies whether a point is in front of or behind with respect to a given camera is called *chirality* of the point.[1]

The MATLAB implementation is given in Listing 7.2. Note that for the chirality test we solved (7.6) using the function `icomb` that implements Proposition A.18.

[1] A geometric configuration is said to be *chiral* if it is not superimposable on its mirror image, such as right hand and left hand.

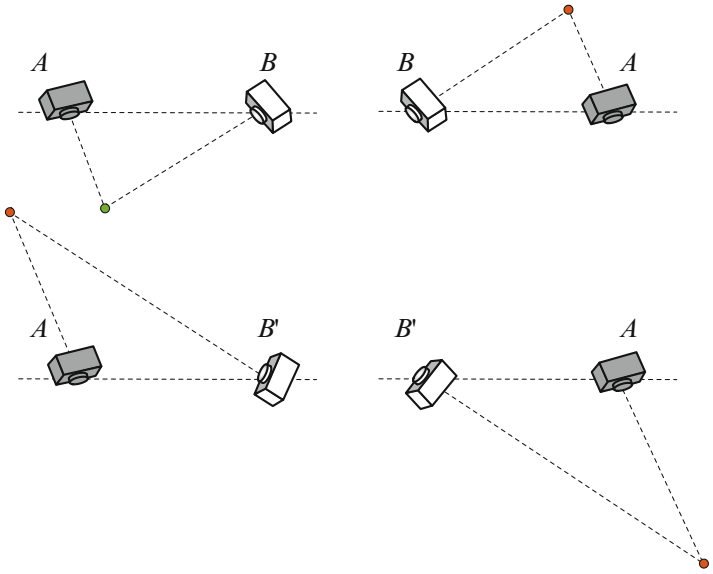


Fig. 7.3 The four possible solutions of the factorisation of E. Between the left and right columns there is a left-right inversion, while between the top and bottom rows the camera B rotates 180° around the baseline. Only in the top-left case the triangulated point lies in front of both cameras

Listing 7.2 Relative orientation from the factorisation of E

```
function [R,t] = relative_lin(q2, q1, K2, K1)
%RELATIVE_LIN Relative orientation

q1 = K1\ensure_homogeneous(q1);
q2 = K2\ensure_homogeneous(q2);
% now q1 and q2 are homogeneous NIC

E = eight_points(q2(1:2,:), q1(1:2,:)) ;

[U,~,V] = svd(E);

S1 = [0 1 0; -1 0 0; 0 0 0];
R1 = [0 -1 0; 1 0 0; 0 0 1];

for j = 1:4
    % 4 combinations
    S = (-1)^j * U*S1*U';

    if j==3   R1 = R1'; end

    t=[S(3,2) S(1,3) S(2,1)]';
    R = det(U*V')*U*R1*V';

    % solve t = z2*q2 - z1*R*q1 for z1 and z2
    z = [ q2(:,1), -R*q1(:,1)] \t;
    % 3D points must be in front of both cameras
    if z(1)>0 && z(2)>0  break,  end

end
```

7.3.1 Closed Form Factorisation of the Essential Matrix

Horn (1990a) proved that the factorisation of E can also be obtained without SVD, in closed form.

We will preliminarily note that (based on Problem 7.5) multiplying E by the factor $\sqrt{2}/\|E\|_F$ leads us back to the case where $\|\mathbf{t}\| = 1$. We therefore assume that this is the case.

Let us write $R = [\mathbf{r}_1, \mathbf{r}_2, \mathbf{r}_3]$ and $E = [\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3]$, then

$$E = [\mathbf{t}]_\times R = [\mathbf{t} \times \mathbf{r}_1, \mathbf{t} \times \mathbf{r}_2, \mathbf{t} \times \mathbf{r}_3] \tag{7.12}$$

means that each column of E is perpendicular to $\mathbf{t}$ so the latter is parallel to the cross product of any pair of columns of E, for example:

$$\mathbf{e}_1 \times \mathbf{e}_2 = (\mathbf{t} \times \mathbf{r}_1) \times (\mathbf{t} \times \mathbf{r}_2) = (\mathbf{t}^\mathsf{T}(\mathbf{r}_1 \times \mathbf{r}_2))\mathbf{t} \simeq \mathbf{t} \tag{7.13}$$

by a triple product property.

As for the recovery of R given $\mathbf{t}$, let us turn to the *adjoint matrix*, defined as

$$\text{adj}(E)^\mathsf{T} = \left[\mathbf{e}_2 \times \mathbf{e}_3, \mathbf{e}_3 \times \mathbf{e}_1, \mathbf{e}_1 \times \mathbf{e}_2\right] \tag{7.14}$$

$$\begin{aligned}
\text{adj}(E)^\mathsf{T} &= \text{adj}([\mathbf{t} \times \mathbf{r}_1,\ \mathbf{t} \times \mathbf{r}_2,\ \mathbf{t} \times \mathbf{r}_3])^\mathsf{T} \\
&= [(\mathbf{t} \times \mathbf{r}_2) \times (\mathbf{t} \times \mathbf{r}_3),\ (\mathbf{t} \times \mathbf{r}_3) \times (\mathbf{t} \times \mathbf{r}_1),\ (\mathbf{t} \times \mathbf{r}_1) \times (\mathbf{t} \times \mathbf{r}_2)] \\
&= [\mathbf{t}^\mathsf{T}(\mathbf{r}_2 \times \mathbf{r}_3)\mathbf{t},\ \mathbf{t}^\mathsf{T}(\mathbf{r}_3 \times \mathbf{r}_1)\mathbf{t},\ \mathbf{t}^\mathsf{T}(\mathbf{r}_1 \times \mathbf{r}_2)\mathbf{t}] \\
&= [\mathbf{t}^\mathsf{T}\mathbf{r}_1\mathbf{t},\ \mathbf{t}^\mathsf{T}\mathbf{r}_2\mathbf{t},\ \mathbf{t}^\mathsf{T}\mathbf{r}_3\mathbf{t}] = [\mathbf{t}\mathbf{t}^\mathsf{T}\mathbf{r}_1,\ \mathbf{t}\mathbf{t}^\mathsf{T}\mathbf{r}_2,\ \mathbf{t}\mathbf{t}^\mathsf{T}\mathbf{r}_3] = \mathbf{t}\mathbf{t}^\mathsf{T}R \\
&= ([\mathbf{t}]_\times^2 + \mathbf{t}^\mathsf{T}\mathbf{t}I_3)R = [\mathbf{t}]_\times^2 R + \mathbf{t}^\mathsf{T}\mathbf{t}R = [\mathbf{t}]_\times E + R
\end{aligned} \tag{7.15}$$

From which we get

$$R = \text{adj}(E)^\mathsf{T} - [\mathbf{t}]_\times E \tag{7.16}$$

Regarding the ambiguity of the solutions, we have two solutions of opposite sign for translation, which generate two solutions for rotation, respectively. Combining them yields four solutions. This factorisation is implemented in Listing 7.3.

Listing 7.3 Relative orientation from E in closed form

```
function [R, t] = motion_from_E(E)
% MOTION_FROM_E Extract R and t from E in closed form (Horn's method)
% Returns all the 4 possible solutions as cell arrays

% compute t
E = sqrt(2) * E./(norm(E,'fro'));
b = cross(E(:,1), E(:,2));
t{1} =  b./norm(b);
t{2} = - t{1};

% compute R
R{1} = adjoint(E)' - skew(t{1}) * E;
R{2} = adjoint(E)' - skew(t{2}) * E;

end
```

7.4 Relative Orientation from the Calibrated Homography

In analogy to what we saw in the previous section with reference to the essential matrix, knowledge of the homography induced by a plane between two images, along with the intrinsic parameters of the cameras, can also serve to recover the relative orientation of the two images, plus the parameters of the plane that induced the homography.

From the definition of H_Π (6.23) we immediately get:

$$\underbrace{K'^{-1} H_\Pi K}_{\bar{H}} = R + \frac{\mathbf{t}\mathbf{n}^\mathsf{T}}{d}. \tag{7.17}$$

This means the left-hand side term—which we dub *calibrated homography* $\bar{H}$—encodes the motion parameters $(R, \mathbf{t})$ and also the plane parameters $\mathbf{n}$ and d. As a matter of fact, this equation can be solved (with a double ambiguity in the solution) to obtain: (i) the normal to the plane $\mathbf{n}$, (ii) the rotation R and (iii) the translation scaled with the distance from the plane $(\mathbf{t}/d)$. The algorithm we will describe follows, to some extent, the one introduced by Faugeras and Lustman (1988).

The calibrated homography $\bar{H}$ will in general contain an unknown scaling factor, which we can however eliminate by dividing it by its second singular value, σ_2 (Problem 7.7). We therefore assume that $\sigma_2 = 1$.

We begin by deriving the normal to the plane, and to this end we consider the SVD of $\bar{H}$:

$$\bar{H} = U D V^\mathsf{T} \tag{7.18}$$

with $D = \mathrm{diag}(\sigma_1, 1, \sigma_3)$.

Therefore:

$$D = (U^\mathsf{T} R V) + \left(U^\mathsf{T} \frac{\mathbf{t}}{d}\right) (V^\mathsf{T} \mathbf{n})^\mathsf{T} = R' + \mathbf{t}' \mathbf{n}'^\mathsf{T}. \tag{7.19}$$

Note that R' is only orthogonal, to get a rotation one would need to fix the determinant, but this is not necessary.

Let us multiply both members by

$$\left[\mathbf{n}'\right]_\times = \begin{bmatrix} 0 & -n'_3 & n'_2 \\ n'_3 & 0 & -n'_1 \\ -n'_2 & n'_1 & 0 \end{bmatrix} \tag{7.20}$$

getting

$$D\left[\mathbf{n}'\right]_\times = R'\left[\mathbf{n}'\right]_\times \tag{7.21}$$

and so

$$\left[\mathbf{n}'\right]_\times^\mathsf{T} D^\mathsf{T} D\left[\mathbf{n}'\right]_\times = \left[\mathbf{n}'\right]_\times^\mathsf{T} R'^\mathsf{T} R'\left[\mathbf{n}'\right]_\times . \tag{7.22}$$

Since $\left[\mathbf{n}'\right]_\times^\mathsf{T} = -\left[\mathbf{n}'\right]_\times$ and $R'^\mathsf{T} R' + I$ we have

$$\left[\mathbf{n}'\right]_\times (D^2 - I)\left[\mathbf{n}'\right]_\times = 0 \tag{7.23}$$

and finally

$$\mathrm{diag}^{-1}\left(\left[\mathbf{n}'\right]_\times (D^2 - I)\left[\mathbf{n}'\right]_\times\right) = 0 \tag{7.24}$$

where diag^{-1} consumes a square matrix and returns the vector containing the diagonal. Expanding the components, the last equation becomes:

$$\begin{bmatrix} (\sigma_2{}^2 - 1)\, n'_3{}^2 + (\sigma_3{}^2 - 1)\, n'_2{}^2 \\ (\sigma_1{}^2 - 1)\, n'_3{}^2 + (\sigma_3{}^2 - 1)\, n'_1{}^2 \\ (\sigma_1{}^2 - 1)\, n'_2{}^2 + (\sigma_2{}^2 - 1)\, n'_1{}^2 \end{bmatrix} = 0 \tag{7.25}$$

By collecting the unknowns $n'_1{}^2, n'_2{}^2, n'_3{}^2$ we obtain a homogeneous linear system in the squares of the components of $\mathbf{n}'$ (recall that σ_2=1):

$$\begin{bmatrix} 0 & \sigma_3{}^2 - 1 & \sigma_2{}^2 - 1 \\ \sigma_3{}^2 - 1 & 0 & \sigma_1{}^2 - 1 \\ \sigma_2{}^2 - 1 & \sigma_1{}^2 - 1 & 0 \end{bmatrix} \begin{bmatrix} n'_1{}^2 \\ n'_2{}^2 \\ n'_3{}^2 \end{bmatrix} = 0 \tag{7.26}$$

whose solution, up to a scale, is: $|n'_1| = \sqrt{\frac{\sigma_1^2 - 1}{1 - \sigma_3^2}}$, $|n'_2| = 0$, $|n'_3| = 1$.

Since $n'_2 = 0$, we have a total of four possibilities for the signs of n'_1 and n'_3. The solution is up to a scale, which we can fix by imposing $\mathbf{n}'^{\mathsf{T}}\mathbf{n}' = 1$.

Finally, we get: $\mathbf{n} = V\mathbf{n}'$, with $\mathbf{n}^{\mathsf{T}}\mathbf{n} = 1$, V being orthogonal.

We then proceed to determine the rotation and multiply both members of (7.17) by $[\mathbf{n}]_\times$ in order to eliminate the term containing the translations:

$$\bar{H}\,[\mathbf{n}]_\times = R\,[\mathbf{n}]_\times\,. \tag{7.27}$$

Transposing the previous equation

$$[\mathbf{n}]_\times\,\bar{H}^{\mathsf{T}} = [\mathbf{n}]_\times\,R^{\mathsf{T}}, \tag{7.28}$$

we observe that it is a matter of decomposing the left-hand term into an antisymmetric matrix and a rotation matrix, as we did in Sect. 7.3.1. We then use (7.16) where E is substituted by $[\mathbf{n}]_\times\,\bar{H}^{\mathsf{T}}$, therefore:

$$\begin{aligned} R &= \mathrm{adj}([\mathbf{n}]_\times\,H^{\mathsf{T}}) + ([\mathbf{n}]_\times\,H^{\mathsf{T}})^{\mathsf{T}}\,[\mathbf{n}]_\times \\ &= \mathrm{adj}(H)^{\mathsf{T}}\,\mathrm{adj}([\mathbf{n}]_\times) - H\,[\mathbf{n}]_\times^2 = \mathrm{adj}(H)^{\mathsf{T}}\mathbf{n}\mathbf{n}^{\mathsf{T}} - H\,[\mathbf{n}]_\times^2\,. \end{aligned} \tag{7.29}$$

Finally, from (7.17) we derive:

$$\bar{H} - R = \frac{\mathbf{t}}{d}\mathbf{n}^{\mathsf{T}}. \tag{7.30}$$

To obtain $\mathbf{t}/d$ we multiply both members by $\mathbf{n}$, remembering that $\mathbf{n}^{\mathsf{T}}\mathbf{n} = 1$:

$$(\bar{H} - R)\mathbf{n} = \frac{\mathbf{t}}{d} \tag{7.31}$$

From each possible $\mathbf{n}$ (there are four of them) we get one R, but since the sign of $\mathbf{n}$ does not affect R, we actually have only two different R. From each pair $(\mathbf{n}, R)$ we get a $\mathbf{t}$, so we have a total of four possible solutions that return $\bar{H}$. To these we must add four more that return $-\bar{H}$, for a total of eight solutions. In fact, of these two groups only one satisfies a physical constraint involving $\mathbf{n}, R, \mathbf{t}$. Listing 7.4 implements this factorisation.

The solutions can be further reduced to two by using constraints on the visibility of the points, in analogy to the previous section. See Malis and Vargas (2007) for details on eliminating impossible solutions and other methods for decomposition.

Listing 7.4 Relative orientation from calibrated H

```
function [n, t, R] = motion_from_H(H)
% MOTION_FROM_H Extract R, t and n from H
% Based on Faugeras-Lustman's method

[~,D,V] = svd(H);

% the second sing.val. of a normalized H is 1
H = H./D(2,2);
D = D./D(2,2);

n = sqrt([ (D(1,1)^2 - 1)/(1- D(3,3)^2); 0; 1]);
n = n./norm(n);

n1 = V * n;
n2 = V * (n .* [1 1 -1]');
% the other two solutions have opposite sign
% n1,n2 have unit norm

% compute R (Horn's formula)
R1 = adjoint( H)' * (n1*n1') - H * skew(n1)^2;
R2 = adjoint( H)' * (n2*n2') - H * skew(n2)^2;
R3 = adjoint(-H)' * (n1*n1') + H * skew(n1)^2;
R4 = adjoint(-H)' * (n2*n2') + H * skew(n2)^2;
% changing the sign of n does not change the rotation

t1 = (H-R1)*n1 ;
t2 = (H-R2)*n2 ;
t4 = (-H-R4)*n2 ;
t3 = (-H-R3)*n1 ;
% the last two have the same direction as the first two,
% but opposite sign and different modulus.
% Since we want to recover t/d, the modulus also matters

% these are the 8 combinations
R = { R1 R2  R1  R2  R3  R4  R3  R4 };
t = { t1 t2 -t1 -t2  t3  t4 -t3 -t4 };
n = { n1 n2 -n1 -n2  n1  n2 -n1 -n2 };

% of these, only 4 are valid, either the first 4 or the last 4
for i=1:8
    d(i) = (1+n{i}'*R{i}'*t{i});
end
bad = d<=0;
n(bad)=[]; t(bad)=[]; R(bad)=[];

end
```

Problems

7.1 Derive the essential matrix with two *generic* PPMs and observe that: a) the translation and the rotation that appear in the expression for E are the relative ones; b) by applying the same isometry to the right of each PPM, the expression for E remains unchanged. This allows us, without loss of generality, to fix $P = [I, \mathbf{0}]$.

7.2 Prove that if S' is antisymmetric, then $US'U^\mathsf{T}$ is also antisymmetric.

7.3 Prove that given a matrix of rotation R and two orthogonal matrices U and V, then $\det(UV^\mathsf{T})URV^\mathsf{T}$ is a rotation matrix (i.e. has positive determinant).

7.4 Obtain the direction of relative translation in the case where the relative rotation is given. Two points are enough.

7.5 Prove that:

$$\|E\|_F^2 = \text{tr}(EE^\mathsf{T}) = 2\|\mathbf{t}\|^2. \tag{7.32}$$

7.6 Derive the relationship between the fundamental matrix and the essential matrix.

7.7 Prove that any 3×3 matrix A of the form $A = Q + \mathbf{ab}^\mathsf{T}$ with Q orthogonal must have $\sigma_2 = 1$ where σ_2 is the second singular value.

References

O. Faugeras and S. Maybank. Motion from point matches: multiplicity of solutions. *International Journal of Computer Vision*, 4(3):225–246, June 1990.

O.D. Faugeras and F. Lustman. Motion and structure from motion in a piecewise planar environment. *International Journal of Pattern Recognition and Artificial Intelligence*, 2:485–508, 1988.

Berthold Horn. Recovering baseline and orientation from essential matrix. Unpublished, 1990a.

B.K.P. Horn. Relative orientation. 4(1):59–78, January 1990b.

T.S. Huang and O.D. Faugeras. Some properties of the E matrix in two-view motion estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11(12):1310–1312, December 1989.

Hongdong Li and Richard Hartley. Five-point motion estimation made easy. In *Proceedings of the International Conference on Pattern Recognition*, pages 630–633, Washington, DC, USA, 2006. IEEE Computer Society.

H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293(10):133–135, September 1981.

Ezio Malis and Manuel Vargas. Deeper understanding of the homography decomposition for vision-based control. Research Report RR-6303, INRIA, 2007. https://hal.inria.fr/inria-00174036.

David Nister. An efficient solution to the five-point relative pose problem. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, page 195, Los Alamitos, CA, USA, 2003. IEEE Computer Society.

Chapter 8
Reconstruction from Two Images



8.1 Introduction

We will deal in this chapter with *triangulation*, which allows us to obtain the 3D coordinates of the *tie points*, that is points whose projections have been matched in the images. The set of 3D points obtained from triangulation is also called *model*. We will see that the model differs from the real one by transformations that reflect our degree of knowledge about the sensor and/or the scene.

Triangulation and resection are somehow dual operations: in one case observing a point from at least two different known locations one derives the position of the point, while in the other case one orients the camera by measuring the position of at least three points.

8.2 Triangulation

Given homologous points in the two images and given the two PPMs, *triangulation* (also known as forward *intersection*) aims to reconstruct the position in object space of the respective tie points (refer to Fig. 6.1).

Consider m, the projection of point M onto the camera whose PPMs is P. From the perspective projection equation (3.26), we derived in Sect. 4.2 that:

$$[\mathbf{m}]_{\times} P\mathbf{M} = \mathbf{0}. \tag{8.1}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_8

These are three equations in the four unknowns of $\mathbf{M}$ (the fourth component of $\mathbf{M}$ is left free and figures among the unknown), but only two of them are linearly independent, being $\mathrm{r}([\mathbf{m}]_\times) = 2$.

Let us now also consider m', the homologous point of m in the second image, and let P' be the second PPMs. Since m and m' are projections of the same tie point M, the equations given by both can be stacked, yielding a homogeneous linear system of six equations (of which no more than four are independent) in four unknowns:

$$\underbrace{\begin{bmatrix} [\mathbf{m}]_\times P \\ [\mathbf{m}']_\times P' \end{bmatrix}}_{A} \mathbf{M} = \mathbf{0}_{4\times 1}. \tag{8.2}$$

The solution is the null space of the 6×4 matrix A, which therefore must have rank three, otherwise, the only solution would be the trivial one $\mathbf{M} = \mathbf{0}$.

In the presence of noise, however, the condition on rank is not satisfied exactly, and so a least-squares solution is obtained as the least right singular vector of A (see Proposition A.13). Hartley and Sturm (1997) call this method *linear-eigen*, because the right singular vectors are the eigenvectors of the matrix $A^\mathsf{T}A$.

The above generalises to the case of $m > 2$ images: each image adds two equations and we obtain a homogeneous system of $2m$ equations in four unknowns. The MATLAB code for the triangulation of one point in multiple images is shown in Listing 8.1.

Listing 8.1 Triangulation

```
function M = triang_lin(P, m)
%TRIANG_LIN Triangulation for one point in multiple images

    L=[]; % stack equations for every camera
    for j=1:length(P)
        L = [L; skew([m{j};1])*P{j} ]  ;
    end
    [~,~,V] = svd(L);
    M =  V(:,end)./V(end,end);
    M = M(1:3,:);
end
```

Iterative refinement of the *linear-eigen* method. With reference—for example—to the second equation of the linear system 8.2, the residual that is minimised is $\widehat{\chi} = \mathbf{p}_1^\mathsf{T}\mathbf{M} - u\mathbf{p}_3^\mathsf{T}\mathbf{M}$, while instead we would like to minimise the geometric residual, that is the difference between the measured u coordinate and the projection of $\mathbf{M}$: $\chi = \frac{\mathbf{p}_1^\mathsf{T}\mathbf{M}}{\mathbf{p}_3^\mathsf{T}\mathbf{M}} - u$ (see also Chap. 9). Observe that $\chi = \frac{\widehat{\chi}}{\mathbf{p}_3^\mathsf{T}\mathbf{M}}$, that is if the equation were weighted with $\frac{1}{\mathbf{p}_3^\mathsf{T}\mathbf{M}}$, we would be

(continued)

minimising exactly the geometric residual. The problem is that the weights depend on $\mathbf{M}$, which is precisely what we want to compute. The solution is to iterate the *linear-eigen* method by weighting the equations with values calculated from the coordinates of $\mathbf{M}$ in the previous step.

8.3 Ambiguity of Reconstruction

Consider a set of 3D points, framed by two cameras represented by P and P'. Let $\mathbf{m}_j$ be the (homogeneous) coordinates of the projection of the j-th point in the first image and $\mathbf{m}'_j$ the (homogeneous) coordinates of the projection of the j-th point in the second image.

The *reconstruction* problem can be posed as follows: given the coordinates (in pixels) of the homologous points $\mathbf{m}_j$ and $\mathbf{m}'_j$ find the PPMs $\{P, P'\}$ and the model $\{\mathbf{M}_j\}$ such that:

$$\mathbf{m}_j \simeq P\mathbf{M}_j \text{ and } \mathbf{m}'_j \simeq P'\mathbf{M}_j. \tag{8.3}$$

We will also denote by the term *reconstruction* the pair formed by the set of PPMs and the 3D model such that (8.3) holds.

This reconstruction can be unique or ambiguous depending on what is known about the PPMs. We shall see that the reconstruction differs from the true one by transformations that become progressively more general as the information on the PPMs decreases.

We will preliminarily note that if $\{P, P'\}$ and $\{\mathbf{M}^j\}$ are a reconstruction, that is they satisfy (8.3), then also $\{P, P'\}T$ and $\{T^{-1}\mathbf{M}^j\}$ satisfy (8.3) for any non-singular 4×4 matrix T, which specifies a linear transformation of the 3D projective space.

If the PPMs are unconstrained (no information), any T produces a valid reconstruction. On the other hand, if we know that the PPMs must satisfy certain constraints, this limits the generality of T to those transformations that preserve the constraints.

At one extreme of the spectrum there is the full knowledge of $\{P, P'\}$ (barring the usual irrelevant scaling factor). In this case T can only be the identity; in fact, any other T would change at least one of the two PPMs (see Problem 8.1). The reconstruction is identical to the true one, expressed in the same reference frame in which the exterior orientation of the camera is given.

If the two PPMs are known modulo a direct isometry (the same for both), it means that the exterior orientation is unknown but the relative orientation is assigned. In this case T is constrained to be a direct isometry, which leaves the relative orientation unchanged (see Problem 7.1). Correspondingly, the points are

pre-multiplied by T^{-1}, which is itself a direct isometry. The reconstruction that is obtained differs from the true one by a (unknown) direct isometry and is called *metric reconstruction*. Note that in this case the interior orientation is known, so the PPMs must have the structure $K[R, t]$ with $R \in SO(3)$, a property that a direct isometry preserves.

In practice, the modulus of translation is hardly known (only in special cases such as a pair of cameras rigidly mounted on a bar). Much more frequent is the case of one camera that moves freely, so one can only recover the relative orientation up to the translation modulus, as seen in Chap. 7. In this case T is a similitude and the global scaling that it contains only affects the translation component of the PPM (see Problem 8.2). The resulting reconstruction differs from the true one by a similitude and is called *Euclidean reconstruction*.

The opposite extreme is when nothing is known about the PPM. In this case, as already observed, T can be any non-singular matrix, and we get a reconstruction differing from the true one by an arbitrary projectivity, which is why it is called *projective reconstruction*.

At the end of the chapter we will pick up this discussion and summarise it in a table.

8.4 Euclidean Reconstruction

Consider a single moving camera; the intrinsic parameters (i.e. K) are known, but the motion of the camera is unknown (i.e. the extrinsic parameters are missing). In this case, a Euclidean reconstruction is obtained, as we have already discussed. The procedure to obtain such a reconstruction is also called Structure from Motion (SFM) in the literature. It is based on recovering the relative orientation $(R, \hat{\mathbf{t}})$ (up to the modulus of translation) from E and the observation that this pair of *canonical* PPM

$$P = K[I, \mathbf{0}] \text{ and } \quad P' = K[R, \hat{\mathbf{t}}] \tag{8.4}$$

yields the essential matrix: $E = \left[\hat{\mathbf{t}}\right]_{\times} R$. This pair is not the only one to have this property: as already noted we can obtain the same E by multiplying the two PPMs by the same similitude T. The Euclidean reconstruction we obtain is defined up to a similitude, which is arbitrarily fixed by choosing $P = K[I, \mathbf{0}]$ and $\|\mathbf{t}\| = 1$. Once the two PPMs are instantiated, the model is obtained by triangulation. The procedure is illustrated in Listing 8.2, while Fig. 8.1 shows a sample of the output.

The upgrade from Euclidean to metric reconstruction is obtained as soon as the length of the baseline, or the distance between two control points, is known, which fix the unknown scale of the model. In order to upgrade this to the identical reconstruction, it is necessary to determine also the unknown isometry, through the knowledge of the coordinates of at least three control points, by solving an *absolute orientation* problem (Sect. 5.2)

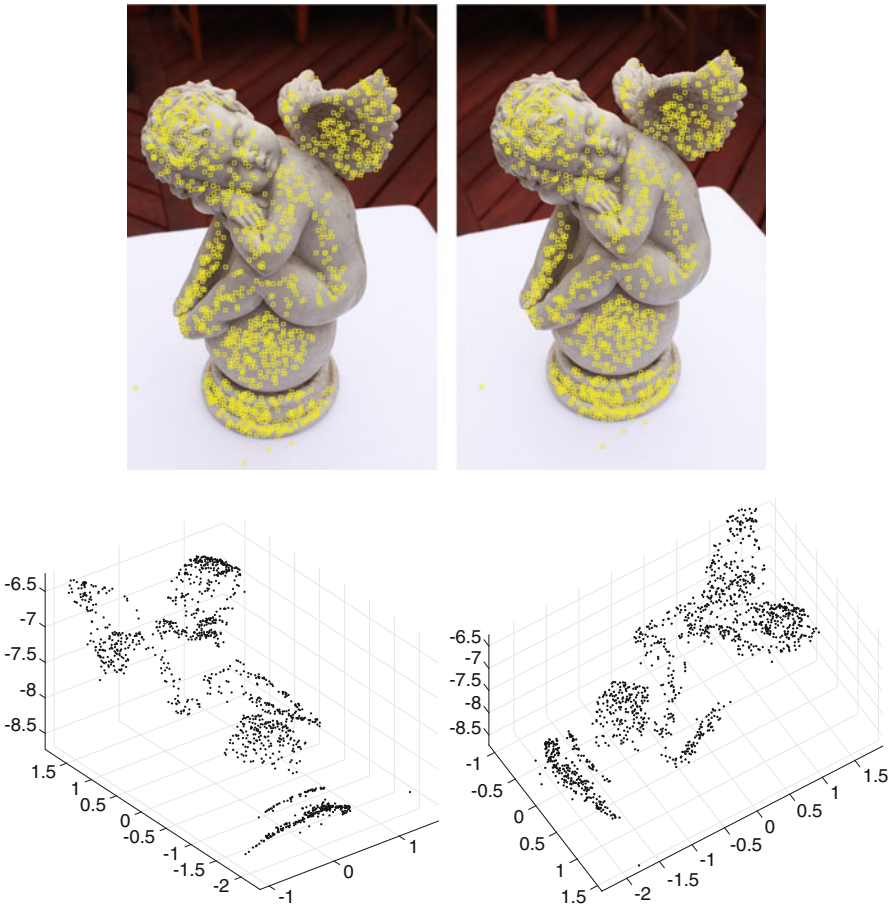


Fig. 8.1 Two images with the homologous points (top row). Two views of the reconstructed 3D model (bottom row). The units on the axes are meaningless, as the model is known up to a scale

This is not the only route to follow in order to obtain an identical reconstruction when a sufficient number of control points is given. Other procedures are well studied and codified in photogrammetry (Kraus 2007). One alternative is to solve for the exterior orientation of each image separately (Chap. 5). While in the previous case (relative + absolute orientation) three control points in all are sufficient, for exterior orientation three control points must be visible in each image, which makes three to six control points in total, depending on visibility. A third path would be to run bundle adjustment (see Chap. 14) with three control points.

Listing 8.2 Reconstruction from two images

```
%% Two views reconstruction

% Compute (load) homologous points  m1 and m2
load matches

% Intrinsic parameters from calibration
K =  [2864.8, 0, 636.7; 0, 2864.8, 931.9; 0,0,1];

% Relative orientation
[R,t] = relative_lin(m2, m1, K, K);
[R,t] = relative_nonlin(R, t ,m2, m1, K, K);

% Canonical camera pair
PPM{1} = K*[eye(3),zeros(3,1)];
PPM{2} = K*[R,t];

% Triangulation
M_out = triang_lin_batch(PPM, {m1,m2});
M_out = triang_nonlin_batch(M_out, PPM, {m1,m2});
% Done
```

8.5 Projective Reconstruction

When the intrinsic parameters are unknown, the only information that can be extracted from a pair of images are the homologous points, and from them it is possible to compute the fundamental matrix. By following *mutatis mutandis* the procedure of Sect. 8.4 based on the essential matrix, we arrive at a reconstruction of the scene, defined however up to a projectivity of the space.

A fundamental matrix can be factorised into

$$F = [\mathbf{e}']_\times A \tag{8.5}$$

with an appropriate matrix A, which is said to be *compatible* with F. The reader will note a resemblance with the factorisation $E = [\mathbf{t}]_\times R$. Unfortunately, however, for the fundamental matrix there are infinitely many such factorisations. In fact, if a matrix A is compatible with F, then also $A + \mathbf{e}'\mathbf{v}^\mathsf{T}$ is compatible, for every vector $\mathbf{v} \in \mathbb{R}^3$, since

$$[\mathbf{e}']_\times (A + \mathbf{e}'\mathbf{v}^\mathsf{T}) = [\mathbf{e}']_\times A + [\mathbf{e}']_\times \mathbf{e}'\mathbf{v}^\mathsf{T} = [\mathbf{e}']_\times A. \tag{8.6}$$

If A is any matrix compatible with F, it is easily verified that the following pair of PPM:

$$P = [I, \mathbf{0}] \quad \text{and} \quad P' = [A, \mathbf{e}'] \tag{8.7}$$

yields the given fundamental matrix. Once the two PPMs are instantiated, the (projective) model is obtained by triangulation.

We are left with the problem of obtaining a matrix A compatible with F. We will see in Problem 8.3 that any homography induced by a plane, including H_∞,

is compatible with F. Another matrix that is compatible with F but is not a homography (being singular) is the following:

$$S = -\frac{1}{\|\mathbf{e}'\|}[\mathbf{e}']_\times F. \tag{8.8}$$

Substituting $A = S$ into the (8.7), we obtain the so-called *canonical representation*, introduced by Luong and Viéville (1996).

Upgrade from projective to Euclidean reconstruction is achieved by knowledge of the intrinsic parameters (Sect. 8.6) or through a process known as self-calibration, which we will study in Chap. 16. Otherwise, one can step directly to the identical reconstruction through the knowledge of the coordinates of at least five control points, which are sufficient to compute a projectivity of the space, for example with the DLT algorithm.

8.6 Euclidean Upgrade from Known Intrinsic Parameters

In this section we shall see how to upgrade to Euclidean a projective pair of PPM when the intrinsic parameters are given. The problem was addressed by Bougnoux (1998) who formulated it as a system of linear equations. The closed form reported here is taken from Gherardi and Fusiello (2010).

We start with a projective reconstruction:

$$P = [I|\mathbf{0}] \qquad P' = \left[A|\mathbf{e}'\right] \tag{8.9}$$

and suppose we know the intrinsic parameters of the two PPMs, K and K', respectively. There must exists a non-singular 4×4 matrix T, which transforms the projective reconstruction into the Euclidean one, that is:

$$PT \simeq K\,[I|\mathbf{0}] \qquad P'T \simeq K'\,[R|\mathbf{t}]\,. \tag{8.10}$$

It is easy to see that the expression for P in (8.9) fixes the following form of T:

$$T = \begin{bmatrix} K & \mathbf{0} \\ \mathbf{v}^\mathsf{T} & s \end{bmatrix} \tag{8.11}$$

where $\mathbf{v}^\mathsf{T}$ is a vector of three elements and s is a scalar. From the expression for P' in (8.9) instead we get:

$$P'T = [AK + \mathbf{e}'\mathbf{v}^\mathsf{T}, \mathbf{e}'] \simeq K'\,[R|\mathbf{t}]\,, \tag{8.12}$$

and then, for each block:

$$\begin{cases} R \simeq K'^{-1}\left(AK + \mathbf{e}'\mathbf{v}^\mathsf{T}\right) = K'^{-1}AK + K'^{-1}\mathbf{e}'\mathbf{v}^\mathsf{T} \\ \mathbf{t} \simeq K'^{-1}\mathbf{e}'. \end{cases} \tag{8.13}$$

While translation (up to a scale) is immediately derived, for rotation it is necessary to also recover the unknown vector $\mathbf{v}$. A solution can be found by observing that there always exists a rotation Q such that: $Q\mathbf{t} = [\|\mathbf{t}\| \; 0 \; 0]^\mathsf{T}$.

Then, after multiplying both members of (8.13) by Q:

$$QR \simeq \underbrace{Q\,K^{-1}AK}_{W} + [\|\mathbf{t}\| \; 0 \; 0]^\mathsf{T}\,\mathbf{v}^\mathsf{T}, \tag{8.14}$$

calling W the first right-hand term and $\mathbf{w}^\mathsf{T}$ its rows, we arrive at an expression where the last two rows of QR are explicit, since they are independent of $\mathbf{v}$:

$$QR = \begin{bmatrix} \mathbf{w}_1^\mathsf{T} + \|\mathbf{t}\|\,\mathbf{v}^\mathsf{T} \\ \mathbf{w}_2^\mathsf{T} \\ \mathbf{w}_3^\mathsf{T} \end{bmatrix} / \|\mathbf{w}_3\|. \tag{8.15}$$

The first row of QR is obtained as the cross product of the last two, and from this we finally derive R.

In the presence of errors (8.15) may be only approximately verified; in that case we cannot assume that $\mathbf{w}_2$ and $\mathbf{w}_3$ are orthonormal, so before computing the first line they must be forced to be so, for example via SVD, as we did in Listing 8.3.

Even if one proceeds to Euclidean reconstruction starting from the fundamental matrix and performing the final upgrade with the known interior orientation, the chirality problem that we addressed in the factorisation of E cannot be avoided. Thus, the second PPM of the resulting pair must eventually be adjusted so that the triangulated points are in front of the camera.

8.7 Stratification

This section may be challenging because of the technical detail, but it is supplementary and can be omitted without affecting the understanding of the rest.

We have seen in the previous sections that depending on the information held on the cameras—that provide images from which measurements are made—one can have access to different descriptions of the three-dimensional structure of physical space, via the reconstruction. This hierarchy is called *stratification* by Faugeras (1995), and each of the levels is called *stratum*.

Listing 8.3 Euclidean upgrade of a projective reconstruction

```
function [P1,P2, T] = eucl_upgrade(P1, P2, K1, K2)
%EUCL_UPGRADE Upgrade projective reconstruction given K

% normalize canonical pair
P2 = P2/[P1; [0 0 0 1]]; % P1 = [I,0] is implicit

t = K2\P2(:,4);
Q = rotvec(t,cross(t,rand(3,1))); % 2nd arg is any vector ~= t

W = Q * ( K2\P2(:,1:3)*K1 );
[U,D,V] = svd(W(2:3,:)); % svd takes care of scale
D(1,1)=1; D(2,2)=1;
R(2:3,:)=U*D*V';
R(1,:) = cross(R(2,:),R(3,:));
% upgrade matrix
v =  (R(1,:)*norm(W(3,:))-W(1,:)) / norm(t);
T =  ([P1; [0 0 0 1]]) \ [K1,[0;0;0]; [v , 1]];

% euclidean cameras
R = Q'* R;
P1 = K1*eye(3,4); % result is up to an isometry
P2 = K2*[R t];
end

function R = rotvec(x,y)
% rotation mapping x to [1 0 0]*norm(x) and y in the z=0 plane

R(1,:) = x(:)'/norm(x);
R(3,:) = cross(R(1,:),y(:)');  R(3,:) = R(3,:)/norm(R(3,:));
R(2,:) = cross(R(3,:),R(1,:));
end
```

The table below summarises the strata that are relevant to our discussion. The second row reports the transformation that links the obtained reconstruction to the true one, while the third shows the *invariants*, that is quantities that remain unchanged after this transformation. They are defined as follows:

$$R = K'^{-1}\widetilde{\widetilde{P}}'\widetilde{\widetilde{P}}^{-1}K \tag{8.16}$$

$$\mathbf{t} = K'^{-1}(\mathbf{p}_{2,4} - \widetilde{\widetilde{P}}'\widetilde{\widetilde{P}}^{-1}\mathbf{p}_{1,4}) \tag{8.17}$$

$$E \simeq [\hat{\mathbf{t}}]_\times R \tag{8.18}$$

$$H_\infty \simeq \widetilde{P}'\widetilde{P}^{-1} \tag{8.19}$$

$$\mathbf{e}' \simeq \mathbf{p}_{2,4} - \widetilde{P}'\widetilde{P}^{-1}\mathbf{p}_{1,4} \tag{8.20}$$

$$F \simeq [\mathbf{e}']_\times \widetilde{P}'\widetilde{P}^{-1} \tag{8.21}$$

The $\widehat{\ }$ denotes a normalisation. For the PPM it means dividing by the norm of the third row of $\widetilde{P}$.

The last row gives the canonical pair of PPM that can be written in that case, using only the invariants. Transforming by T the canonical pair yields another possible pairs of cameras that are consistent with the invariant above.

Rows 4 and 5 describe how the canonical pair is obtained from what is known or measurable. "Match" stand for "a sufficient number of homologous points".

Reconstruction	Identical	Metric	Euclidean	Affine	Projective
Transformation T	Identity	Isometry	Similitude	Affinity	Projectivity
Unchanged by T	$P\ P'$	$K, K', [R, \mathbf{t}]$	$K, K', [R, \hat{\mathbf{t}}]$ (E)	$[\widehat{H}_\infty, \hat{\mathbf{e}}']$	$\widehat{F}$
Known	$P\ P'$	K, K' + match + baseline	K, K' + match	match + pts $\in H_\infty$	match
Procedure	Void	Relative o. + set scale	Relative o.	Compute $H_\infty, \hat{\mathbf{e}}'$	Compute F, $\hat{\mathbf{e}}'$
Canonical pair	P, P'	$K[I, \mathbf{0}], K'[R, \mathbf{t}]$	$K[I, \mathbf{0}], K'[R, \hat{\mathbf{t}}]$	$[I, \mathbf{0}], [\widehat{H}_\infty, \hat{\mathbf{e}}']$	$[I, \mathbf{0}], [[\hat{\mathbf{e}}']_\times \widehat{F}, \hat{\mathbf{e}}']$

The affine stratum corresponds to the knowledge of the plane at infinity and its homography, that can be used to instantiate a camera pair. The corresponding transformation T that preserves the structure of the cameras is an affinity, that does not change the plane at infinity. More on this in Luong and Viéville (1996).

Problems

8.1 Prove that, given a PPM P, any transformation $T = \alpha I + \mathbf{C}\mathbf{v}^\mathsf{T} \quad \forall \mathbf{v} \in \mathbb{R}^4, \alpha \in \mathbb{R}$, leaves it unchanged (modulo the scale), that is $PT = \alpha P$. However, if the same T must leave unaltered *two* different PPMs, then it can only be $\mathbf{v} = \mathbf{0}$.

8.2 Show that if T is a scaling and P a PPM, the product PT only affects the fourth column of P.

8.3 Prove that, given any plane Π, the induced homography H_Π is always compatible.

8.4 Compare the statement of the previous problem with the observation about the degrees of freedom of H_Π in Sect. 6.5, and show that $F = [\mathbf{e}']_\times H$ if and only if $H^\mathsf{T} F + F^\mathsf{T} H = 0$.

References

S. Bougnoux. From projective to Euclidean space under any practical situation, a criticism of self-calibration. In *Proceedings of the International Conference on Computer Vision*, pages 790–796, Bombay, 1998.

Olivier Faugeras. Stratification of three-dimensional vision: projective, affine, and metric representations. *J. Opt. Soc. Am. A*, 12(3):465–484, Mar 1995.

Riccardo Gherardi and Andrea Fusiello. Practical autocalibration. In *Proceedings of the European Conference on Computer Vision*, Lecture Notes in Computer Science, pages 790–801. Springer, Berlin, 2010.

R. I. Hartley and P. Sturm. Triangulation. *Computer Vision and Image Understanding*, 68(2):146–157, November 1997.

Karl Kraus. *Photogrammetry - Geometry from Images and Laser Scans - 2nd edition*. Walter de Gruyter, Berlin, 2007.

Q.-T. Luong and T. Viéville. Canonical representations for the geometries of multiple projective views. *Computer Vision and Image Understanding*, 64(2):193–229, 1996.

Chapter 9
Non-linear Regression



9.1 Introduction

In this chapter we will see how to refine the results obtained from the linear algorithms studied so far in order to compute better estimates (in terms of precision and accuracy) in the face of input data affected by errors. We will do this at the cost of having to use non-linear minimisation iterative algorithms, such as the Levenberg-Marquardt algorithm, which involve a greater computational burden and have no guarantee of global convergence to the optimal value. Our job is to provide the cost function and its derivatives, which will be the bulk of the work done in this chapter. Dealing with derivatives forces us to be overly pedantic. However, the chapter is not essential to the economy of the text if one wishes to limit oneself to linear methods. At least the first two sections (Sects. 9.3 and 9.4) are necessary, though, if one wants to understand bundle adjustment in Chap. 14.

9.2 Algebraic Versus Geometric Distance

Consider the problem of regressing a conic on a set of points $\mathbf{x}_i$ of the plane. The conic is represented by the implicit equation: $\nu(\mathbf{x}) = a\,x^2 + 2\,b\,x\,y + 2\,c\,x + d\,y^2 + 2\,e\,y + f = 0$, where $\mathbf{x} = (x, y)$. Because $\nu(\mathbf{x})$ is a polynomial, this is a special case of an algebraic variety.[1]

The immediate approach for regressing the conic ν on the points $\mathbf{x}_i$ is based on minimising the following cost function:

[1] Set of zeros of a family of polynomials.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_9

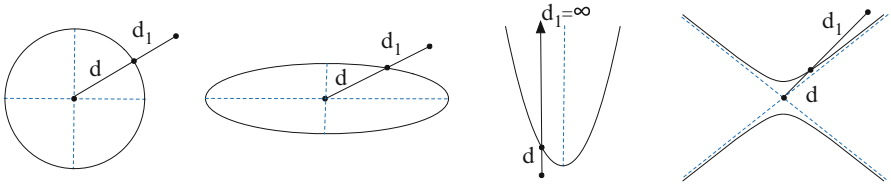


Fig. 9.1 Algebraic distance in the regression of conics (taken from Bookstein 1979)

$$\sum_{i=1}^{n} \nu(\mathbf{x}_i)^2. \tag{9.1}$$

The term $\nu(\mathbf{x}_i)$ is a polynomial in $\mathbf{x}_i$, but it is linear in the unknown parameters $[a \dots f]$, so this is a linear least-squares problem (see Sect. C.1).

If $\nu(\mathbf{x}) = 0$ also $\lambda\nu(\mathbf{x}) = 0 \; \forall\lambda \neq 0 \in \mathbb{R}$, hence ν needs to be somehow normalised, typically by setting to a constant value a function of its coefficients, for example $a + d = 1$, $f = 1$ or $\|[a \dots f]\|_2 = 1$. The latter case corresponds to the least-squares solution described in Proposition A.13.

Let us call $\hat{\nu}$ the polynomial after normalisation and define the *algebraic distance* of $\mathbf{x}_i$ from the conic $\nu(\mathbf{x}) = 0$ as $\hat{\nu}(\mathbf{x}_i)$. Different normalisations yield different algebraic distances. The essential feature of an algebraic distance is that it is computed by evaluating a normalised polynomial $\hat{\nu}(\mathbf{x}_i)$ where the normalisation does not depend on the point $\mathbf{x}_i$. An important aspect that makes algebraic distances attractive is that they always lead to a regression that is solved by eigendecomposition (or by SVD).

This approach, however, has some problems. First, not all algebraic distances that can be defined are invariant to the change of reference frame, although Bookstein (1979) proposes one that is invariant to a similitude. Second, algebraic distances yield *biased* regression models. Bookstein shows that $\nu(\mathbf{x})$ is proportional to $1 + ((d + d_1)^2/d^2)$ where d and d_1 are defined as in Fig. 9.1. This causes, for example, deviations from an ellipse to be overweighted where the curvature is low and underweighted elsewhere, with the effect of producing conics that fit points better in the low curvature sections than in the high curvature sections: this is the bias effect.

What we would like to minimise instead is the *geometric distance*, defined as the distance from point $\mathbf{x}_i$ to the conic $\nu(\mathbf{x}) = 0$, that is, the distance from point $\mathbf{x}_i$ to the closest point on the conic. This distance is invariant to the choice of the reference frame and produces unbiased regression models. The price to pay for this is that the expression of the point-conic distance is non-linear and thus one must solve a non-linear least-squares problem.

In addition, the point-conic distance is cumbersome to calculate. For this reason one must be content with surrogates, in the form of *non-algebraic* distances such as the *Sampson distance*, defined as the distance of the point $\mathbf{x}_i$ to the first-order approximation of the conic. Let us consider the Taylor expansion of ν around $\mathbf{x}_0$,

the point on the conic that realises the minimum distance from $\mathbf{x}_i$:

$$\nu(\mathbf{x}) \approx \nu(\mathbf{x}_0) + (\mathbf{x} - \mathbf{x}_0)\nabla\nu(\mathbf{x}). \tag{9.2}$$

Since $\mathbf{x}_0$ belongs to the conic: $\nu(\mathbf{x}_0) = 0$ and the normal in $\mathbf{x}_0$ points towards $\mathbf{x}_i$, that is, $\nabla\nu(\mathbf{x}_0)$ has the direction of $(\mathbf{x}_i - \mathbf{x}_0)$. Confusing $\nabla\nu(\mathbf{x}_0)$ with $\nabla\nu(\mathbf{x}_i)$ we get:

$$\nu(\mathbf{x}_i) \approx \|(\mathbf{x}_i - \mathbf{x}_0)\|\|\nabla\nu(\mathbf{x}_i)\| \tag{9.3}$$

from which

$$\|(\mathbf{x}_i - \mathbf{x}_0)\| \approx \frac{\nu(\mathbf{x}_i)}{\|\nabla\nu(\mathbf{x}_i)\|}. \tag{9.4}$$

Since the normalisation depends on the current point $\mathbf{x}_i$, this is not an algebraic distance, although it is also not geometric.

It offers the following advantages: first, the Sampson distance is computationally more tractable than the geometric distance, while being invariant for translations and rotations and varying in proportion to the geometric distance under uniform scaling. Second, it coincides with the geometric distance for flat surfaces and approximates it to the first order for non-flat surfaces (in the sense that it approximates the surface). Finally, it is invariant to scaling of ν.

In this section we used the example of conics to introduce some general concepts. In particular, the fact that the geometric distance enjoys good properties but leads to non-linear least-squares problems, while algebraic distances give rise to linear least-squares problems but yield biased (hence less accurate) regression models, is a notion that applies to all the problems we will address.

On the other hand, the fact that the geometric distance is onerous to compute is not general. It is true for conics, homographies and fundamental matrices (for which we will use the Sampson distance, indeed), while for other problems (e.g. calibration) the geometric distance is easy to compute and there is no need to resort to surrogates.

The eight-point method minimises an algebraic distance defined by the polynomial $\mathbf{m}'^{\mathsf{T}} F \mathbf{m}$ with the normalisation $\|F\|_F = 1$. It is not invariant to the reference frame, but preconditioning in practice mitigates this aspect. In analogy to the work of Bookstein (1979) on conics, Torr and Fitzgibbon (2004) suggest fixing the scale of F via the Frobenius norm of its principal submatrix consisting of its first two rows and columns, so as to obtain an algebraic error invariant to the reference frame. Be aware though that this parameterisation fails when the principal submatrix vanishes, which corresponds to parallel epipolar lines.

9.3 Non-linear Regression of the PPM

The DLT method for calibration by resection illustrated in Sect. 4.2 is simple and fast but minimises an algebraic distance (4.2), which, although being invariant to similitudes, overweights distant points and it is biased towards small focal lengths (Hartley and Zisserman 2003). To get things right we shall therefore minimise a geometric distance.

Let us write the perspective projection as a composition of transformations, in order to exploit the chain rule to compute its derivative. We begin by introducing the function δ that performs the so-called *perspective division*:

$$\delta : \mathbb{R}^3 \longrightarrow \mathbb{R}^3 \quad \delta([x, y, z]^\mathsf{T}) = \left[\frac{x}{z}, \frac{y}{z}, 1\right]^\mathsf{T}. \tag{9.5}$$

Therefore, the perspective projection writes:

$$\mathbf{m} = \delta(P\mathbf{M}) = \delta(K[R, \mathbf{t}]\mathbf{M}) = K\delta([R, \mathbf{t}]\mathbf{M}). \tag{9.6}$$

The last step is justified by the fact that K is affine and will allow us to insert the radial distortion before the transformation into pixels (Sect. 9.5.3). Finally, to step to the Cartesian coordinates, we have to remove the third component, through pre-multiplication by the matrix

$$P_2 = [I_2, \mathbf{0}] = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}, \tag{9.7}$$

thus obtaining:

$$\widetilde{\mathbf{m}} = \widetilde{\delta}(P\mathbf{M}) = P_2\delta(P\mathbf{M}) = P_2K\delta([R, \mathbf{t}]\mathbf{M}) \tag{9.8}$$

where we defined $\widetilde{\delta}$ as the composition of P_2 and δ:

$$\widetilde{\delta} : \mathbb{R}^3 \longrightarrow \mathbb{R}^2 \quad \widetilde{\delta}([x, y, z]^\mathsf{T}) = \left[\frac{x}{z}, \frac{y}{z}\right]^\mathsf{T}. \tag{9.9}$$

9.3.1 Residual

The relevant distance for this problem is the distance between the image point and the projection of the corresponding control point, also called *reprojection error*. In formula, the cost function is:

$$\chi(P) = \sum_{i=1}^{n} \| \widetilde{\delta}(P\mathbf{M}_i) - \widetilde{\mathbf{m}}_i \|^2. \tag{9.10}$$

The minimisation of this cost function corresponds to the least-squares solution of a system of $2n$ non-linear equations (two for each point) where each pair of equations is given by $\widetilde{\delta}(P\mathbf{M}_i) - \widetilde{\mathbf{m}}_i = \mathbf{0}$ (the left part of the equation is the *residual* vector).

The solution involves iterative techniques with local convergence—such as the Levenberg-Marquardt algorithm (see Sect. C.2.2)—which therefore require an initial estimate of the solution, achievable, for example, with the DLT method.

9.3.2 Parameterisation

Since P has 11 degrees of freedom, while the matrix has 12 entries, it must be appropriately parameterised with 11 parameters $\mathbf{v} = [\omega, \varphi, \kappa, t_1, t_2, t_3, \alpha_u, u_0, v_0, \tau, \theta]^\mathsf{T}$, which are the 6 extrinsic parameters and the 5 intrinsic parameters.

We will refer, nominally, to this set of five intrinsic parameters, but it is clear that any parameterisation of K (also a reduced one) will do.

9.3.3 Derivatives

The Levenberg-Marquardt algorithm needs the derivative of the residual with respect to the parameters that determine the unknowns to find its way to the solution. In this specific case the unknown is the matrix P, assigned through its parameterisation $\mathbf{v}$; therefore, the derivative that must be computed is:

$$\frac{\partial \left(\widetilde{\delta}(P\mathbf{M}_i) - \widetilde{\mathbf{m}}_i \right)}{\partial \mathbf{v}^\mathsf{T}} = \frac{\partial P_2 \delta(P\mathbf{M}_i)}{\partial \mathbf{v}^\mathsf{T}} = P_2 \frac{\partial \delta(P\mathbf{M}_i)}{\partial \mathbf{v}^\mathsf{T}} \tag{9.11}$$

This Jacobian matrix can be expanded into (we will henceforth omit the index i):

$$\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{v}^\mathsf{T}} = \underbrace{\begin{bmatrix} \frac{\partial u}{\partial \omega} & \frac{\partial u}{\partial \varphi} & \frac{\partial u}{\partial \kappa} & \frac{\partial u}{\partial t_1} & \frac{\partial u}{\partial t_2} & \frac{\partial u}{\partial t_3} \\ \frac{\partial v}{\partial \omega} & \frac{\partial v}{\partial \varphi} & \frac{\partial v}{\partial \kappa} & \frac{\partial v}{\partial t_1} & \frac{\partial v}{\partial t_2} & \frac{\partial v}{\partial t_3} \end{bmatrix}}_{\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{g}^\mathsf{T}}} \underbrace{\begin{bmatrix} \frac{\partial u}{\partial \alpha_u} & \frac{\partial u}{\partial u_0} & \frac{\partial u}{\partial v_0} & \frac{\partial u}{\partial \tau} & \frac{\partial u}{\partial \theta} \\ \frac{\partial v}{\partial \alpha_u} & \frac{\partial v}{\partial u_0} & \frac{\partial v}{\partial v_0} & \frac{\partial v}{\partial \tau} & \frac{\partial v}{\partial \theta} \end{bmatrix}}_{\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{k}^\mathsf{T}}}.$$

The Jacobian matrix of the system (9.10) is obtained by stacking n of these 2×11 matrices, one for each equation of the system.

If we consider separately the extrinsic parameters $\mathbf{g} = [\omega, \varphi, \kappa, t_1, t_2, t_3]^\mathsf{T}$ and the intrinsic ones $\mathbf{k} = [\alpha_u, u_0, v_0, \tau, \theta]^\mathsf{T}$, the Jacobian is partitioned into two blocks, corresponding respectively to intrinsic and extrinsic parameters:

$$\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{v}^\mathsf{T}} = \left[\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{g}^\mathsf{T}}, \frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{k}^\mathsf{T}} \right]. \tag{9.12}$$

We will work out these two blocks separately.

Thanks to the chain rule, the derivative with respect to the extrinsic parameters expands to:

$$\begin{aligned} \frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{g}^\mathsf{T}} &= P_2 \frac{\partial (K\delta([R,\mathbf{t}]\mathbf{M}))}{\partial \mathbf{g}^\mathsf{T}} = \\ &= P_2 K \left. \frac{\partial \delta(\mathbf{q})}{\partial \mathbf{q}^\mathsf{T}} \right|_{\mathbf{q}=[R,\mathbf{t}]\mathbf{M}} \left. \frac{\partial G\mathbf{M})}{\partial (\operatorname{vec} G)^\mathsf{T}} \right|_{G=[R,\mathbf{t}]} \frac{\partial \operatorname{vec}[R,\mathbf{t}]}{\partial \mathbf{g}^\mathsf{T}} \\ &= P_2 K \; \mathsf{D}\,\delta([R,\mathbf{t}]\mathbf{M}) \, (\mathbf{M}^\mathsf{T} \otimes I_4) \begin{bmatrix} \mathsf{D}\,R & 0 \\ 0 & I_3 \end{bmatrix} \end{aligned} \tag{9.13}$$

because

$$\frac{\partial G\mathbf{M})}{\partial (\operatorname{vec} G)^\mathsf{T}} = \mathsf{D}(G\mathbf{M}) = \mathbf{M}^\mathsf{T} \otimes I_4, \tag{9.14}$$

where

$$\frac{\partial \delta(\mathbf{q})}{\partial \mathbf{q}^\mathsf{T}} = \mathsf{D}\,\delta(\mathbf{q}) = \begin{bmatrix} \frac{1}{z} & 0 & -\frac{x}{z^2} \\ 0 & \frac{1}{z} & -\frac{y}{z^2} \\ 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{q} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \tag{9.15}$$

and $\mathsf{D}\,R$, the Jacobian of the rotation matrix with respect to the Euler angles, has been derived in Sect. B.2.

As for the intrinsic parameters, thanks to the chain rule, we have:

$$\begin{aligned} \frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{k}^\mathsf{T}} &= P_2 \frac{\partial (K\delta([R,\mathbf{t}]\mathbf{M}))}{\partial \mathbf{k}^\mathsf{T}} = \frac{\partial (K\delta([R,\mathbf{t}]\mathbf{M}))}{\partial (\operatorname{vec} K)^\mathsf{T}} \frac{\partial \operatorname{vec} K}{\partial \mathbf{k}^\mathsf{T}} = \\ &= P_2 \left(\delta([R,\mathbf{t}]\mathbf{M})^\mathsf{T} \otimes I_3 \right) \mathsf{D}\,K \end{aligned} \tag{9.16}$$

where $\mathsf{D}\,K$ is the Jacobian of the parameterisation of K and is easily computed entry-wise.

Listing 9.1 reports the implementation of non-linear regression of PPM or *non-linear calibration*. Notice how the value of initial estimate for P is used to transform the control points. In this way the Euler angles are close to zero and singularities—that occurs at $\varphi = \pm\pi/2$—are avoided.

Listing 9.1 Non-linear calibration

```
function [P_out] = resect_nonlin(P,m,M)
    %RESECT_NONLIN  Non-linear refinement of resection
    % see also BUNDLEADJ

    [K0,R0,t0] = krt(P);
    M = htx ([R0,t0; 0 0 0 1],M); % transform M to work near origin

    v_out = lsq_nonlin(@(x)fobj(x,M,m),[0;0;0;0;0;0; K2par(K0)]);

    R_out = eul(v_out(1:3));
    P_out = par2K(v_out(7:end)) * [R_out*R0, R_out*t0 + v_out(4:6)];
end


function [res,J]  = fobj(v,M,m)

    [K,DK] = par2K(v(7:end));
    [R,DR] = eul(v(1:3)); t = v(4:6);

    J = []; res = [];
    for i = 1:size(M,2)
        [r,JA, ~, JC] = reproj_res(K,[],[R,t] ,M(:,i),m(:,i));
        J = [J; JA * blkdiag(DR,eye(3)), JC * DK];
        res = [res; r];
    end
end
```

Listing 9.2 Computation of the residual of reprojection and its derivatives

```
function [res,A,B,C,D]  = reproj_res(K,kappa,G,M,m)
    %REPROJ_RES Reprojection residual and jacobian for 1 point - 1 camera
    % There are two residual (u and v)
    % G = [R,t]

    M = [M(:);1];   % make homogeneous
    q = G * M;

    % perspective division occurs here
    De = [ 1/q(3), 0, -q(1)/q(3)^2; 0, 1/q(3), -q(2)/q(3)^2];
    [qp,Dr,Dkappa] = rdx(kappa,q(1:2)./q(3),eye(3));

    res =  htx(K,qp) - m(:);

    A = K(1:2,1:2) * Dr * De * kron( M',eye(3) ); % wrt extrinsic par.
    B = K(1:2,1:2) * Dr * De * G(:,1:3);          % wrt 3D point
    C = kron([qp;1]', eye(3)); C(3,:)=[];         % wrt intrinsic par.
    D = K(1:2,1:2) * Dkappa;                      % wrt distortion par.
end
```

The computation of the reprojection residual and its derivatives prior to the parameterisation of the unknowns is implemented in Listing 9.2. Block `A` in the code implements (9.13), while block `C` implements (9.16), without the derivative of the parameterisation, that will be added by the caller.

This function will be invoked in other context, so it contains more stuff than needed in this section. In particular, block `B` will be introduced in Sect. 9.5, while the radial distortion and block `D` will be introduced in Sect. 9.5.3.

Please note that the removal of the third line for blocks `A` and `B` is obtained by using a 2×2 submatrix of K in place of K.

9.3.4 General Remarks

This first example of non-linear regression gives us an opportunity to focus on the elements that are needed in similar cases, which have been highlighted with subsections, namely:

1. Identify a significant residual to be minimised
2. Define a parameterisation of the unknowns
3. Select a minimisation scheme
4. Compute the derivative of the residual (if required).

As for point 1, the least-squares cost function is a summation:

$$\sum_{i=1}^{n} \|\mathbf{r}_i\|_2^2 \tag{9.17}$$

where $\mathbf{r}_i$ is the *residual*, in general a vector. Its norm $\|\mathbf{r}_i\|_2$ is interpreted as a *distance* (algebraic, geometric or other), so the cost is a sum of squared distances. It is evident that the cost function is equal to the sum of squares of the components of each residual, so it is equivalent to the Frobenius norm of the matrix whose columns are the residual vectors:

$$\sum_{i=1}^{n} \|\mathbf{r}_i\|_2^2 = \|[\mathbf{r}_1, \ldots, \mathbf{r}_n]\|_F^2 \tag{9.18}$$

As for point 3, we choose once and for all the Levenberg-Marquardt (LM) algorithm (Sect. C.2.3), and therefore, the derivatives (point 4) are always required.

9.4 Non-linear Regression of Exterior Orientation

Regressing the exterior orientation is similar to calibration, what changes is the fact that the intrinsic parameters are known and therefore image points are represented in normalised image coordinates $\mathbf{q}$. The cost function is:

$$\chi(P) = \sum_{i=1}^{n} \|\widetilde{\delta}([R, \mathbf{t}]\mathbf{M}_i) - \widetilde{\mathbf{q}}_i\|^2. \tag{9.19}$$

The $2n \times 6$ Jacobian matrix of the residuals corresponds to that defined in (9.13) with $K = I$ since we work in normalised coordinates.

The MATLAB implementation is shown in Listing 9.3. As in the non-linear calibration, the transformation of the control points avoids the singularities of Euler angles representation.

Listing 9.3 Non-linear regression of exterior orientation

```
function [R,t] = exterior_nonlin(R0,t0,q,M,K)
    %EXTERIOR_NONLIN Non-linear refinement of exterior orientation

    q = K\ensure_homogeneous(q);  % Homogeneous NIC

    M = htx ([R0,t0; 0 0 0 1],M); % transform M to work near origin

    g_out = lsq_nonlin(@(x)fobj(x,M,q(1:2,:)),[0;0;0;0;0;0]);

    R_out = eul(g_out(1:3));
    R =  R_out*R0;  t =  R_out*t0 + g_out(4:6);
end

function [res,J]  = fobj(g,M,q)

    [R,JR] = eul(g(1:3));  t = g(4:6);

    J =[];  res = [];
    for i = 1:size(M,2)
        [r,JA] = reproj_res(eye(3),[],[R,t], M(:,i),q(:,i));
        J = [J ; JA * blkdiag(JR,eye(3))];
        res = [res; r];
    end
end
```

9.5 Non-linear Regression of a Point in Space

The point triangulation introduced in Sect. 8.2 leads to a linear regression but minimises an algebraic distance. In the same section it has been noted that the difference between algebraic and geometric distance lies in the weights that are given to the equations, and these weights bias the algebraic solutions. In addition, the algebraic distance is not invariant to affine transformations (Hartley and Zisserman 2003), a feature that makes the method unsuitable to operate when the reference is not Euclidean.

9.5.1 Residual

Let m and m' be projections of the same 3D point M through two PPMs P and P' respectively. As in the calibration (Sect. 9.3), we consider the *reprojection error*, that is, the distance between the image point and the projection of the corresponding 3D point:

$$\chi(\widetilde{\mathbf{M}}) = \|\widetilde{\delta}(P\mathbf{M}) - \widetilde{\mathbf{m}}\|^2 + \|\widetilde{\delta}(P'\mathbf{M}) - \widetilde{\mathbf{m}}'\|^2. \tag{9.20}$$

In this case the unknowns are not the intrinsic and extrinsic parameters that determine the PPM but the coordinates of the point $\mathbf{M}$ in space.

The previous equation generalises to m images:

$$\chi(\widetilde{\mathbf{M}}) = \sum_{j=1}^{m} \|\widetilde{\delta}(P^{(j)}\mathbf{M}) - \widetilde{\mathbf{m}}^{(j)}\|^2 \tag{9.21}$$

where (j) is the image index.

This is a non-linear least-squares problem, each term is the squared distance between the given point in the j-th image and the projection of $\mathbf{M}$ through $P^{(j)}$; the residuals are the differences $\widetilde{\delta}(P^{(j)}\mathbf{M}) - \widetilde{\mathbf{m}}^{(j)}$.

Since the unknown $\mathbf{M}$ does not require any parameterisation, we proceed directly to calculate the derivatives.

9.5.2 Derivatives

Let us compute the derivative of the residual, namely (9.21), or:

$$\frac{\partial(\widetilde{\delta}(P^{(j)}\mathbf{M}) - \widetilde{\mathbf{m}}^{(j)})}{\partial\widetilde{\mathbf{M}}^\intercal} = \frac{\partial\widetilde{\delta}(P^{(j)}\mathbf{M})}{\partial\widetilde{\mathbf{M}}^\intercal} = \begin{bmatrix} \frac{\partial u_j}{\partial X} & \frac{\partial u_j}{\partial Y} & \frac{\partial u_j}{\partial Z} \\ \frac{\partial v_j}{\partial X} & \frac{\partial v_j}{\partial Y} & \frac{\partial v_j}{\partial Z} \end{bmatrix}. \tag{9.22}$$

The Jacobian matrix of the system (9.21) is obtained stacking m of these matrices 2×3, one for each image.

Let us see in detail how the derivatives are computed. By applying the chain rule (and omitting the index) we have:

$$\begin{aligned} \frac{\partial\widetilde{\delta}(P\mathbf{M})}{\partial\widetilde{\mathbf{M}}^\intercal} &= P_2 \frac{\partial(K\delta([R,\mathbf{t}]\mathbf{M}))}{\partial\widetilde{\mathbf{M}}^\intercal} = P_2 K \left.\frac{\partial\delta(\mathbf{q})}{\partial\mathbf{q}^\intercal}\right|_{\mathbf{q}=[R,\mathbf{t}]\mathbf{M}} \frac{\partial([R,\mathbf{t}]\mathbf{M})}{\partial\widetilde{\mathbf{M}}^\intercal} \\ &= P_2 K \; \mathsf{D}\,\delta([R,\mathbf{t}]\mathbf{M}) \frac{\partial(R\widetilde{\mathbf{M}} + \mathbf{t})}{\partial\widetilde{\mathbf{M}}^\intercal} = P_2 K \; \mathsf{D}\,\delta([R,\mathbf{t}]\mathbf{M})\, R \end{aligned} \tag{9.23}$$

where $\mathsf{D}\,\delta$ is given in (9.15).

The MATLAB code of the non-linear regression of one point, or *non-linear triangulation*, is reported in Listing 9.4. The Jacobian (9.23) is implemented in block `B` in Listing 9.2.

Listing 9.4 Non-linear regression of a point (triangulation)

```
function M = triang_nonlin(M0, P, m)
    %TRIANG_NONLIN Non-linear refinement of triangulation

    M = lsq_nonlin(@(x)fobj(x,P,m),M0,'GaussNewton');
end

function [res,J] = fobj(M,P,m)
    J=[];  res=[];
    for j=1:length(P)
        [K,R,t ] = krt(P{j});
        [r,~,JB] = reproj_res(K, [], [R,t], M, m{j});
        J = [J ; JB];
        res = [res; r];
    end
end
```

9.5.3 Radial Distortion

With reference to the projection model defined by (9.6), the radial distortion ρ (defined by (3.38)) is introduced before the transformation into pixels operated by K, that is:

$$\mathbf{m} = K\rho(\delta([R,\mathbf{t}]\mathbf{M})) \tag{9.24}$$

Taking this into account, it is easy to verify that the derivatives that we previously computed for the reprojection residual are modified as follows:

$$\frac{\partial(K\rho(\delta([R,\mathbf{t}]\mathbf{M})))}{\partial \mathbf{g}^\mathsf{T}} = K\ \mathsf{D}\,\rho(\hat{\mathbf{q}})\ \mathsf{D}\,\delta(\mathbf{q})(\mathbf{M}^\mathsf{T}\otimes I)\begin{bmatrix}\mathsf{D}\,R & 0\\ 0 & I_{3.}\end{bmatrix} \tag{9.25}$$

$$\frac{\partial(K\ \rho(\delta([R,\mathbf{t}]\mathbf{M})))}{\partial \widetilde{\mathbf{M}}^\mathsf{T}} = K\,\mathsf{D}\,\rho(\hat{\mathbf{q}})\ \mathsf{D}\,\delta(\mathbf{q})R \tag{9.26}$$

$$\frac{\partial(K\rho(\delta([R,\mathbf{t}]\mathbf{M})))}{\partial \mathbf{k}^\mathsf{T}} = \left(\rho(\delta([R,\mathbf{t}]\mathbf{M}))^\mathsf{T}\otimes I\right)\mathsf{D}\,K \tag{9.27}$$

where $\mathbf{q} = [R,\mathbf{t}]\mathbf{M}$ and $\hat{\mathbf{q}} = \delta([R,\mathbf{t}]\mathbf{M})$.

Finally, we have to compute the derivative of the reprojection residual with respect to the radial distortion coefficients, gathered in the vector $\boldsymbol{\kappa}$:

$$\frac{\partial(K\rho(\delta([R,\mathbf{t}]\mathbf{M})))}{\partial \boldsymbol{\kappa}^\mathsf{T}} = K\frac{\partial\rho(\delta([R,\mathbf{t}]\mathbf{M}))}{\partial \boldsymbol{\kappa}^\mathsf{T}} = K\ \frac{\partial\rho(\hat{\mathbf{q}})}{\partial \boldsymbol{\kappa}^\mathsf{T}}\bigg|_{\hat{\mathbf{q}}=\delta([R,\mathbf{t}]\mathbf{M})}. \tag{9.28}$$

The derivative of the radial distortion with respect to the parameters is easily computed component by component. This expression corresponds to block `D` in Listing 9.2.

9.6 Regression in the Joint Image Space

In the case of homography and fundamental matrix, we are in a context much more similar to the regression of conics introduced at the beginning. The analogy becomes apparent when we map the input data into a suitable *joint image space*, containing the direct sum (juxtaposition) of the input coordinates.

In fact, estimating the homography (and the fundamental matrix) between two sets of homologous points can be interpreted as a non-linear regression problem in $\mathbb{R}^4$, where we want to determine the algebraic variety (we can visualise it as a surface) that best approximates (i.e. passes as closely as possible) a set of points in $\mathbb{R}^4$. These points are given by the juxtaposition of the Cartesian coordinates of the homologous points $\mathbf{m}$ and $\mathbf{m}'$ in the two images, and the variety is defined by the implicit equation $\mathbf{m}' \times H\mathbf{m} = \mathbf{0}$ or $\mathbf{m}'^{\mathsf{T}} F\mathbf{m} = 0$. What should be minimised in this case is the square of the (geometric) distance between the points and this variety of $\mathbb{R}^4$.

This distance is onerous to compute, so we settle for an approximation of it, introduced by Sampson (1982) in the context of the regression of conics.

It is proved in Hartley and Zisserman (2003) that the Sampson distance between a point $\mathbf{x}$ and a variety of $\mathbb{R}^n$ defined by the implicit equation $\boldsymbol{\nu}(\mathbf{x}) = \mathbf{0}$ is given by

$$d = \|-(\mathsf{D}\,\boldsymbol{\nu})^{+}\boldsymbol{\nu}(\mathbf{x})\|_2 \tag{9.29}$$

where

$$\mathsf{D}\,\boldsymbol{\nu} = \frac{\partial \boldsymbol{\nu}(\mathbf{x})}{\partial \mathbf{x}^{\mathsf{T}}}. \tag{9.30}$$

and $(\mathsf{D}\,\boldsymbol{\nu})^{+}$ is the pseudoinverse of $\mathsf{D}\,\boldsymbol{\nu}$, which, if $\mathsf{D}\,\boldsymbol{\nu}$ has full row rank, is given by:

$$(\mathsf{D}\,\boldsymbol{\nu})^{+} = \mathsf{D}\,\boldsymbol{\nu}^{\mathsf{T}}(\mathsf{D}\,\boldsymbol{\nu}\mathsf{D}\,\boldsymbol{\nu}^{\mathsf{T}})^{-1} \tag{9.31}$$

The Sampson residual is the vector inside the norm:

$$\mathbf{r} = -(\mathsf{D}\,\boldsymbol{\nu})^{+}\boldsymbol{\nu}(\mathbf{x}). \tag{9.32}$$

The regression on n points thus minimises the following function cost:

$$\chi = \sum_{i=1}^{n} \|\mathbf{r}_i\|_2^2 = \sum_{i=1}^{n} \|-\left(\mathsf{D}\,\boldsymbol{\nu}|_{\mathbf{x}=\mathbf{x}_i}\right)^{+}\boldsymbol{\nu}(\mathbf{x}_i)\|_2^2. \tag{9.33}$$

This general form of the residual holds for both homographies and fundamental matrices; in the specific cases we need to instantiate $\boldsymbol{\nu}$ and its derivative $\mathsf{D}\,\boldsymbol{\nu}$, which we will do in the next two paragraphs.

9.7 Non-linear Regression of the Homography

Let us see how to use the Sampson residual in the regression of the homography.

9.7.1 Residual

In the case of homography, if $\mathbf{m}$ and $\mathbf{m}'$ are two homologous points (from now on we shall leave out the subscript i):

$$\boldsymbol{\nu}(\mathbf{x}) = \boldsymbol{\nu}([\widetilde{\mathbf{m}}; \widetilde{\mathbf{m}}']) = P_2(\mathbf{m}' \times H\mathbf{m}) \quad \text{with} \quad P_2 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \tag{9.34}$$

where the pre-multiplication by the matrix P_2 has the effect of removing the third row, since only two of the equations are independent, as we noted with regard to DLT. So in this case $\boldsymbol{\nu} : \mathbb{R}^4 \to \mathbb{R}^2$ and the algebraic variety is defined by the zeros of *two* polynomials.

As regards the computation of $\mathsf{D}\,\boldsymbol{\nu}$, we first observe that the derivative of the cross product is:

$$\frac{\partial(\mathbf{a} \times \mathbf{b})}{\partial \mathbf{b}^\mathsf{T}} = \frac{\partial([\mathbf{a}]_\times \mathbf{b})}{\partial \mathbf{b}^\mathsf{T}} = [\mathbf{a}]_\times \tag{9.35}$$

(in the derivative with respect to $\mathbf{a}$ the sign changes). So we get:

$$\frac{\partial(\mathbf{m}' \times H\mathbf{m})}{\partial \mathbf{m}^\mathsf{T}} = \left[\mathbf{m}'\right]_\times H \quad \text{and} \quad \frac{\partial(\mathbf{m}' \times H\mathbf{m})}{\partial \mathbf{m}'^\mathsf{T}} = -\left[H\mathbf{m}\right]_\times \tag{9.36}$$

and consequently:

$$\frac{\partial \boldsymbol{\nu}}{\partial \mathbf{m}^\mathsf{T}} = P_2\left(\left[\mathbf{m}'\right]_\times H\right) \quad \text{and} \quad \frac{\partial \boldsymbol{\nu}}{\partial \mathbf{m}'^\mathsf{T}} = P_2\left(-\left[H\mathbf{m}\right]_\times\right) \tag{9.37}$$

These derivatives are with respect to the homogeneous vector; to get the derivatives with respect to Cartesian coordinates alone, we must eliminate the third column, through post-multiplication by the projection matrix P_2^T, that is:

$$\mathsf{D}\,\boldsymbol{\nu} = P_2\left[\left[\mathbf{m}'\right]_\times H P_2^\mathsf{T},\ -\left[H\mathbf{m}\right]_\times P_2^\mathsf{T}\right]. \tag{9.38}$$

We observed that the DLT method for calibration and that for calculating homography are very similar. In contrast, the respective non-linear regressions minimise different errors. In other words, why is the Sampson distance not also used in the calibration? Because in the calibration, the corresponding points do not have the same nature. In particular, the control points are assumed to be known with infinite precision.

9.7.2 Parameterisation

The parameterisation of H is straightforward; it only needs to take homogeneity into account, that is, fix its scale. The simplest (and most common) choice would be to fix $H_{3,3} = 1$, but this would exclude all the homographies that have this element set to zero (these are non-affine mappings that send the origin to infinity). Note, however, that the entries of the third row cannot be all zero; otherwise, the matrix would be singular. It is therefore safe to fix the scale setting the norm of the last row of H to one, as this would exclude only singular matrices (which are not homographies).

Therefore, the third row of H is identified with a point on the unitary sphere of $\mathbb{R}^3$ parameterised with two angles ϕ_7, ϕ_8:

$$H = \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 \\ \phi_4 & \phi_5 & \phi_6 \\ \cos\phi_7 & \sin\phi_7\cos\phi_8 & \sin\phi_7\sin\phi_8 \end{bmatrix}. \tag{9.39}$$

9.7.3 Derivatives

To proceed to the solution with Gauss-Newton, we must compute the derivative of the Sampson residual:

$$\mathbf{r} = -(\mathsf{D}\,\boldsymbol{\nu})^{+}\boldsymbol{\nu} \tag{9.40}$$

with $\boldsymbol{\nu}$ given by (9.34) and $\mathsf{D}\,\boldsymbol{\nu}$ given by (9.38).

Setting $V = \mathsf{D}\,\boldsymbol{\nu}$, the above equation rewrites:

$$\mathbf{r} = -V^{+}\boldsymbol{\nu} \tag{9.41}$$

Let us apply the rule for the derivative of the product and the chain rule to obtain:

$$\begin{aligned}\frac{\partial \mathbf{r}}{\partial(\operatorname{vec} H)^{\mathsf{T}}} &= -(\boldsymbol{\nu}^{\mathsf{T}} \otimes I_4)\frac{\partial \operatorname{vec} V^{+}}{\partial(\operatorname{vec} H)^{\mathsf{T}}} - V^{+}\frac{\partial \boldsymbol{\nu}}{\partial(\operatorname{vec} H)^{\mathsf{T}}} \\ &= -(\boldsymbol{\nu}^{\mathsf{T}} \otimes I_4)\frac{\partial \operatorname{vec} V^{+}}{\partial(\operatorname{vec} V)^{\mathsf{T}}}\frac{\partial \operatorname{vec} V}{\partial(\operatorname{vec} H)^{\mathsf{T}}} - V^{+}\frac{\partial \boldsymbol{\nu}}{\partial(\operatorname{vec} H)^{\mathsf{T}}}.\end{aligned} \tag{9.42}$$

There are still three terms to be expanded in (9.42).

The first one is the derivative of the pseudoinverse that is given by (see Sect. B.1):

$$\frac{\partial \operatorname{vec} V^{+}}{\partial(\operatorname{vec} V)^{\mathsf{T}}} = -V^{+\mathsf{T}} \otimes V^{+} + \left((I_2 - VV^{+}) \otimes V^{+}V^{+\mathsf{T}} + V^{+\mathsf{T}}V^{+} \otimes (I_4 - V^{+}V)\right) K_{2,4} \tag{9.43}$$

The second one is

$$\begin{aligned}\frac{\partial \operatorname{vec} V}{\partial(\operatorname{vec} H)^{\mathsf{T}}} &= (I_4 \otimes P_2)\left[\frac{\partial([\mathbf{m}']_{\times} H P_2^{\mathsf{T}})}{\partial(\operatorname{vec} H)^{\mathsf{T}}} ; \frac{\partial(-[H\mathbf{m}]_{\times} P_2^{\mathsf{T}})}{\partial(\operatorname{vec} H)^{\mathsf{T}}}\right] \\ &= (I_4 \otimes P_2)\begin{bmatrix} P_2 \otimes [\mathbf{m}']_{\times} \\ -(P_2 \otimes I_3)\frac{\partial [\mathbf{u}]_{\times}}{\partial \mathbf{u}^{\mathsf{T}}}(\mathbf{m}^{\mathsf{T}} \otimes I_3). \end{bmatrix}\end{aligned} \tag{9.44}$$

where $\frac{\partial [\mathbf{u}]_{\times}}{\partial \mathbf{u}^{\mathsf{T}}}$ is a 9×3 matrix of zeros and ones which is easily derived element by element.

The last term is the derivative of $\boldsymbol{\nu}$, which is computed straightforwardly starting from:

$$\frac{\partial([\mathbf{m}']_{\times} H\mathbf{m})}{\partial(\operatorname{vec} H)^{\mathsf{T}}} = \mathbf{m}^{\mathsf{T}} \otimes [\mathbf{m}']_{\times}. \tag{9.45}$$

Considering that the residual is given only by the first two components of $[\mathbf{m}']_{\times} H\mathbf{m}$, we pre-multiply by the matrix P_2:

$$\frac{\partial \boldsymbol{\nu}}{\partial(\operatorname{vec} H)^{\mathsf{T}}} = P_2(\mathbf{m}^{\mathsf{T}} \otimes [\mathbf{m}']_{\times}). \tag{9.46}$$

Finally, to obtain the derivative of $\mathbf{r}$ with respect to the parameters which determine H, we must post-multiply (9.42) by the derivative of the parameterisation of H, which is computed from (9.39) element by element.

Listings 9.5, 9.6 and 9.7 implement this method.

Listing 9.5 Non-linear regression of H

```
function  H_out = hom_nonlin(H0, m2, m1)
    %HOM_NONLIN Non-linear refinement of H

    % pre-conditioning
    [T1,m1] = precond(m1); [T2,m2] = precond(m2);
    H0 = T2*H0/T1;

    h_out = lsq_nonlin(@(x)fobj(x,m1,m2),H2par(H0));
    H_out = par2H(h_out);

    % post-conditioning
    H_out = T2\H_out * T1;
end

function [res,J] =  fobj(h,m1,m2)
    % build H from parameters and return residual and its jacobian
    [H,Jp] = par2H(h);

    [res, Js] = sampson_hom(H,m1,m2);

    J = Js *Jp;
end
```

Listing 9.6 Parameterisation of H

```
function [H,J] = par2H(h)
%PAR2H Return a homography matrix and its jacobian from parameters

[v,Jv] = ang2cart(h(7:8));
H = reshape([vec(h(1:6)); v],3,[]).';
J = commutation(3,3)* [eye(6), zeros(6,2); zeros(3,6), Jv];
end
```

Listing 9.7 Sampson distance for H

```
function [rs, Js] = sampson_hom(H,m1,m2)
%SAMPSON_HOM Sampson residual for H (and its Jacobian)

m1=ensure_homogeneous(m1);
m2=ensure_homogeneous(m2);

Q = [1 0; 0 1; 0 0]; [~,DS] = skew([1 1 1]);
K24 = commutation(2,4);
rs = [];  Js = [];
for i = 1:size(m1,2)
    ra = Q'*(skew(m2(:,i))*H*m1(:,i)); % algebraic residual

    V = Q'*[skew(m2(:,i))*H*Q, -skew(H*m1(:,i))*Q];
    pV = pinv(V);  % pV = V'*inv(V*V');
    rs = [rs; - pV * ra]; % Sampson residual

    if nargout >    1  % Jacobian required
        % derivative of pinv(V)
        DY = - kron(pV',pV) + ( kron((eye(2)-V*pV), pV*pV') + ...
            kron(pV'*pV, (eye(4)-pV*V)) )* K24;
        % derivative of ra
        De = Q'*kron(m1(:,i)',skew(m2(:,i)));
        % derivative of V
        DV = kron(eye(4),Q') * [ kron(Q',skew(m2(:,i)))
            - kron(Q',eye(3)) * DS * kron(m1(:,i)', eye(3)) ];

        Js = [Js; - kron(ra',eye(4)) * DY*DV - pV*De];
    end
end
end
```

9.8 Non-linear Regression of the Fundamental Matrix

As already anticipated, the case of the fundamental matrix falls in the same class with conics and homography.

9.8.1 Residual

In the case of the fundamental matrix, ν is a scalar function $\boldsymbol{\nu} : \mathbb{R}^4 \to \mathbb{R}$

$$\nu(\mathbf{x}) = \nu([\widetilde{\mathbf{m}}; \widetilde{\mathbf{m}}']) = \mathbf{m}'^{\mathsf{T}} F \mathbf{m} \tag{9.47}$$

Therefore, $\mathsf{D}\,\nu = \nabla\nu$ is a row vector of four columns.

We shall now obtain the expression for $\nabla\nu$. Remembering that $\frac{\partial \mathbf{a}^{\mathsf{T}}\mathbf{x}}{\partial \mathbf{x}^{\mathsf{T}}} = \mathbf{a}^{\mathsf{T}}$, we see that:

$$\frac{\partial(\mathbf{m}'^{\mathsf{T}} F \mathbf{m})}{\partial \mathbf{m}^{\mathsf{T}}} = \mathbf{m}'^{\mathsf{T}} F \quad \text{e} \quad \frac{\partial(\mathbf{m}'^{\mathsf{T}} F \mathbf{m})}{\partial \mathbf{m}'^{\mathsf{T}}} = \mathbf{m}^{\mathsf{T}} F^{\mathsf{T}} \tag{9.48}$$

These derivatives are with respect to the homogeneous vector; to obtain the derivatives with respect to the Cartesian components, we need to eliminate the third component, via the projection matrix P_2^{T} introduced earlier, hence:

$$\nabla\nu = \left[\mathbf{m}'^{\mathsf{T}} F P_2^{\mathsf{T}},\ \mathbf{m}^{\mathsf{T}} F^{\mathsf{T}} P_2^{\mathsf{T}}\right] \tag{9.49}$$

The Sampson residual is then:

$$\mathbf{r} = -(\nabla\boldsymbol{\nu})^{+}\boldsymbol{\nu} = -\frac{\nabla\nu^{\mathsf{T}}}{\nabla\nu\nabla\nu^{\mathsf{T}}}\nu \tag{9.50}$$

with $\boldsymbol{\nu}$ given by (9.47) and $\nabla\boldsymbol{\nu}$ given by (9.49).

Consider the square norm of the residual, that is, the square of the Sampson distance:

$$\begin{aligned} \|\mathbf{r}\|_2^2 &= \frac{\nu(\nabla\nu\nabla\nu^{\mathsf{T}})\nu}{(\nabla\nu\nabla\nu^{\mathsf{T}})^2} = \frac{\nu^2}{\nabla\nu\nabla\nu^{\mathsf{T}}} \\ &= \frac{(\mathbf{m}'^{\mathsf{T}} F \mathbf{m})^2}{[F\mathbf{m}]_1^2 + [F\mathbf{m}]_2^2 + [F^{\mathsf{T}}\mathbf{m}']_1^2 + [F^{\mathsf{T}}\mathbf{m}']_2^2}. \end{aligned} \tag{9.51}$$

This expression highlights that the Sampson distance coincides with ν suitably scaled (as we have seen for the conics).

9.8.2 Parameterisation

We must represent F by a set of parameters equal in number to its degrees of freedom, that is, seven, so that F is singular and the scaling factor is fixed.

To fix the scale, we can arbitrarily fix the norm of a subset (possibly improper) of F entries. Note, however, that any choice excludes from parameterisation all F that have those entries null. The singularity of the matrix can be obtained by writing one row or column as a linear combination of the other two. This parameterisation does not exhaust all matrices of rank two; in fact, by choosing—for example—to write the first column of F as a linear combination of the other two, we give up representing matrices that have the second or third column null or linearly dependent on each other. To explore the pros and cons of these choices, consult (Luong and Faugeras 1996; Zhang 1998; Torr and Fitzgibbon 2004).

As for us, we will follow the approach of Bartoli and Sturm (2004), which avoids the problems just mentioned by imposing the rank and scale constraints through the SVD. The fundamental matrix is decomposed as: $F = U\,\mathrm{diag}(\sigma_1, \sigma_2, 0)V^\mathsf{T}$, where U and V are rotation matrices and σ_1, σ_2 are two positive reals. The scaling is fixed by setting the norm of the vector $[\sigma_1, \sigma_2]$ to one, that is, by parameterising its two entries as the sine and cosine of an angle. The two rotation matrices are parameterised by three Euler angles each. The careful reader might object that U and V in the SVD are only orthogonal; however, any sign change required to make their determinant positive is absorbed by the homogeneity of F.

The parameterisation of rotations with Euler angles has some singularities; to avoid them we work in a neighbourhood of the origin, making the parameterisation incremental, that is:

$$F = U_0 U(\phi_1, \phi_2, \phi_3)\,\mathrm{diag}(\cos\phi_7, \sin\phi_7, 0)\,V(\phi_4, \phi_5, \phi_6)^\mathsf{T} V_0^\mathsf{T} \tag{9.52}$$

where U_o and V_o are constant rotation matrices, obtained from a initial estimate of F such that the parameters $(\phi_1 \ldots \phi_6)$ close to zero (see Listing 9.9).

9.8.3 Derivatives

To proceed to the solution with LM, we must compute the derivative of the Sampson residual:

$$\mathbf{r} = -\frac{\nabla\nu^\mathsf{T}\nu}{\nabla\nu\nabla\nu^\mathsf{T}} \tag{9.53}$$

with $\nu = (\mathbf{m}'^\mathsf{T} F \mathbf{m})$ e $\nabla\nu = \left[\mathbf{m}'^\mathsf{T} F P_2^\mathsf{T}, \mathbf{m}^\mathsf{T} F^\mathsf{T} P_2^\mathsf{T}\right]$. The procedure is similar to the one followed for the homography, with the difference that in this case ν is a scalar and $\nabla\nu$ a row vector, which will make expressions simpler.

We define $Y = \frac{\nabla\nu^\mathsf{T}}{\nabla\nu\nabla\nu^\mathsf{T}}$ and apply the rule for the derivative of the product and the chain rule to obtain:

$$\frac{\partial \mathbf{r}}{\partial(\text{vec}\, F)^\mathsf{T}} = -\nu \frac{\partial Y}{\partial \nabla\nu} \frac{\partial \nabla\nu^\mathsf{T}}{\partial(\text{vec}\, F)^\mathsf{T}} - Y \frac{\partial \nu}{\partial(\text{vec}\, F)^\mathsf{T}}. \tag{9.54}$$

Since $\nabla\nu$ is a row vector, the expression for the derivative of Y is simple:

$$\frac{\partial Y}{\partial \nabla\nu} = \frac{1}{\nabla\nu\nabla\nu^\mathsf{T}} I_4 - \frac{2}{(\nabla\nu\nabla\nu^\mathsf{T})^2} \nabla\nu^\mathsf{T}\nabla\nu \tag{9.55}$$

The other two derivatives are:

$$\frac{\partial \nabla\nu^\mathsf{T}}{\partial(\text{vec}\, F)^\mathsf{T}} = \left[\frac{\partial(\mathbf{m}'^\mathsf{T} F P_2^\mathsf{T})}{\partial(\text{vec}\, F)^\mathsf{T}} ; \frac{\partial(\mathbf{m}^\mathsf{T} F^\mathsf{T} P_2^\mathsf{T})}{\partial(\text{vec}\, F)^\mathsf{T}} \right] = \begin{bmatrix} P_2 \otimes \mathbf{m}'^\mathsf{T} \\ (P_2 \otimes \mathbf{m}^\mathsf{T}) K_{3,3} \end{bmatrix} \tag{9.56}$$

$$\frac{\partial(\mathbf{m}'^\mathsf{T} F \mathbf{m})}{\partial(\text{vec}\, F)^\mathsf{T}} = \mathbf{m}^\mathsf{T} \otimes \mathbf{m}'^\mathsf{T}. \tag{9.57}$$

To obtain the derivative of $\mathbf{r}$ with respect to the parameters determining F, it is necessary finally to post-multiply (9.54) by the derivative of the parameterisation of F given by (9.52). It is easy to derive that:

$$\frac{\partial\, \text{vec}\, F}{\partial[\phi_1 \ldots \phi_7]} = (V_o \otimes U_0) \left[\frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial[\phi_1, \phi_2, \phi_3]}, \frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial[\phi_4, \phi_5, \phi_6]}, \frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial \phi_7} \right], \tag{9.58}$$

with

$$\frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial[\phi_1, \phi_2, \phi_3]} = (V D \otimes I_3)\, \mathsf{D}\, U \tag{9.59}$$

$$\frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial[\phi_4, \phi_5, \phi_6]} = (I_3 \otimes U D) K_3\, \mathsf{D}\, V \tag{9.60}$$

$$\frac{\partial\, \text{vec}(U D V^\mathsf{T})}{\partial \phi_7} = (V \otimes U) \frac{\partial\, \text{vec}\, D}{\partial \phi_7}. \tag{9.61}$$

Since $D = \text{diag}(\cos\phi_7, \sin\phi_7, 0)$, the derivative of D is straightforward.

Listings 9.8, 9.9 and 9.10 implement the method.

Listing 9.8 Non-linear regression of F

```
function  F_out = fund_nonlin(F0, m2, m1)
    %FUND_NONLIN Non-linear refinement of fundamental matrix

    % pre-conditioning
    [T1,m1] = precond(m1); [T2,m2] = precond(m2);
    F0 = T2'\(F0/T1);

    % the parametrization is local around U0*V0',
    % so we have to carry them over
    [f0,U0,V0] = F2par(F0);
    f_out = lsq_nonlin(@(x)fobj(x,m1,m2,U0,V0),f0);
    F_out = par2F(f_out,U0,V0);

    %post-conditioning
    F_out = T2' * F_out * T1;

end

function [r,J] =  fobj(f,m1,m2,U0,V0)
    % build F from parameters and return residual and its jacobian
    [F,Jp] = par2F(f,U0,V0);
    [r,Js] = sampson_fund(F,m1,m2);
    J = Js * Jp;
end
```

Listing 9.9 Parameterisation of F

```
function [F,J] = par2F(f,U0,V0)
    %PAR2F Return a fundamental matrix and its jacobian from parameters

    % the parametrization is local around U0*V0'

    [U,JU] = eul(f(1:3));
    [V,JV] = eul(f(4:6));
    [s,Js] = ang2cart(f(7));
    D = diag([s;0]);
    % this 0-1 matrix is the jacobian of the diag([s;0]) operator
    JD = [ 1 0 0 0 0 0 0 0 0
           0 0 0 0 1 0 0 0 0]' *Js;

    F = U0*(U*D*V')*V0';

    K3 = commutation(3,3);
    J = kron(V0,U0)*[kron(V*D,eye(3))*JU, ...
        kron(eye(3),U*D)*K3*JV, kron(V,U)*JD ];
end
```

Preconditioning, in this case, does not serve to mitigate the malconditioning problem associated with the algebraic distance since we are using Sampson's distance, but it does however make the problem better numerically conditioned.

9.9 Non-linear Regression of Relative Orientation

The regression of the fundamental matrix is easily adapted to the essential matrix, one only has to replace the pixel coordinates with the normalised image coordinates and F with E.

Listing 9.10 Sampson distance for F

```
function  [rs, Js] = sampson_fund(F,m1,m2)
%SAMPSON_FUND Sampson residual for F (and its Jacobian)

m1=ensure_homogeneous(m1);
m2=ensure_homogeneous(m2);

K3 = commutation(3,3); Q = [1 0; 0 1; 0 0];
rs = [];  Js = [];
for i = 1:size(m1,2)
    ra = m2(:,i)'*F*m1(:,i); % algebraic residual

    v = [ m2(:,i)'*F*Q , m1(:,i)'*F'*Q ];
    pv =  v'/(v*v');
    rs = [rs; -pv * ra];   % Sampson residual

    if nargout > 1 % Jacobian required
        %  derivative of J'*(inv(J*J'))
        DY = eye(4)/(v*v')  - 2*(v'*v)/(v*v')^2;
        % derivative of ra
        De =  kron(m1(:,i)', m2(:,i)');
        % derivative of v
        Dv = [kron(Q', m2(:,i)'); kron(Q', m1(:,i)')*K3];

        Js =  [Js;  - ra*DY*Dv - pv*De];
    end
end
end
```

9.9.1 Parameterisation

The essential matrix has five degrees of freedom and is factored into $E = \left[\hat{\mathbf{t}}\right]_\times R$. Therefore, it can be parameterised with three Euler angles ω, φ, κ for the rotation R and two angles ϕ_1 e ϕ_2 which identify $\hat{\mathbf{t}}$ as a point on the unit sphere of $\mathbb{R}^3$:

$$\hat{\mathbf{t}} = \hat{\mathbf{t}}(\phi_1, \phi_2) = [\cos\phi_1, \sin\phi_1 \cos\phi_2, \sin\phi_1 \sin\phi_2,]^\mathsf{T}. \tag{9.62}$$

9.9.2 Derivatives

In this section we will only have to provide the derivative of the parameterisation, while the rest is as in the case of F (Sect. 9.8). Thanks to the rule for the derivative of the product:

$$\begin{aligned}\frac{\partial \operatorname{vec} E}{\partial[\omega, \varphi, \kappa, \phi_1, \phi_2]} &= \frac{\partial \operatorname{vec}\left(\left[\hat{\mathbf{t}}\right]_\times R\right)}{\partial[\omega, \varphi, \kappa, \phi_1, \phi_2]} \\ &= \left(I_3 \otimes \left[\hat{\mathbf{t}}\right]_\times\right) \frac{\partial \operatorname{vec} R}{\partial[\omega, \varphi, \kappa, \phi_1, \phi_2]} + (R^\mathsf{T} \otimes I_3) \frac{\partial \operatorname{vec}\left[\hat{\mathbf{t}}\right]_\times}{\partial[\omega, \varphi, \kappa, \phi_1, \phi_2]}\end{aligned} \tag{9.63}$$

Because the first term depends only on the first three parameters and the second term depends only on the last two, their respective derivatives have blocks of zeros in complementary positions, so the sum of two terms gives rise to a partitioned matrix:

$$\frac{\partial \operatorname{vec} E}{\partial[\omega, \varphi, \kappa, \phi_1, \phi_2]} = \left[\left(I_3 \otimes \left[\hat{\mathbf{t}}\right]_\times \right) \frac{\partial \operatorname{vec} R}{\partial[\omega, \varphi,]} \,,\, (R^\mathsf{T} \otimes I_3) \frac{\partial \operatorname{vec} \left[\hat{\mathbf{t}}\right]_\times}{\partial[\phi_1, \phi_2]} \right] \tag{9.64}$$

The derivative of the rotation was obtained in Sect. B.2, while as regards the right block, we observe that, by the chain rule, the derivative is:

$$\frac{\partial \operatorname{vec} \left[\hat{\mathbf{t}}\right]_\times}{\partial[\phi_1, \phi_2]} = \left. \frac{\partial \operatorname{vec} [\mathbf{u}]_\times}{\partial \mathbf{u}} \right|_{\mathbf{u}=\hat{\mathbf{t}}} \frac{\partial \hat{\mathbf{t}}}{\partial[\phi_1, \phi_2]}. \tag{9.65}$$

The derivative of the vector $\hat{\mathbf{t}}$ with respect to its parameters is computed easily:

$$\frac{\partial \hat{\mathbf{t}}}{\partial[\phi_1, \phi_2]} = \begin{bmatrix} -\sin\phi_1 & 0 \\ \cos\phi_1 \cos\phi_2 & -\sin\phi_1 \sin\phi_2 \\ \cos\phi_1 \sin\phi_2 & \sin\phi_1 \cos\phi_2 \end{bmatrix}. \tag{9.66}$$

as well as the Jacobian of the constructor of the antisymmetric matrix (already encountered in the regression of H).

Listing 9.11 reports the MATLAB implementation of the algorithm. Also note in this case the transformation that brings Euler angles near zero (followed by the anti-transformation of the result). The parameterisation is implemented in Listing 9.12.

Listing 9.11 Non-linear regression of relative orientation

```
function   [R,t]  = relative_nonlin(R0, t0, q2, q1, K2, K1)
    %RELATIVE_NONLIN Non-linear refinement of relative orientation

    q1 = K1\ensure_homogeneous(q1);
    q2 = K2\ensure_homogeneous(q2);
    % now q1 and q2 are homogeneous NIC

    q1 = R0*q1; % tranform q1 to work near origin

    e_out = lsq_nonlin(@(x)fobj(x,q1,q2),[0;0;0;cart2ang(t0)]);

    R = eul(e_out(1:3))*R0;  t = ang2cart(e_out(4:5));
end

function [res,J] =  fobj(e,q1,q2)
    % build F from parameters and return residual and its jacobian
    [E,Jp] = par2E(e);

    [res,Js] = sampson_fund(E,q1,q2);

    J = Js *Jp;
end
```

Listing 9.12 Parameterisation of E

```
function [E,J] = par2E(g)
    %PAR2E Return an essential matrix and its jacobian from parameters

    % build E from parameters (5 angles)
    [R,DR] = eul(g(1:3));
    [t,Dt] = ang2cart(g(4:5));
    [S,DS] = skew(t);
    E = S*R;

    J = [kron(eye(3),S) * DR , kron(R',eye(3)) * DS * Dt];
end
```

In light of this parameterisation of E, this method can also be interpreted as the non-linear regression of relative orientation.

In the functions for calibration, exterior orientation and relative orientation, we avoided the singularities of the Euler representation by means of a transformation of the *points* leading to work close to the origin, whereas with F we adopted an incremental parameterisation. How come? They are essentially the same thing. The approach followed for F is more general, but the implementation is less clean because of the static part of the parameterisation that has to be carried along (U_0 and V_0 in the specific case). Point transformation has a more straightforward implementation, but it is not always viable. It can be done when the transformation is rigid, for other transformations would distort the noise statistics on the image points, and when only one rotation is involved (e.g. it cannot be done in Bundle Adjustment (BA)).

9.10 Robust Regression

In conclusion, let us briefly touch upon the problem of anomalous samples, or *outliers*. The methods of least-squares regression, either linear or non-linear, assume that all point correspondences are inherently correct and that the measurements are affected by a certain random error distributed as a Gaussian around the true value. If even one of these matches is wrong, the estimate can be arbitrarily skewed, to the point of being useless. These mismatches are *outliers*, for the purposes of the regression problem.

In practice, we have to assume that data coming from automatic matching *will* be corrupted by outliers; therefore, a *robust* regression method is needed to deal with these outliers. This topic is dealt with in more details in Sect. C.3

Let us take as an example the regression of the fundamental matrix. Such example is of considerable importance, for computing F serves to validate a set of homologous points with respect to epipolar geometry even in situations in which the fundamental matrix is not of interest per se.

A more stringent validation of matches can be obtained by imposing the so-called trifocal constraint (Hartley 1997) on image triples.

Listing 9.13 Robust estimate of F

```
function   [F, inliers] = fund_rob(m2,m1,varargin)
    %FUND_ROB  Robust fundamental matrix

    s = 8; % MSS size

    sampson_d = @(F,x)...
    sqrt(sum(reshape(sampson_fund(F,x(1:2,:),x(3:4,:)),4,[]).^2))';

    [~,inliers] = robust_fit([m1;m2],@(x,w)...
        fund_lin(x(3:4,:),x(1:2,:),w),sampson_d,s,varargin{:});

    if sum(inliers) < s
        warning('Fund_rob: not enough inliers');
    end

    % final linear fit
    F = fund_lin(m2(:,inliers), m1(:,inliers));
end
```

Listing 9.13 reports a function for the robust estimation of the fundamental matrix with three different methods described in Sect. C.3. In the case of Random Sample Consensus (RANSAC) (or M-estimator SAmple Consensus MSAC), in each iteration a test fundamental matrix is calculated on a random sample of seven matches (or eight if we use the linear algorithm). Consensus is calculated by counting how many of the remaining corresponding points are reciprocally on the epipolar line determined by the test fundamental matrix, with a certain tolerance in Sampson distance. When finished, the solution with the greatest consensus is selected and matches outside the consensus set are discarded (Fig. 9.2).

RANSAC can be applied likewise to homographies, as reported in Listing 9.14.

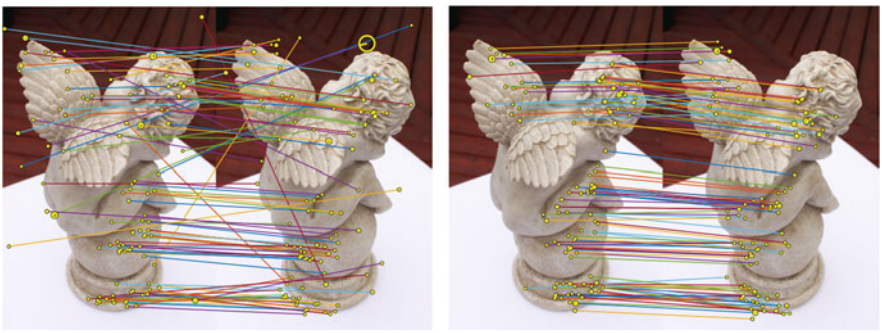


Fig. 9.2 Scale Invariant Feature Transform SIFT point correspondences before (right) and after (left) validation of epipolar geometry through RANSAC

Listing 9.14 Robust estimate of H

```
function   [H, inliers] = hom_rob(m2,m1,varargin)
    %HOM_ROB  Robust homography fit

    s = 4; % MSS size

    sampson_d = @(H,x)sqrt(sum(reshape(sampson_hom(H,x(1:2,:),x(3:4,:)),4,[]).^2))
        ';

    [~,inliers] = robust_fit([m1;m2],@(x,w)hom_lin(x(3:4,:),x(1:2,:),w),...
        sampson_d, s, varargin{:});

    if sum(inliers) < s
        warning('Hom_rob: not enough inliers');
    end

    % final linear fit
    H = hom_lin(m2(:,inliers), m1(:,inliers));
end
```

Note that in this context, methods that use the minimum number of points (seven for F, five for E, four for H, three for exterior orientation) are important to reduce the computational burden. Indeed, the number of iterations k required to achieve a confidence α given the size of the test sample p and the fraction of outliers ϵ is given by:

$$k = \frac{\log(1-\alpha)}{\log(1-(1-\epsilon)^p)}. \tag{9.67}$$

While in the case of homographies the minimal solver is linear, in the other cases it entails finding roots of polynomials. Our implementation of the robust estimate of F uses the linear algorithm for simplicity.

In summary, errors and rogue measurements must be taken both into account, using a *robust* procedure (e.g. RANSAC) to discard outliers, followed by a non-linear least-squares regression on clean data to obtain the optimal estimate.

Finally, beware that errors that inevitably affect measures propagate to the regression results, which should therefore always be accompanied by an indication of the error, usually in the form of standard deviation. Section C.4 deals with this topic.

Problems

9.1 Verify that the conic equation can be written as a quadratic form with the matrix

$$C = \begin{bmatrix} a & b & c \\ b & d & e \\ c & e & f \end{bmatrix}$$

in homogeneous coordinates.

9.2 Write a conic regression algorithm that minimises an algebraic distance.

9.3 Derive the Sampson distance for the conic and write an algorithm that minimises the latter using the Gauss-Newton method.

References

A. Bartoli and P. Sturm. Nonlinear estimation of the fundamental matrix with minimal parameters. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(3):426–432, March 2004.

Fred L. Bookstein. Fitting conic sections to scattered data. *Computer Graphics and Image Processing*, 9(1):56–71, 1979.

R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2nd edition, 2003.

R.I. Hartley. Lines and Points in Three Views and the Trifocal Tensor. *International Journal of Computer Vision*, 22(2):125–140, 1997.

Q.-T. Luong and O. D. Faugeras. The fundamental matrix: Theory, algorithms, and stability analysis. *International Journal of Computer Vision*, 17:43–75, 1996.

Paul Sampson. Sampson, p.d.: Fitting conic sections to "very scattered" data: An iterative refinement of the Bookstein algorithm. comput. graphics image process. 18, 97-108. *Computer Graphics and Image Processing*, 18:97–108, 01 1982.

P. H. S. Torr and A. W. Fitzgibbon. Invariant fitting of two view geometry. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(5):648–650, May 2004.

Z. Zhang. Determining the epipolar geometry and its uncertainty: A review. *International Journal of Computer Vision*, 27(2):161–195, March/April 1998.

Chapter 10
Stereopsis: Geometry



10.1 Introduction

Traditionally, the term *stereopsis* (from *sterèos* = solid and *opsis* = sight) denotes a human perceptual capacity; Poggio and Poggio (1984) give the following definition:

> What is stereopsis? Because our two eyes are located in different positions in the head, their views of a 3-D scene are slightly disparate. One can easily experience directly this binocular disparity by looking at objects not too distant and noting their different relative positions when closing each eye in turn. The disparity of each "point" depends on its distance from the fixation point of the two eyes. Our brain is capable of measuring this disparity and using it to produce the sensation of depth that is the subjective estimate of relative distance. This is stereopsis and its sole basis is the horizontal disparity between the two retinal images.

Along the lines of this definition, we can say that *computational* stereopsis is the process of obtaining information about the solid structure of a scene from two images taken from two different viewpoints.

We have already addressed the problem of reconstruction from two views in Chap. 8: given the homologous points of the two images and the two PPMs, it is possible to reconstruct by *triangulation* the position in space of the tie points, thus obtaining a solid model consisting of the tie points. In this chapter we tackle the same problem but under a special epipolar geometry, called the *normal configuration*. The substance of the problem does not change, but the normal case allows for the introduction of ad hoc techniques, which opens the way to recovering a denser model, by ideally matching all the pixels.

In the *normal configuration* the homologous points lie on horizontal lines at the same height, and the correspondences can be encoded by the binocular *disparity*, that is, the difference of their respective horizontal coordinates. Searching for homologous points—or calculating disparity, in this context—is straightforward, and triangulation also takes a special form (Sect. 10.2). Given the advantages associated with the normal case, we will introduce in Sect. 10.3 the *epipolar*

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_10

rectification process that virtually brings the cameras into this configuration even where they are not in reality.

We shall assume, for now, that the disparity is known, and we will see in Chap. 12 how to compute it.

10.2 Triangulation in the Normal Case

In the so-called *normal configuration* we consider two identical cameras in which the second is related to the first by a translation along the X-axis (conventionally, the horizontal axis). It is easy to verify that, as a consequence, disparity is purely horizontal and therefore the two-dimensional construction of Fig. 10.1 is justified.

Let b be the distance between the COPs, called *baseline*. Arbitrarily fixed the world reference frame coincident with that of the first camera, and working in normalised image coordinates, we can write the following two PPMs:

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad P' = \begin{bmatrix} 1 & 0 & 0 & b \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tag{10.1}$$

From which we get:

$$\begin{cases} \dfrac{1}{Z} = \dfrac{x}{X} \\ \dfrac{1}{Z} = \dfrac{x'}{X + b} \end{cases} \tag{10.2}$$

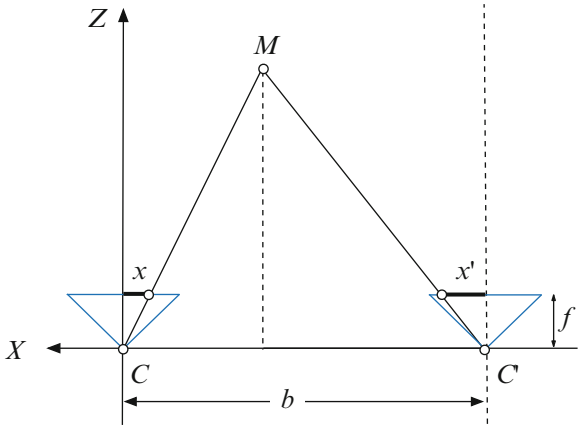


Fig. 10.1 Stereoscopic triangulation in the normal case. Equation (10.3) can also be derived immediately from the similitude of the triangle having for base b and height Z and the one, split into two parts, having for base $|x'| + |x|$ and height $f = 1$

and after processing:

$$Z = \frac{b}{x' - x}. \tag{10.3}$$

Equation (10.3) shows that it is possible to derive the third coordinate Z once the binocular disparity is known ($d_x = x' - x$). It can also be seen that the baseline b behaves like a scaling factor: the disparity associated with a fixed scene point depends directly on b. If parameter b is unknown, it is possible to reconstruct the three-dimensional model *up to a scaling factor*.

Let us see how to translate what we just saw into matrix form. If we add a third component equal to the disparity d_x to the image coordinates, we can modify (10.1) into:

$$\begin{bmatrix} x \\ y \\ d_x \\ 1 \end{bmatrix} \simeq \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & b \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{Q} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}. \tag{10.4}$$

This 4×4 matrix Q differs from P by one row and can be inverted to derive the 3D coordinates. In essence, if we augment the image coordinates with disparity, we can derive the 3D coordinates with a projective transformation. It is easily verified that the expression for the Z component is (10.3).

> This is closely reminiscent of the way projection is done in computer graphics, where instead of a true projection, which irreversibly erases the depth information, an invertible transformation is applied in 3D space that produces a vector whose first two components are the image coordinates and the third is a monotonic function of the depth (called *pseudo-depth*), which is thus preserved for further processing.

If we want to make the situation more realistic, we need to take into account the intrinsic parameters of the two cameras. We relax the assumption that the two cameras are identical and allow for a different horizontal coordinate of the principal point, u_0 and u'_0, respectively (the reason will be made clear in Sect. 10.3).

The disparity in image space (measured in pixels) is derived as:

$$d_u = u' - u = (u'_0 + \alpha_u x') - (u_0 + \alpha_u x) = (u'_0 - u_0) - \alpha_u \underbrace{(x' - x)}_{d_x} \tag{10.5}$$

then the matrix that accounts for (10.5) is the following:

$$\begin{bmatrix} u \\ v \\ d_u \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha_u & 0 & 0 & u_0 \\ 0 & \alpha_v & 0 & v_0 \\ 0 & 0 & -\alpha_u & (u'_0 - u_0) \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ d_x \\ 1 \end{bmatrix}. \tag{10.6}$$

Combining it with Q yields:

$$\begin{bmatrix} u \\ v \\ d_u \\ 1 \end{bmatrix} \simeq \underbrace{\begin{bmatrix} \alpha_u & 0 & u_0 & 0 \\ 0 & \alpha_v & v_0 & 0 \\ 0 & 0 & (u'_0 - u_0) & \alpha_u b \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{T} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \tag{10.7}$$

And finally, taking the inverse:

$$\begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \simeq \underbrace{\begin{bmatrix} \frac{1}{\alpha_u} & 0 & 0 & -\frac{u_0}{\alpha_u} \\ 0 & \frac{1}{\alpha_v} & 0 & -\frac{v_0}{\alpha_v} \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{1}{\alpha_u b} & -\frac{u'_0 - u_0}{\alpha_u b} \end{bmatrix}}_{T^{-1}} \begin{bmatrix} u \\ v \\ d_u \\ 1 \end{bmatrix}. \tag{10.8}$$

Considering the third component we arrive at the expression:

$$Z = \frac{\alpha_u\, b}{d_u - (u'_0 - u_0)}. \tag{10.9}$$

Listing 10.1 implements the procedure just described, and Fig. 10.2 shows a sample result.

Listing 10.1 Triangulation from disparity

```
function M  = triang_disp( u,v, d,  P1, P2 )
    %TRIANG_DISP Triangulation from disparity

    [K1,R1,t1] = krt(P1); [K2,R2,t2] = krt(P2);

    % baseline
    c1 = - R1'*t1; c2 = - R2'*t2;
    t = c1(1)-c2(1);

    T = [ 1/K1(1,1), 0,       0,          -K1(1,3)/K1(1,1)
        0,        1/K1(2,2),  0,          -K1(2,3)/K1(2,2)
        0,            0,      0,                1
        0,            0,  1/(K1(1,1)*t), (K1(1,3)-K2(1,3))/(K1(1,1)*t)];


    M = htx(T, [u(:)';v(:)';d(:)']) ;
end
```

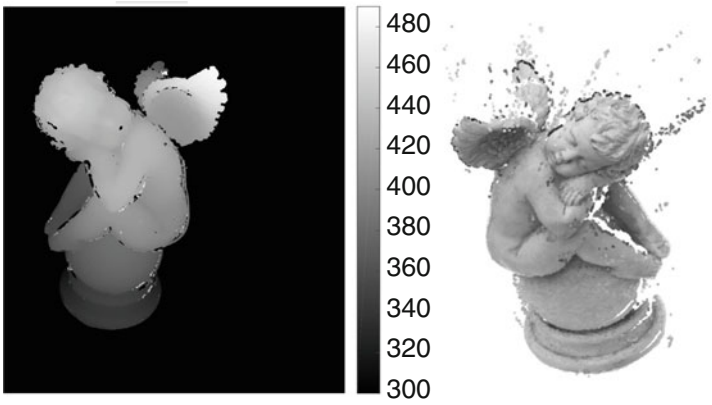


Fig. 10.2 Disparity map and corresponding 3D model

The normal configuration is obtained in practice after the rectification process (Sect. 10.3); the matrix T^{-1} can then be applied directly to the disparity referred to the rectified pair to obtain the 3D coordinates of the points.

10.3 Epipolar Rectification

The *epipolar rectification* consists in transforming a pair of generic stereoscopic images into a pair in *normal* configuration (Sect. 10.2) while keeping the two COPs unchanged.

In this configuration, image planes are parallel to the baseline and the epipolar lines are parallel and horizontal. This is a particularly favourable situation for calculating disparity, since the homologous points lie on the same image line.

We will describe a method that applies to calibrated images (PPMs are known) introduced by Fusiello et al. (2000) and one that applies to uncalibrated images (PPM unknown) proposed by Fusiello and Irsara (2011).

10.3.1 Calibrated Rectification

Let P_o and P'_o be the two PPMs of the two cameras. The idea behind the rectification is to define two new PPMs P_n and P'_n obtained by rotating the original cameras around their COPs until the focal planes become coplanar (and in doing so they

will both contain the baseline). This implies that the epipoles are at infinity, so the epipolar lines are *parallel*. To have *horizontal* epipolar lines, the baseline must be parallel to the new X-axis of both cameras. In addition, we want to ensure a stronger rectification property by requiring that the homologous points have *the same vertical coordinates*. This is achieved by forcing the two new cameras to have the same intrinsic parameters.[1] It can be seen that since the focal length is the same, the two rectified image planes are coincident, as shown in Fig. 10.3.

To summarise: the positions (i.e. the COPs) of the new PPMs are the same as those of the old cameras, while the new angular attitude (the same for both cameras) must be defined ad hoc; the intrinsic parameters are the same for both cameras.[1] Thus, the two resulting PPMs can be thought of as a single camera translated along the X-axis of its reference frame as in the normal case.

Let us write the new PPMs in terms of the COPs and the rotation:

$$P_n = K_n[R_n, -R_n\,\widetilde{\mathbf{C}}], \quad P'_n = K_n[R_n, -R_n\,\widetilde{\mathbf{C}}']. \tag{10.10}$$

The intrinsic parameters matrix K is the same for both PPMs and can be fixed arbitrarily. The COPs $\widetilde{\mathbf{C}}$ and $\widetilde{\mathbf{C}}'$ are given by the old COPs computed with (3.29). The matrix R_n, which determines the camera angular attitude, is the same for both PPMs. Let us write it with its rows:

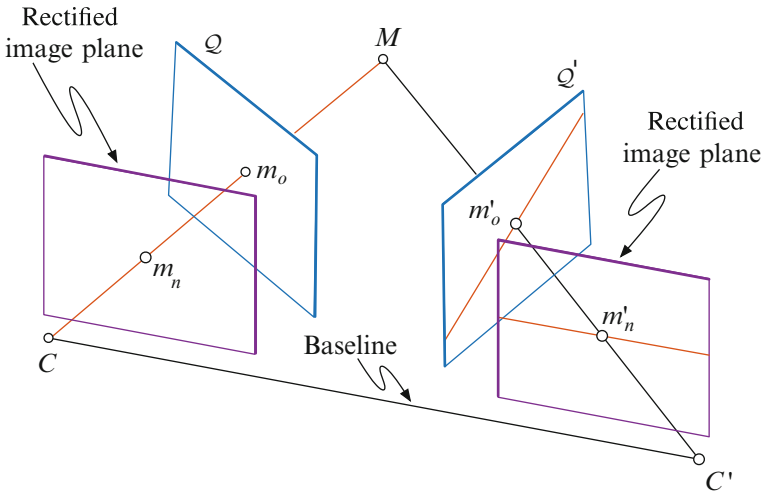


Fig. 10.3 After rectification, the image planes are coincident and parallel to the baseline

[1] Actually the horizontal coordinate of the principal point, u_0, can be different in the two images.

$$R_n = \begin{bmatrix} \mathbf{r}_1^\top \\ \mathbf{r}_2^\top \\ \mathbf{r}_3^\top \end{bmatrix} \tag{10.11}$$

we have that $\mathbf{r}_1^\top$, $\mathbf{r}_2^\top$, $\mathbf{r}_3^\top$ are, respectively, the X, Y and Z axes of the camera reference frame, expressed in world coordinates. In accordance with the above, we determine R_n by setting:

1. the new X-axis parallel to the baseline:

$$\mathbf{r}_1 = (\widetilde{\mathbf{C}}' - \widetilde{\mathbf{C}})/\|\widetilde{\mathbf{C}}' - \widetilde{\mathbf{C}}\|;$$

2. the new Y-axis orthogonal to X and an arbitrary versor $\mathbf{k}$:

$$\mathbf{r}_2 = (\mathbf{k} \times \mathbf{r}_1)/\|\mathbf{k} \times \mathbf{r}_1\|;$$

3. the new Z-axis orthogonal to XY (necessarily):

$$\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2.$$

At point 2, $\mathbf{k}$ fixes the position of the new Y-axis in the plane orthogonal to X (the vertical direction). We take it equal to the Z versor of the old camera, thus forcing the new Y-axis to be orthogonal to both the new X-axis and the old Z-axis.

This algorithm fails when the optical axis is parallel to the baseline, that is, when there is a purely forward motion.

The MATLAB implementation is given in Listing 10.2.

To rectify the image produced by camera P_o (e.g.), we must compute the transformation that brings the image plane of P_o into the image plane of P_n (for P_o' we proceed likewise).

For any 3D point $\mathbf{M}$ we can write:

$$\begin{cases} \mathbf{m}_o \simeq P_o \mathbf{M} \\ \mathbf{m}_n \simeq P_n \mathbf{M} \end{cases} \tag{10.12}$$

In agreement with (3.32), the optical ray equations are as follows (since rectification does not displace the COP):

$$\begin{cases} \widetilde{\mathbf{M}} = \widetilde{\mathbf{C}} + \lambda_o \widetilde{P}_o^{-1} \mathbf{m}_o \quad \lambda_o \in \mathbb{R} \\ \widetilde{\mathbf{M}} = \widetilde{\mathbf{C}} + \lambda_n \widetilde{P}_n^{-1} \mathbf{m}_n \quad \lambda_n \in \mathbb{R} \end{cases} \tag{10.13}$$

therefore:

$$\mathbf{m}_n \simeq \widetilde{P}_n \widetilde{P}_o^{-1} \mathbf{m}_o. \tag{10.14}$$

Listing 10.2 Calibrated epipolar rectification

```
function [T1,T2,Pn1,Pn2] = rectifyP(P1,P2)
% RECTIFYP Epipolar rectification (calibrated)

    % factorise old PPM
    [K1,R1,t1] = krt(P1); [K2,R2,t2] = krt(P2);

    % optical centers (unchanged)
    c1 = - R1'*t1;
    c2 = - R2'*t2;

    % new x axis is the baseline
    v1 = (c2-c1) * sign(R1(1,:)*(c2-c1));

    % new y axes (orthogonal to old z and new x)
    v2 = cross(R1(3,:)',v1);
    % new z axes (no choice)
    v3 = cross(v1,v2);

    % rotation matrix (normalize rows)
    R = [v1(:)/norm(v1), v2(:)/norm(v2), v3(:)/norm(v3)]';

    % new intrinsic (arbitrary)
    Kn1 = K2; Kn1(1,2)=0; Kn2 = Kn1;

    % new projection matrices
    Pn1 = Kn1*[R -R*c1];  Pn2 = Kn2*[R -R*c2];

    % rectifying image transformation
    T1 = Pn1(1:3,1:3)/(P1(1:3,1:3)); T2 = Pn2(1:3,1:3)/(P2(1:3,1:3));
```

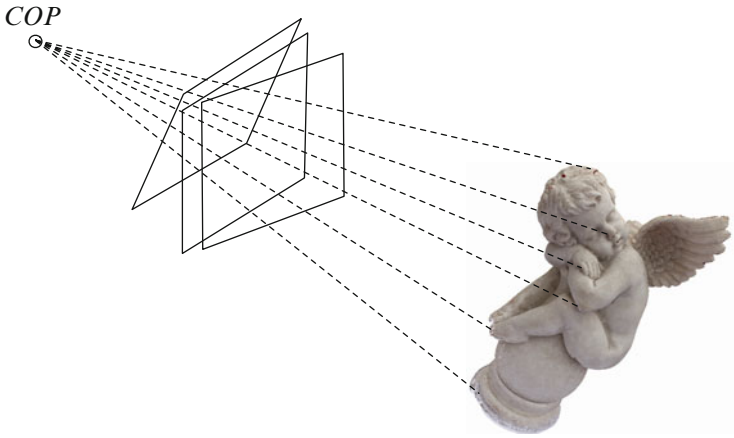


Fig. 10.4 An image is the intersection of a plane with the projective bundle

The sought transformation is the *homography* defined by the 3×3 matrix: $H = \widetilde{P}_n \widetilde{P}_o^{-1}$.

Recalling that $\widetilde{P}_o = K_o R_o$ and $\widetilde{P}_n = K_n R_n$, we have that

$$H = K_n R_n R_o^{\mathsf{T}} K_o^{-1}. \tag{10.15}$$

So the homography that, applied to the original image, produces the rectified image has the form of the homography of the plane at infinity associated with the rotation

$R_n R_o^\top$. The latter is precisely the rotation that must be applied to the camera to bring it into the normal configuration.

To understand rectification it is useful to think of an image as the intersection of the image plane with the projective bundle, consisting of the rays passing through the COP and through each visible point in the scene (Fig. 10.4). By rotating the image and holding the projective bundle steady, we will get a new image of the same visible points.

The rectified images together with their respective PPMs P_n, P'_n can be used in all circumstances in place of the original images and PPMs.

In conclusion, let us address the problem of the size of the rectified images. The pixel coordinates of the rectified images can, in principle, vary in regions of the plane very different from the window occupied by the original image. In the example in Fig. 10.5 (top) the origin of the coordinates has been kept in $(0, 0)$, and the size of the images has been increased to fit the content. However, the part corresponding to the negative coordinates is cut off and there are empty areas on the sides that unnecessarily take up space. The first problem is solved by applying a vertical translation (change of v_o) to the coordinates of both images so that they become positive; the second problem is addressed by applying two different horizontal translations by changing u_o independently in the two images, since this do not affect the rectification. The result is shown in Fig. 10.5 (bottom).

10.3.2 Uncalibrated Rectification

The previous method relies on the knowledge of the PPM, which is why it is called "calibrated". When this assumption is not verified, we can assume that only homologous points in the two images are available, which allows to compute the fundamental matrix F.

It is easy to verify (Problem 10.3) that the fundamental matrix of a pair of cameras in normal configuration is the antisymmetric matrix associated with the versor $\mathbf{e}_1 = [1, 0, 0]$:

$$[\mathbf{e}_1]_\times = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \tag{10.16}$$

Let H and H' be the two unknown rectifying homographies. When they are applied to the homologous points $\mathbf{m}, \mathbf{m}'$, respectively, the pair of transformed points must



Fig. 10.5 Top: Rectified stereo pair in original coordinate space. Bottom: Rectified stereo pair after centring; the two reference frames differ by a horizontal translation of 994 pixels

satisfy the epipolar geometry of a camera pair in normal configuration, that is:

$$(H'\mathbf{m}')^{\mathsf{T}}\,[\mathbf{e}_1]_\times\,(H\mathbf{m}) = \mathbf{m}'^{\mathsf{T}} H'^{\mathsf{T}}\,[\mathbf{e}_1]_\times\,H\mathbf{m} = 0. \tag{10.17}$$

This equation implies that in any factorisation of the fundamental matrix F of the original camera pair of the type

$$F = H'^{\mathsf{T}}\,[\mathbf{e}_1]_\times\,H, \tag{10.18}$$

with H'^{T} and H invertible 3×3 matrices, H'^{T} and H are rectifying homographies. These are not unique, and there are several ways to constrain the problem, each leading to solutions with different characteristics (Hartley 1999; Isgrò and Trucco 1999; Hsien-Huang P. Wu Jan 2005; Mallon and Whelan 2005), where the underlying goal is to limit the perspective distortion.

The method of Fusiello and Irsara (2011) is based on the idea of approximating what happens in the calibrated case. Indeed, we know that rectification occurs

by rotating the camera around its COP, and thus, the induced homographies are (cf. (10.15)):

$$H' = K'_n R' K'^{-1}_o \quad H = K_n R K_o^{-1}. \tag{10.19}$$

The rotations R, R' are the unknowns of the calibrated rectification while in the uncalibrated case also the intrinsic parameters are part of the unknowns. We can reduce the number of unknowns by making some educated guesses about the intrinsic parameters: no skew, unit aspect ratio and principal point in the centre of the image. The only remaining parameter is the focal length in pixels α, which we also assume to be identical in the two cameras. Therefore:

$$K'_o = K_o = \begin{bmatrix} \alpha & 0 & w/2 \\ 0 & \alpha & h/2 \\ 0 & 0 & 1 \end{bmatrix} \tag{10.20}$$

where w and h are the dimensions (in pixels) of the image.

Regarding rotations, it should be noted that if R_x is any rotation about the X-axis, we have:

$$R_x^\mathsf{T} [\mathbf{e}_1]_\times R_x = [\mathbf{e}_1]_\times \tag{10.21}$$

In fact, a rotation around the X-axis, which coincides with the baseline, of both cameras does not alter the rectification, but only the portion of the scene that is embraced. Therefore, we reduce the unknowns by fixing the angle of rotation around X in one of the two cameras to zero.

The intrinsic parameters of the rectified images (K_n, K'_n) can be determined arbitrarily, as long as the vertical focal length and the vertical coordinate of the principal point are the same. In fact, it is easily verified that

$$K'^\mathsf{T}_n [\mathbf{e}_1]_\times K_n \simeq [\mathbf{e}_1]_\times , \tag{10.22}$$

as soon as the second (and third) rows of K'_n and K_n are equal. For this reason, they are not included in the parameterisation of homographies.

To summarise, the method aims at solving

$$\mathbf{m}'^\mathsf{T} K_o^{-\mathsf{T}} R'^\mathsf{T} [\mathbf{e}_1]_\times R K_o^{-1} \mathbf{m} = 0 \tag{10.23}$$

for the rotation matrices $R(\varphi, \kappa)$, $R'(\omega', \varphi', \omega')$ and the focal length α in (10.20). In practice, a least-squares solution is sought that minimises the squares of the Sampson residuals of the original homologous points with respect to the fundamental matrix F (see Sect. 9.8):

$$F = K_o^{-\mathsf{T}} R'^\mathsf{T} [\mathbf{e}_1]_\times R K_o^{-1}. \tag{10.24}$$

As with the non-linear regression problems addressed in Chap. 9, we need to compute the derivative of the residuals and to hand the minimisation over an iterative method such as Levenberg-Marquardt (LM). This derivative is obtained by applying the chain rule to the composition of the Sampson residual for F with the parameterisation given by (10.24); only the derivative of the latter needs to be computed here, while for the former we refer to Sect. 9.8:

$$\frac{\partial \operatorname{vec} F}{\partial [\varphi, \kappa, \omega', \varphi', \omega', \alpha]} =$$

$$\left[\frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} R)^\mathsf{T}} \mathsf{D}\, R^*, \frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} R')^\mathsf{T}} \mathsf{D}\, R', \frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} K_o^{-1})^\mathsf{T}} \frac{\partial \operatorname{vec} K_o^{-1}}{\partial \alpha} \right] \tag{10.25}$$

where $\mathsf{D}\, R^*$ contains the last two columns of $\mathsf{D}\, R$, being $\omega = 0$. The three terms are:

$$\frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} R)^\mathsf{T}} = K_o^{-\mathsf{T}} \otimes K_o^{-\mathsf{T}} R'^\mathsf{T} [\mathbf{e}_1]_\times \tag{10.26}$$

$$\begin{aligned} \frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} R')^\mathsf{T}} &= \left(([\mathbf{e}_1]_\times R K_o^{-1})^\mathsf{T} \otimes K_o^{-\mathsf{T}} \right) K_3 \\ &= - \left(K_o^{-\mathsf{T}} R^\mathsf{T} [\mathbf{e}_1]_\times \otimes K_o^{-\mathsf{T}} \right) K_3 \end{aligned} \tag{10.27}$$

$$\begin{aligned} \frac{\partial \operatorname{vec} F}{\partial (\operatorname{vec} K_o^{-1})^\mathsf{T}} &= K_3 \left(I_3 \otimes K_o^{-\mathsf{T}} (R'^\mathsf{T} [\mathbf{e}_1]_\times R)^\mathsf{T} \right) + (I_3 \otimes K_o^{-\mathsf{T}} R'^\mathsf{T} [\mathbf{e}_1]_\times R) \\ &= -K_3 \left(I_3 \otimes K_o^{-\mathsf{T}} R^\mathsf{T} [\mathbf{e}_1]_\times R' \right) + \left(I_3 \otimes K_o^{-\mathsf{T}} R'^\mathsf{T} [\mathbf{e}_1]_\times R \right) \end{aligned} \tag{10.28}$$

and $\frac{\partial \operatorname{vec} K_o^{-1}}{\partial \alpha}$ is computed element by element, since K_o^{-1} is easily written in closed form.

In the absence of any a priori estimates, it makes sense to assign zero as the initial value of the rotation angles; however, in this way the last column of the Jacobian matrix (corresponding to α) cancels out, making it singular (as noted by Monasse 2011). We can solve the problem by assigning non-zero values to the rotation angles (e.g. we can always expect a small rotation around Y to compensate for vergence) or by performing two cascading minimisations, the first with α fixed and the second with all the unknowns.

The MATLAB implementation is given in Listing 10.3.

Note that the focal length α is parameterised with the field of view, which is an angle as the other unknowns.

Listing 10.3 Uncalibrated epipolar rectification

```
function [H1,H2, K] = rectifyF(m1, m2, sz)
% RECTIFYF Epipolar rectification (uncalibrated)

m1=ensure_homogeneous(m1);
m2=ensure_homogeneous(m2);

% first optimize with fixed f (to avoid jacobian singularity)
p_init = lsq_nonlin(@(x)costRectif(x,sz,m1,m2), [0 0 0 0 0]);

% then add f to the optimization
p_out = lsq_nonlin(@(x)costRectif(x,sz,m1,m2), [p_init(:); -1.2]);

% if extreme focal, fall back to default one
if abs(p_out(6)) <.26 || abs(p_out(6)) > 2.3
    p_out = [p_init(:); -1.2];
end

% recover K
K = par2K(p_out(6), sz);

% compute homographies
H1 = K * eul([0;p_out(1:2)]) /K;
H2 = K * eul(p_out(3:5)) /K;

end

function [res,J] = costRectif(a_in, sz, m1, m2)
% costRectif  Compute rectification cost

K3 = commutation(3,3); S = skew([1 0 0]);

n = length(a_in);     % number of variable parameters
a = [0;0;0;0;0;-1.2]; % default values
a(1:n) = a_in;        % overwrite default

% intrinsic parameters of the original cameras
[K, JK] = par2K(a(6),sz);
iK = inv(K);

% and its Jacobian
DiK = -kron(iK', iK) * JK;

% rotation angle X-left is always 0
[R1,DR1] = eul([0;a(1:2)]);
[R2,DR2] = eul(a(3:5));

% fundamental matrix btw original points
F = iK'*R2'*S*R1*iK;

% Jacobian of this parametrization of F
Jp = [kron(iK', iK'*R2'*S)*DR1(:,2:3),...
    -kron(iK'*R1'*S, iK')*K3*DR2,...
    (-K3*kron(eye(3),iK'*R1'*S*R2) + kron(eye(3),iK'*R2'*S*R1))*DiK];
Jp(:,n+1:end) = []; % remove fixed parameters

% Sampson residual
[res, Js] = sampson_fund(F,m1,m2);
J = Js * Jp;
end
```

Problems

10.1 With reference to the triangulation in the normal case, how is the computed depth Z affected by an error Δd on the disparity?

10.2 Referring to triangulation in the normal case, plot the iso-disparity surfaces in space.

10.3 Prove that a pair of cameras in normal configuration possesses the following fundamental matrix:

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}.$$

10.4 In light of the result of Problem 10.3, show that the epipolar rectification procedure illustrated in Sect. 10.3 is correct.

10.5 The rectification can be easily extended to the *trifocal* case (i.e. three images). In such a case the focal plane of the new PPMs is completely determined by the three COPs, and thus, the versor **k** is the normal to this plane.

10.6 What about rectifying $n > 3$ images?

References

A. Fusiello and L. Irsara. Quasi-euclidean epipolar rectification of uncalibrated images. *Machine Vision and Applications*, 22(4):663–670, 2011.

A. Fusiello, E. Trucco, and A. Verri. A compact algorithm for rectification of stereo pairs. *Machine Vision and Applications*, 12(1):16–22, 2000.

R.I. Hartley. Theory and practice of projective rectification. *International Journal of Computer Vision*, 35(2):1–16, November 1999.

Yu-Hua Yu Hsien-Huang P. Wu. Projective rectification with reduced geometric distortion for stereo vision and stereoscopic video. *Journal of Intelligent and Robotic Systems*, 42(1):Pages 71–94, Jan 2005.

F. Isgrò and E. Trucco. Projective rectification without epipolar geometry. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages I:94–99, Fort Collins, CO, June 23-25 1999.

John Mallon and Paul F. Whelan. Projective rectification from the fundamental matrix. *Image and Vision Computing*, 23(7):643–650, 2005.

Pascal Monasse. Quasi-Euclidean Epipolar Rectification. *Image Processing On Line*, 1:187–199, 2011.

Gian F. Poggio and T. Poggio. The analysis of stereopsis. *Annual Review of Neuroscience*, 7:379–412, 1984.

Chapter 11
Features Points



11.1 Introduction

In the discussion so far, we have always assumed that it was possible to perform the preliminary computation of a number of point correspondences. In this chapter we will address the practical problem of how to obtain such correspondences. We begin by noting that not all points in an image are equally suitable for computing correspondences. The *salient points* or *feature points* are points belonging to a region of the image that differs from its neighbourhood and therefore can be detected repeatably and with positional accuracy.

The definition of salient point that we have given is forcibly vague, because it depends implicitly on the algorithm under consideration: a posteriori we could say that the salient points are those that the algorithm extracts.

In order to match such points in different images, we need to characterise them. Since the intensity of a single point is poorly discriminative, we typically abstract some property that is a function of the pixel intensities of a surrounding region. The vector that summarises the local structure around the salient point is called the *descriptor*. Point matching thus reduces to descriptor comparison. For this to be effective, it is important that the descriptors remain invariant (to some extent) to changes in viewpoint and illumination, and possibly even to some degradation of the image.

We will illustrate in the following two different and complementary approaches to detecting features. The first consists of identifying image points that can best be matched with others based on local window autocorrelation analysis (Sect. 11.4). The second (Sect. 11.5) is based on scale invariant detection of high contrast regions, a.k.a. *blob*. Points of the former type will be matched simply by comparing pixel values in a local window, while for the latter, a descriptor invariant to geometric and radiometric transformations will be computed. The second approach is more complex but is better suited to images taken under very different conditions.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_11

For a complete critical review on detectors and descriptors, see Mikolajczyk and Schmid (2005), Mikolajczyk et al. (2005).

11.2 Filtering Images

The image I is a matrix of integers (we shall consider monochromatic images, for simplicity). In this context we treat it as a two-dimensional digital signal $I : \mathbb{N}^2 \to \mathbb{N}$ which is a discrete representation of an underlying analogue (continuous) signal $f : \mathbb{R}^2 \to \mathbb{R}$. In other words, we regard it as obtained by sampling and quantisation of a surface whose height is proportional to the grey value.

Analogous to what is commonly done for one-dimensional signals, it is possible to define a *convolution* operation for images. Linear *filtering* consists in the *convolution* of the image with a constant matrix called *kernel* or mask.

Let I be a $N \times M$ image and let K be a $m \times m$ kernel. The filtered version I_K of I is given by the discrete convolution (denoted by $*$):

$$I_K(u, v) = I * K = \sum_{h=-m/2}^{m/2} \sum_{k=-m/2}^{m/2} K(h, k) I(u - h, v - k). \tag{11.1}$$

A linear filter replaces the value $I(u, v)$ with a weighted sum of values of I itself in a $m \times m$ neighbourhood of (u, v), where the weights are the kernel entries.

Convolution enjoys the following properties:

- Commutative: $I * K = K * I$
- Associative: $(H * K) * I = H * (K * I)$
- Linear: $(\alpha I + \beta H) * K = \alpha I * K + \beta H * K$
- Commutative with differential $\mathsf{D}(K * I) = (\mathsf{D}\, K) * I$

Note that due to the minus sign in the neighbourhood indexing in I, the convolution coincides with the *cross-correlation* defined as

$$I_K(u, v) = \sum_{h=-m/2}^{m/2} \sum_{k=-m/2}^{m/2} K(h, k) I(u + h, v + k) \tag{11.2}$$

provided that the mask K is flipped both up-down and left-right before combining with I. If the mask is symmetric the convolution coincides with the correlation.

It is worth noting that cross-correlation can be interpreted as a comparison between two signals, and its maximum indicates the position where the two match best. In other words, it is like searching within the image I for the position of a

smaller image K, which in this context is called *template*, and the whole operation is referred to as *template matching*.

The effects of a linear filter on a signal can best be appreciated in the frequency domain. By the convolution theorem, the Fourier transform of the convolution of I and K is simply the product of their Fourier transforms $\mathcal{F}(I)$ and $\mathcal{F}(K)$. Therefore, the result of convolving a signal with K is to attenuate or suppress the frequencies of the signal corresponding to small or zero values of $|\mathcal{F}(K)|$. From this point of view, the kernel is the (finite) impulsive response of the filter.

11.2.1 Smoothing

If all entries of K are positive or zero, a *smoothing* effect is obtained from the convolution: linear filtering replaces the pixel value with the weighted average of its surroundings. The simplest kernel of this type is the *box filter* or average filter:

$$K_{box} = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}. \tag{11.3}$$

The practical effect of such a filter is to smooth out the noise, since the mean intuitively tends to level out small variations. Formally, we see that averaging m^2 noisy values divides the standard deviation of the noise by m. The obvious disadvantage is that smoothing reduces the sharp details of the image, thereby introducing *blurring*. The size of the kernel controls the amount of blurring: a larger kernel produces more blurring, resulting in greater loss of detail.

In the frequency domain, we know that the Fourier transform of a 1D box signal is the *sinc* function: something similar holds in 2D (Fig. 11.1). Since the frequencies of the signal that fall within the main lobe are weighted more than the frequencies that fall in the secondary lobes, the average filter is a low-pass filter. However, it is a very poor low-pass filter as the frequency cut-off is not sharp, due to the secondary lobes.

We then consider the Gaussian smoothing filter, which corresponds to the function:

$$G = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/2\sigma^2}. \tag{11.4}$$

The Fourier transform of a Gaussian is still a Gaussian and therefore has no secondary lobes. The following mask corresponds to a Gaussian with $\sigma = 1$:

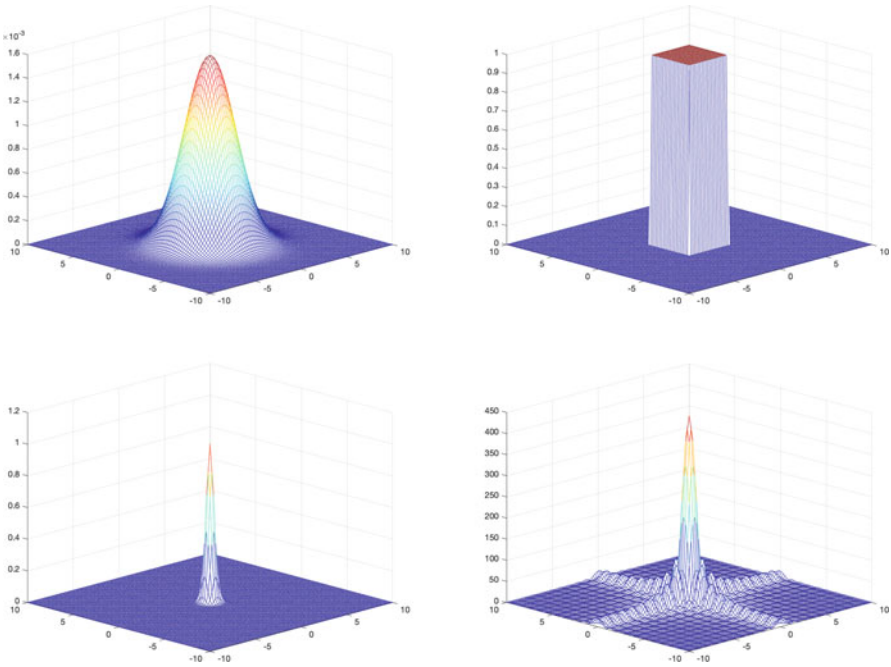


Fig. 11.1 Top: kernel (impulsive response). Bottom: Fourier transform

$$G = \frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix} \tag{11.5}$$

Gaussian smoothing can be implemented efficiently due to the fact that the kernel is separable, that is, that $G = (gg^{\mathsf{T}})$ where g denotes a 1D vertical Gaussian kernel. Indeed, for example:

$$G = \frac{1}{273}\begin{bmatrix} 1 & 4 & 7 & 4 & 1 \\ 4 & 16 & 26 & 16 & 4 \\ 7 & 26 & 41 & 26 & 7 \\ 4 & 16 & 26 & 16 & 4 \\ 1 & 4 & 7 & 4 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.06 \\ 0.24 \\ 0.39 \\ 0.24 \\ 0.06 \end{bmatrix} \begin{bmatrix} 0.06 & 0.24 & 0.39 & 0.24 & 0.06 \end{bmatrix}. \tag{11.6}$$

In the case of two 1D kernels oriented in orthogonal directions, their convolutions reduce to an outer product and therefore, due to the associativity of the convolution, we have:

$$G * I = (gg^{\mathsf{T}}) * I = (g * g^{\mathsf{T}}) * I = g * (g^{\mathsf{T}} * I) \tag{11.7}$$

This means that convolving an image I with a 2D Gaussian kernel G is the same as convolving first all rows and then all columns with a 1D Gaussian kernel. The advantage is that the time complexity depends linearly on the mask size rather than quadratically.

Another interesting property of Gaussian filtering is that repeated convolution with a Gaussian kernel is equivalent to convolution with a larger Gaussian kernel:

$$G_{\sigma_1} * (G_{\sigma_2} * I) = (G_{\sigma_1} * G_{\sigma_2}) * I = G_{\sqrt{\sigma_1^2+\sigma_2^2}} * I. \tag{11.8}$$

To construct a discrete Gaussian mask one must sample a continuous Gaussian (due to the separability of the Gaussian kernel we can only consider 1D masks). The width of the mask and the variance of the Gaussian are not independent: fixed one the other is determined. A rule of thumb is: $w = 4\sigma + 1$.

11.2.1.1 Non-linear Filters

Gaussian smoothing does a good job in removing the noise, but since it cancels the high frequencies, it also smooths the image content such as the sharp variations of intensity, called *edges*.

The *bilateral filter* is a non-linear smoothing filter, which reduces image noise but preserves edges. Like any linear filter, it replaces the intensity of each pixel with a weighted average of the intensity values of neighbouring pixels. The difference is that these weights depend not only on the distance of the pixels from the centre of the window, as in a Gaussian kernel, but also on differences in intensities. For this reason, it is *non-linear*.

In flat regions, the pixel values in a small neighbourhood are similar to each other and the bilateral filter basically acts as a standard linear filter. Let us now consider a sharp boundary between a dark and a bright region. When the bilateral filter is centred, for example, on a pixel on the bright side of the boundary, the weights are high for pixels on the same side and low for pixels on the dark side. Consequently, the filter replaces the bright central pixel with an average of the bright pixels in its neighbourhood, essentially ignoring the dark pixels. Conversely, when the filter is centred on a dark pixel, bright pixels are ignored. Thus, the bilateral filter preserves sharp edges.

Another example of a non-linear filter is the *median filter*, in which the value of each pixel is replaced by the median of the values of its neighbours. Since the median cannot be expressed as a weighted sum, the filter is *non*-linear.

11.2.2 Derivation

Let $f : \mathbb{R}^2 \to \mathbb{R}$ be a differentiable function. Its gradient at point (x_0, y_0) is the vector whose components are the partial derivatives of f at (x_0, y_0):

$$\nabla f(x_0, y_0) = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}\right] = [f_x, f_y] \tag{11.9}$$

The vector ∇f points in the direction of steepest ascent and is perpendicular to the tangent to the contour line.

The modulus of the gradient $\sqrt{f_x^2 + f_y^2}$ is related to the slope at the point and thus takes high values corresponding to steep changes in the value of the function. The phase of the gradient $\arctan(f_y/f_x)$ represents its direction instead.

Returning now to images, we can think of using the gradient to detect edges or edge points, which are the pixels at—or around—which the image intensity changes sharply (Fig. 11.2): edges are detected as maxima of the modulus of the gradient (Fig. 11.3).

There are several reasons for our interest in edges. The main one is that edges often (but not always) define the contours of solid objects.

To calculate the gradient we need the directional derivatives. Since the image is a function assigned on a discrete domain, we will need to consider its *numerical* derivatives. Combining the truncated Taylor expansions of $f(x + h)$ and $f(x - h)$ we get:

$$\frac{\partial f}{\partial x} \approx \frac{f(x + h) - f(x - h)}{2h} \tag{11.10}$$

Considering the image I as the discrete representation of f, setting $h = 1$ and neglecting the $\frac{1}{2}$ factor, we have that:

$$I_u = \frac{\partial I}{\partial u} \approx I(u + 1, v) - I(u - 1, v) \tag{11.11}$$

$$I_v = \frac{\partial I}{\partial v} \approx I(u, v + 1) - I(u, v + 1). \tag{11.12}$$

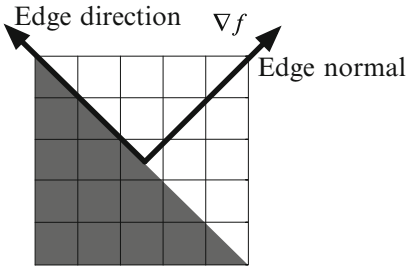


Fig. 11.2 Edges are points of sharp contrast in an image where the intensity of the pixels changes abruptly. The edge direction points to the direction in which intensity is constant, while the normal to the edge points in the direction of the maximal change in intensity

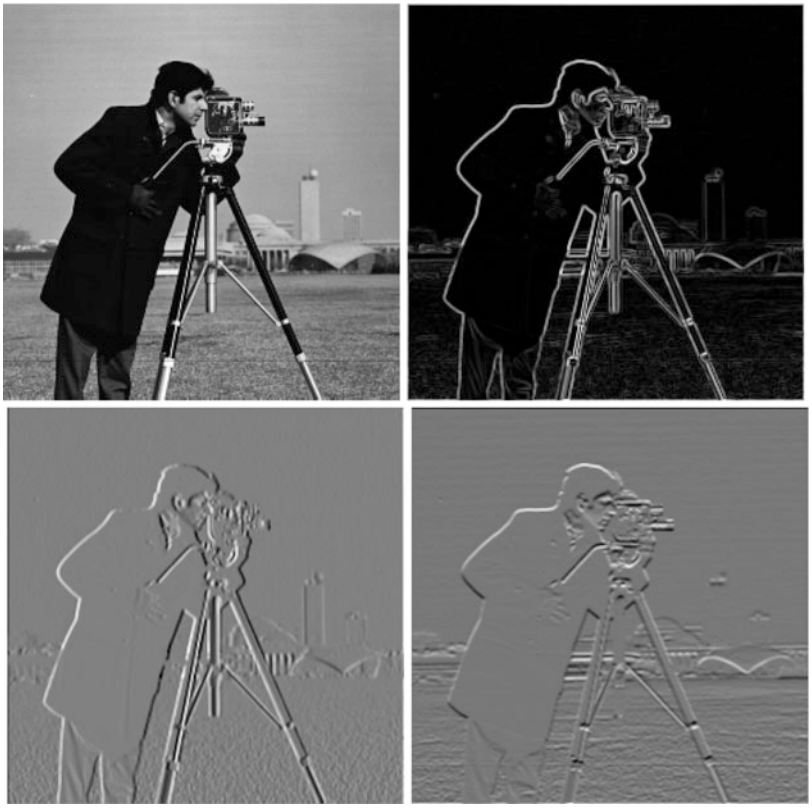


Fig. 11.3 Top: Original image and gradient magnitude. Bottom: Directional derivatives in u and v

We immediately see that the numerical derivation of an image is implemented as a linear filtering, namely a convolution with the mask $[1, 0, -1]$ for I_u and with its transpose for I_v.

The frequency interpretation of the convolution with a derivative mask is a high-pass filtering, which amplifies the noise. The solution is to cut down the noise before deriving by smoothing the image. Let D be a derivation mask and S a smoothing mask. For the associativity of the convolution:

$$D * (S * I) = (D * S) * I. \tag{11.13}$$

This means that one needs to filter the image only once with a kernel given by $(D * S)$.

In the case of two 1D kernels D and S oriented in orthogonal directions, their convolution reduces to an outer product and thus results in a separable kernel. For example, the Prewitt operator is obtained from the box filter [1 1 1] convolved with the derivative mask [1 0 −1]:

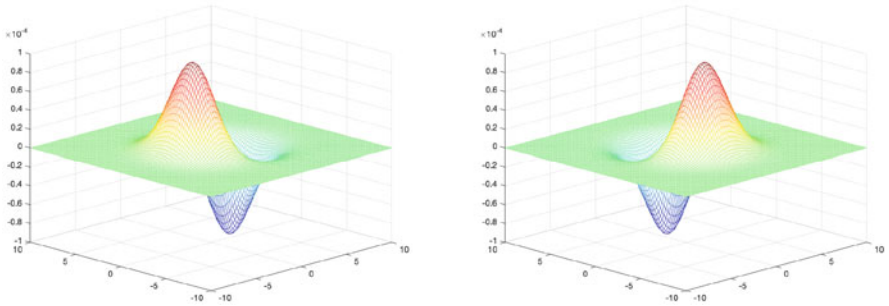


Fig. 11.4 Partial derivatives of the Gaussian kernel

$$P = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}. \tag{11.14}$$

Sobel's operator, on the other hand, is a rudimentary Gaussian filter $[1 \;\; 2 \;\; 1]$ convolved with $[1 \;\; 0 \;\; -1]$:

$$S = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & -1 \end{bmatrix}. \tag{11.15}$$

A 2D Gaussian can be used for smoothing: we can either filter with a Gaussian kernel and then calculate the derivatives or utilise the commutativity property between differential operators and convolution and simply convolve with the derivative of the Gaussian kernel:

$$\mathsf{D}(G * f) = (\mathsf{D}\, G) * f. \tag{11.16}$$

In practice, convolution is carried out with the two partial derivatives of the Gaussian (Fig. 11.4).

11.3 LoG Filtering

Abrupt changes in the grey level of the input image correspond to the extreme points (maxima and minima) of the first derivative. If we consider the second derivative instead, these same points correspond to the zero crossings of the latter (Fig. 11.5).

To obtain the numerical approximation of the second derivative, we proceed similarly to what we did for the first derivative, obtaining:

$$\frac{\partial^2 f}{\partial x^2} \approx \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} \tag{11.17}$$

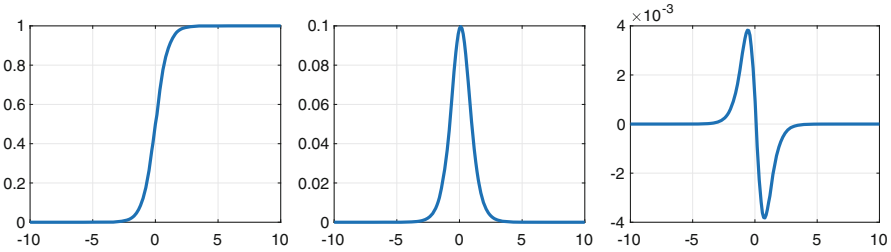


Fig. 11.5 From left: signal (edge), first derivative, second derivative

and thus the corresponding convolution mask is $[1 \quad -2 \quad 1]$, so the second partial derivatives are:

$$I_{uu} = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix} * I \quad \text{and} \quad I_{vv} = \begin{bmatrix} 1 & -2 & 1 \end{bmatrix}^{\mathsf{T}} * I. \tag{11.18}$$

Summing them yields the Laplacian operator:

$$\nabla^2 I = I_{uu} + I_{vv} = \left(\begin{bmatrix} 1 & -2 & 1 \end{bmatrix} + \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix} \right) * I = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} * I \tag{11.19}$$

$\nabla^2 I$ is a scalar and can be found using a single mask. Since it is a second-order differential operator, it is very sensitive to noise, more than the first derivative. Thus, it is always combined with a smoothing operation, for example with a Gaussian kernel. An input image $I(u, v)$ is first convolved with a Gaussian kernel $G(u, v; \sigma)$ (at some scale σ) to obtain a smoothed version of it:

$$\widehat{I}(u, v) = G(u, v; \sigma) * I(u, v), \tag{11.20}$$

then the Laplacian operator is applied:

$$\nabla^2 \widehat{I} = \widehat{I}_{uu} + \widehat{I}_{vv}. \tag{11.21}$$

Due to the commutativity between the differential operators and the convolution, the Laplacian of the smoothed image can be equivalently computed as the convolution with the Laplacian of the Gaussian kernel, hence the name *Laplacian of Gaussian* (LoG):

$$\nabla^2 \widehat{I}(u, v) = \underbrace{\left(\nabla^2 G(u, v; \sigma) \right)}_{\text{LoG}} * I(u, v). \tag{11.22}$$

In addition to identifying the edges as zero crossings (Marr and Hildreth 1980), observe that LoG filtering provides strong positive responses for dark blobs and

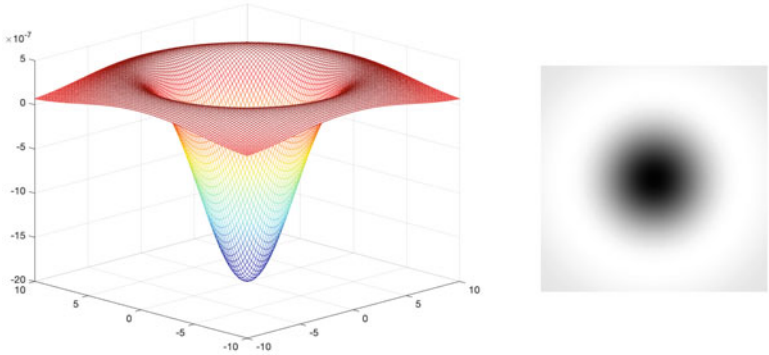


Fig. 11.6 Two plots of the LoG filter kernel, with the classic "sombrero" shape

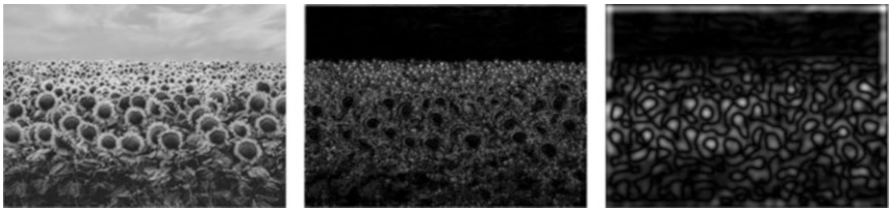


Fig. 11.7 Test image and response of the Gaussian Laplacian with $\sigma = 3$ and $\sigma = 15$. As can be seen, the peaks of the response correspond to dark blobs of a certain size, which depends on the σ. Sunflowers photo by Todd Trapani on Unsplash

strong negative responses for bright blobs (Fig. 11.7). In fact, the LoG kernel is precisely the template of the dark blob on a light background (Fig. 11.6), and we already noted that the correlation can be read as an index of similarity with the mask (a.k.a. template matching). Moreover, we see that the response is dependent on the ratio of the blob size to the size of the Gaussian kernel and is maximal for blobs of diameter close to σ. In summary, extremal points of the LoG detect blobs at a scale given by σ (Fig. 11.7).

LoG can be approximated by a difference of two Gaussians (DoG) with different scales (Fig. 11.8). The separability and cascadability of Gaussians applies to DoG, so an efficient implementation is achieved:

$$\nabla^2 G_\sigma \approx G_{\sigma_1} - G_{\sigma_2} \quad \text{with } \sigma_1 = \frac{\sigma}{\sqrt{2}}, \sigma_2 = \sqrt{2}\sigma. \tag{11.23}$$

From a frequency point of view, the LoG operator is a bandpass filter, and in fact, it responds well to spots characterised by a spatial frequency in its passband, while attenuating the rest. The approximation with DoG further confirms this interpretation, as it shows that the LoG is the difference of two low-pass filters.

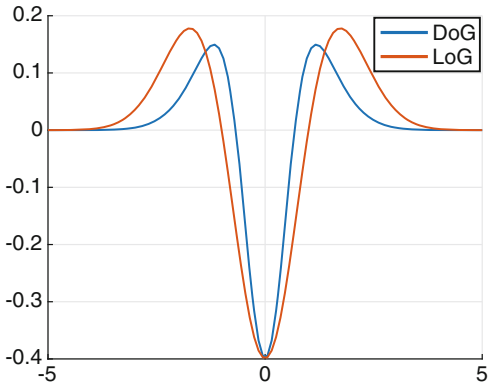


Fig. 11.8 Approximation of LoG as difference of Gaussians (DoG)

11.4 Harris-Stephens Operator

This operator detects features points with a discontinuity in intensity in two directions, which are traditionally referred to as "corners", as it also responds to the junction of two edges. This discontinuity can be detected by measuring the amount of variation when a small region centred on the point is translated in its neighbourhood. If the region can be translated in one direction without significant variation, then there is an edge in that direction. If, however, there is a significant variation in all directions, then a "corner" is present.

Let us calculate the variation, in terms of sum of squared differences (SSD), that is obtained by translating in the direction $\mathbf{x}$ a region Ω centred at the point $\mathbf{m} = (u, v)$ of the image I:

$$e(\mathbf{m}, \mathbf{x}) = \sum_{\mathbf{d} \in \Omega} [I(\mathbf{m} + \mathbf{d}) - I(\mathbf{m} + \mathbf{d} + \mathbf{x})]^2. \tag{11.24}$$

By truncating the Taylor series we obtain:

$$\begin{aligned} e(\mathbf{m}, \mathbf{x}) &= \sum_{\mathbf{d} \in \Omega} [\nabla I(\mathbf{m} + \mathbf{d})\mathbf{x}]^2 \\ &= \sum_{\mathbf{d} \in \Omega} \mathbf{x}^\mathsf{T} (\nabla I(\mathbf{m} + \mathbf{d}))^\mathsf{T} (\nabla I(\mathbf{m} + \mathbf{d}))\mathbf{x} \\ &= \sum_{\mathbf{d} \in \Omega} \mathbf{x}^\mathsf{T} \begin{bmatrix} I_u^2 & I_u I_v \\ I_u I_v & I_v^2 \end{bmatrix} \mathbf{x} \\ &= \mathbf{x}^\mathsf{T} \underbrace{\begin{bmatrix} \sum_{\mathbf{d} \in \Omega} I_u^2 & \sum_{\mathbf{d} \in \Omega} I_u I_v \\ \sum_{\mathbf{d} \in \Omega} I_u I_v & \sum_{\mathbf{d} \in \Omega} I_v^2 \end{bmatrix}}_{S(\mathbf{m})} \mathbf{x} \end{aligned} \tag{11.25}$$

where $\nabla I(\mathbf{m}+\mathbf{d}) = [I_u \;\; I_v]^\mathsf{T}$. The matrix $S(\mathbf{m})$ is often referred to in the literature as the *structure tensor*.

So, moving a region centred on $\mathbf{m}$ in the direction $\mathbf{x}$ yields an SSD equal to:

$$e(\mathbf{m}, \mathbf{x}) = \mathbf{x}^\mathsf{T} S(\mathbf{m})\, \mathbf{x}. \tag{11.26}$$

Since $\mathbf{x}$ encodes a direction, it can be assumed to have unit norm, and thanks to Proposition A.8:

$$\lambda_1 < e(\mathbf{m}, \mathbf{x}) < \lambda_2 \tag{11.27}$$

where λ_1 and λ_2 are the minimum and maximum eigenvalues of $S(\mathbf{m})$, respectively. Thus, if we consider all possible directions $\mathbf{x}$, the maximum of the variation is λ_2, while the minimum is λ_1. We can then classify the image structure around each pixel by analysing the eigenvalues λ_1 and λ_2. The cases that can occur are the following:

- flat (no structure): $\lambda_1 \approx \lambda_2 \approx 0$
- edge: $\lambda_1 \approx 0$, $\lambda_2 \gg 0$
- corner: λ_1 and λ_2 both $\gg 0$

The condition for having a corner point then translates to $\lambda_1 > c$ where c is a predefined threshold.

Harris and Stephens (1988), however, do not explicitly compute the eigenvalues but the quantity:

$$r(\mathbf{m}) = \det(S(\mathbf{m})) - k \;\; \mathrm{tr}^2(S(\mathbf{m})) \tag{11.28}$$

and consider as corners the points where the value of r exceeds a certain threshold (the constant k is set to 0.04). The rationale for the above formula is that

$$\mathrm{tr}(S(\mathbf{m})) = \lambda_1 + \lambda_2 = \sum_{\mathbf{d}} I_u^2 + \sum_{\mathbf{d}} I_v^2 \tag{11.29}$$

and

$$\det(S(\mathbf{m})) = \lambda_1 \lambda_2 = \sum_{\mathbf{d}} I_u^2 \sum_{\mathbf{d}} I_v^2 - \left(\sum_{\mathbf{d}} I_u I_v \right)^2. \tag{11.30}$$

The Harris-Stephens operator responds with positive values at corners, negative values at edges and values close to zero in uniform regions (Fig. 11.9).

Starting from the same considerations about eigenvalues, Noble (1988) proposes an operator also based on the trace and the determinant of the matrix $S(\mathbf{m})$ but without parameters. For each point in the image, the following ratio is computed:

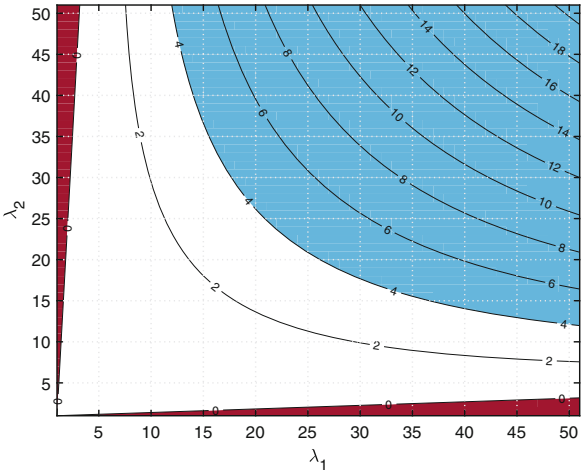


Fig. 11.9 Response of the Harris-Stephens operator as a function of the two eigenvalues of $S(\mathbf{m})$, λ_1 and λ_2. There are two curves of interest: the zero-level one separates flat points from edges, while a threshold-level curve (4.0 in the figure) separates flat points from those considered as corners

$$n(\mathbf{m}) = \frac{\det(S(\mathbf{m}))}{\mathrm{tr}(S(\mathbf{m}))} \tag{11.31}$$

which responds with high values at corners (Fig. 11.10). Zuliani et al. (2004) provide an insightful comparison of corner detectors.

Observe that the matrix $S(\mathbf{m})$ can be defined alternatively (up to a constant) as:

$$S(\mathbf{m}) = \begin{bmatrix} (I_u^2 * K_{box})(\mathbf{m}) & (I_u I_v * K_{box})(\mathbf{m}) \\ (I_u I_v * K_{box})(\mathbf{m}) & (I_v^2 * K_{box})(\mathbf{m}) \end{bmatrix} \tag{11.32}$$

where K_{box} is a box filter implementing the sum over Ω. In practice, the box is replaced by a Gaussian smoothing kernel whose standard deviation determines the spatial scale at which the corners are detected.

In the Harris-Stephens operator there are two scales: the integration scale and the derivation scale (Zuliani et al. 2004). The former is the standard deviation of the Gaussian kernel convolved with the images I_u^2, $I_u I_v$, I_v^2 and is the one that most affects the scale at which the angular points are detected. The derivation scale, on the other hand, is the standard deviation of the Gaussian with which the image is filtered before calculating the derivatives, which in our implementation is fixed by the Sobel 3×3 window. In order to deal with the scale in a more principled way, one should consider the latter as well

(continued)

and vary it along with the former, typically setting them to be equal. See, for example, (Gueguen and Pesaresi 2011).

Listing 11.1 reports the implementation of the Harris-Stephens operator. To obtain a detector, we need to select the points where the operator (i) has a value above a certain threshold and (ii) is a local maximum, that is, in its surroundings there are no points with a larger response.

Figure 11.10 shows an example of corner detection.

Listing 11.1 Harris-Stephens operator

```
function C = imhs(I,s)
%IMHS Harris-Stephens corner strength
% s is the integration scale

    S = fspecial('sobel');
    G = fspecial('gaussian',2*ceil(2*s)+1, s);

    % directional derivatives
    Iu = filter2(S, I, 'same');
    Iv = filter2(S',I, 'same');

    % convolve with Gaussian
    Iuv = filter2(G, Iu.*Iv,'same');
    Ivv = filter2(G, Iv.^2, 'same');
    Iuu = filter2(G, Iu.^2, 'same');

    % trace and determinant
    tr = Iuu + Ivv;
    dt = Iuu.*Ivv - Iuv.^2;

    C = dt - 0.04 *tr.^2; % H-S version
    % C = dt./(1+tr);     % Noble version
end
```

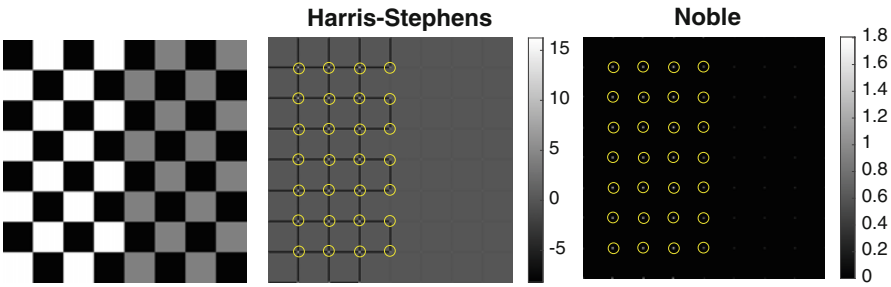


Fig. 11.10 From left: test image, Harris-Stephens operator response (note that it takes negative values), Noble operator response. The circles indicate the corners detected after selecting local maxima larger than a threshold (4.0 for HS and 1.0 for Noble). With these thresholds, low contrast corners on the right side of the image are not detected

11.4.1 Matching and Tracking

Once corners have been detected as described in the previous section, they can be matched, typically taking into account the proximity (position) and/or the similarity of a small region Ω centred on the corner itself, similarly to stereo matching (see Chap. 12).

When a video sequence needs to be processed, the matching in each pair of frames is repeated; this is referred to as *features tracking*. This process can be difficult, as features may vanish or reappear because of occlusions or detector errors.

11.4.2 Kanade-Lucas-Tomasi Algorithm

The tracking problem for small displacements can be characterised as follows: given the position of a feature point in a frame at time t, we want to find the position of the same feature point in the frame at time $t + \Delta t$. Assuming that the content of a $n \times n$ window of pixels around the feature point remains unchanged, Tomasi and Kanade (1991) formalise the problem as the least-squares solution of the following system of non-linear equations in the unknown speed $\mathbf{x}$:

$$r_i(\mathbf{x}) = I(\mathbf{m}_i + \mathbf{x}\Delta t, t + \Delta t) - I(\mathbf{m}_i, t) = 0 \quad i = 1 \ldots n \times n. \tag{11.33}$$

In other words, it is a matter of finding the translation $\mathbf{x}\Delta t$ that produces the best alignment of the window centred on the feature point between the frame at time t and the frame at time $t + \Delta t$.

A non-linear least-squares problem can be solved by the Gauss-Newton method, which involves calculating the Jacobian of the system:

$$J = \begin{bmatrix} \dfrac{\partial r_1}{\partial \mathbf{x}} \\ \vdots \\ \dfrac{\partial r_{n\times n}}{\partial \mathbf{x}} \end{bmatrix}. \tag{11.34}$$

In our case

$$\frac{\partial r_i}{\partial \mathbf{x}} = \Delta t \ \nabla I(\mathbf{m}_i + \mathbf{x}\Delta t, t + \Delta t) \tag{11.35}$$

where ∇I represents the spatial gradient of the image.

The solution according to the Gauss-Newton method (See Appendix C.2.2) proceeds iteratively by computing at each step an increment $\Delta\mathbf{x}$ as the solution of the normal equations:

$$J^\top J \Delta\mathbf{x} = -J^\top \mathbf{r}(\mathbf{x}) \tag{11.36}$$

with

$$\mathbf{r}(\mathbf{x}) = \begin{bmatrix} r_1(\mathbf{x}) \\ \vdots \\ r_{n\times n}(\mathbf{x}) \end{bmatrix}. \tag{11.37}$$

In the iterations after the first one, it is necessary to compute $I(\mathbf{m} + \mathbf{x}\Delta t, t + \Delta t)$, which will not, in general, be found on the pixel grid, and therefore, an interpolation is needed.

Note that the points suitable to be tracked are those that guarantee good conditioning of the system of normal equations, namely those for which the eigenvalues λ_1 and λ_2 of $J^\top J$ are both far from zero and not too different: $\min(\lambda_1, \lambda_2) > c$ where c is a fixed threshold. Since the matrix $J^\top J$ is essentially identical to the matrix S of the Harris-Stephens method, these points coincide with the corners detected by Harris-Stephens.

The KLT algorithm, proposed by Tomasi and Kanade (1991) and based on previous work by Lucas and Kanade (1981), is extended by Shi and Tomasi (1994) with an *affine* model for window deformation (instead of just translational). In fact, one can complicate the motion model in (11.33) at will, as long as one is able to compute its Jacobian.

11.4.3 Predictive Tracking

The KLT method illustrated in the previous section assumed a small displacement between two consecutive images. When this is instead significant, one can compensate the motion by exploiting the spatio-temporal coherence to predict where the points should be in the next frame.

The classical scheme is based on the iteration of three steps: extraction, prediction and association, and typically includes a *Kalman filter*, which uses a model of motion to predict where the point will be at the next time instant, given its previous positions. In addition, the filter maintains an estimate of the uncertainty in the prediction, thus allowing a search window to be defined. Features are detected within the search window, one of them is associated with the tracked point, and the filter updates the position estimate using the position of the found point and its uncertainty.

The association operation may present ambiguities (Fig. 11.11). When more than one point is found in the search window, more sophisticated filters are needed, like the *Joint Probabilistic Data Association Filter*. See (Bar-Shalom and Fortmann 1988) for a discussion of this problem.

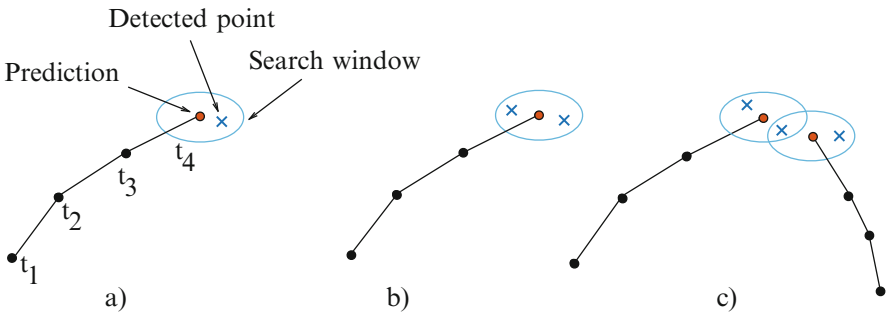


Fig. 11.11 Predictive tracking (**a**) and data association of single-track (**b**) and multiple-tracks (**c**)

11.5 Scale Invariant Feature Transform

The Scale Invariant Feature Transform (SIFT) operator introduced by Lowe (2004) detects blob-like feature points characterised by high contrast via an approximation of the *LoG filter* (Laplacian of Gaussian) applied to a *scale-space* representation of the image.

11.5.1 Scale-Space

Recall that both the LoG and the HS operator detect feature points (of different nature) *at a given scale*. The choice of scale is not secondary, and although initially overlooked, it must be addressed at some point. In fact, real scenes are composed of different structures that possess different intrinsic scales. Moreover, the projected size of objects varies due to the distance from the camera. This implies that a real object may appear in different ways depending on contingent factors. Since there is no way to know a priori which scale is appropriate to describe each of the interesting structures in the images, one approach is to embrace them all in a multiscale description.

For this purpose, we introduce the notion of *scale-space*. The image is represented as a family of smoothed versions, parameterised by the size σ of the smoothing kernel. The parameter σ is referred to as *scale* and together with the two spatial variables (u, v) locates a point (u, v, σ) in scale-space.

We will use the linear scale-space, which is obtained with a Gaussian kernel. The choice is not arbitrary but derives as a necessary consequence from the formalisation of the criterion that filtering should not create new spurious structures when moving from a fine to a coarser scale. The reader can refer to Lindeberg (2012) for further details.

For a given image $I(u, v)$, its linear scale-space representation is the family of images $L(u, v, \sigma)$ defined by the convolution of $I(u, v)$ with a Gaussian kernel of variance σ^2:

$$L(u, v, \sigma) = G(u, v; \sigma) * I(u, v) \tag{11.38}$$

where $*$ denotes the convolution and

$$G(u, v; \sigma) = \frac{1}{2\pi\sigma^2} e^{-(u^2+v^2)/2\sigma^2}. \tag{11.39}$$

As the variance σ^2 of the Gaussian kernel increases, there is an increasing removal of image detail, for L results from the convolution of I with a low-pass kernel of increasing spatial support (Fig. 11.12). In particular, blobs that are significantly smaller than σ are faded out in $L(u, v; \sigma)$.

This framework will enable us to define operations (such as feature detection) that are scale invariant, in the next section.

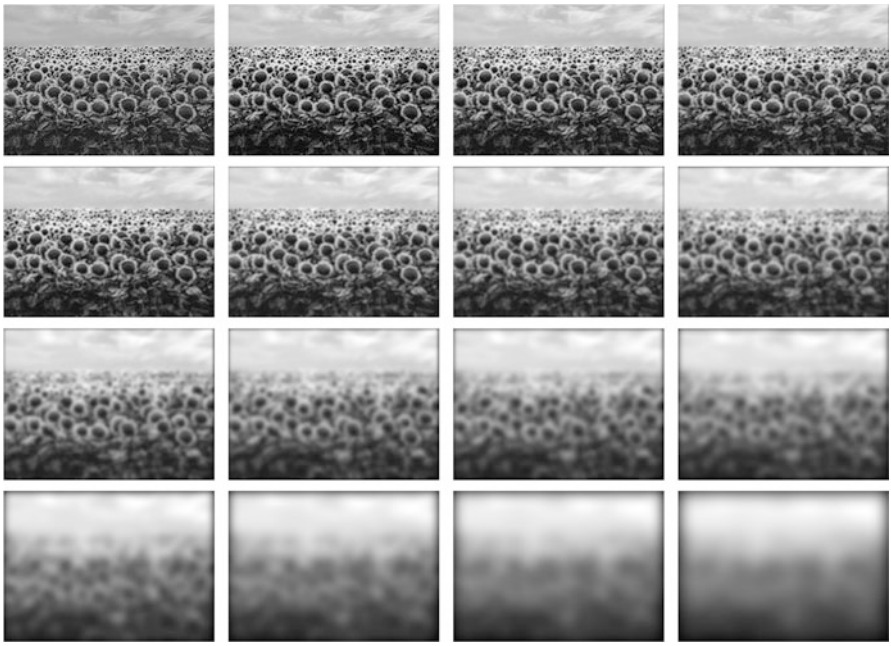


Fig. 11.12 Scale-space for an image of a field of sunflowers. The first one in the upper left is the original. The others correspond to increasing variances of the Gaussian kernel, in geometric progression (standard deviation doubles every three images). Sunflowers photo by Todd Trapani on Unsplash

In (11.38) we defined the Gaussian scale-space with reference to the original image. A more precise way to define it is to refer to an ideal image of infinite resolution, which we call I^*. We then define the scale-space as the collection of filtered images:

$$L(u, v, \sigma) = G(u, v, \sigma) * I^*(u, v), \quad \sigma \geq 0.$$

However, I^* is not accessible, and we must therefore anchor the scale-space to the input image. The latter is conventionally considered as the result of filtering I^* with a kernel at σ_{pix}=0.5 to account for the finite pixel size: $I(u, v) = G(u, v, \sigma_{\text{pix}}) * I^*(u, v)$. So, in practice, the scale-space is calculated with:

$$L(u, v, \sigma) = G\left(u, v, \sqrt{\sigma^2 - \sigma_{\text{pix}}^2}\right) * I(u, v), \quad \sigma \geq \sigma_{\text{pix}}.$$

11.5.2 SIFT Detector

As we have seen, the Laplacian combined with Gaussian filtering detects feature points (or rather blobs) at the scale set by the Gaussian kernel.

In order to obtain a multiscale blob detector, Lindeberg (1998) proposed to apply the (normalised) Laplacian to the entire linear scale-space:

$$\nabla^2_{\text{norm}} L(u, v, \sigma) = \sigma^2 \nabla^2 \left(G(u, v; \sigma) * I(u, v)\right). \tag{11.40}$$

and to look for its extremal points, that is, local maxima or minima (in scale-space).

Normalisation is an essential detail when dealing with scale-space analysis. Going from finer to coarser scales, the image will become increasingly blurred, thus resulting in the decrease of the amplitude of the image derivatives. Without normalisation, the maximum amplitude will always be found at the finest scale and the minimum at the coarsest scale. As demonstrated by Lindeberg (1998), the correct scaling factor for this is σ^2.

In the same paper it is shown that this method of detecting feature point leads to scale invariance in the sense that under scale transformations the feature points are preserved and the scale of a point of interest transforms accordingly.

The SIFT detector proposed by Lowe (2004) works similarly to the method of Lindeberg (1998), of which it can be seen as a computationally efficient variant, due to the replacement of the normalised Laplacian scale-space with a pyramid of differences of Gaussian (DoG):

$$\mathrm{DoG}(u, v, \sigma_i) = L(u, v, \sigma_{i+1}) - L(u, v, \sigma_i) \tag{11.41}$$

where the scale levels follow a geometric progression $\sigma_{i+1} = k\,\sigma_i$ with a fixed k and a given σ_0. Indeed, it can be seen that:

$$\mathrm{DoG}(u, v, \sigma_i) \approx \frac{(k^2 - 1)}{2} \nabla^2_{\mathrm{norm}} L(u, v, \sigma_i). \tag{11.42}$$

The constant k does not affect the location of the extremal points, and it is typically chosen as $k = \sqrt[n]{2}$ so that the scale σ doubles every n levels (interval which is denoted as *octave*).

The Gaussian pyramid is constructed by repeatedly applying Gaussian filtering and subsampling at each octave change so that in the end we obtain a stepped pyramid in which the size changes at each octave. Differences (DoGs) are computed between adjacent levels within the same octave, and then the extremal points in space and scale in the DoG pyramid are detected through comparisons with neighbours in a small window. Only extremal points that have a response above a given threshold (in absolute value) are considered.

SIFT employs $n = 3$ levels per octave, so $k = \sqrt[3]{2}$, and the value of σ_0 is set to 1.6. It uses a $3 \times 3 \times 3$ window, which requires a DoG overlay image with the previous and the next octave, for a total of $n + 2$ DoG images per octave. The resulting pyramid is shown in Fig. 11.13.

Where does the original image I fit in the Gaussian pyramid of SIFT? The first image at the bottom of the pyramid (index -1) is not I but instead corresponds to filtering I^* with a standard deviation Gaussian $\frac{\sigma_0}{k}$. According to the box at page 157, the image at level -1 is obtained from I after filtering with a Gaussian with standard deviation $\sqrt{\left(\frac{\sigma_0}{k}\right)^2 - \sigma^2_{\mathrm{pix}}}$. With the values used by SIFT, this makes 1.16, so the image at level -1 can be confused with I to the extent that 1.16 can be confused with 1.

The DoG (like the LoG) also has opposite maxima and minima near the edges (where it vanishes), but these are of little use for correspondences and should be eliminated. We therefore want to select the extremal points of the DoG that exhibit strong localisation, that is, such that the two principal curvatures[1] are both large. This criterion can be expressed in terms of the eigenvalues of the Hessian matrix of the DoG image (denoted D):

[1] Maximum and minimum curvature of a curve contained in the surface and passing through the point.

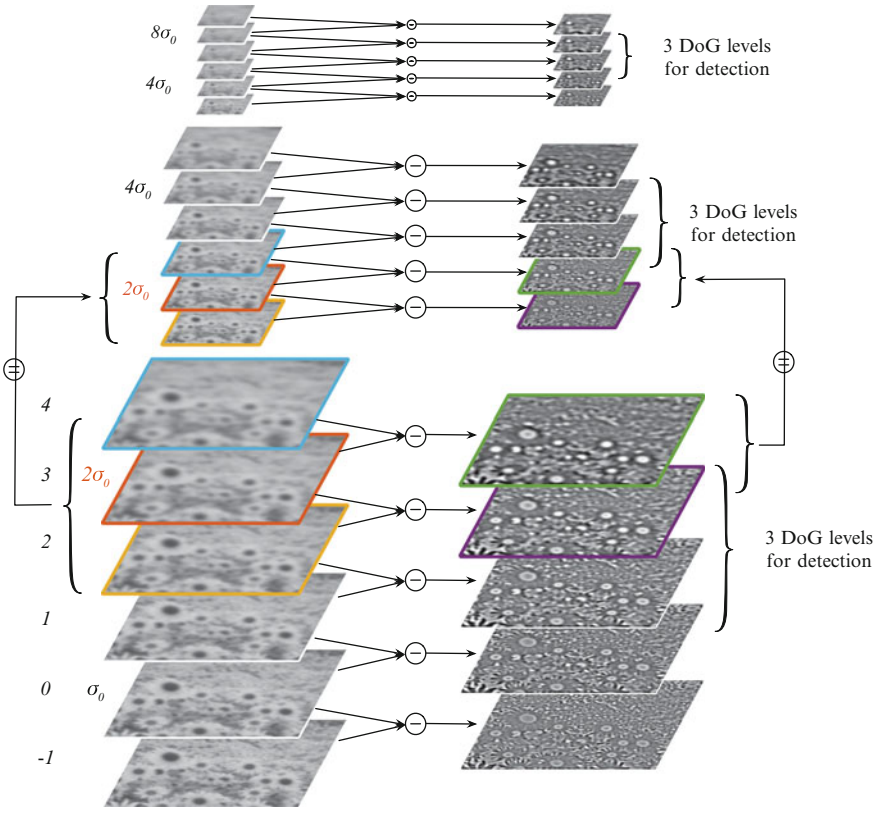


Fig. 11.13 SIFT Gaussian pyramid with three levels per octave. Actually, in order to search for extremal points on three levels of DoG for each octave, there must be five DoGs (right pyramid) per octave (to always have one above and one below) and consequently, there must be six filtered images (left pyramid) per octave. Note that the last three images of an octave correspond to the first three of the upper octave (subsampled). DoG images are equalised to improve intelligibility. Images courtesy of R. Toldo

$$\mathcal{H}_D = \begin{bmatrix} D_{uu} & D_{uv} \\ D_{uv} & D_{vv} \end{bmatrix} \tag{11.43}$$

calculated at the location and scale of the feature point.

In particular, the two principal curvatures are the eigenvalues of $\mathcal{H}_D$, which are therefore both required to be large. As with the Harris-Stephens operator, this criterion can be reformulated in terms of the trace and determinant of the matrix $\mathcal{H}_D$ to make the calculation more efficient:

$$\frac{\det \mathcal{H}_D}{\text{trace}^2 \mathcal{H}_D} = \frac{D_{uu} D_{vv} - D_{uv}^2}{(D_{uu} + D_{vv})^2} \geq \frac{r}{(r+1)^2} \tag{11.44}$$

where $r \geq 1$ denotes the allowed upper bound on the ratio of maximum to minimum eigenvalue (SIFT uses $r = 10$).

Finally, in order to increase the localisation accuracy in space and scale, a second-degree polynomial is fitted to the values in the neighbourhood of the extremal point and its maximum is taken, which in general will correspond to a sub-pixel (and sub-scale) position.

The relationship between Hessian matrix and structure tensor. The Hessian matrix of a given point of image I is

$$H = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix}$$

with $\mathrm{tr}(H) = \nabla^2$ being the Laplacian.

The Hessian matrix contains information on the curvature of the function. The two main curvatures (maximum and minimum) are the two eigenvalues λ_1, λ_2 of H, and since a feature point correspond to peak of the function, both should be large. There are several indicators of this condition; one of these is the Gaussian curvature $K = \det(H) = \lambda_1 \lambda_2$. The product is large when both factors are large; therefore, we are interested in maxima of K. The criterion is reminiscent of the Harris and Stephens operator, which however applies to the structure tensor S, which is not the same as H. In fact, barring the convolution with the low-pass kernel, the structure tensor (or second-moment matrix) writes

$$S = \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} = \nabla I \nabla I^\mathsf{T}.$$

The entries of S are product of first-order derivatives, whereas the entries of H are second-order derivatives, so they are different.

Nevertheless, a relationship exists, for one can regard $\nabla I \nabla I^\mathsf{T}$ as an approximation of H in the same way as $J J^\mathsf{T} \simeq H$ in the derivation of the Gauss-Newton algorithm.

11.5.3 SIFT Descriptor

Each SIFT feature point is associated with a descriptor, which essentially reduces to a histogram of the gradient direction in a neighbourhood. In principle, to achieve scale invariance, the neighbourhood size must be appropriately normalised by

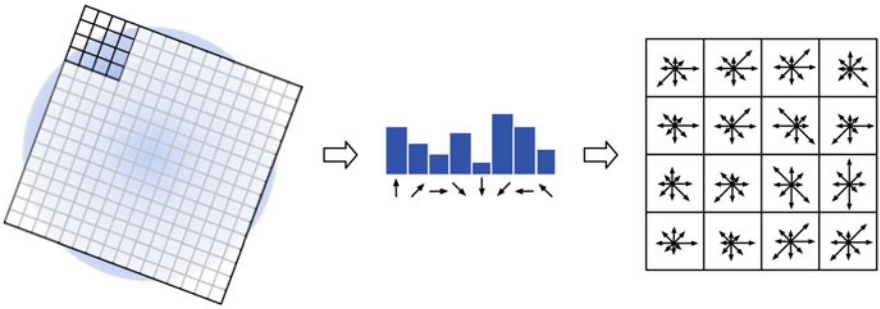


Fig. 11.14 SIFT descriptor. In each 4 × 4 quadrant of the 16 × 16 window, a histogram of the eight-bin gradient directions is computed

making it scale dependent. Equivalently, SIFT considers a window of fixed size 16 × 16 in the level of the Gaussian pyramid where the point was detected.

To achieve invariance to rotation, a dominant direction of the gradients in the 16 × 16 window is estimated. Specifically, a histogram of the direction of the gradients discretised into 36 bins is constructed where votes are weighted by the magnitude of the gradient and by a Gaussian centred on the window. The maximum of the histogram identifies the dominant direction. This value is refined by parabolic interpolation with the two neighbouring bins to improve the 10-degree resolution. Multiple dominant directions are accepted within 80% of the maximum. In this case one descriptor is produced for each direction.

To construct the descriptor (Fig. 11.14) the 16 × 16 window is virtually rotated to align it with the dominant direction, by adding a constant angle to the gradients. In each 4 × 4 sub-quadrant of the first, an eight-bin histogram of the gradient directions is computed, weighting the votes by the magnitude of the gradient itself. The Gaussian-weighted gradients used in the computation of the dominant direction are considered also in this step, to gradually reduce their contribution as they move away from the centre of the 16 × 16 window.

The resulting 128 non-negative values are collected into a vector that is normalised (to unit length) to make the descriptor invariant to changes in contrast, while additive variations in grey levels are already removed by the gradient. Finally, the values below 0.2 are set to zero and the vector is normalised again to obtain the SIFT descriptor.

11.5.4 Matching

Descriptors associated with feature points in two images are matched using a simple *closest neighbour* strategy: for each descriptor in one image, we search for the descriptor in the other image that minimises the Euclidean distance in the 128-dimensional space.

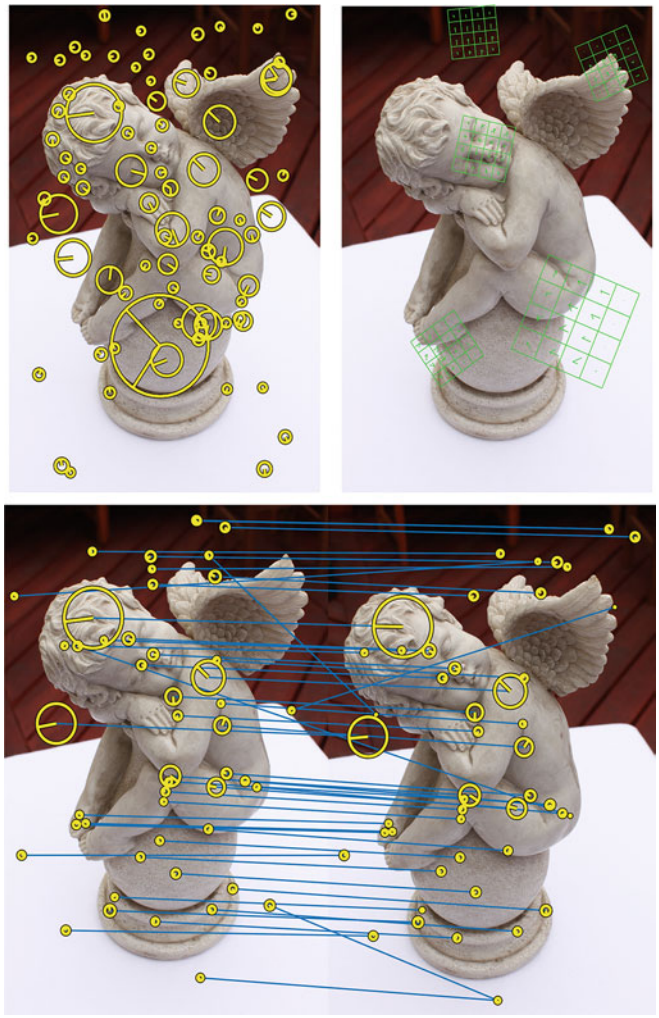


Fig. 11.15 Top left: test image with a subset of 50 SIFT points highlighted (the radius of the circle indicates scale while the segment represents the orientation). Top right: some descriptors are represented as 4×4 grids. Bottom: a subset of matches

To suppress matching that can be considered ambiguous, pairs for which the ratio of the distance of the nearest to the second-nearest descriptor is above a certain threshold (typically between 0.6 and 0.8) are rejected. Figure 11.15 shows an example of SIFT point matching.

The images in this chapter were created using the SIFT implementation by A. Vedaldi (http://www.vlfeat.org). See also https://www.vlfeat.org/api/sift.html for detailed documentation on the implementation.

References

Y. Bar-Shalom and T. E. Fortmann. *Tracking and data Association*. Academic Press, Waltham, MA, 1988.

Lionel Gueguen and Martino Pesaresi. Multi scale Harris corner detector based on differential morphological decomposition. *Pattern Recognition Letters*, 32: 1714–1719, 10 2011. https://doi.org/10.1016/j.patrec.2011.07.021.

C. Harris and M. Stephens. A combined corner and edge detector. *Proceedings of the 4th Alvey Vision Conference*, pages 189–192, August 1988.

T. Lindeberg. Scale invariant feature transform. *Scholarpedia*, 7 (5): 10491, 2012.

Tony Lindeberg. Feature detection with automatic scale selection. *International Journal of Computer Vision*, 30: 79–116, 1998.

David G. Lowe. Distinctive image features from scale-invariant keypoints. *International Journal of Computer Vision*, 60 (2): 91–110, 2004.

B. D. Lucas and T. Kanade. An iterative image registration technique with an application to stereo vision. In *Proceedings of the International Joint Conference on Artificial Intelligence*, pages 674–679, 1981.

D. Marr and E. Hildreth. Theory of edge detection. *Proceedings of the Royal Society of London, Series B*, 207: 187–217, February 1980.

Krystian Mikolajczyk and Cordelia Schmid. A performance evaluation of local descriptors. *IEEE Transactions on Pattern Analysis & Machine Intelligence*, 27 (10): 1615–1630, 2005.

Krystian Mikolajczyk, Tinne Tuytelaars, Cordelia Schmid, Andrew Zisserman, J. Matas, F. Schaffalitzky, T. Kadir, and L. Van Gool. A comparison of affine region detectors. *International Journal of Computer Vision*, 65 (1/2): 43–72, 2005.

J.A. Noble. Finding corners. *Image and Vision Computing*, 6: 121–128, May 1988.

J. Shi and C. Tomasi. Good features to track. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 593–600, June 1994.

C. Tomasi and T. Kanade. Detection and tracking of point features. Technical Report CMU-CS-91-132, Carnegie Mellon University, Pittsburg, PA, April 1991.

M. Zuliani, C. Kenney, and B.S. Manjunath. A mathematical comparison of point detectors. In *2004 Conference on Computer Vision and Pattern Recognition Workshop*, pages 172–172, 2004. https://doi.org/10.1109/CVPR.2004.282.

Chapter 12
Stereopsis: Matching



12.1 Introduction

Having already discussed the geometric aspect of stereopsis in Chap. 10, we will focus here on *stereo matching*, an important technique used in computer vision for finding correspondences between two images of a scene taken from different perspectives. The goal is to establish a one-to-one correspondence between pixels in the two images, which can be customarily represented as a *disparity map* (Fig. 12.1).

For the reader's convenience, we are providing here some definitions given in the previous chapters. A *homologous pair* consists of two points in two different images that are projections of the same object point. The binocular *disparity* is the (vector) difference between two homologous points, assuming that the two images are superimposed.

12.2 Constraints and Ambiguities

The matching of homologous points is made possible by the assumption that, locally, the two images differ only slightly, so that a detail of the scene appears similar in the two images (Sect. 12.1). However, based on this similarity constraint alone, many false matchings are possible, thus necessitating additional constraints to avoid them.

The most important of these is the *epipolar constraint* (Chap. 6), which states that the homologous of a point in one image is found on the epipolar line in the other image. Thanks to this, the search for correspondences becomes one-dimensional, instead of two-dimensional.

We assume, without loss of generality, that the images are rectified (Sect. 10.3), meaning that the epipolar lines are parallel and horizontal in both images. This allows us to search for homologous points along horizontal scan lines of the

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_12



Fig. 12.1 Pair of stereo images (rectified) and ideal disparity map (images taken from http://vision.middlebury.edu/stereo/)

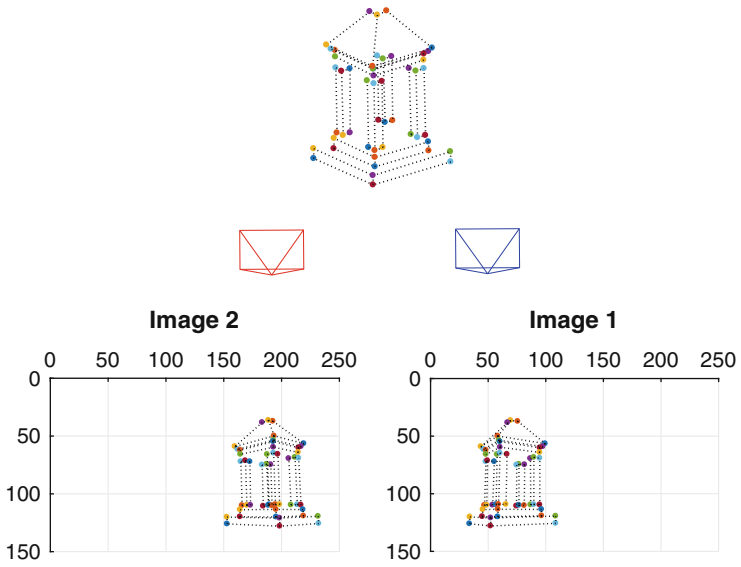


Fig. 12.2 Cameras in normal configuration and corresponding rectified images

same height. The disparity is then reduced to a scalar value, which can be easily represented in a *disparity map* (see Fig. 12.1). In each pixel of the map, the horizontal translation between the same point of the reference image and its homologous point is recorded.

Consider the example in Fig. 12.2: moving from image 1 (right camera) to image 2 (left camera), the horizontal coordinates of the points in image 2 are greater than those of the corresponding points in image 1. It follows that the disparity, defined as $\mathbf{m}_2 - \mathbf{m}_1$, is positive in this case. When computing disparity we will always assume that it is positive, so we will have to identify images 1 and 2 so that this assumption is verified.

In addition to the similarity constraint and the epipolar constraint, there are other constraints that can be exploited in the calculation of correspondences:

smoothness: the scene is composed of regular surfaces, so the disparity is almost everywhere smooth (transitions between different objects are excluded);

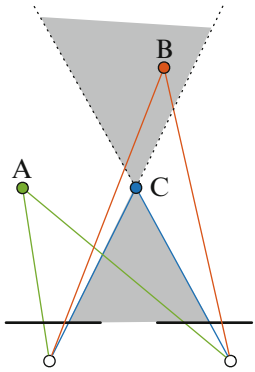


Fig. 12.3 Point B violates the ordering constraint with respect to C, while point A fulfil it. The grey cone is the forbidden zone of C

uniqueness: a point in one image can be matched with only one point in the other image, and vice versa (fails with transparent objects or occlusions);

ordering: if the point $\mathbf{m}_1$ in one image matches $\mathbf{m}'_1$ in the other, the homologous of a point $\mathbf{m}_2$ lying to the right (left) of $\mathbf{m}_1$ must lie to the right (left) of $\mathbf{m}'_1$. It fails for points that lie in the "forbidden zone" of a given point (corresponding to the grey area in Fig. 12.3). Normally, for an opaque, cohesive surface, points in the forbidden zone are not visible, so the constraint holds.

The constraints just listed, however, are not sufficient to make the matching unambiguous. Even if they are fulfilled, a point in one image can be matched with many points in the other image: this is the problem of *false matches*. In addition to this, there are other problems that plague the calculation of correspondences, due to the fact that the scene is framed from two different points of view:

occlusions: due to discontinuities in the surfaces, there are parts of the scene that appear in only one of the images, that is, there are points in one image that do not have a corresponding in the other image. Clearly no disparity can be defined for such points;

non-Lambertian surfaces: due to surfaces violating the Lambertian assumption (see Chap. 2), the intensity observed by the two cameras (the *radiance*) is different for the same point in the scene;

perspective distortion: perspective projections of geometric shapes are different in the two images.

All these problems are exacerbated the more the cameras are apart. On the other hand, in order to have a meaningful disparity, the cameras *must* be well separated from each other.

All matching methods attempt to pair pixels in one image with pixels in the other image by taking advantage of the constraints listed above and trying to work around the problems just mentioned. *Local* methods consider only a small window surrounding the pixel to be matched, while *global* methods impose constraints on the entire image.

In addition to the local/global dichotomy we introduced, it is possible to classify more finely stereo matching methods, according to a scheme introduced by Scharstein and Szeliski (2002) that uses four dimensions corresponding to as many phases of most algorithms:

- computation of the matching cost (SSD, SAD, NCC, etc.).
- cost aggregation (square window, adaptive window, etc.)
- disparity calculation (WTA, SO, SGM, Graph-cut, etc.).
- disparity refinement (sub-pixel, occlusion detection, etc.).

12.3 Local Methods

The methods of this class, also called *block matching* or *correlation based* , compute an aggregate matching cost on a local support Ω, called *window*, and determine *for each pixel* the disparity that yields the lowest cost. This cost is a measure of difference or dissimilarity between Ω and the underlying image.

In more detail, a small area of one image is considered, and the most similar area in the other image is identified by minimising a matching cost that depends on the grey levels, or a function of them (see Fig. 12.4). This process is repeated for each point, resulting in a dense disparity map.

In formulae, for each pixel (u, v) in image I_1, let us consider a window Ω centred in (u, v) of size $(2N + 1)(2N + 1)$. This is compared with a window of the same size in I_2 moving along the epipolar line corresponding to (u, v); since the images are rectified, we consider the positions $(u + d, v), d \in [d_{\min}, d_{\max}]$. Let $c(u, v, d)$



Fig. 12.4 Illustration of the *block matching* method. A rectangular window Ω is (ideally) cropped in the reference image and slid over the other image along the scan line until it finds the translation value d that yields the maximum similarity between the window and the underlying image

be the value of the resulting matching cost; the computed disparity for (u, v) is the displacement d_o that corresponds to the minimum cost c:

$$d_o(u, v) = \arg\min_d c(u, v, d). \tag{12.1}$$

12.3.1 Matching Cost

The matching costs we will analyse in this section fall into three categories:

- based on correlation (NCC, ZNCC);
- based on intensity differences (SSD, SAD);
- based on transformations of intensities (e.g. census transform).

One of the most common, especially in photogrammetry, is Normalised Cross Correlation (NCC). It can be seen as the scalar product of the two vectorised windows divided by the product of their respective norms:

$$\mathrm{NCC}(u, v, d) = \frac{\sum_{(k,l)\in\Omega} I_1(u+k, v+l) I_2(u+k+d, v+l)}{\sqrt{\sum_{(k,l)\in\Omega} I_1(u+k, v+l)^2} \sqrt{\sum_{(k,l)\in\Omega} I_2(u+k+d, v+l)^2}}. \tag{12.2}$$

where $I(u, v)$ denotes the grey level of the pixel (u, v). Be aware that NCC is not actually a cost, being instead a similarity measure between 0 and 1. To convert it to a cost, just take 1-NCC. The MATLAB implementation is given in Listing 12.1.

To obtain invariance to brightness changes between the two images (of additive type), one can subtract from each window its average, obtaining the Zero-mean NCC or ZNCC.

Listing 12.1 Stereo with NCC

```
function [dmap,cost]=imstereo_ncc(imL,imR,drange,ws)
    %IMSTEREO_NCC Stereo block-matching with NCC

    dmin=drange(1); dmax=drange(2);
    s1 = filter2(ones(ws),imL.^2);
    ncc = ones([size(imL),dmax-dmin+1])*Inf;
    for d=0:dmax-dmin
        imR_d = circshift(imR,[0, -(dmin+d)]);
        imR_d(:, end-(dmin+d):end) = 0;
        prod =  imL.*imR_d; % product
        s2 = filter2(ones(ws),imR_d.^2);
        ncc(:,:,d+1) = 1-(filter2(ones(ws),prod)./sqrt(s1.*s2));
    end
    [cost,dmap]=min(ncc,[],3);
    dmap=dmap+dmin-1;
end
```

Another cost, popular especially in computer vision, is the Sum of Squared Difference (SSD):

$$\mathrm{SSD}(u, v, d) = \sum_{(k,l)\in\Omega} (I_1(u+k, v+l) - I_2(u+k+d, v+l))^2 \tag{12.3}$$

It can be seen as the squared norm of the difference of the vectorised windows.

The smaller the value of (12.3), the more similar the portions of the images considered. The MATLAB implementation is given in Listing 12.2.

Listing 12.2 Stereo with SSD

```
function [dmap,cost] = imstereo_ssd(imL,imR,drange,ws)
    %IMSTEREO_SSD Stereo block-matching with normalized SSD

    dmin=drange(1); dmax=drange(2);
    s1 = filter2(ones(ws),imL.^2);
    ssd = ones([size(imL),dmax-dmin+1])*Inf;
    for d = 0:dmax-dmin
        imR_d = circshift(imR,[0, -(dmin+d)]); % shift
        imR_d(:, end-(dmin+d):end) = 0;
        sd = (imL-imR_d).^2; % squared differences
        s2 = filter2(ones(ws),imR_d.^2);
        ssd(:,:,d+1) = filter2(ones(ws),sd)./sqrt(s1.*s2); % SSD
    end
    [cost,dmap]=min(ssd,[],3);
    dmap=dmap+dmin-1;
end
```

Similar to SSD is Sum of Absolute Difference (SAD), where the square is replaced by the absolute value. In this way the cost is less sensitive to impulsive noise: for example, two windows that are equal in all but one pixel are more similar according to SAD than according to SSD, since the square weighs much more the differences than the absolute value:

$$\mathrm{SAD}(u, v, d) = \sum_{(k,l)\in\Omega} |I_1(u+k, v+l) - I_2(u+k+d, v+l)|. \tag{12.4}$$

Following the same line of reasoning, one could replace the absolute value with a more robust penalty function, such as an M-estimator (Sect. C.1). This class includes, for example, truncated cost functions proposed by (Scharstein and Szeliski 2002).

12.3.2 Census Transform

We now look at a more sophisticated technique, in which first a transformation based on local sorting of grey levels is applied to the images and then the similarity of the windows on the transformed images is measured.

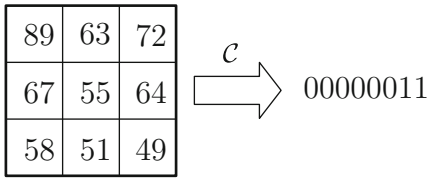


Fig. 12.5 Example of census transform with $\rho = 1$

The *census transform* (Zabih and Woodfill 1994) is based on the comparison of intensities. Let $I(p)$ and $I(p')$ be the intensity values of pixels p and p', respectively.

If we denote the concatenation of bits by the symbol $\odot$, the census transform for a pixel p in the image I is the bit string:

$$\mathcal{C}I(p) = \bigodot_{p' \in \mathcal{W}(p,\rho)} [\![I(p) > I(p')]\!] \tag{12.5}$$

where $\mathcal{W}(p, \rho)$ denotes a window centred in p of radius ρ and $[\![]\!]$ are the Iverson parenthesis.

The census transform summarises the local spatial structure. In fact, it associates with a window a bit string that encodes its intensity in relation to the central pixel, as exemplified in Fig. 12.5.

The matching takes place between windows of the transformed images, comparing strings of bits. By denoting with the symbol $\ominus$ the Hamming distance between two bit strings, that is, the number of bits in which they differ, the SCH (Sum of Census Hamming Distances) matching cost is written:

$$\mathrm{SCH}(u, v, d) = \sum_{(k,l) \in \Omega} \mathcal{C}I_1(u + k, v + l) \ominus \mathcal{C}I_2(u + k + d, v + l). \tag{12.6}$$

Each term of the summation is the number of pixels inside the transformation window $\mathcal{W}(p, \rho)$, whose relative order (i.e. having higher or lower intensity) with respect to the considered pixel changes from I_1 to I_2.

This method is invariant to any monotonic transformation of intensities, whether linear or not. In addition, this method is tolerant to errors due to occlusions. In fact, the study of Hirschmuller and Scharstein (2007) identifies SCH as the best matching cost for stereo matching.

The MATLAB implementation is reported in Listing 12.3.

The census transform is also efficiently computable: the basic operations are simple integer comparisons, thus avoiding floating point operations or even integer multiplications. The computation is purely local and the same function is evaluated in every pixel, so it can be applied in parallel.

In Fig. 12.6 we show, as an example, the disparity maps obtained with SSD, NCC and SCH on the pair in Fig. 12.1.

Listing 12.3 Stereo with SCH

```
function [dmap,cost] = imstereo_sch(imL,imR,drange,ws)
    %IMSTEREO_SCH Stereo block-matching with SCH (Census)

    % compute Census transform 5x5
    fun = @(x) bin2dec(num2str(x(:)'>x(13)));
    imL = uint32(nlfilter(imL,[5 5],fun));
    imR = uint32(nlfilter(imR,[5 5],fun));

    dmin=drange(1); dmax=drange(2);
    sch=ones([size(imL),dmax-dmin+1])*Inf;
    for d = 0:dmax-dmin
        imR_d = circshift(imR,[0, -(dmin+d)]); % shift
        imR_d(:, end-(dmin+d):end) = 0;
        z = bitxor(imL, imR_d);
        ch = 0; % count the "1" in the xor
        for i=1:32 % 32 bits
            ch = ch + bitget(z,i);
        end
        sch(:,:,d+1)=filter2(ones(ws),ch);
    end
    [cost,dmap]=min(sch,[],3);
    dmap=dmap+dmin-1;
end

%% Warning: SCH needs 'nlfilter' from the Image Processing Toolbox
```

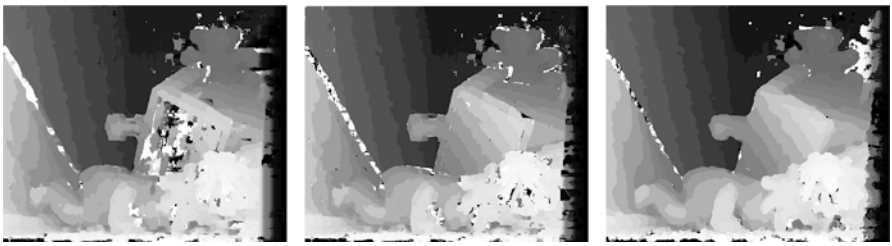


Fig. 12.6 Disparity maps produced by SSD, NCC and SCH (left to right) with window 9×9

These block matching methods inevitably fail in the presence of uniform areas or repetitive structures because in both cases the considered window does not have a unique match in the other image. They also implicitly assume that all pixels belonging to the window Ω have the same disparity (i.e. the same distance from the focal plane). This implies that the surfaces of the scene must be oriented like the focal plane. Tilted surfaces and depth discontinuities lead to mismatched portions of the image being included in Ω, and thus the calculated disparity will be fatally wrong.

Robust cost functions can partially mitigate this problem by treating samples of one of the two surface as outliers; better still can do non-parametric cost functions like SCH. However, the problem can be addressed more systematically by revising the cost aggregation step: changing the shape, size and weighting of the support Ω.

12.4 Adaptive Support

We observed that if the window Ω covers a region where the depth varies, the computed disparity will inevitably be affected by error, since there is no *one* disparity that can be attributed to *all* the support. Ideally, therefore, one would like Ω to include only points with the same disparity, but this disparity is unknown when we set out to compute it.

Reducing the window size helps mitigate the problem; however, this choice makes the matching problem more ambiguous, especially when dealing with uniform regions and repetitive patterns.

To appreciate the phenomenon in a controlled experiment, we consider the random-dots stereogram of Fig. 12.7, consisting of a synthetic stereo pair obtained in the following way: a background image is generated by assigning random intensity values to the pixels. Using the same method, we generate a square (or a region of any shape). Then, we copy the square over the background in two different positions that differ by a horizontal translation. The pair thus constructed has a horizontal disparity: observing one image with the right eye and the other with the left eye, it is possible to perceive the square as it was in the foreground.

Figure 12.8 shows the results of applying the *block matching* algorithm with different window sizes to the random-dots stereogram. It can be seen that a small window yields more accurate disparity on edges but with random errors (less reliable), while large windows remove random errors but introduce systematic errors at disparity discontinuities (less accurate). Thus, we are in the presence of two opposing demands for simultaneous reliability and accuracy. An ideal cost aggregation strategy should have a support that includes only points at the same depth—which is not known—and extends as far as possible to maximise the signal (intensity variation) to noise ratio.

Several solutions have been proposed to adapt the shape and size of the support to meet the two requirements.

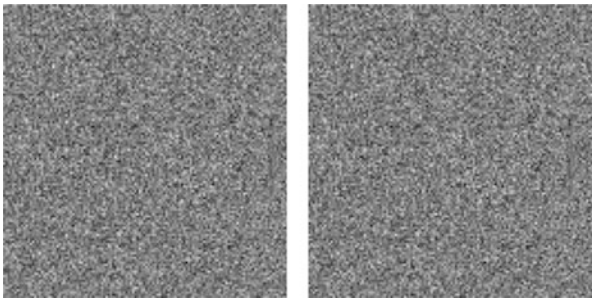


Fig. 12.7 Random-dots stereogram. In the right image, a central square is translated to the right by five pixels



Fig. 12.8 Disparity maps obtained with SSD correlation on a random-dots stereogram with Gaussian noise $\sigma^2 = 10.0$ added. The grey level—normalised—represents the disparity. The correlation window sizes are, from left to right, 3×3, 7×7 and 11×11

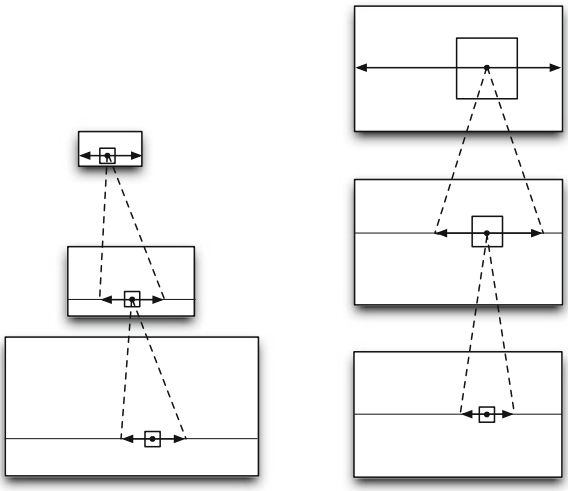


Fig. 12.9 Coarse-to-fine (left) and fine-to-fine (right) methods

12.4.1 Multiresolution Stereo Matching

Multiresolution stereo matching address the problem of window size (but not shape). They are hierarchical methods that operate at different resolutions. The idea is that at the coarse level large windows provide an inaccurate but reliable result. At the finer levels, smaller windows and smaller search intervals improve accuracy. We distinguish two techniques (Fig. 12.9):

coarse-to-fine: the search interval is the same but operates on images at gradually increasing resolutions. The disparity obtained at one level is used as the center for the interval at the higher level; the disparity obtained at one level is used as the center for the interval at the next level;

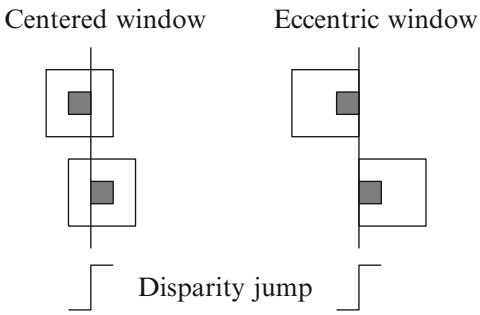


Fig. 12.10 An eccentric window can cover a constant disparity zone even near a disparity jump

fine-to-fine: always operates on the same image but with smaller windows and intervals. As before, the disparity at one level is used as the center for the interval at the next level.

12.4.2 Adaptive Windows

In this category we find methods that adapt window shape and/or size based on the image content. The ancestor of this class is the method proposed by Kanade and Okutomi (1994), which implies a window whose size is locally selected based on the signal-to-noise ratio and disparity variation. The ideal window should include as much intensity variation and as little disparity variation as possible. Since the disparity is initially unknown, we start with an estimate obtained with a 3×3 fixed window and iterate, approximating at each step the optimal window for each point, until convergence (if any) is achieved. Building on this, a simpler algorithm based on fixed size but eccentric windows was proposed by Fusiello et al. (1997). Nine windows are employed for each point, each with the center at a different location (Fig. 12.10). The window among the nine with the smallest SSD is the one most likely to cover the area with the least variation in disparity.

It is possible to improve the efficiency of this scheme by not computing the matching cost nine times. Instead, the costs for eccentric windows can be computed using the costs of the neighbouring points calculated with the centred support. By relaxing the disparity of the point in its neighbourhood, the disparity with the lowest matching cost is selected.

This method does not adjust the window size, which is assumed to be set to a value that provide a sufficient signal-to-noise ratio. Veksler (2003) adds a strategy to adjust the window size, while Hirschmüller et al. (2002) introduce a scheme that adapts the shape by assembling the support Ω from the aggregation of small square windows with the lowest matching score.

Other strategies include segmenting the image into regions of similar intensity (assuming that the depth in these regions is also nearly constant) and intersecting

the window with the region to which the pixel belongs, so as to obtain an irregularly shaped support (Gerrits and Bekaert 2006). Similar to this strategy is the one proposed by Kweon (2005) that assigns a weight to the pixels of the support based on a segmentation criterion (as in the bilateral filtering illustrated in Chap. 11).

12.5 Global Matching

We have observed how cost aggregation near depth discontinuities is a source of error. An orthogonal approach to window adaptation is to forgo aggregating costs altogether. However, this loses reliability, since the individual pixels do not contain enough information for an unambiguous match, and thus constraints must be added to regularise the problem. This results in the global optimisation of an objective function that includes the matching cost and a penalty for discontinuities.

These methods, generically called *global*, can be well understood by referring to the so-called Disparity Space Image (DSI). This is a three-dimensional image (a volume) c, where $c(u, v, d)$ is the value of the matching cost between the pixel (u, v) in the first image and the pixel $(u, v+d)$ in the second image. In principle, the cost is computed pixel-wise, meaning that the support reduces to one pixel, although a small Ω can sometimes be used.

The disparity map we expect the algorithm to produce can be seen as a surface inside the DSI, described by a function $d = d(u, v)$ (Fig. 12.11). In this formulation, we look for the disparity map $d(u, v)$ that minimises an objective function $E(d)$:

$$E(d) = \sum_{\mathbf{m}} c(\mathbf{m}, d(\mathbf{m})) + \sum_{\mathbf{m},\mathbf{q}\in\mathcal{N}_{\mathbf{m}}} V(d(\mathbf{m}), d(\mathbf{q})) \tag{12.7}$$

where $\mathcal{N}_{\mathbf{m}}$ denotes a neighbourhood of $\mathbf{m}$. The first term of the function sums all pixel matching costs over the entire image, while the second term V adds a penalty for all pixels with neighbours that have a different disparity; in the simplest case it could be

$$V(d, d') = C_1 [\![|d - d'| \geq 1]\!]. \tag{12.8}$$

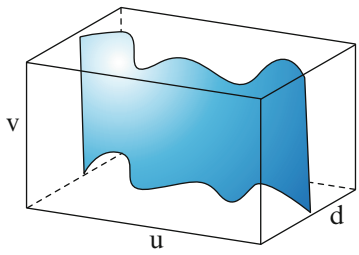


Fig. 12.11 The disparity map as a surface in DSI

In this way, discontinuities are allowed if the pixel match is stronger than the penalty C_1, that is, if the signal strongly indicates a discontinuity.

Note that the second term links all pixels in the image, making the problem a global one. If this term were omitted, the sum of the (positive) costs would be minimised by a local strategy—known as Winner Takes All (WTA)—which consist in taking the lowest cost for each pixel. This is exactly what is applied in our MATLAB implementation of the local methods reported in the previous section (plus the aggregation of the cost over Ω).

Among the best methods in this area are those based on the minimum graph cut (Roy and Cox 1998; Kolmogorov and Zabih 2001). In a nutshell, they create a flow network, where nodes correspond to cells of the DSI and arcs connect adjacent cells, with an associated capacity that is a function of the costs of the incident cells. The minimum cost cut represents the sought surface. The disadvantage of these and many other global methods is the high computational cost in both time and memory consumption.

Although the problem, as we have formulated it, is inherently two-dimensional (the solution is a surface), a compromise to reduce the computational cost is to decompose it into many independent (simpler) one-dimensional problems.

Scanline Optimisation (SO) operates on individual $u-d$ sections of the DSI (i.e. on scanlines $v = \text{cost}$) and optimises one scanline at a time independently of the others. A disparity value d is assigned at each point u such that the overall cost along the scanline is minimised with respect to a cost function that incorporates the matching cost and the discontinuity penalty. Therefore, a SO algorithm optimises the cost function 12.7 with the only difference being that the neighbourhood $\mathcal{N}$ is one-dimensional (it only extends horizontally) and thus vertical discontinuities are not penalised. If the horizontal discontinuities were not penalised as well, this would be equivalent to forgo the regularisation term and would result in a WTA, as we have already observed.

In this class there are algorithms that operate in $u-d$ sections of the DSI (Intille and Bobick 1994; Hirschmuller 2005) or in the so-called match space, the matrix containing the costs of each pixel pair of two corresponding scanlines (Cox et al. 1996; Ohta and Kanade 1985), as illustrated in Fig. 12.12.

In both cases, it is a matter of computing a minimum cost path through a cost matrix, and *dynamic programming* has been shown to be particularly well suited for this task. However, since the optimisation is performed independently along the horizontal scanlines, horizontal artefacts in the form of *streakings* are present. Several authors add other penalties to the cost function for violating, e.g. uniqueness or ordering constraints.

A compromise between global methods, which undoubtedly produce better results (Fig. 12.13) but at high computational cost, and those operating on a single scanline is the Semi Global Matching (SGM)) of Hirschmuller (2005) which can be regarded as a variation of SO that considers many different scanlines instead of just one (Fig. 12.14). The method minimises a one-dimensional cost function over n (typically $n = 8$) sections of the DSI along the cardinal directions of the image (the horizontal one corresponds to the $u-d$ section of the SO methods described

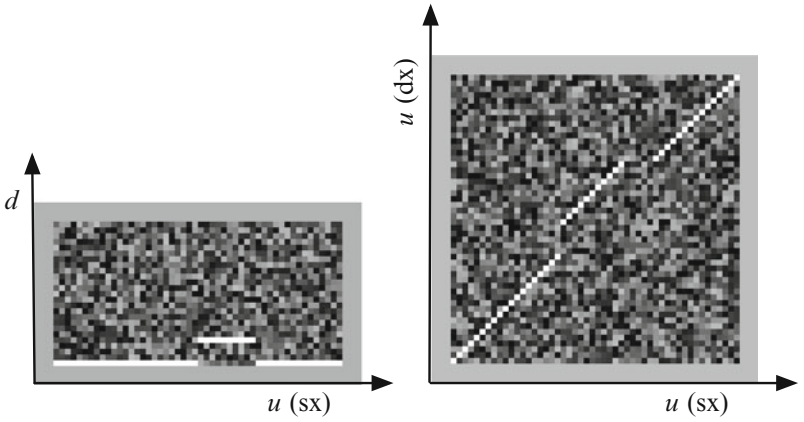


Fig. 12.12 Idealised cost matrices for the random point stereogram. Section (u, d) of the DSI (left) and match space. Intensities represent cost, with white corresponding to the minimum cost. Choosing a point in the matrix is equivalent to setting a match

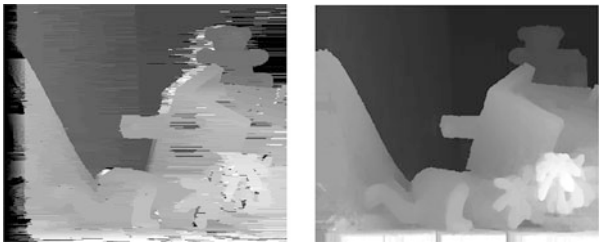


Fig. 12.13 Disparity maps produced by a SO algorithm (left) and SGM (right). Note the characteristic streaks in the left map

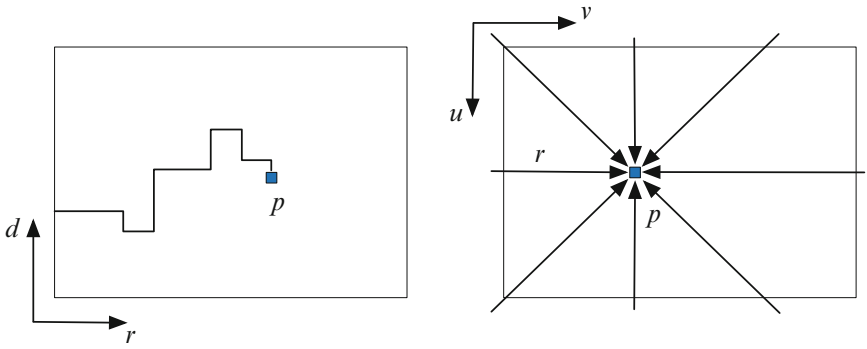


Fig. 12.14 To the left, section of the DSI along the $\mathbf{r} - d$ plane; to the right, the eight scan lines considered for one pixel

above). This way of proceeding can be seen as an approximation of the optimisation of the global cost function.

In addition, the semi-global method introduces a specific cost $C_2 < C_1$ function to penalise differently small disparity jumps, which are often part of sloped surfaces, and real discontinuities of surfaces:

$$V(d, d') = C_1[\![|d - d'| > 1]\!] + C_2[\![|d - d'| = 1]\!]. \tag{12.9}$$

SGM computes, for each direction $\mathbf{r}$, an aggregate cost $L_{\mathbf{r}}(u, v, d)$ defined recursively as follows (starting from the borders):

$$L_{\mathbf{r}}(\mathbf{m}, d) = c(\mathbf{m}, d) + \min_{d'}(L_{\mathbf{r}}(\mathbf{m} - \mathbf{r}, d') + V(d, d')) \tag{12.10}$$

For each pixel and each disparity, the costs are summed over the eight paths, resulting in an aggregate cost volume:

$$L(\mathbf{m}, d) = L_{\mathbf{r}}(\mathbf{m}, d) \tag{12.11}$$

in which per-pixel minima (as in WTA) are chosen as the computed disparity.

Hirschmuller (2005) described SGM in conjunction to a matching cost based on mutual information; however, any cost can be plugged in. This algorithm achieves an excellent trade-off between result quality and execution time, as well as being suitable for parallel implementations in hardware. Therefore, it has been widely adopted in application contexts such as robotics and driver assistance systems, where there are real-time constraints and limited computational capabilities.

When the two images of a stereo pair have different lighting conditions, it is necessary to normalise them by matching their respective histograms, that is, to transform one histogram so that it is as similar as possible to the other.

Let F_X and F_Y be cumulative histogram (counts the total number of pixels in all of the bins up to the current bin) of images X and Y, respectively. Let us assume that $Y = g(X)$ where g is an unknown map that we would like to retrieve.

The functions F_Y^{-1} and F_X^{-1} are the *quantile functions*, that is, $F^{-1}(\alpha)$ is the $\alpha - th$ quantile of F. If g is monotone increasing, then $F_Y(y) = F_X(g^{-1}(y))$, and also $F_Y^{-1}(u) = g(F_X^{-1}(u))$. This means that the quantile functions transform according to g; hence, the graph of g is described by the pairs $(F_Y^{-1}(\alpha), F_X^{-1}(\alpha))$ for all $\alpha \in [0, 1]$. This graph, where the quantiles of two images are plotted one against the other, is called the *Q-Q plot*. This allows to find the exact transformation that makes the quantiles of X and Y to match. For example, if g is an affine map, the Q–Q plot will be a line.

12.6 Post-Processing

Downstream of the calculation of the "raw" disparity, there are several steps of optimisation and post-processing that can be applied. First, interpolating the cost function near the minimum (e.g. with a parabola) can yield a sub-pixel resolution corresponding to fractional values of disparity. Further, various image processing techniques such as median filtering, morphological operators and bilateral filtering (Chap. 11) can be used to reduce isolated pixels and make the map more regular, particularly if it has been created using non-global methods. In the following paragraphs, we will explore two important post-processing steps: the computation of reliability indicators and occlusion detection.

12.6.1 Reliability Indicators

The depth information provided by stereopsis is not everywhere equally reliable. In particular, there is no information for occlusion zones and uniform intensity (or non-textured) areas. This incomplete information can be integrated with information from other sensors, but then it needs to be accompanied by a reliability or confidence estimate, which plays a primary role in the integration process.

We now briefly account for the most popular confidence indicators; more details are found in (Hu and Mordohai 2012). In the following $c(d)$ denotes the matching cost, normalised in [0, 1], related to the disparity d (in the case of NCC we take 1-NCC). The minimum of the cost function is denoted by c_o and the corresponding disparity value by d_o. The second best cost is denoted by c_2, while the best second local minimum is denoted by $c_{2\mathrm{m}}$ (Fig. 12.15). The confidence $\phi(i)$ varies in [0, 1], where 0 means unreliable and 1 reliable.

Matching cost	$\phi_{\mathrm{MSM}} = 1 - c_o$
Curvature	$\phi_{\mathrm{CUR}} = \dfrac{2 + (-2c_o + c(d_o - 1) + c(d_o + 1))}{4}$
Peak Ratio	$\phi_{\mathrm{PKR}} = 1 - \dfrac{c_o}{c_{2\mathrm{m}}}$
Maximum Margin	$\phi_{\mathrm{MMN}} = \dfrac{c_2 - c_o}{c_2}$
Winner Margin	$\phi_{\mathrm{WMN}} = \dfrac{c_{2\mathrm{m}} - c_o}{\sum_d c(d)}$

12.6.2 Occlusion Detection

Occlusions generate points lacking homologous counterparts, and are related to depth discontinuities. For example, in one image of a random-dots stereogram, there

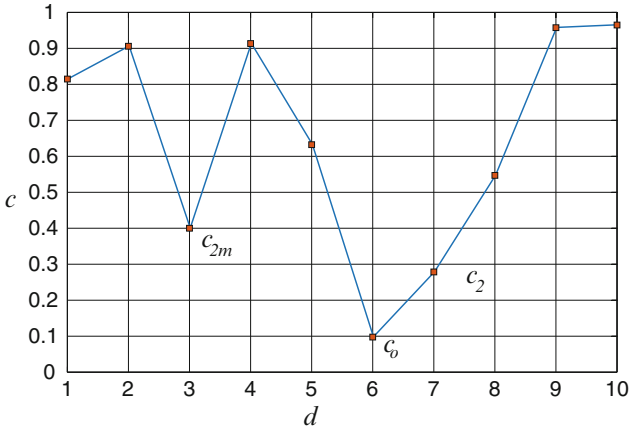


Fig. 12.15 Matching cost profile. In the example $d_o = 6$. Courtesy of F. Malapelle

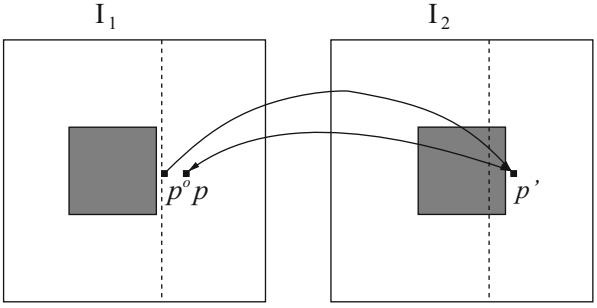


Fig. 12.16 Left-right consistency. The point p^o being occluded does not have a one-to-one correspondence with p', while p does

are two depth discontinuities along a horizontal scanline, located on the left and right edges of the square (see Fig. 12.12).

By definition, occlusions can be identified when points in one image fail to have corresponding homologous points in the other image. The matching procedure, however, is going to find a best match for any point, even if the pairing is weak. To detect occlusions one can analyse the quality of the matches and remove any matches that are likely to be inaccurate, using, e.g. the metrics introduced in the previous section. However, a more stringent detection can be implemented by exploiting the uniqueness constraint.

In the matching procedure, for each point in I_1, the corresponding point in I_2 is searched (see Fig. 12.16). If, for example, a portion of the scene is visible in I_1 but not in I_2, a pixel $p^o \in I_1$, whose homologous is occluded, will be paired with a certain pixel $p' \in I_2$ according to the matching cost employed. If the true

homologous of p' is $p \in I_1$—and assuming that the matching operates correctly—p is also matched to p', violating the uniqueness constraint. This can be easily detected by checking the injectivity of the disparity map.

The next problem is to determine which of the two potential homologous is the correct one, and this can be done by selecting the lowest matching cost (or the most reliable match, in general).

More effective[1] (but also more expensive) is the left-right consistency check (or bidirectional matching), which prescribes that if p is coupled to p' by performing the search from I_1 to I_2, then p' must be coupled to p by performing the search from I_2 to I_1. Thus, in the previous example, p' is paired with its true homologous p, so the point p^o can be recognised as occluded and left without a correspondent.

Eventually, for points without a correspondent, one can estimate a disparity by interpolation of neighbouring values, or leave them as they are.

Scharstein and Szeliski (2002) introduced a standard protocol for evaluating of stereo algorithms. The results can be found on the web http://vision.middlebury.edu/stereo/.

References

I. J. Cox, S. Hingorani, B. M. Maggs, and S. B. Rao. A maximum likelihood stereo algorithm. *Computer Vision and Image Understanding*, 63 (3): 542–567, May 1996.

L. Di Stefano, M. Marchionni, S. Mattoccia, and G. Neri. Quantitative evaluation of area-based stereo matching. In *7th International Conference on Control, Automation, Robotics and Vision, 2002. ICARCV 2002.*, volume 2, pages 1110–1114 vol.2, 2002.

A. Fusiello, V. Roberto, and E. Trucco. Efficient stereo with multiple windowing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 858–863, Puerto Rico, June 1997. IEEE Computer Society Press.

Mark Gerrits and Philippe Bekaert. Local stereo matching with segmentation-based outlier rejection. In *Proceedings of the The 3rd Canadian Conference on Computer and Robot Vision*, CRV '06, page 66, USA, 2006. IEEE Computer Society.

Heiko Hirschmüller, Peter R. Innocent, and Jonathan M. Garibaldi. Real-time correlation-based stereo vision with reduced border errors. *Int. J. Comput. Vis.*, 47 (1-3): 229–246, 2002.

H. Hirschmuller and D. Scharstein. Evaluation of cost functions for stereo matching. In *2007 IEEE Conference on Computer Vision and Pattern Recognition*, pages 1–8, 2007.

Heiko Hirschmuller. Accurate and efficient stereo processing by semi-global matching and mutual information. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 807–814, Washington, DC, USA, 2005. IEEE Computer Society.

X. Hu and P. Mordohai. A quantitative evaluation of confidence measures for stereo vision. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 34 (11): 2121–2133, Nov 2012.

[1] The two approaches are equivalent in the proposed example, but in more complex cases they are not, as pointed out by Di Stefano et al. (2002).

S. S. Intille and A. F. Bobick. Disparity-space images and large occlusion stereo. In Jan-Olof Eklundh, editor, *European Conference on Computer Vision*, pages 179–186. Springer, Berlin, May 1994.

T. Kanade and M. Okutomi. A stereo matching algorithm with an adaptive window: Theory and experiments. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16 (9): 920–932, September 1994.

Vladimir Kolmogorov and Ramin Zabih. Computing visual correspondence with occlusions using graph cuts. *Proceedings of the International Conference on Computer Vision*, 2: 508, 2001.

Kuk-Jin Yoon; In-So Kweon. Locally adaptive support-weight approach for visual correspondence search. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, volume 2, pages 924–931, 2005.

Y. Ohta and T. Kanade. Stereo by intra- and inter-scanline search using dynamic programming. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 7 (2): 139–154, March 1985.

Sébastien Roy and Ingemar J. Cox. A maximum-flow formulation of the n-camera stereo correspondence problem. In *Proceedings of the International Conference on Computer Vision*, page 492, Washington, DC, USA, 1998. IEEE Computer Society.

D. Scharstein and R. Szeliski. A taxonomy and evaluation of dense two-frame stereo correspondence algorithms. *International Journal of Computer Vision*, 47 (1): 7–42, May 2002.

Olga Veksler. Fast variable window for stereo correspondence using integral images. In *Proceedings of the 2003 IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, CVPR'03, page 556–561, USA, 2003. IEEE Computer Society.

R. Zabih and J. Woodfill. Non-parametric local transform for computing visual correspondence. In *Proceedings of the European Conference on Computer Vision*, volume 2, pages 151–158. Springer, Berlin, 1994.

Chapter 13
Range Sensors



13.1 Introduction

The recovery of a 3D model, or *3D shape acquisition*, can be achieved through a variety of approaches that do not necessarily involve cameras and images. These techniques include contact (probes), destructive (slicing), transmissive (tomography) and reflective non-optical (SONAR, RADAR) methods. Reflective optical techniques, which rely on back-scattered visible (including near infrared) electromagnetic radiation, have been the focus of our discussion so far, as they offer several advantages over other methods. These include not requiring contact, speed and cost-effectiveness. However, these methods also have some limitations, such as only being able to acquire the visible portion of surfaces and dependency on surface reflectance.

Within optical techniques, we distinguish between active and passive ones. The attribute *active* refers to the fact that they involve a control on the illumination of the scene, that is, radiating it in a specific and structured way (e.g. by projecting a pattern of light or a laser beam), and exploit this knowledge in the 3D model reconstruction. In contrast, the *passive* methods seen so far rely only on analysing the images as they are, without any assumptions about the illumination, except that there is enough to let the camera see.

Active methods are reified as stand-alone sensors that incorporate a light source and a light detector, which is not necessarily a camera. These are also called *range sensors*, for they return the *range* of visible points in the scene, that is, their distance from the sensor (Fig. 13.1). This definition can be extended to include passive sensors as well, such as stereo heads.

Full-field sensors return a matrix of depths called a *range image* which is acquired in a single temporal instant, similar to how a camera with a global shutter operates.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_13



Fig. 13.1 Colour image and range image of the same subject, captured by a Microsoft Kinect device. Courtesy of U. Castellani

Scanning sensors (or *scanners*) sweep the scene with a beam or a light plane, and capture a temporal sequence of different depth measurements. This sequence can then be used to generate a range image or a point cloud, depending on the desired output. Similarly to rolling shutter cameras, artefacts will appear if the sensor or the object moves.

Global shutter captures an entire image simultaneously, while rolling shutter captures the image line by line in a temporal sequence. This can cause distortions in the image, such as objects appearing skewed or slanted if they are moving quickly.

13.2 Structured Lighting

The techniques of structured lighting have in common that they purposively project a light pattern characterised by an informative content on the scene. This "special" lighting has the advantage of making the result virtually independent of the texture of the object surface (which is not true for passive methods, such as stereopsis).

Among these techniques, the first two that we will illustrate are based on triangulation: in *active stereo* structured light is simply used to better condition the stereo matching in a binocular setup, while in *active triangulation* there is only one camera and the triangulation takes place with the projector. Finally, we will mention *photometric stereo*, based on the acquisition of many images in which the viewpoint is static but the lighting direction varies in a controlled way (in this sense we consider it to be "structured").

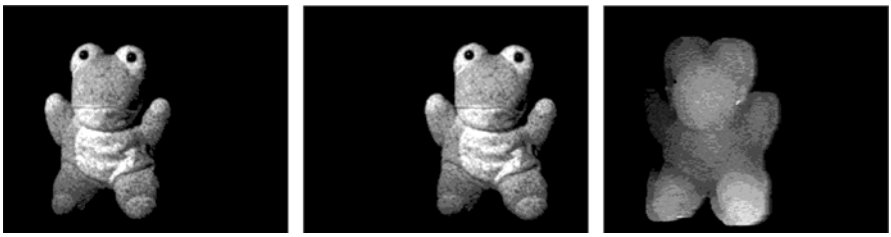


Fig. 13.2 Active stereo with random dots. Stereo pair with projected "salt and pepper" artificial texture and resulting disparity map

13.2.1 Active Stereopsis

Active stereopsis works similarly to its passive counterpart (Chap. 12), but uses structured light projected onto the scene to improve results in textureless areas. Structured light can take several forms, as will be illustrated in the examples below:

1. A random *dot pattern* is projected onto the scene, facilitating the matching, as in Fig. 13.2.
2. A *laser beam* scans the scene, projecting a point onto the surfaces that is easily detected and matched in the two images. It is necessary to take many images since a single pair of images is used to calculate the depth of a single point.
3. A laser *sheet* is swept over the scene[1] (Fig. 13.3). The lines determined by the light plane in the two images intersected with their respective epipolar lines provide the corresponding points. In this way, for each pair of images, disparity is assigned to the points in the lighted line. It is faster than the previous solution, but still many images are needed.

The first solution configures a full-field sensor, while the other two involve a scanning behaviour. Active stereo does not have much practical impact, but it paves the way the next method, which is active triangulation.

13.2.2 Active Triangulation

Active triangulation, like stereopsis, is based on the principle of triangulation between two devices, with one device being the light projector and the other the camera. The projector is treated as an inverse camera, where the light rays exit from the Centre of Projection (COP) instead of entering, but the underlying geometry

[1] A plane or sheet can be obtained by passing a laser beam through a cylindrical lens.

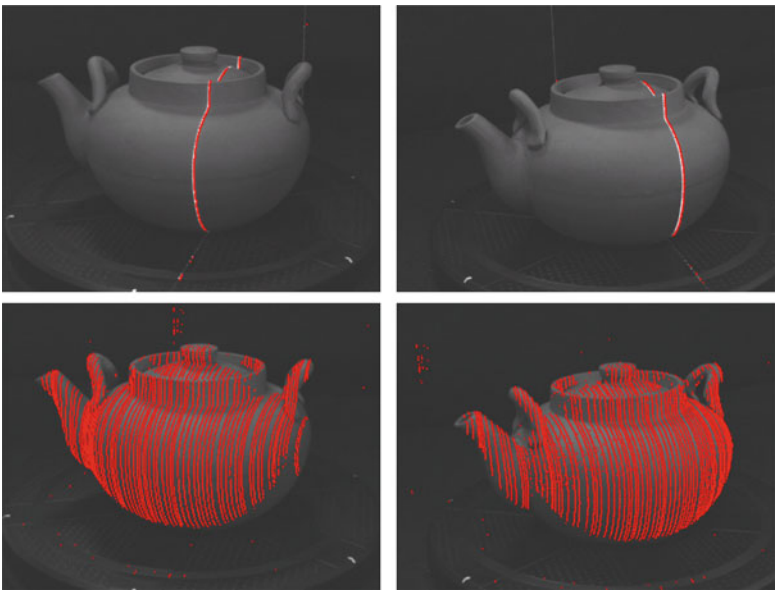


Fig. 13.3 Example of active stereo system with laser sheet. The top row shows the two images acquired by the cameras, in which the line formed by the laser sheet is visible. Below are shown, in overlay, the points detected after a sweep

remains the same. Thus, a calibrated camera-projector system is geometrically equivalent to a calibrated camera pair.

A projector of planes can be modelled, analogous to the pinhole camera, with a 2×4 P projection matrix mapping 3D points in 2D lines:

$$P \simeq \begin{bmatrix} \alpha_u & 0 & u_0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R & \mathbf{t} \end{bmatrix}. \tag{13.1}$$

The calibration and triangulation are entirely analogous to the passive stereo case (Trobina 1995).

The camera shoots a scene in which a device projects a pattern of *structured* light, that is, that contains the information necessary to identify its elements. The range of points in the scene is obtained by intersecting the optical ray of an image point with the corresponding light ray or plane emitted by the projector. Calibration is necessary to orient the the light plane in object space. The matching phase is avoided, as correspondences are obtained from the information contained in the pattern itself. Some examples follow:

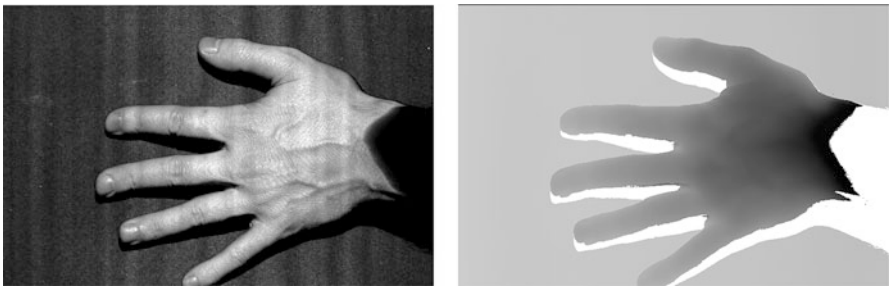


Fig. 13.4 Image of intensity and range image obtained from a commercial laser active triangulation system. The missing parts in the range image (in white) are due to the different position of the laser source and the camera. Courtesy of S. Fantoni

1. The projector scans the scene with a beam (or sheet) of *laser* light. The laser dot (or line) in the image is uniquely identified.
2. Instead of one plane, one can project many planes simultaneously using a projector of light bands. In this case, the stripes must be encoded in some way to distinguish them from each other in the image. Compared to the previous solution, more points are measured with a single image.
3. The projector illuminates the scene with a *pattern of random dots*, so that the configuration of dots in a neighbourhood of a point identifies it uniquely.[2]

Because the camera and projector see the scene from different positions, shadow areas are created where the range cannot be measured (Fig. 13.4), similarly to occlusions in stereo.

13.2.3 Ray-Plane Triangulation

By knowing the geometry of the system, the equation of the optical ray from the camera and the equation of the corresponding light plane, the 3D coordinates of the observed scene point can be calculated by intersection.

Consider a point M with coordinates $\widetilde{\mathbf{M}}_c = [X_c, Y_c, Z_c]^\mathsf{T}$ in the camera reference frame. The direct isometry $(R, \mathbf{t})$ that brings the camera reference frame onto the projector reference frame is known from calibration; therefore, the coordinates of the same point in the projector reference frame are

$$\widetilde{\mathbf{M}}_p = R\widetilde{\mathbf{M}}_c + \mathbf{t}. \tag{13.2}$$

[2] This is also the working principle of the first Microsoft Kinect sensor. The pattern is generated by an infrared laser through appropriate diffraction grating that creates a speckle pattern.

The projection of point M onto the camera reference frame is (in normalised coordinates) $\mathbf{q}_c = [u_c, v_c, 1]^\mathsf{T}$, and is obtained from the projection equation:

$$\mathbf{q}_c = \begin{bmatrix} u_c & v_c 1 \end{bmatrix} = \begin{bmatrix} \frac{X_c}{Z_c} & \frac{Y_c}{Z_c} 1 \end{bmatrix} = \frac{1}{Z_c} \widetilde{\mathbf{M}}_c. \tag{13.3}$$

As for the projector, we model it as a camera, in which the vertical coordinate of the projected point is undefined (we assume that the light sheets are vertical, in the interior reference frame of the projector). Let x_p be the coordinate of the plane illuminating M, then, keeping with the language of the camera, we will say that M is projected onto the point (in normalised coordinates) $\mathbf{q}_p = [x_p, y_p, 1]^\mathsf{T}$, in the reference frame of the projector, where y_p is only a placeholder for and undefined value:

$$\mathbf{q}_p = \frac{1}{Z_p} \widetilde{\mathbf{M}}_p. \tag{13.4}$$

Using Eqs. (13.2), (13.3) and (13.4), we get:

$$Z_p \mathbf{q}_p - Z_c R \mathbf{q}_c = \mathbf{t}, \tag{13.5}$$

which decomposes into a system of three scalar equations:

$$\begin{cases} Z_p x_p - Z_c \mathbf{r}_1^\mathsf{T} \mathbf{q}_c = t_1 \\ Z_p y_p - Z_c \mathbf{r}_2^\mathsf{T} \mathbf{q}_c = t_2 \\ Z_p - Z_c \mathbf{r}_3^\mathsf{T} \mathbf{q}_c = t_3 \end{cases} \tag{13.6}$$

of which the second cannot be used since y_p is undefined. We derive Z_p from the third equation and substitute it into the first, obtaining, after a few steps, the result we sought, namely, the depth Z_c of point M (in the camera reference frame):

$$Z_c = \frac{t_1 - t_3 x_p}{(x_p \mathbf{r}_3^\mathsf{T} - \mathbf{r}_1^\mathsf{T}) \mathbf{q}_c}. \tag{13.7}$$

13.2.4 Scanning Methods

In laser-based methods, the determination of the ray-plane correspondence is straightforward due to the fact that only one light plane is projected. Consequently, the sensor measures the 3D position of one line at a time. To obtain a range image, it is necessary to mechanically move the sensor in a controlled manner and with great precision. This results in a laser scanning sensor, or *scanner*, in brief.

An alternative to the controlled scanning is to move the sensor freely (e.g. by hands) and use a coupled coordinate-measuring machine (CMM) that, through a magnetic, optical or ultrasonic device, returns the position and attitude of the sensor in a local reference frame.

13.2.5 Coded-Light Methods

In the *coded-light* methods, the stripes or the points encode the information which serves to uniquely identify them. This has the advantage that moving parts are eliminated and the number of measurable points in a frame increases. See Salvi et al. (2010) for a review on this topic.

As for the stripes, which are one of the most widely used methods, the simplest encoding is accomplished by assigning a different colour (Boyer and Kak 1987) to each stripe, so that they can be distinguished from each other in the image.

The colour of a pixel is determined by the combination of light and surface reflectance. Although we can control the light, the surface reflectance is usually unknown, making it difficult to accurately predict the resulting pixel colour. To ensure accurate identification, one can associate the identity of the stripe not only with its colour but also with that of neighbouring stripes, thus providing more reliable results.

A de Bruijn sequence of order n on an alphabet A of length k, denoted by $B(k, n)$, is a cyclic (start and end coincide) sequence k^n long in which all possible strings of length n of A occur exactly once as substrings of $B(k, n)$. For example, on $A = \{0, 1\}$, we have that $B(2, 3) = 00010111$. The binary strings of length 3 occur only once as substrings of 00010111 (cyclic).

For example, if the colours of the bands are arranged according to a de Bruijn sequence, each one can be uniquely identified by its colour and that of the neighbouring bands. Another very robust technique is the so-called temporal coding, which is accomplished by projecting, a temporal sequence of n black-and-white stripe patterns, appropriately generated by a computer-controlled projector. Each projection direction is thus associated with a code of n bits, where the i-th bit indicates whether the corresponding stripe was in light or shadow in the i-th pattern (see Fig. 13.5). This method allows to distinguish 2^n different directions of projection.

A camera acquires n grey-level images of the object lighted with the stripe pattern. These are then converted into binary form so as to separate the areas illuminated by the projector from the dark areas, and for each pixel the n-bit string is stored, which, by the above, uniquely encodes the illumination direction of the

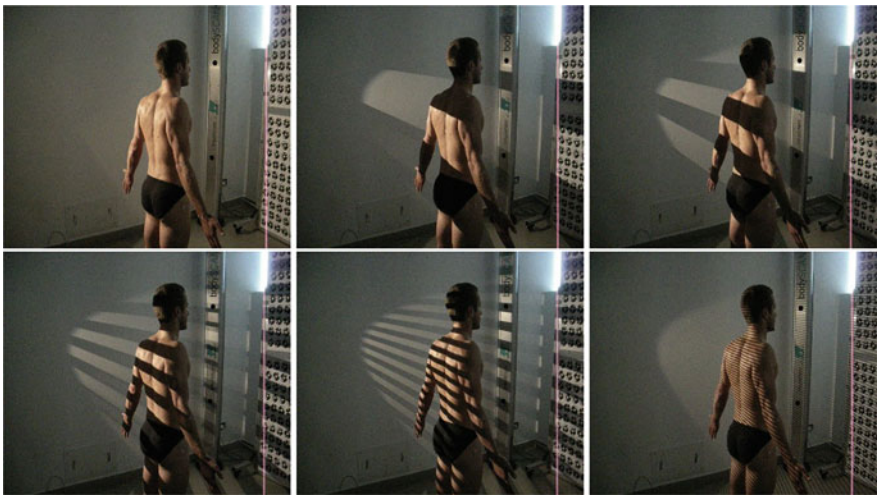


Fig. 13.5 Coded light. Images of a subject with stripes of projected light. In the lower right image, the stripes are very thin and are not visible at reduced resolution. Courtesy of A. Giachetti

thinnest stripe the pixel belongs to. To minimise the effect of errors Gray coding is customarily used, so that adjacent stripes differ by one bit only.

13.3 Time-of-Flight Sensors

A Time of Flight (TOF) laser sensor, commonly referred to as LiDAR (an acronym for Light Detection And Ranging), works on the same principle as RADAR (Radio Detection And Ranging). The surface of an object reflects laser light back to a receiver, which measures the time elapsed between the transmission and reception of the light—the time of flight. This time can be measured in two ways: directly as Δt or indirectly after conversion to a phase delay $\Delta\phi$ through amplitude modulation.

In the first case, also called pulsed wave (PW) LiDAR, the laser source emits a pulse, the receiver records the instant of time when it returns after being reflected by the target and compute the TOF Δt. A simple calculation allows to recover the distance: $r = \frac{c\Delta t}{2}$ where c indicates the speed of light. As the reader can easily deduce, the depth resolution of the sensor is related to its ability to measure extremely small timescales. Light travels 1 mm in 3.3 ps, and it is difficult to measure intervals smaller than 5–10 ps.

To obtain a more versatile sensor, a continuous wave (CW) is emitted instead of a pulse; this is an amplitude-modulated radiation with a sine wave, and the phase of the reflected signal is measured. Let therefore $f_{AM} = \frac{c}{\lambda_{AM}}$ be the modulation frequency. The distance to the target is related to the phase $\Delta\phi$ of the reflected signal

by

$$\Delta\phi = 2\pi f_{AM} \frac{2r}{c}. \tag{13.8}$$

Therefore, we derive:

$$r = n\frac{\lambda_{AM}}{2} + \frac{\Delta\phi}{4\pi}\lambda_{AM} \quad \forall n \in \mathbb{Z}. \tag{13.9}$$

The phase $\Delta\phi$ is measured by the cross-correlation of the emitted and received signals. Note the periodicity ($n \in \mathbb{Z}$): since the phase is measured modulo 2π, the distances also turn out to be measured modulo $\Delta r = \frac{\lambda_{AM}}{2}$.

The choice of modulation frequency depends on the type of application. A choice of f_{AM}=10 MHz, which corresponds to an unambiguous range of 15 m, is typically suitable for indoor robotic vision. For close-range object modelling, we will choose a lower f_{AM}. This ambiguity can be eliminated by scanning with decreasing wavelengths, or equivalently with a *chirp* type signal.

What we described so far is a laser range finder, a sensor that emits a single beam and therefore obtains distance measurements of only one point in the scene at a time. A laser *scanner*, instead, uses a beam of laser light to scan the scene in order to derive the distance to all visible points.

In a typical terrestrial laser scanner, an actuator rotates a mirror along the horizontal axis (thus scanning in the vertical direction), while the instrument body, rotating on the vertical axis, scans horizontally. Each position then corresponds to a 3D point detected in polar coordinates (two angles and the distance).

In an airborne laser scanner, on the other hand, the instrument performs an angular scan (or "swath") along the perpendicular to the flight direction, while the other scan direction is given by the motion of the aircraft itself, which must therefore be able to measure its orientation with extreme precision, thanks to inertial sensors (accelerometer and gyroscope) and a global navigation satellite system (GNSS), such as the GPS. The same principle is also applied in the terrestrial domain (mobile mapping) to different carriers such as road vehicles, robots or humans.

Simultaneous Localization and Mapping (SLAM) is a powerful tool for navigation in GNSS-denied environments, such as indoors. It utilises 3D registration techniques (see Chap. 15) to compute the incremental rigid motion between consecutive scans. For further information, see (Thrun and Leonard 2008).

Recently, full-field time-of-flight sensors, also known as Flash LiDAR or TOF cameras, have been developed. These sensors allow for simultaneous time-of-flight detection of a 2D matrix of points through an array of electronic circuits on the chip. While the lateral resolution of these sensors is low (e.g. 128x128), it is compensated for by their high acquisition rate (e.g. 50 fps).

13.4 Photometric Stereo

Photometric stereo (Woodham 1980) is a technique for deriving information about the shape of an object from a series of photographs taken by illuminating the scene from different angles. Typically, the input to photometric stereo consists of a series of images captured by a fixed camera, where the illumination changes are due to the displacement of a single light source and the output are the normals of the 3D surface under observation.

The idea behind this approach is that the light radiation (radiance) reflected from the object falling on the camera depends mainly on the direction of the light illuminating it and on the normals of the surface: the more the direction of the light coincides with the normal, the more light will be reflected and therefore the higher the intensity of the pixel in the corresponding image.

The brightness of point (u, v) on the image is equal to the radiance L of point (X, Y, Z) of the scene projecting into (u, v):

$$I(u, v) = L(X, Y, Z). \tag{13.10}$$

The radiance at point (X, Y, Z), in turn, depends on the shape (the normal), the reflectance of the surface and the light sources.

We assume that the surface is *Lambertian*, so we apply (2.9), which we rewrite as

$$L(X, Y, Z) = \rho' \mathbf{n}^\mathsf{T}\mathbf{s} \, [\![\mathbf{n}^\mathsf{T}\mathbf{s} \geq 0]\!] \tag{13.11}$$

where ρ' is the effective albedo (incorporates the incident radiance) and $\mathbf{s}$ is the versor of the illumination direction (points in the direction of the light source), $\mathbf{n}$ is the versor of the normal; all these quantities depend, in principle, on the point (X, Y, Z) considered. If we assume constant albedo and parallel illumination (light source at infinite distance), then only the normal depends on the point (X, Y, Z), and we can therefore write:

$$I(u, v) = \rho' \, \mathbf{n}(X, Y, Z)^\mathsf{T}\mathbf{s} \tag{13.12}$$

where we have neglected the Iverson bracket, so that the problem can be solved using simple linear algebra tools.

In practice, given f greyscale images taken under $\mathbf{s}_1, \ldots, \mathbf{s}_f$ light directions, we consider the matrix B of size $p \times f$ obtained by juxtaposing by column the p pixels of each image. It is important that the images are radiometrically calibrated, that is, that the pixel intensities correspond to the physical values of the scene radiance. This can be ensured if one works directly with images in linear RGB format. Assuming the photographed object is Lambertian, the reflectance equation (2.6) translates to matrix form as

$$B = \mathrm{diag}([\rho'_1 \ldots \rho'_p])N^\top S \tag{13.13}$$

$N = [\mathbf{n}_1, \ldots, \mathbf{n}_p]$ is the $3 \times p$ matrix that collects for columns the normals and $S = [\mathbf{s}_1, \ldots, \mathbf{s}_f]$ is the $3 \times f$ matrix that has for columns the directions of the light sources used during the acquisition. If the light directions S are known, we speak of a *calibrated lights array*. In this situation, we are led to solve a linear least-squares system of the type:

$$\arg\min_X \|B - X^\top S\|_2^2 \tag{13.14}$$

which is determined if we have more than two images available ($f \geq 3$). The rows of the matrix X that realise the minimum, once normalised, give us the normal versors of $N^\top$, while the albedo corresponds to the norm of each row of X. The MATLAB implementation is given in Listing 13.1, and an example result is shown in Fig. 13.6.

Listing 13.1 Photometric stereo

```
function [N, a] = photostereo(B,S)
%PHOTOSTEREO Photometric stereo
    X = B/S;
    a = sqrt(sum(X.^2,2));
    N  = diag(1./a)*X;
end
```



Fig. 13.6 Top: 12 images of a figurine taken with different light source positions. Bottom: the normals map obtained by photometric stereo (the hue encodes the normal), on the right the reconstructed surface. Courtesy of L. Magri

The solution of the problem becomes more complicated if the directions of the light sources are not known. In this case, the problem is called *uncalibrated photometric stereo*. A possible solution consists in iteratively estimating X and the lighting directions S, starting from the Singular Value Decomposition (SVD) factorisation of the matrix B. It can also be shown that, if the lighting directions are not known, the normals field can only be determined up to an invertible transformation called bas-relief ambiguity. The name is due to the fact that this ambiguity is exploited in the bas-relief technique to give the feeling of seeing a subject that is in low relief as if it were in the round. Intuitively, the matrix B can be factorised both as $B = XS$ and as $B = XAA^{-1}S$ where A is the transformation matrix representing the ambiguity. More details can be found in (Belhumeur et al. 1999).

A further complication is due to the presence of phenomena that violate the Lambertian model, such as shadows and specularity. Specular reflection departs from the Lambertian model in that light is not scattered uniformly in space, but is reflected primarily along one direction. Shadows, on the other hand, correspond to those pixels for which $\mathbf{n}^\top \mathbf{s} \leq 0$ (self- shadows), or to points that are not reached by the light source due to occlusions (cast shadows). In some cases, specular and shadowed pixels can be handled by treating them as outliers.

For example, after observing that the matrix B must have at most rank 3 (because it is the product of two matrices with at most rank 3), one can exploit matrix factorisation techniques to clean up the data as explained by Wu et al. (2010). The idea is to robustly determine the linear subspace of dimension three that represents the Lambertian measurements so as to discard specularities as rogue measurements and treat shadows as missing data.

If, on the other hand, non-diffusive phenomena are dominant, a different ρ function must be used to more closely approximate the surface reflectance.

13.4.1 From Normals to Coordinates

Once the normals field is obtained, the surface of the observed object can be reconstructed. An overview of some of the more efficient methods is given in (Agrawal et al. 2006). Here we present a simple but not robust method that assumes orthographic projection. If we denote by $Z(x, y)$ the depth of the surface at pixel (x, y), in the orthographic projection case we have that the vector

$$\mathbf{v} = \begin{bmatrix} x+1 \\ y \\ Z(x+1, y) \end{bmatrix} - \begin{bmatrix} x \\ y \\ Z(x, y) \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \\ Z(x+1, y) - Z(x, y) \end{bmatrix} \tag{13.15}$$

approximates the tangent vector to the surface $(x, y, Z(x, y))$ in the direction x and must therefore be orthogonal to the normal, that is, $\mathbf{v}^\mathsf{T}\mathbf{n} = 0$ (Fig. 13.7).

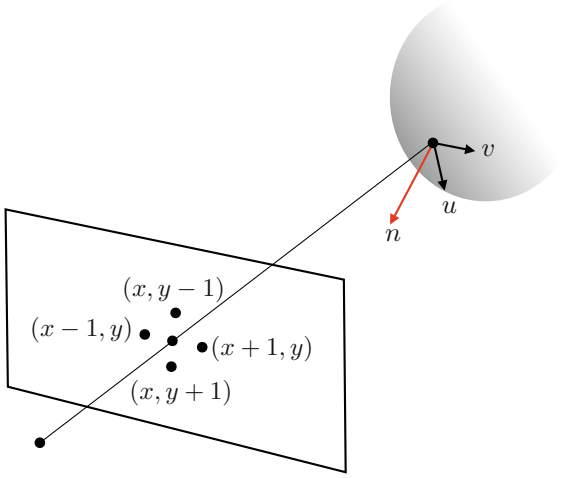


Fig. 13.7 Discrete approximation of normal. Courtesy of L. Magri

Developing the calculations we obtain a link between the depths and the entries of the normal

$$n_1 + n_3 \left(Z(x+1, y) - Z(x, y)\right) = 0. \tag{13.16}$$

A similar relationship can be derived for the vector:

$$\mathbf{u} = \left[x \; y+1 \; Z(x, y+1)\right] - \left[x \; y \; Z(x, y)\right] \tag{13.17}$$

which approximates the tangent vector to the surface $(x, y, Z(x, y))$ in the y-direction, yielding

$$n_2 + n_3 \left(Z(x, y+1) - Z(x, y)\right) = 0. \tag{13.18}$$

In this way, as the normals vary, we can derive conditions on the depths Z. It is then possible to determine Z by solving the corresponding overdetermined sparse system of dimension $2p \times p$.

Among the structured light methods, we also mention the method of the *moiré fringes*. The idea of this method is to project a grid onto an object and take an image of it through a second reference grid. This image interferes with the reference grid and creates interference figures known as *moiré fringes*, which appear as bands of light and shadow. Analysis of these bands provides information about the depth variation.

13.5 Practical Considerations

Figure 13.8 summarises the taxonomy of the active methods discussed in this chapter. It is usually noted that when discussing range sensors, the most important distinction is between active triangulation and time-of-flight, with the class of "structured lighting" and "active triangulation" being considered coincident. This implies neglecting active stereo and photometric stereo, which are not implemented in any material range sensor.

Active triangulation and time-of-flight sensors have fairly well separated operating ranges. Triangulation sensors are the most accurate, reaching a maximum resolution of the order of tens of micrometres, but they work within a few metres of the object. This is because the baseline must be of the same order of magnitude as the sensor-object distance. Time-of-flight sensors reach much greater distances, with lower resolutions. In particular, PW class sensors reach distances of several kilometers, with a resolution of approximately 10 cm at 1 km. In any case the the resolution does not go below 2 mm. CW sensors, on the other hand, reach only hundreds of meters with a resolution (approximately) of 1 mm at 300 m. The smallest achievable resolution is of the order of hundredths of millimetres. Figure 13.9 illustrates the different application areas. It should be noted that the boundaries of the boxes representing the different classes of sensors are not sharp and that there are partial overlaps.

In summary, for small to medium measurement volumes (up to human size) and consequent working distances, triangulation sensors are used, while for larger

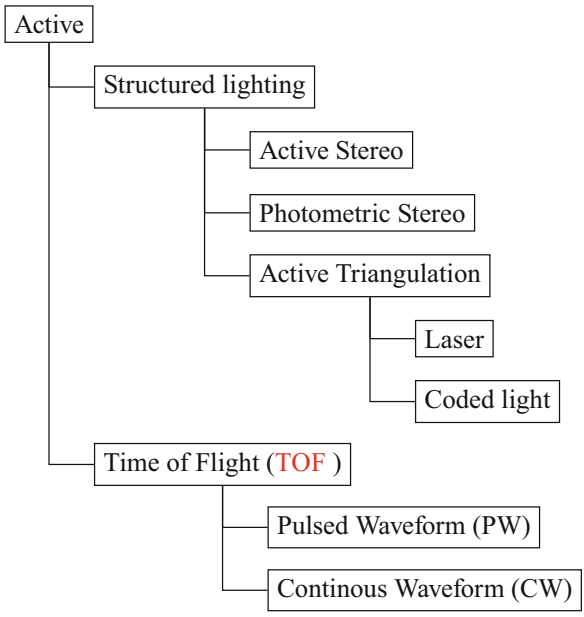


Fig. 13.8 Taxonomy of the active methods we covered in this chapter

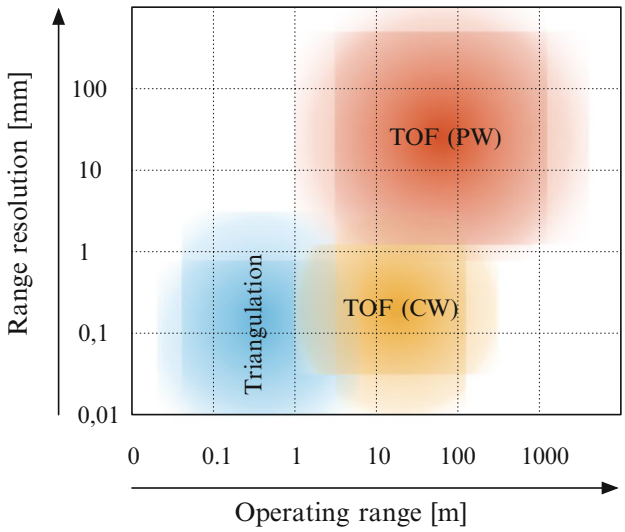


Fig. 13.9 Comparison of characteristics of active systems. Adapted from (Guidi et al. 2009)

volumes and distances (buildings, geographical areas), time-of-flight sensors are used. The former can be eventually coupled to CMMs while the latter to GPS and inertial sensors.

The most important figures of merit that characterise a range sensor (like any other sensor) are:

resolution: the smallest change in measured quantity that the sensor can detect;
accuracy: (measure the systematic error) difference between measured value (average of repeated measurements) and true value;
accuracy: (measures the dispersion of measurements around the mean) standard deviation of repeated measurements of the same quantity;
speed: number of measurements per second;
operating range: minimum and maximum value of the measurement.

In the specific case of range sensors, a distinction is made between lateral and depth resolutions. The lateral resolution is defined as the smallest detectable distance between two adjacent points measured in a plane perpendicular to the sensor axis. It therefore depends on the distance at which the plane is placed. Often the size in points (or pixels) of the image is given and the resolution is applied by dividing the size of the area imaged by the size of the image. Of course, sensors based on different principles will have different operating characteristics.

References

Amit Agrawal, Ramesh Raskar, and Rama Chellappa. What is the range of surface reconstructions from a gradient field? In *European Conference on Computer Vision*, pages 578–591. Springer, Berlin, 2006.

Peter N Belhumeur, David J Kriegman, and Alan L Yuille. The bas-relief ambiguity. *International Journal of Computer Vision*, 35 (1): 33–44, 1999.

K. Boyer and A. Kak. Color-encoded structured light for rapid active ranging. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 9 (10): 14–28, 1987.

Gabriele Guidi, Michele Russo, and Jean-Angelo Beraldin. *Acquisizione 3D e modellazone poligonale*. McGraw-Hill, Milano, 2009.

Joaquim Salvi, Sergio Fernandez, Tomislav Pribanic, and Xavier Llado. A state of the art in structured light patterns for surface profilometry. *Pattern Recognition*, 43 (8): 2666–2680, August 2010.

Sebastian Thrun and John J. Leonard. *Simultaneous Localization and Mapping*, pages 871–889. Springer Berlin Heidelberg, Berlin, Heidelberg, 2008. ISBN 978-3-540-30301-5. https://doi.org/10.1007/978-3-540-30301-5_38.

Marjan Trobina. Error model of a coded-light range sensor. Technical Report BIWI-TR-164, ETH-Zentrum, 1995.

Robert J. Woodham. Photometric method for determining surface orientation from multiple images. *Optical Engineering*, 19 (1): 191139, 1980.

Lun Wu, Arvind Ganesh, Boxin Shi, Yasuyuki Matsushita, Yongtian Wang, and Yi Ma. Robust photometric stereo via low-rank matrix completion and recovery. In *Asian Conference on Computer Vision*, pages 703–717. Springer, Berlin, 2010.

Chapter 14
Multi-View Euclidean Reconstruction



14.1 Introduction

In this chapter we will deal with the problem of reconstruction from many *calibrated* images, which is the most relevant in practice and leads to a Euclidean reconstruction.

Consider a set of 3D points, viewed by m cameras with matrices $\{P_i\}_{i=1\ldots m}$. Let $\mathbf{m}_i^j$ be the (homogeneous) coordinates of the projection of the j-th point into the i-th camera. The problem of *reconstruction* can be posed as follows: given the set of pixel coordinates $\{\mathbf{m}_i^j\}$, find the set of Perspective Projection Matrix (PPM) $\{P_i\}$ and the 3D points $\{\mathbf{M}^j\}$ (called *model* in this context) such that:

$$\mathbf{m}_i^j \simeq P_i \mathbf{M}^j. \tag{14.1}$$

As already observed in the case of two images, without further constraints one will obtain, in general, a reconstruction defined up to an arbitrary projectivity, and for this reason it is called projective reconstruction. Indeed, if $\{P_i\}$ and $\{\mathbf{M}^j\}$ are a reconstruction, that is, they satisfy (14.1), then also $\{P_i T\}$ and $\{T^{-1}\mathbf{M}^j\}$ satisfy (14.1) for every nonsingular 4×4 matrix T that represents a projectivity. If the intrinsic parameters are known, a Euclidean reconstruction can be achieved, which differs from the true one by a similitude.

In this chapter, we will focus on the problem of reconstruction in the case of multiple calibrated images, also known in the literature as Structure from Motion (SFM) (Fig. 14.1). Here, when we refer to "structure", we mean the 3D model and by "motion", we mean the exterior orientation of a set of cameras. The uncalibrated case will be dealt with in Chap. 16.

For the case of $m = 2$ images with known interior orientation, Chap. 8 discussed how to achieve a Euclidean reconstruction through the computation of

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_14



Fig. 14.1 Visualisation of the cherub reconstruction, consisting of a 3D model and the oriented images, shown as pyramids having the vertex in the COP



Fig. 14.2 Some tracks overlaid on the cherub images

the essential matrix, its factorisation and triangulation. Now, the question is: how do we generalise this process for $m > 2$?

14.1.1 Epipolar Graph

Given a set of $m > 2$ unordered images, we assume that for a number of pairs of these images (ideally whenever possible), a set of corresponding points is provided. Such points are typically produced by a detector and are matched automatically, so they are perturbed by small errors and contain a significant percentage of outliers.

This leads to the initial problem of establishing which of them form epipolar pairs and to compute the essential or fundamental matrix (depending on whether the interior orientation is or is not known) in order to geometrically validate the matches, discarding the incorrect ones.

The correspondences between pairs of images are then propagated to the whole set, resulting in so-called tracks, which span multiple images (Fig. 14.2). The

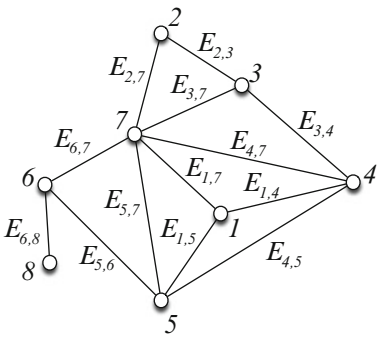


Fig. 14.3 An epipolar graph. The nodes are the images and the edges correspond to the epipolar pairs, that is, they are labelled with an essential matrix

corresponding 3D points are called *tie points*: they are the ones that will constitute the *model*.

It is useful in this context to represent the available information by means of the *epipolar graph* $G = (V, E)$, whose nodes V are the m images and edges E correspond to the computed fundamental or essential matrices (Fig. 14.3). The adjacency matrix A of the graph contains one in the entry (i, j) if E_{ij} is available, zero otherwise. We can also associate a weight in $[0, 1]$ with the edges proportional to the number of corresponding points between the two images. In some cases it will be convenient to invert this value, turning it into a cost.

The edge (i, j) is labelled with the essential matrix $E_{ij} = \left[\mathbf{t}_{ij}\right]_\times R_{ij}$, defined by $\mathbf{q}_i^\mathsf{T} E_{ij} \mathbf{q}_j = 0$. Observe that the essential matrix between two PPMs $P = [I, \mathbf{0}]$, $P' = [R, \mathbf{t}]$, which in the previous chapters we simply called $E = [\mathbf{t}]_\times R$, with the notation introduced above corresponds to $E_{21} = [\mathbf{t}_{21}]_\times R_{21}$ being $P_1 = [I, \mathbf{0}]$, $P_2 = [R_{21}, \mathbf{t}_{21}]$.

In the construction of the epipolar graph, we assumed that for a certain number of image pairs point correspondences are provided. If the number n of images is small, one can think of attempting to match of all $\binom{m}{2}$ pairs of images, which has a quadratic cost, but in general one should aim to find out in advance a fixed number of images to attempt point matching with. In some cases one has auxiliary information about the camera trajectory (e.g. in aerial photogrammetry, or in rotating table acquisitions), but when this information is missing, one has to exploit the images themselves.

A possible approach to the construction of the epipolar graph. Extract salient points and descriptors (e.g. Scale Invariant Feature Transform (SIFT)) in all images and store them in a data structure that supports spatial queries (e.g. k-d tree). For each descriptor determine its first k neighbours and the images in which they occur. In a two-dimensional $n \times n$ histogram, increment the accumulator (i, j) whenever a descriptor of image i has a descriptor of image

(continued)

j among its first k neighbours. In this way, the histogram approximates the number of features that the two images have in common, and provides a first approximation of the adjacency matrix of the epipolar graph. We must now decide for which edges of this graph to attempt point matching, and, to avoid quadratic growth, we must do this for a linear number of edges. Brown and Lowe (2003) choose k neighbours for each node, obtaining kn candidate pairs. (Toldo et al. 2015) instead sequentially construct k spanning trees for the graph, removing each time the edges that were considered in the tree. The union of the resulting k disjoint spanning trees, each containing $n - 1$ edges, gives $k(n - 1)$ candidate pairs. The first strategy favours the creation of loosely connected cliques, whereas overlapping spanning trees guarantee a more strongly connected graph.

14.1.2 The Case of Three Images

It is instructive to start by considering the case of $m = 3$ images. We assume that we know the essential matrices related to the image pairs (1,2) and (2,3), as shown in Fig. 14.4. We outline three ways of attacking the problem, which will be expanded in the following sections

Strategy 1 Let us proceed in an *incremental* manner. We apply the reconstruction process from two images (Listing 8.2) to the pair (I_1, I_2), obtaining a model. Assuming that some points in the model are visible in I_3 (the dots in Fig. 14.5), these can be used to solve for the exterior orientation of I_3 by resection (Listing 5.2). The model is then updated and augmented by the contribution of I_3 allowing new points to be triangulated (Listing 8.1).

Strategy 2 Let us *independently* apply the procedure for the reconstruction from two images (Listing 8.2) to each of the two pairs (I_1, I_2) and (I_2, I_3) obtaining two independent models separated by a similitude, and merge them into one by solving an absolute orientation problem (Listing 5.1).

Strategy 3 We aim to instantiate three mutually consistent PPMs and proceed to the final triangulation.

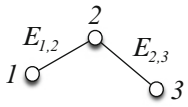


Fig. 14.4 Epipolar graph for the case of three images

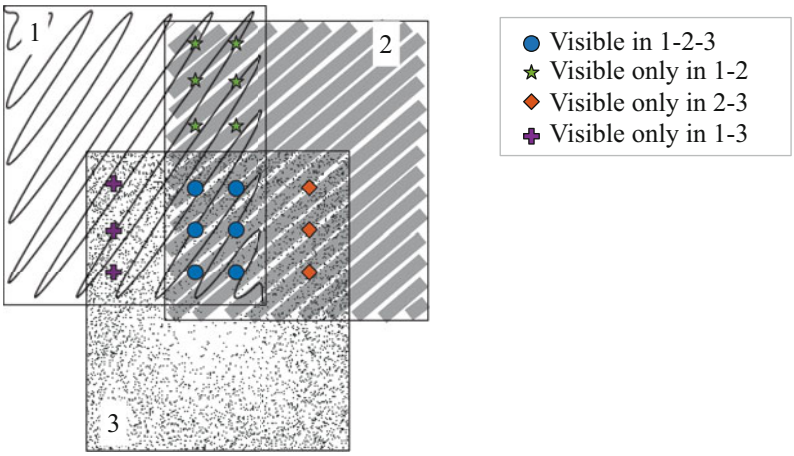


Fig. 14.5 The points shared by I_1 and I_2 (stars and dots) are those that make up the initial model. Image I_3 is added, thanks to the model points visible in I_3 (dots); their position will be refined by triangulation from three views. Other tie points are added to the model: those visible from I_1 and I_3 (crosses) and those visible from I_2 and I_3 (rhombuses)

From the factorisation of the essential matrices E_{21} and E_{32} (Listing 7.2), we obtain two rigid motions $(R_{21}, \hat{\mathbf{t}}_{21})$ and $(R_{32}, \hat{\mathbf{t}}_{32})$ in which each translation is known up to the modulus, and for this we consider the versor, denoted by $\widehat{\ }$.

If the moduli of the translations were known, we could concatenate the isometries in the following way:[1]

$$R_{31} = R_{32}R_{21}, \tag{14.2}$$

$$\mathbf{t}_{31} = \mathbf{t}_{32} - R_{32}\mathbf{t}_{21} \tag{14.3}$$

and then instantiate three mutually consistent PPMs, $P_1 = [I, \mathbf{0}]$, $P_2 = [R_{21}, \mathbf{t}_{21}]$ and $P_3 = [R_{31}, \mathbf{t}_{31}]$ and reconstruct the model by triangulation (Listing 8.1).

Unfortunately, however, the composition does not work for versors. In other words, while the modulus of the only translation involved was ignored in the two-view case (yielding a scaled reconstruction), in the three (or more)-view case, the moduli of each relative translation cannot be fixed arbitrarily, since the relative translations are not independent of each other.

Adding the edge 1–3 to the graph solves this problem. Let us rewrite (14.3) such as the ratios between the moduli are explicited:

$$\lambda\hat{\mathbf{t}}_{31} = \mu\hat{\mathbf{t}}_{32} - R_{32}\hat{\mathbf{t}}_{21} \tag{14.4}$$

[1] Why the composition follows these formulae will be clear in light of the notation we will introduce in Sect. 14.3.

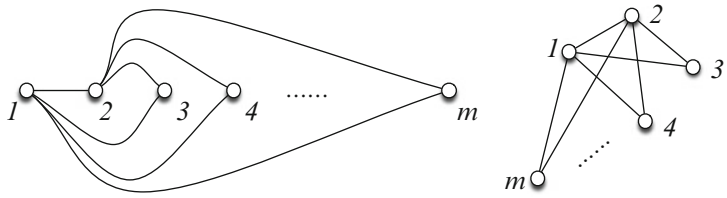


Fig. 14.6 Epipolar graph for the Zeller-Faugeras method: in the right-hand arrangement, we see that it is made of $m - 2$ circuits of length 3 sharing edge 1–2

where $\lambda = \|\mathbf{t}_{31}\|/\|\mathbf{t}_{21}\|$ and $\mu = \|\mathbf{t}_{32}\|/\|\mathbf{t}_{21}\|$. From the matrix E_{31}, one obtains $\hat{\mathbf{t}}_{31}$ and solves the above equation for the unknowns λ and μ as described in Proposition A.18.

Note, however, that a global scaling factor remains undetermined, and it is fixed arbitrarily through the norm of $\mathbf{t}_{21}$, as in the case of two images.

This method is generalised by Zeller and Faugeras (1996) to $m > 3$ images provided that the essential matrices 2-1, i-1 and i-2 are available for $i \geq 3$. In essence, the epipolar graph must have the particular structure illustrated in Fig. 14.6.

14.1.3 Taxonomy

Methods for reconstruction from many images can be divided, at a first level, between those that are global and those that are based on partial reconstructions (Fig. 14.7). The former are those that simultaneously consider all images and all tie points, such as projective factorisation (Sect. 16.1.1) and bundle adjustment (Sect. 14.4).

In the second class fall the methods illustrated in the previous examples. They are characterised by processes that employ subsets of the points and/or cameras. The final result is obtained by an alignment or fusion of partial results. Among these we distinguish point-based methods from reference frame-based methods. The former (Sect. 14.2) employ the points to compute the alignment, that is, they use them to solve some orientation problems (relative, exterior, absolute), while the latter (Sect. 14.3) align (*synchronise*) the reference frames of the cameras, without taking the points into account, and compute the model only at the end.

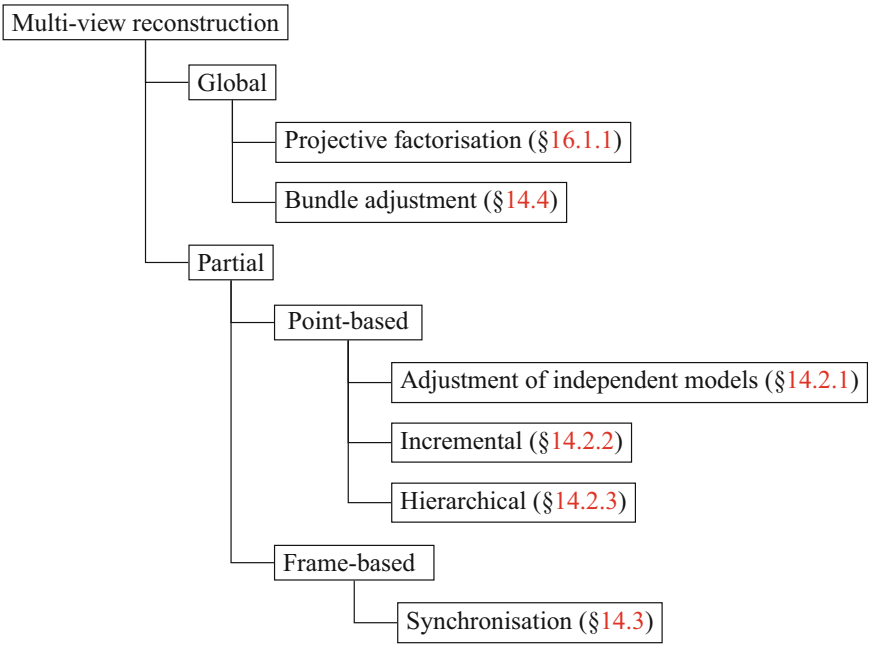


Fig. 14.7 Taxonomy of reconstruction methods from many images

Triples of images can be used instead of pairs in the partial strategies.

14.2 Point-Based Approaches

This class includes methods that stem from the first two strategies outlined in Sect. 14.1.2.

14.2.1 Adjustment of Independent Models

The Adjustment of Independent Models (AIM) method, well known in photogrammetry, is based on the construction and subsequent integration of many independent models built from image pairs. In summary:

- the images are grouped in pairs (with overlaps), and many independent models are computed by reconstruction from two frames;

- these models are related to each other by similitude transformations; the 3D points they have in common are used to bring them to a single reference frame.

In the first step, the image pairs correspond to a subset of the edges of the epipolar graph. For example, one can take the edges of a minimum spanning tree, where the cost associated with an edge is inversely proportional to the overlap.

With the tools we have seen so far, the last step can be implemented with a series of cascading Orthogonal Procrustes analysis (OPA); however, this would be suboptimal. We will see in Sect. 15.1.1 how the alignment of two point clouds with OPA can be generalised to the *simultaneous* alignment of many point clouds, with Generalised Procrustes analysis (GPA) (Crosilla and Beinat 2002).

14.2.2 Incremental Reconstruction

Starting with an initial reconstruction from two images, the incremental approach grows it by iterating between the following two steps:

- orient one image with respect to the current model via *resection*;
- update the model through triangulation (or *intersection*).

This is why it is also called the *resection-intersection* method.

Initialisation. Two images are chosen, with a criterion that must balance a high number of homologous points with sufficient separation of the two cameras (if too close, the triangulation is poorly conditioned). Reconstruction is then performed with these two frames, as described in Sect. 8.4. After this initialisation, the algorithm enters the main resection-intersection loop, in which the next image $i > 2$ is added to the reconstruction:

Resection. The correspondences relative to the points already reconstructed are used to obtain 2D-3D matches. Based on these, the exterior orientation of camera i with respect to the current model is estimated;

Intersection. The current model is then updated in two ways:

- the position of existing 3D points that were observed in image i is recomputed, by adding one equation to the triangulation;
- new 3D points are triangulated, thanks to the correspondences that become available as a result of adding image i.

The cycle ends when all images have been considered.

The sequential order of processing successive images can be obtained, for example, by a depth-first visit of the minimum spanning tree of the epipolar graph (a.k.a. preordering), starting with the second image of the initial pair.

Some comments and clarifications on the incremental method:

- immunity to outliers is crucial: one must use Random Sample Consensus (RANSAC) (or other robust techniques) not only in calculating epipolar geometry but also in resection;
- error containment is also essential: in triangulation it is advisable to keep an eye on some indicator of bad conditioning (near-parallel rays) and possibly discard the ill-conditioned 3D points. In addition, Bundle Adjustment (BA) should be run with some frequency during the iteration and not only at the end.

14.2.3 Hierarchical Reconstruction

A variant of the incremental reconstruction has been proposed by Gherardi et al. (2010). Instead of being ordered sequentially, the images are grouped hierarchically, resulting in a tree (or *dendrogram*) in which the leaves are the images and the nodes represent progressively larger cluster (Fig. 14.8). The distances used for clustering are derived from the weighted adjacency matrix of the epipolar graph, so the images are grouped according to their overlap.

The algorithm proceeds from the leaves to the root of the tree:

1. in the nodes where two leaves are joined (image pairs), we proceed with the reconstruction as described in Sect. 8.4;

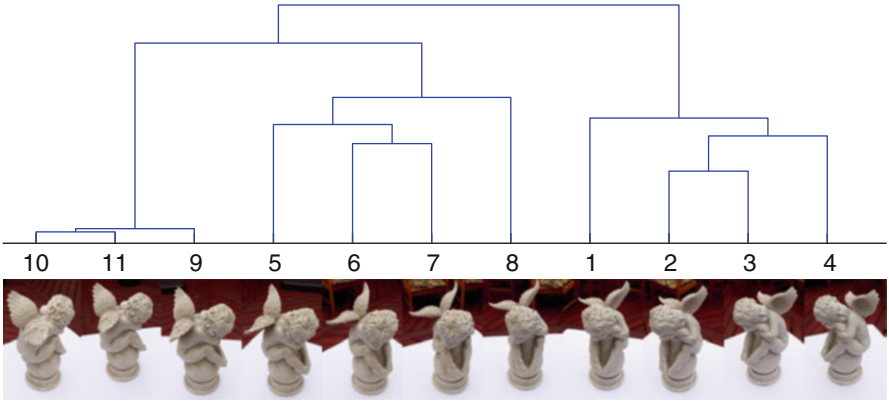


Fig. 14.8 Dendrogram relative to the cherub reconstruction. Pairs (10,11), (6,7) and (2,3) initiate the reconstructions; other images are added by resection; in the root node and its left child, two reconstructions are merged

2. in the nodes where a leaf joins an internal node, the reconstruction corresponding to the latter is increased by adding a single image via resection and updating the model via intersection (as in the previous sequential method);
3. in the nodes where two internal nodes join, the two independent reconstructions are merged by solving an absolute orientation (with OPA).

Step 1 is identical to the initialisation of the sequential method, but is performed on many pairs instead of just one; step 2 coincides with the iteration of the sequential method. Step 3, on the other hand, is reminiscent of the AIM. In fact, if the tree is perfectly balanced, step 2 is never executed and we obtain a cascade of absolute orientations that align the independent models built in step 1.

If, on the contrary, the tree is totally unbalanced, step 3 is never performed and we get the sequential method.

The approach is computationally less expensive and exhibits better error containment than the sequential approach, while also mitigating the dependence on the initial pair.

14.3 Frame-Based Approach

The approach of this section is based on the network of relative transformations between reference frames, and computes the model only at the end. It follows the third strategy outlined in Sect. 14.1.2, so it aims to instantiate the PPMs:

$$P_i = K_i[R_i, \mathbf{t}_i] \quad i = 1 \dots m > 3 \tag{14.5}$$

given the (error-prone) estimates of a number of relative orientations $\{R_{ij}, \hat{\mathbf{t}}_{ij}\}$, which correspond to the edges of the epipolar graph (Fig. 14.9).

If we fix the image I_1 as a reference, the rotations $R_{i1} = R_i$ and translations $\mathbf{t}_{i1} = \mathbf{t}_i$ that are needed to instantiate the PPMs are not directly available, since, in general, image I_1 does not overlap with all the other $m - 1$ images. They must be computed by composing the relative orientations (we assume for now that the moduli of the translations are known). An immediate solution is to compute a spanning tree for the epipolar graph with root in I_1 and concatenate the relative orientations along the tree. In this way, each $[R_{i1}, \mathbf{t}_{i1}]$ is uniquely determined (there is only one path between two nodes in a tree), but not all (redundant) measures are taken into account, resulting in the loss of the ability to compensate for error. The *synchronisation* procedure, which we will see in Sects. 14.3.1 and 14.3.2, instead implements a global compensated solution, which takes into account *all* the relative orientations available as edges of the epipolar graph.

Before proceeding, we fix the notation and derive some relations. We associate to each node i of the epipolar graph the exterior orientation of the corresponding camera, expressed as position and angular attitude with respect to a world reference frame. This orientation can be assigned by a matrix M_i representing a direct

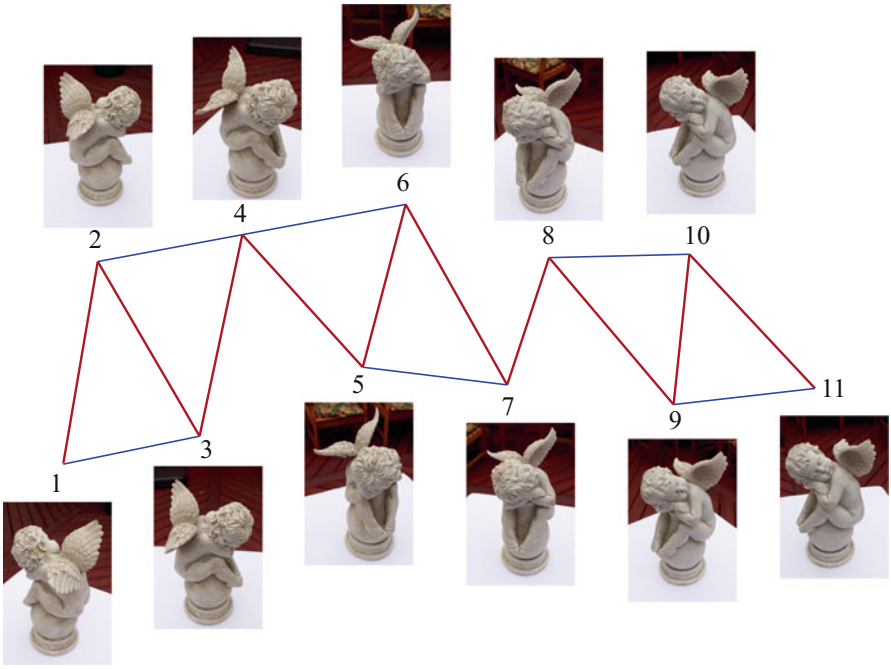


Fig. 14.9 Epipolar graph for cherub images. The path touching the images in numerical order forms a spanning tree. If the relative orientations were composed using only these elements, precious redundancy would be eliminated

isometry, which is the inverse of the usual matrix G_i found in the definition of the PPM $P_i = K[I, \mathbf{0}]G_i$, that is:

$$M_i = G_i^{-1} = \begin{bmatrix} R_i & \mathbf{t}_i \\ \mathbf{0} & 1 \end{bmatrix}^{-1} = \begin{bmatrix} R_i^\top & -R_i^\top \mathbf{t}_i \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} R_i^\top & \widetilde{\mathbf{C}}_i \\ \mathbf{0} & 1 \end{bmatrix} \tag{14.6}$$

Observe that the COP $\widetilde{\mathbf{C}}_i$ appears in M_i.

The edge (i, j) is labelled with the relative orientation:

$$M_{ij} = \begin{bmatrix} R_{ij} & \mathbf{t}_{ij} \\ \mathbf{0} & 1 \end{bmatrix} \tag{14.7}$$

inferred from E_{ij}. The following compatibility relationship exists:

$$M_{ij} = M_i^{-1} M_j = G_i G_j^{-1} \tag{14.8}$$

which, considering rotation and translation separately, becomes

$$R_{ij} = R_i R_j^\top \tag{14.9}$$

$$\mathbf{t}_{ij} = -R_{ij}\mathbf{t}_j + \mathbf{t}_i = R_i\widetilde{\mathbf{C}}_j - R_i\widetilde{\mathbf{C}}_i = R_i(\widetilde{\mathbf{C}}_j - \widetilde{\mathbf{C}}_i). \tag{14.10}$$

In the following paragraphs, we will focus first on rotations and then on translations. As we observed in the case of three images, the composition of isometries only works if the translations are known with their moduli, but this condition is false for the relative orientations computed from the essential matrices. We will deal with this problem in Sect. 14.3.3.

14.3.1 Synchronisation of Rotations

Let us focus initially on recovering the angular attitude of the cameras: the goal is to compute the rotations $R_i \in SO(3)$ that satisfy the compatibility constraint (14.9), that is:

$$R_{ij} = R_i R_j^\top \quad \forall (i, j). \tag{14.11}$$

The problem is known in the literature as *rotation averaging* (Hartley et al. 2013) or rotations synchronisation (Singer 2011). We immediately note that the solution is defined up to an arbitrary rotation of all R_i. To remain consistent with the solution for two images, we fix it so that $R_1 = I$.

In the presence of noise, we want to solve a minimum problem instead:

$$\min_{R_1,\ldots,R_m \in SO(3)} \sum_{(i,j)} \| R_{ij} - R_i R_j^\top \|_F^2. \tag{14.12}$$

We will now devise a solution based on the eigendecomposition of a matrix that solves a relaxation of problem (14.12). For this purpose, we introduce the following $3m \times 3m$ matrix that contains all relative rotations (we initially assume that all $\binom{m}{2}$ image pairs allow the computation of relative orientation):

$$Z = \begin{bmatrix} I & R_{12} & \ldots & R_{1m} \\ R_{21} & I & \ldots & R_{2m} \\ \ldots & & & \ldots \\ R_{m1} & R_{m2} & \ldots & I \end{bmatrix}. \tag{14.13}$$

Note that $R_{ij} = R_{ji}^\top$, so Z is symmetric. Let X be the $3m \times 3$ matrix constructed by stacking the unknown rotation matrices:

$$X = \begin{bmatrix} R_1 \\ R_2 \\ \cdots \\ R_m \end{bmatrix}. \tag{14.14}$$

From the compatibility constraint (14.9), it follows that Z can be written as

$$Z = XX^{\mathsf{T}}. \tag{14.15}$$

So $\text{rank}(Z) = \text{rank}(X) = 3$ and Z possesses three nonzero eigenvalues.

We multiply both members by X obtaining

$$ZX = XX^{\mathsf{T}}X = mX \tag{14.16}$$

since $X^{\mathsf{T}}X = mI$.

The relation (14.16) tells us that the columns of X are the three eigenvectors corresponding to the nonzero eigenvalues of Z, which are equal to m.

In a practical scenario hardly all relative rotations are available; in such a case, the blocks of Z corresponding to the unknown rotations are set to zero

The i-th block of ZX writes:

$$\begin{bmatrix} R_{i1} & R_{i2} & \ldots & R_{im} \end{bmatrix} \begin{bmatrix} R_1 \\ R_2 \\ \cdots \\ R_m \end{bmatrix} = R_{i1}R_1 + R_{i2}R_2 + \cdots + R_{im}R_m = d_i R_i \tag{14.17}$$

where d_i is the number of nonzero blocks in block-row i of Z, which is equal to m if all the relative orientations are available. Let us collect these d_i into a diagonal matrix $D = \text{diag}(d_1, d_2, \cdots, d_n)$, then (14.16) becomes:

$$ZX = (D \otimes I_3)X \tag{14.18}$$

where the Kronecker product creates a diagonal matrix where each d_i occurs three consecutive times on the diagonal, because each d_i must multiply the 3×3 matrix R_i.

In other words, the columns of X are the three eigenvectors of $(D \otimes I)^{-1}Z$ corresponding to eigenvalue 1, the other eigenvalues being zero.

In the presence of noise, we take the three dominant eigenvectors of $(D \otimes I)^{-1}Z$, and this is the solution to a relaxation of the problem (14.12), where the constraint $R_i \in SO(3)$ is disregarded. The 3×3 blocks of X corresponding to $R_1, R_2, \ldots, R_m$ are thus projected onto $SO(3)$ only at the end by computing the nearest rotation matrix according to Proposition A.14.

Considering the epipolar graph, it is easy to see that d_i is the degree of node i and D is the *degree matrix* of the graph. In the complete graph, all nodes have degree

m, if we assume that each node has a loop, which in the Z corresponds to the I on the diagonal.

The MATLAB implementation is given in Listing 14.1.

Listing 14.1 Rotation synchronisation

```
function R = rotation_synch(Z,A)
%ROTATION_SYNCH Rotation syncronization

    n = size(A,1);

    iD = diag(1./sum(A,2));              % inverse degree matrix
    [Q,~] = eigs( kron(iD,eye(3))*Z,3,'lr'); % top 3 eigenvectors

    Q = real(Q/(Q(1:3, 1:3))); % normalize first rotation to I
    % and guard against spurious complex valuess from roundoff

    R=cell(1,n);
    for i=1:n  % Projection onto SO(3)
        [U,~,V] = svd(Q(3*i-2:3*i,:));
        R{i} = U*diag([1,1,det(U*V')])*V';
    end
end
```

14.3.2 Synchronisation of Translations

We now deal with the recovery of the COPs $\mathbf{C}_i$ under the assumption that the relative translations are known with the moduli. The translation components of the PPM $\mathbf{t}_i \in \mathbb{R}^3$ will eventually be derived from the relation $\widetilde{\mathbf{C}}_i = -R_i^\mathsf{T}\mathbf{t}_i$, where $\widetilde{\mathbf{C}}_i$ is the centre of the i-th camera in Cartesian coordinates.

We consider the compatibility equation for translations (14.10):

$$\mathbf{t}_{ij} = R_i(\widetilde{\mathbf{C}}_j - \widetilde{\mathbf{C}}_i). \tag{14.19}$$

which we rewrite as

$$R_i^\mathsf{T}\mathbf{t}_{ij} = \widetilde{\mathbf{C}}_j - \widetilde{\mathbf{C}}_i = \mathbf{u}_{ij}, \tag{14.20}$$

where $\mathbf{u}_{ij}$ represents the relative translation expressed in the world reference frame, which we fixed integral to the first PPM.

We now consider the $3 \times m$ matrix X obtained from the juxtaposition of the COPs: $X = [\widetilde{\mathbf{C}}_1, \widetilde{\mathbf{C}}_2, \ldots \widetilde{\mathbf{C}}_m]$. Then (14.20) is written:

$$X\mathbf{b}_{ij} = \mathbf{u}_{ij} \tag{14.21}$$

where the indicator vector

$$\mathbf{b}_{ij} = (0, \ldots, \underset{\uparrow}{-1}_{i}, \ldots, \underset{\uparrow}{1}_{j}, \ldots, 0)^\mathsf{T} \tag{14.22}$$

selects columns i and j from X. We see that $\mathbf{b}_{ij}$ represents an edge of the epipolar graph G, and in fact it is one of the ℓ columns of the *incidence matrix* $B(G)$. Hence, the ℓ equations that we can write, one for each edge, are expressed in matrix form as

$$XB = U \tag{14.23}$$

where the $3 \times \ell$ matrix U contains the $\mathbf{u}_{ij}$ as columns. The matrix B is $m \times \ell$, but its rank is at most $m - 1$, if the epipolar graph is connected, so the system is underdetermined. However, the solution is clearly defined up to a translation of all centres, which is why we can arbitrarily fix $\widetilde{\mathbf{C}}_1 = \mathbf{0}$ (to remain consistent with the solution for two images) and remove it from the unknowns, thus leaving a $m - 1 \times \ell$ B_1 matrix of full rank. Thanks to the "vec-trick", we can write:

$$(B_1^{\mathsf{T}} \otimes I) \operatorname{vec} X_1 = \operatorname{vec} U. \tag{14.24}$$

If one considers the errors that inevitably plagues the measurements, then an approximate solution must be sought. Russell et al. (2011) show that the least-squares solution of (14.24) among all those satisfying the compatibility constraints is the one closest to the measurements, in Euclidean norm.

The MATLAB implementation is given in Listing 14.2.

Listing 14.2 Translation synchronisation

```
function T = translation_synch(U,B)
%TRANSLATION_SYNCH  Translation synchronization

    B(1,:) = []; % remove node 1

    X=kron(B',eye(3)) \ U(:);
    X=[0;0;0;X]; % add node 1
    X=reshape(X,3,[]);

    T = num2cell(reshape(X,3,[]),[1,size(X,2)]);

end
```

The two examples of synchronisation that we have seen can be extended to other cases (Arrigoni and Fusiello 2020). In fact, abstracting from the two specific problems, synchronisation emerges when one has a graph whose nodes are labelled with elements of a *group* and the edges are labelled with differences (or ratios, depending on the group) of the elements associated with the two adjacent nodes. The name comes from clock synchronisation, an application in which pairs of devices measure their time lags, and all must agree on a single "global" time that accounts for the time lags.

14.3.3 Localisation from Bearings

In the context of SFM, the synchronisation of translations does not solve the problem of localizing the cameras, since the moduli of the translations are unknowns. In other words, instead of the translation $\mathbf{u}_{ij}$, we only can measure the unit vector:

$$\hat{\mathbf{u}}_{ij} = R_i^{\mathsf{T}} \hat{\mathbf{t}}_{ij} \tag{14.25}$$

which is called *bearing*,[2] and represents the direction under which the i-camera sees the j-camera, expressed in the world reference frame, while the $\hat{\mathbf{t}}_{ij}$ corresponds to the same direction in the i-camera reference frame .

Following Strategy 3 one should first compute the unknown moduli of the translations and then run translation synchronisation. This has been done by Arrigoni et al. (2015), but here we will follow a simpler approach (Brand et al. 2004) that directly computes the positions of the cameras $\widetilde{\mathbf{C}}_i$.

We start with the translation synchronisation equation (14.24) and multiply both members by the $3\ell \times 3\ell$ block diagonal matrix $\widehat{S}$:

$$\widehat{S} = \text{blkdiag}\left(\left\{ \left[\hat{\mathbf{u}}_{ij} \right]_\times \right\}_{(i,j)\in E} \right) \tag{14.26}$$

obtaining

$$\widehat{S}(B_1^{\mathsf{T}} \otimes I) \,\text{vec}\, X_1 = \widehat{S}\,\text{vec}\, U = 0. \tag{14.27}$$

The right-hand member cancels out as each $\mathbf{u}_{ij}$ (column of U) is multiplied on the left by the corresponding $\left[\hat{\mathbf{u}}_{ij}\right]_\times$, which is equivalent to the cross product of a vector by its own versor. The net effect is to replace U, which contains the unknown moduli, with $\widehat{S}$, which depends only on the known bearings. In the noise-free case, (14.27) has a unique solution up to a scale if and only if $\widehat{S}(B_1^{\mathsf{T}} \otimes I)$ possesses a one-dimensional null space.

> While translation synchronisation has solution as soon as the epipolar graph is connected, the conditions on the graph topology under which (14.27) is solvable are more complex and call into question the notion of *rigidity* of the graph (Arrigoni and Fusiello 2019). Generally, in order to be rigid, the graph must have more edges than those ensuring connectivity. For example, in the three- view case, edge 1–3 was needed to solve the problem.

[2] In analogy to the bearing angle of a point, which in topography is defined as the angle between the cardinal north direction and the observer-point direction.

Since equation (14.27) is homogeneous, the solution is defined up to a scale, which potentially includes a sign, and this introduces a reflection of the solution (i.e. if X is solution so is $-X$). Geometrically it means that the solution of (14.27) considers the bearings collected in U only as orientation and neglects the sense.[3] The latter can be taken into account a posteriori by choosing between X and $-X$ the solution that agrees with the direction of the bearings (Listing 14.3).

In the case of noise-affected bearings, $\widehat{S}(B_1^\mathsf{T} \otimes I)$ will in general have full rank and a solution is sought that solves

$$\min_{\|X\|=1} \|\widehat{S}(B_1^\mathsf{T} \otimes I) \operatorname{vec} X_1\|^2. \tag{14.28}$$

The least right singular vector of the coefficients matrix is the solution, as usual.

Listing 14.3 Localisation from bearings

```
function C = cop_from_bearings(U,B)
%COP_FROM_BEARINGS  Camera locations from bearings

    B(1,:) = []; % remove node 1

    S=kron(eye(size(B,2)),ones(3,3));
    for i=1:size(U,2)
        S(3*i-2:3 *i,3*i-2:3*i) = skew(U(:,i)) ;
    end

    [~,~,V] = svd(S*kron(B',eye(3)));
    X =  reshape(V(:,end),3,[]);
    s = sign(U(:,1)'*X*B(:,1)); % sign
    X = s * [[0;0;0],X]; % add node 1 and fix sign

    C = num2cell(reshape(X,3,[]),[1,size(X,2)]);
end
```

The COPs, along with the rotations previously recovered, allow us to instantiate the PPMs:

$$P_1 = [I, \mathbf{0}],\ P_2 = [R_2, -R_2\widetilde{\mathbf{C}}_2], \ldots P_m = [R_m, -R_m\widetilde{\mathbf{C}}_m] \tag{14.29}$$

and then proceed to triangulate the points.

14.4 Bundle Adjustment

Both point-based and frame-based methods are not statistically optimal, the former because they produce the final model incrementally, the latter because they are global only in compensating for relative orientations but neglect points, which only

[3] Orientation and sense together determine the direction of a vector.

played a role in determining the relative orientations between pairs of images. A genuinely global method should optimise a cost that simultaneously includes all images and all tie points.

Optimal reconstruction from many views is naturally formulated as the Bundle Adjustment (BA) problem: one wants to minimise the overall reprojection error of the tie points in the images in which they are visible, with respect to the 3D coordinates of the tie points and the orientation (exterior and interior) of the images.

The name comes from a geometric interpretation of the problem: each image with its optical rays corresponding to the tie points can be seen as a bundle of rays, and "adjustment" refers to the process of orienting bundles in space until homologous optical rays intersect. If control points are present, the rays should also touch them.

Analytically, it is a matter of adjusting both the m cameras and the n tie points so that the sum of squared distances between the j-th tie point reprojected via the i-th camera $P_i\mathbf{M}^j$ and the measured point $\mathbf{m}_i^j$ is as small as possible, in each image where the point appears. In formulae (the notation resumes that introduced in (9.8)):

$$\chi(\widetilde{\mathbf{M}}^j, P_i) = \sum_{i=1}^{m}\sum_{j=1}^{n}\|\widetilde{\delta}(P_i\mathbf{M}^j) - \widetilde{\mathbf{m}}_i^j\|^2. \tag{14.30}$$

This is the reprojection error that we have already employed in Chap. 9, and in fact BA is nothing more than a simultaneous regression of the PPMs (Sect. 9.3) and the 3D points (Sect. 9.5).

Equation (14.30) is the cost function of a non-linear least-squares problem, for which there are no closed-form solutions, but only iterative techniques (such as the Levenberg-Marquardt (LM) algorithm) that require a starting point close enough to the global minimum. Thus, BA does not solve the SFM problem by itself, but can be understood as a way to improve suboptimal solutions, such as those obtained by the methods described in the first part of the chapter.

Let us consider BA in the case of two images. We know that five points are sufficient in principle to determine the essential matrix, which leads to a reconstruction up to a similarity. Let us confirm this result by a simple equations count. Five tie points in two images give 20 equations; the unknowns are the 3D coordinates of the tie points, which makes 15, plus the 12 unknowns corresponding to the orientations of the 2 cameras, thus giving 27 unknowns. The remaining seven degrees of freedom corresponds to those of a similitude. In other words, out of 12 unknowns, only 5 can be determined, which represent the degrees of freedom of the relative orientation up to scale.

For further discussion of BA, see Triggs et al. (2000), Engels et al. (2006), Konolige (2010).

14.4.1 Jacobian of Bundle Adjustment

The BA residual is the reprojection error, and the unknowns are the camera parameters $\mathbf{v}$ (intrisic and extrinsic) and the point coordinates $\mathbf{M}$. As such, the derivatives with respect to each of these two sets of unknowns have already been computed in Chap. 9. In particular expressions (9.12), (9.13), (9.16) and (9.23) are summarised here for the readers' convenience:

$$\begin{aligned}
\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \mathbf{v}^{\mathsf{T}}} &= P_2 \left[K \, \mathsf{D}\delta([R,\mathbf{t}]\mathbf{M}) \, (\mathbf{M}^{\mathsf{T}} \otimes I_4) \begin{bmatrix} \mathsf{D}R & 0 \\ 0 & I_3 \end{bmatrix}, \left(\delta([R,\mathbf{t}]\mathbf{M})^{\mathsf{T}} \otimes I_3\right) \mathsf{D}K \right] \\
\frac{\partial \widetilde{\delta}(P\mathbf{M})}{\partial \widetilde{\mathbf{M}}^{\mathsf{T}}} &= P_2 K \; \mathsf{D}\delta([R,\mathbf{t}]\mathbf{M}) \, R.
\end{aligned} \tag{14.31}$$

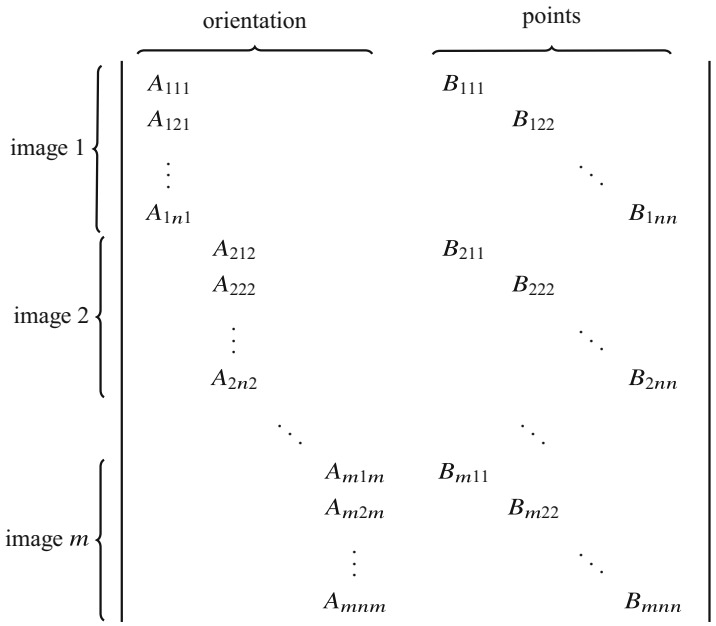
The Jacobian of the least-squares problem corresponding to (14.30) is therefore composed of blocks of this type. Let us define:

$$A_{ijk} = \frac{\partial \widetilde{\delta}(P_i \mathbf{M}^j)}{\partial \mathbf{v}_k^{\mathsf{T}}} \tag{14.32}$$

the matrix of partial derivatives of the residual of point j in image i with respect to the parameters of camera k. And similarly:

$$B_{ijk} = \frac{\partial \widetilde{\delta}(P_i \mathbf{M}^j)}{\partial \widetilde{\mathbf{M}}_k^{\mathsf{T}}} \tag{14.33}$$

are the partial derivatives of the residual of point j in image i with respect to the coordinates of point k. It is easy to see that $A_{ijk}{=}0 \quad \forall i{\neq}k$ and $B_{ijk}{=}0 \quad \forall j{\neq}k$. Thus, the Jacobian matrix has a sparse block structure, called *primary structure* and represented graphically in the following diagram:



If we consider only one image and control points instead of tie points, only the blocks A_{111} to A_{1n1} remain that correspond to the Jacobian of the residual of non-linear calibration (n is the number of control points). If, on the other hand, cameras are known, only the blocks B remain, that correspond to the Jacobian of the residual of the triangulation. This is consistent with the observation that BA is tantamount to a simultaneous regression of the PPMs and of the tie points.

In the beginning of this chapter, we assumed that intrinsic parameters are known, whereas in this section we are dealing with the general case in which both exterior and interior orientations are unknown. It is understood that the calibrated case is easily dealt with by omitting the intrinsic parameters from the unknowns.

Note that in practice the structure of the Jacobian matrix (Fig. 14.10) also reflects the visibility of the points since the two rows corresponding to $\mathbf{m}_i^j$ are only present if the point $\mathbf{M}^j$ is visible in the camera P_i. The shape of the matrix derived from this observation is called *secondary structure*.

Because one can scale, rotate and translate the entire reconstruction without altering the reprojection error, the Jacobian has a rank deficiency corresponding to the degrees of freedom of a similitude, which is seven. As a consequence, the normal equation of the Gauss-Newton method is underdetermined. There are several ways to solve this problem:

- remove the degrees of freedom by fixing the coordinates of some points (which become control points) and/or some COPs;

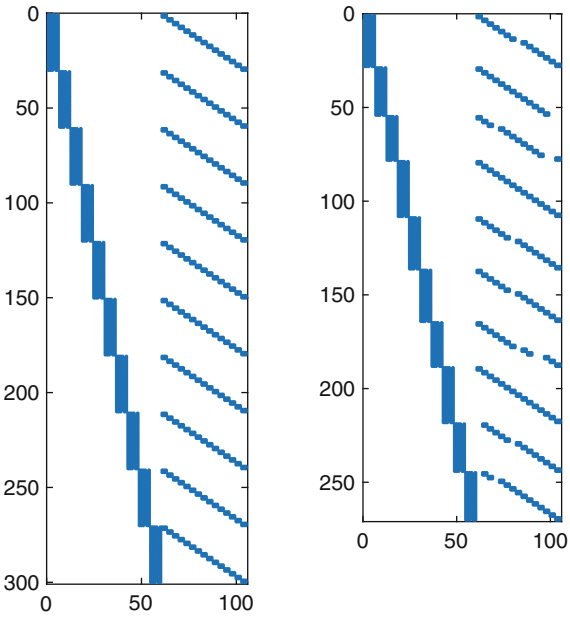


Fig. 14.10 Example of primary (left) and secondary (right) structure for a Jacobian matrix of a BA with 10 images and 15 tie points

- solve the normal equations with the pseudoinverse, which in the underdetermined case returns the minimum norm solution, thus implicitly imposing a constraint on the solution, as in our Gauss-Newton implementation (Listing C.1).
- add a diagonal damping term in the normal equation which makes up for the rank drop, as the LM method does (Sect. C.2.3).

With reference to the terminology introduced in Chap. 8, the first solution leads to an identical reconstruction, while the last two lead to a Euclidean reconstruction.

In photogrammetry, an overabundant number of control points are used in the BA, which—in addition to fixing the degrees of freedom—also allow to "bend" the model and compensate for any systematic errors.

14.4.2 Reduced System

The primary structure of the Jacobian matrix that we observed in the previous section can be exploited to reduce the computational cost of solving the normal equations, through the formulation of a *reduced* system of equations.

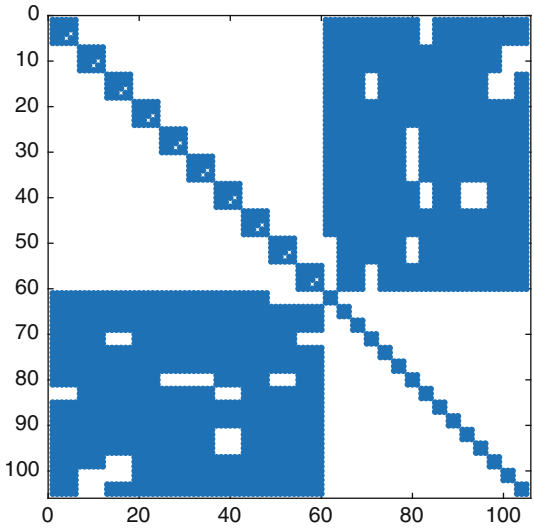


Fig. 14.11 Structure of matrix H for the same example of Fig. 14.10. The fact that the two NW and SE blocks are block diagonals depends on the primary structure, while the sparsity pattern of the NE and SW blocks depends on the secondary structure, that is, visibility

The normal equation that is solved for $\Delta\mathbf{x}$ at each Gauss-Newton step is

$$\underbrace{J^\top J}_{H}\,\Delta\mathbf{x} = -J^\top f(\mathbf{x}) \tag{14.34}$$

where $\mathbf{x}$ is the vector of variables containing $3 \times n$ parameters for the points $\mathbf{M}^j$ and $11 \times m$ parameters for the cameras ($6 \times m$ if we exclude the intrinsic parameters), and $f(\mathbf{x})$ are the residuals $\widetilde{\delta}(P_i\mathbf{M}^j) - \widetilde{\mathbf{m}}_i^j$ in (14.30). The residuals are $2mn$ if all points are visible in all cameras, although typically the number is much smaller. Matrix H has dimension $2mn \times 3n + 6n$: its size is largely dominated by n which can be a few orders of magnitude larger than m.

It is therefore a matter of solving a linear system that can be very large, but one can take advantage of the sparse structure of the Jacobian matrix. In particular, we partition the Jacobian into two parts, the one related to the cameras and the one related to the points:

$$J = \begin{bmatrix} J_c & J_p \end{bmatrix}. \tag{14.35}$$

Then, the matrix H (Fig. 14.11) and the normal equation also turn out to be partitioned as

$$\begin{bmatrix} J_c^\top J_c & J_c^\top J_p \\ J_p^\top J_c & J_p^\top J_p \end{bmatrix} \begin{bmatrix} \Delta\mathbf{x}_c \\ \Delta\mathbf{x}_p \end{bmatrix} = \begin{bmatrix} -J_c^\top f([\mathbf{x}_c, \mathbf{x}_p]) \\ -J_p^\top f([\mathbf{x}_c, \mathbf{x}_p]) \end{bmatrix}. \tag{14.36}$$

To simplify the notation, we rewrite it as

$$\begin{bmatrix} H_{cc} & H_{cp} \\ H_{pc} & H_{pp} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}_c \\ \Delta \mathbf{x}_p \end{bmatrix} = \begin{bmatrix} b_c \\ b_p \end{bmatrix}. \tag{14.37}$$

Note that H_{cc} and H_{pp} are block diagonals. We multiply both members to the left by

$$\begin{bmatrix} I & -H_{cp} H_{pp}^{-1} \\ 0 & I \end{bmatrix} \tag{14.38}$$

thereby transforming the coefficient matrix into lower block triangular:

$$\begin{bmatrix} H_{cc} - H_{cp} H_{pp}^{-1} H_{pc} & 0 \\ H_{pc} & H_{pp} \end{bmatrix} \begin{bmatrix} \Delta \mathbf{x}_c \\ \Delta \mathbf{x}_p \end{bmatrix} = \begin{bmatrix} b_c - H_{cp} H_{pp}^{-1} b_p \\ b_p \end{bmatrix} \tag{14.39}$$

and therefore solve for the block of unknowns $\mathbf{x}_c$ (cf. Gaussian elimination):

$$(H_{cc} - H_{cp} H_{pp}^{-1} H_{pc}) \Delta \mathbf{x}_c = b_c - H_{cp} H_{pp}^{-1} b_p. \tag{14.40}$$

This system is smaller than the original, and the inversion of H_{pp} is straightforward due to its diagonal block structure, so we get a benefit from this rewriting. The other block of unknowns is derived with

$$\Delta \mathbf{x}_p = H_{pp}^{-1} (b_p - H_{pc} \Delta \mathbf{x}_c). \tag{14.41}$$

The secondary structure of J is reflected in the reduced system matrix: $H_{cc} - H_{cp} H_{pp}^{-1} H_{pc}$. If each image sees only a fraction of the points, that matrix will be sparse; in particular if images are ordered such that index differences reflect the degree of overlap, the reduced system matrix has a band structure, and this can be usefully exploited to reduce the computational cost.

The function `bundleadj` (listing omitted for space reasons) implements BA with the LM method and the reduced system. It is possible to choose how many intrinsic parameters to include among the unknowns, and whether they are all the same or different for each image. Also, one can designate some 3D points as control points.

In our implementation of BA, what does that guarantee that we do not run into some singularity in the Euler representation of rotations? Nothing, in fact. We would need to adopt an incremental parameterisation of the exterior orientation of each image, as in the parameterisation of F.

References

F. Arrigoni and A. Fusiello. Bearing-based network localizability: A unifying view. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41 (9): 2049–2069, 2019. ISSN 0162-8828. https://doi.org/10.1109/TPAMI.2018.2848225.

Federica Arrigoni and Andrea Fusiello. Synchronization problems in computer vision with closed-form solutions. *International Journal of Computer Vision*, 128 (1): 26–52, Jan 2020. ISSN 1573-1405. https://doi.org/10.1007/s11263-019-01240-x.

Federica Arrigoni, Andrea Fusiello, and Beatrice Rossi. On computing the translations norm in the epipolar graph. In *Proceedings of the International Conference on 3D Vision (3DV)*, pages 300–308. IEEE, 2015. https://doi.org/10.1109/3DV.2015.41.

M. Brand, M. Antone, and S. Teller. Spectral solution of large-scale extrinsic camera calibration as a graph embedding problem. 2004.

M. Brown and D. Lowe. Recognising panoramas. In *Proceedings of the 9th International Conference on Computer Vision*, volume 2, pages 1218–1225, Nice, October 2003.

F. Crosilla and A. Beinat. Use of generalised procrustes analysis for the photogrammetric block adjustment by independent models. *ISPRS Journal of Photogrammetry & Remote Sensing*, 56 (3): 195–209, April 2002.

C. Engels, H. Stewénius, and D. Nistér. Bundle adjustment rules. In *Photogrammetric Computer Vision (PCV)*. ISPRS, September 2006.

Riccardo Gherardi, Michela Farenzena, and Andrea Fusiello. Improving the efficiency of hierarchical structure-and-motion. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2010)*, 2010.

Richard I. Hartley, Jochen Trumpf, Yuchao Dai, and Hongdond Li. Rotation averaging. *International Journal of Computer Vision*, 2013.

Kurt Konolige. Sparse sparse bundle adjustment. In *British Machine Vision Conference*, Aberystwyth, Wales, 08/2010 2010.

W. J. Russell, D. J. Klein, and J. P. Hespanha. Optimal estimation on the graph cycle space. *IEEE Transactions on Signal Processing*, 59 (6): 2834–2846, June 2011.

A. Singer. Angular synchronization by eigenvectors and semidefinite programming. *Applied and Computational Harmonic Analysis*, 30 (1): 20–36, 2011.

Roberto Toldo, Riccardo Gherardi, Michela Farenzena, and Andrea Fusiello. Hierarchical structure-and-motion recovery from uncalibrated images. *Computer Vision and Image Understanding*, 2015.

Bill Triggs, Philip F. McLauchlan, Richard I. Hartley, and Andrew W. Fitzgibbon. Bundle adjustment—a modern synthesis. In *Proceedings of the International Workshop on Vision Algorithms*, pages 298–372. Springer, Berlin, 2000.

C. Zeller and O. Faugeras. Camera self-calibration from video sequences: the Kruppa equations revisited. Research Report 2793, INRIA, February 1996.

Chapter 15
3D Registration



15.1 Introduction

The purpose of *registration* or *alignment* of partial 3D models is to bring them all into the same reference frame by means of a suitable transformation. This problem has strong similarities with image mosaicking, which we will discuss in Chap. 18.

This chapter takes its starting point from the Adjustment of Independent Models (AIM) introduced in 14 and complete it by describing (in Sect. 15.1.1) how to extend the alignment of two sets of 3D points to the *simultaneous* alignment of many sets, with given correspondences. Such sets of 3D points are also called *point clouds* in this context.

The registration problem arises also when considering model acquisition by a range sensor. In fact, range sensors such as the ones we studied Chap. 13 typically fail to capture the shape of an object with a single acquisition: many are needed, each spanning a part of the object's surface, possibly with some overlap (Fig. 15.1). These partial models of the object are each expressed in its own reference frame (related to the position of the sensor).

While in the AIM the correspondences are given by construction, the partial point clouds produced by range sensors do not come with correspondences. In Sect. 15.2.2 we will present an algorithm called Iterative Closest Point (ICP) that aligns two sets of points without knowing the correspondences, as long as the initial misalignment is moderate. We will finally (Sect. 15.2.3) mention how to extend ICP for registration of many point clouds by exploiting synchronisation (Sect. 14.3).

For a general framing of the problem, see also Bernardini and Rushmeier (2002).

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_15



Fig. 15.1 Registration of partial 3D models. On the right, eight range images of an object, each in its own reference frame. On the left, all images superimposed after registration. Courtesy of F. Arrigoni

15.1.1 Generalised Procrustes Analysis

The registration of more than two point clouds in a single reference frame can be simply obtained by concatenating registrations of overlapping pairs. This procedure, however, does not lead to the optimal solution. For example, if we have three overlapping point clouds and we align sets one and two and then sets one and three, we are not necessarily minimising the same cost function between sets two and three. We therefore need adjustment procedures that operate globally, that is, take into account all sets simultaneously.

If correspondences between the different point clouds are available, we can merge them in a single operation, applying to each the similitude that brings it into a common reference frame, thus generalising the absolute orientation via Orthogonal Procrustes analysis (OPA) that we saw earlier (Sect. 5.2.1). This technique is in fact referred to as Generalised Procrustes analysis (GPA).

Given the matrices $M_1, M_2, \dots M_m$ representing the m sets of points $\mathcal{M}_1, \mathcal{M}_2, \dots \mathcal{M}_m$, the alignment is achieved by minimising the following cost function:

$$\sum_{i=1}^{m} \sum_{j=i+1}^{m} \|s_i(R_i M_i + \mathbf{t}_i \mathbf{1}^\top) - s_j(R_j M_j + \mathbf{t}_j \mathbf{1}^\top)\|_F^2 \tag{15.1}$$

where the unknowns are the parameters of the similitude: s_i, R_i, $\mathbf{t}_i$. At the core of this technique lies the following observation by Commandeur (1991):

$$\sum_{i=1}^{m} \sum_{j=i+1}^{m} \|B_i - B_j\|_F^2 = m \sum_{i=1}^{m} \|B_i - \langle B_i \rangle\|_F^2, \tag{15.2}$$

where $\langle B_i \rangle$ is the mean or centroid:

$$\langle B_i \rangle = \frac{1}{m} \sum_{i=1}^{m} B_i . \tag{15.3}$$

The minimisation of the residual rewritten as in (15.2) turns out to be expressible as an iterative process, in which one alternately compute $\langle B_i \rangle$ and align the sets $\mathcal{M}_i$ on $\langle B_i \rangle$, each separately with OPA, as seen in Listing 15.1. Convergence of the algorithm is proved in (Commandeur 1991), although not necessarily at the global minimum.

In the realistic case that not all correspondences are available in all point clouds, one must multiply each M_i with a diagonal matrix W_i that contains on the diagonal 1 where the corresponding column of M_i is specified and 0 otherwise. The formulae for the solution of the absolute orientation are adapted straightforwardly, with the only caveat that the vector $\mathbf{1}$ must also be replaced with $W\mathbf{1}$ and that the formula for the centroid becomes (Crosilla & Beinat 2002):

$$\langle B_i \rangle = \left(\sum_{i=1}^{m} B_i W_i \right) \left(\sum_{i=1}^{m} W_i \right)^{-1} . \tag{15.4}$$

Listing 15.1 Generalised Procrustean Analysis

```
function  [G,D] = gpa(M,w)
    %GPA Generalized Procrustes analysis
    % with similarity (7 parameters)

    m = length(M);

    % constants
    MaxIterations = 50;
    FunctionTol = 1e-3;
    StepTol = 1e-16;

    G = mat2cell(repmat(eye(4),1,m),4,4*ones(1,m));
    res_n = cell(1,m); res = Inf;
    for iter = 1: MaxIterations

        % compute centroid
        YW = cellfun(@(X,u) bsxfun(@times, u',X), M,w, 'uni',0);
        D = sum(cat(3,YW{:}),3);
        u = 1./sum(cat(3,w{:}),3);  u(isinf(u))=0; % fix divison by 0
        D = bsxfun(@times, u', D);  % same as D = D*diag(u) but faster

        % align each M{i} onto centroid
        for i = 1:m
            [R,t,s] = opa(D,M{i},w{i});
            G{i} = [R, t; 0 0 0 1/s] * G{i};
            M{i} = s*(R*M{i} + t);
            res_n{i} = sum(sum((bsxfun(@times,w{i}',(M{i}-D))).^2));
        end

        prevres=res;
        res = sum([res_n{:}])/m; % norm(res)
        if norm(res-prevres)<StepTol || norm(res)<FunctionTol
            break; end

    end
```

15.2 Correspondence-Less Methods

When correspondences are not given, one can either compute them, and fall back to OPA/GPA methods, or devise different techniques that do not explicitely need them. As for the first option, 3D salient points can be extract and matched, along the lines of what is done in Structure from Motion (SFM) (Chap. 14). See the review of Tombari et al. (2013) in this regard. In this chapter we will deal with the second option.

15.2.1 Registration of Two Point Clouds

The OPA computes the alignment (or registration) of two sets of corresponding 3D points. There are practical cases, however, where two point clouds must be registered for which the correspondences are not known. In such a case, an algorithm called ICP (Besl & McKay 1992; Chen & Medioni 1992) can be used, which simultaneously solves the correspondences problem and estimates the direct isometry.

15.2.2 Iterative Closest Point

Given two sets of points $\mathcal{M}$ and $\mathcal{D}$, with $\mathcal{M} \subseteq \mathcal{D}$, the ICP algorithm computes for each point $\mathbf{M}_i$ of the set $\mathcal{M}$, the point $\mathbf{D}_i \in \mathcal{D}$ closest to it. It then assumes that the closest points are the true correspondents, and computes the direct isometry that best aligns the corresponding points (with OPA, Sect. 5.2). Apply the obtained isometry to $\mathcal{M}$ and iterate the process until convergence.

The idea is that, even if the correspondences assumed in the iterations are not the true ones, the resulting transformation brings $\mathcal{M}$ closer to $\mathcal{D}$. Figure 15.2 shows an example of how the correspondences converge to the correct ones. Besl and McKay (1992) prove that the algorithm converges to a local minimum of the error:

$$\min_{R,\mathbf{t}} \sum_{i=1}^{n} \mathbf{D}_i - (R\mathbf{M}_i + \mathbf{t})\|^2. \tag{15.5}$$

To ensure convergence to the global minimum, a good initial estimate of the isometry between the sets of points is required (in practice we need the initial rotation to differ by no more than 20° from the correct one). The initial alignment can be obtained:

- by measuring the motion of the sensor;
- by sampling the set of all possible transformations;
- by considering the moments of $\mathcal{D}$ and $\mathcal{M}$ (mean and principal axes).

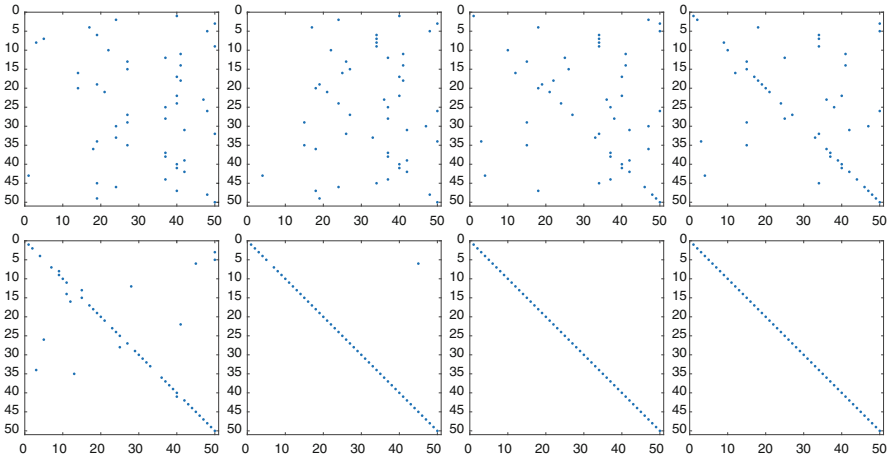


Fig. 15.2 Evolution of correspondences assumed by ICP in subsequent iterations. It can be seen that by starting with very wrong matches, the algorithm quickly converges to correct ones, represented in this example by the identity matrix

The algorithm just described assumes that $\mathcal{M} \subseteq \mathcal{D}$, while in reality the intersection of the two sets is not completely contained in either of them. In this case there will be necessarily mismatches, since the nearest-neighbour-based procedure always finds a match at every point of $\mathcal{M}$, but not every point of $\mathcal{M}$ actually has a corresponding point. These mismatches spoil the estimate obtained by OPA, and prevent convergence: even if the two sets were initially perfectly aligned, ICP could diverge. Mismatches can be treated as outliers and the problem addressed with the tools of robust statistics.

The code shown in Listing 15.2 applies a robust weight function to underweight corresponding points according to their distance (Fig. 15.3).

ICP can also be applied to surfaces, that is, points connected in a polygonal mesh. In the latter case, given a set of points $\mathcal{M}$ and a surface $\mathcal{D}$, for every point $\mathbf{M}_i \in \mathcal{M}$, there exists at least one point $\mathbf{D}_i$ on the surface $\mathcal{D}$ that is closest to $\mathbf{M}_i$, and we take it as the corresponding one.

In addition, it is possible to introduce an heuristic that eliminates matchings on the border of the mesh, based on the observation that, in the case of partially overlapping surfaces, it is likely that points belonging to the unique part of one surface will be matched with points belonging to the border of the other surface. These correspondences are obviously wrong (since these points should not be paired), and therefore are eliminated.

In the case of surfaces, however, to make point matching faster, one can also abandon the search for nearest points by matching a point $\mathbf{M}_i$ with the point of $\mathcal{D}$ encountered along the normal to the surface $\mathcal{M}$ at point $\mathbf{M}_i$ (*normal shooting*). For further details we refer to the reading of (Rusinkiewicz 2001).

Listing 15.2 Iterative closest point

```
function [R,t] = icp(D,M)
%ICP Iterative Closet Point algorithm

    MaxIterations = 400;
    FunctionTol = 1e-6;
    StepTol = 1e-6;
    G=eye(4);  res = Inf;

    for iter = 1: MaxIterations

        prevres = res;
        % compute closest points (in the model) and residuals
        [dist, Dcp] = closestp(M,D);
        res = norm(dist);
        % robust weighting based on distance
        scale =  robstd(dist);
        w = weightfun(dist,scale,'bisquare');

        % compute incremental tranformation
        [R,t] = opa(Dcp,M,w);
        G = [R, t; 0 0 0 1] * G;

        % apply transformation that align M onto D
        M = R*M + t;

        if norm(res-prevres)<StepTol || norm(res)<FunctionTol
            break; end
    end
    R = G(1:3,1:3); t = G(1:3,4);
end

function  [dist, Dcp] = closestp(D,M)
    %CLOSESTP compute closest points

    [dist,I] = min( distmat(D',M'),[],2 );
    Dcp = M(:,I);
end
```

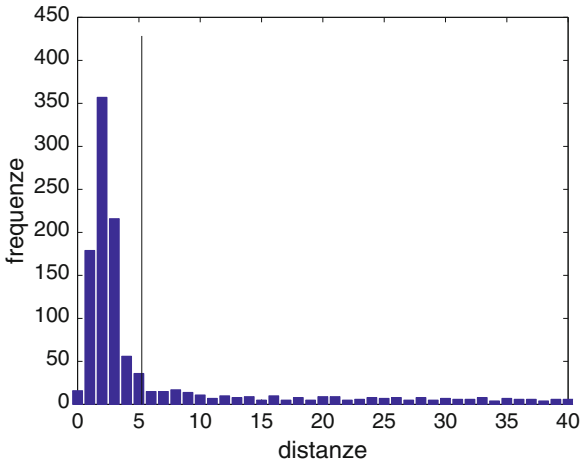


Fig. 15.3 Histogram of distances between closest points in two partially overlapping but aligned sets. Matches with distances higher than the vertical line are deemed as outliers

The step that dominates the computational cost of ICP is the search for matches. The naive method takes time $O(n^2)$, where n is the cardinality of $\mathcal{D}$. Using kD-trees it can be done in $O(n \log n)$. Other speedup techniques include *caching* of the matches (Simon et al. 1994) and computing the distance transform in the space of the set $\mathcal{D}$, which remains stationary (Fitzgibbon 2003).

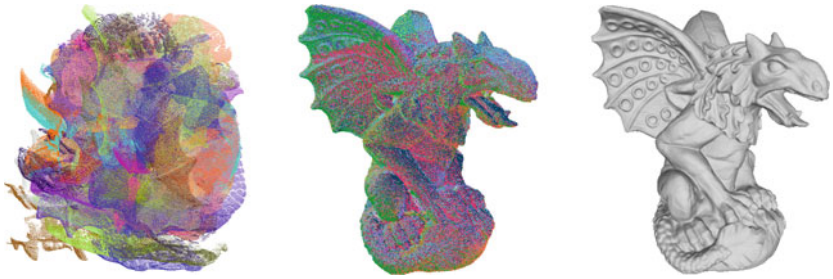


Fig. 15.4 From left to right: 27 point clouds in the initial references frames, point cloud after alignment (points are coloured according to the source image), surface reconstructed using the technique of Kazhdan et al. (2006). Images taken from Fantoni et al. (2012)

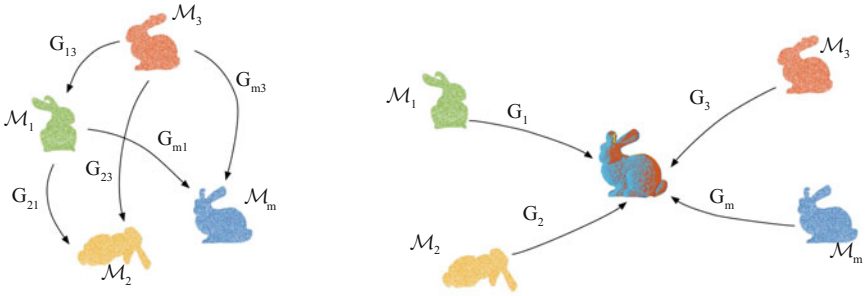


Fig. 15.5 Synchronisation of direct isometries applied to the alignment of 3D point clouds. In the figure $G = (R, \mathbf{t})$. Courtesy of F. Arrigoni

15.2.3 Registration of Many Point Clouds

In the case of simultaneous alignment of many point clouds without correspondences, one is faced with the same problem that ICP solves for two sets, so it would come immediately to think of generalising the latter. In fact this is possible, for example, by extending the cost function (15.5) to many sets and solving numerically the resulting non-linear least-squares problem, as in Fantoni et al. (2012) (Fig. 15.4). Or one can keep the main loop of ICP with nearest point search and replace OPA with GPA, as Toldo et al. (2010).

The methods we have mentioned so far involve points; however, if we wanted to avoid this step and work in the space of reference frames, we could exploit the technique of *synchronisation* already introduced in Chap. 14. In this case the measurements attached to the graph edges are rotations and translations (direct isometries) obtained, for example, by ICP applied to pairs of point clouds (Fig. 15.5). Arrigoni et al. (2016) suggest to proceed as follows:

- apply ICP to all pairs with sufficient overlap to obtain the direct isometries $(R_{ij}, \mathbf{t}_{ij})$ that link the pairs;

- synchronise the resulting graph for rotations and translations, as seen in Sect. 14.3 (here we do not have the complication of unknown moduli);
- to each set $\mathcal{M}_i$ apply the isometry $(R_i, \mathbf{t}_i)$ resulting from synchronisation, to obtain the alignment.

References

Federica Arrigoni, Beatrice Rossi, and Andrea Fusiello. Global registration of 3D point sets via LRS decomposition. In *Proceedings of the 14th European Conference on Computer Vision*, pages 489–504, 2016.

Fausto Bernardini and Holly E. Rushmeier. The 3d model acquisition pipeline. *Computer Graphics Forum*, 21 (2): 149–172, 2002.

P. Besl and N. McKay. A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14 (2): 239–256, February 1992.

Y. Chen and G. Medioni. Object modeling by registration of multiple range images. *Image and Vision Computing*, 10 (3): 145–155, 1992.

J. J. F. Commandeur. *Matching configurations*. DSWO Press, Leiden, 1991.

F. Crosilla and A. Beinat. Use of generalised procrustes analysis for the photogrammetric block adjustment by independent models. *ISPRS Journal of Photogrammetry & Remote Sensing*, 56 (3): 195–209, April 2002.

S. Fantoni, U. Castellani, and A. Fusiello. Accurate and automatic alignment of range surfaces. In *Second Joint 3DIM/3DPVT Conference: 3D Imaging, Modeling, Processing, Visualization and Transmission (3DIMPVT)*, 2012.

A. Fitzgibbon. Robust registration of 2D and 3D point sets. *Image and Vision Computing*, 21 (13-14): 1145 – 1153, 2003.

Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the fourth Eurographics symposium on Geometry processing*, pages 61–70. Eurographics Association, 2006.

M. Levoy S. Rusinkiewicz. Efficient variants of the ICP algorithm. In *IEEE International Conference on 3-D Digital Imaging and Modeling*, Quebec City (Canada), 2001.

D. A. Simon, M. Hebert, and T. Kanade. Real-time 3-d pose estimation using a high-speed range sensor. In *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, pages 2235–2241 vol.3, May 1994.

R. Toldo, A. Beinat, and F. Crosilla. Global registration of multiple point clouds embedding the generalized procrustes analysis into an ICP framework. In *International Symposium on 3D Data Processing, Visualization and Transmission*, 2010.

Federico Tombari, Samuele Salti, and Luigi Di Stefano. Performance evaluation of 3d keypoint detectors. *International Journal of Computer Vision*, 102 (1): 198–220, Mar 2013.

Chapter 16
Multi-view Projective Reconstruction and Autocalibration



16.1 Introduction

In this chapter we will study the problem of computing a projective reconstruction from multiple *uncalibrated* images. The point-based methods of Chap. 14 can be—to some extent—adapted to the uncalibrated case. For example, with the Adjustment of Independent Models (AIM) one would obtain a set of projective reconstructions related to each other by an unknown projectivity (in other words, each of these defines its own projective reference frame). Less obvious is how to modify frame-based techniques.

In the following we will describe a global method proposed by Sturm and Triggs (1996) that is specific for projective reconstruction and is based on the factorisation method of Tomasi and Kanade (1992). We will then address the problem of self-calibration, that is, promoting the reconstruction from projective to Euclidean, which is equivalent to estimating the intrinsic parameters.

16.1.1 Sturm-Triggs Factorisation Method

Consider m cameras framing n points in three-dimensional space, $\mathbf{M}^1 \ldots \mathbf{M}^n$. The usual perspective projection equation (with the scaling factors made explicit) writes

$$\zeta_i^j \mathbf{m}_i^j = P_i \mathbf{M}^j \quad i = 1 \ldots m, \quad j = 1 \ldots n \tag{16.1}$$

or, in matrix form:

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_16

$$\underbrace{\begin{bmatrix} \zeta_1^1\mathbf{m}_1^1 & \zeta_1^2\mathbf{m}_1^2 & \dots & \zeta_1^m\mathbf{m}_1^n \\ \zeta_2^1\mathbf{m}_2^1 & \zeta_2^2\mathbf{m}_2^2 & \dots & \zeta_2^m\mathbf{m}_2^n \\ \vdots & \vdots & \ddots & \vdots \\ \zeta_m^1\mathbf{m}_m^1 & \zeta_m^2\mathbf{m}_m^2 & \dots & \zeta_m^m\mathbf{m}_m^n \end{bmatrix}}_{W} = \underbrace{\begin{bmatrix} P_1 \\ P_2 \\ \vdots \\ P_m \end{bmatrix}}_{P} \underbrace{\begin{bmatrix} \mathbf{M}^1, & \mathbf{M}^2, & \dots & \mathbf{M}^n \end{bmatrix}}_{M}. \tag{16.2}$$

It tells us that W factorises into the product of a $3m \times 4$ matrix P and a $4 \times n$ matrix M, and because of this it has rank four.

Let us rewrite (16.2) in compact form as

$$W = Q \circ (Z \otimes \mathbf{1}_3) = PM, \tag{16.3}$$

where

$$Q = \begin{bmatrix} \mathbf{m}_1^1 & \mathbf{m}_1^2 & \dots & \mathbf{m}_1^n \\ \mathbf{m}_2^1 & \mathbf{m}_2^2 & \dots & \mathbf{m}_2^n \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{m}_m^1 & \mathbf{m}_m^2 & \dots & \mathbf{m}_m^n \end{bmatrix}, \quad Z = \begin{bmatrix} \zeta_1^1 & \zeta_1^2 & \dots & \zeta_1^m \\ \zeta_2^1 & \zeta_2^2 & \dots & \zeta_2^m \\ \vdots & \vdots & \ddots & \vdots \\ \zeta_m^1 & \zeta_m^2 & \dots & \zeta_m^m \end{bmatrix} \tag{16.4}$$

and the symbol $\circ$ denotes the Hadamard product, that is, the element-by-element product. Note that the matrix Z has m rows, while Q has $3m$ since its elements are 3×1 blocks, and so in order to match the dimensions, one must triplicate each row of Z via the Kronecker product by the vector $\mathbf{1}_3 = [1, 1, 1]^\mathsf{T}$.

In this equation only Q is known and everything else is unknown. However, if we provisionally assume that Z is known, the matrix W becomes known, and we can compute its Singular Value Decomposition (SVD):

$$W = UDV^\mathsf{T}. \tag{16.5}$$

In the ideal case where the data is unaffected by error, the rank of W is four and therefore $D = \text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4, 0, \dots 0)$. Then, only the first four columns of U and V contribute to the product between matrices. Let therefore $U_{3m\times 4}$ and $V_{n\times 4}$ be the matrices formed by the first four columns of U and V, respectively. We can then write the compact SVD of W as

$$W = U_{3m\times 4}\ \text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4)\ V_{n\times 4}^\mathsf{T}. \tag{16.6}$$

Comparing (16.6) with (16.2), we can identify:

$$P = U_{3m\times 4}\ \text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4) \quad \text{and} \quad M = V_{n\times 4}^\mathsf{T} \tag{16.7}$$

thus obtaining the sought reconstruction. Note that the choice to include $\text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$ in P is arbitrary. It could have been included in M or split between the two. This however is consistent with the fact that the reconstruction is up to a projectivity, which therefore also absorbs $\text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4)$.

In the real case where the data is affected by errors, the rank of W is not 4. By the Eckart-Young theorem, forcing $D = \text{diag}(\sigma_1, \sigma_2, \sigma_3, \sigma_4, 0, \dots 0)$ yields the solution that minimises the error:

$$\|W - PM\|_F^2 = \sum_{i,j} \|\zeta_i^j \mathbf{m}_i^j - P_i \mathbf{M}^j\|^2. \tag{16.8}$$

We are now left with the problem of estimating the unknowns ζ_i^j that can be calculated once P and M are known. In fact, for image i (16.2) writes:

$$\begin{aligned} &\left[\zeta_i^1 \mathbf{m}_i^1, \zeta_i^2 \mathbf{m}_i^2, \dots, \zeta_i^m \mathbf{m}_i^n\right] = \\ &= \underbrace{\left[\mathbf{m}_i^1, \mathbf{m}_i^2, \dots, \mathbf{m}_i^n\right]}_{Q_i} \text{diag}(\underbrace{[\zeta_i^1, \zeta_i^2, \dots, \zeta_i^m]}_{\boldsymbol{\zeta}_i}) = P_i M \end{aligned} \tag{16.9}$$

or:

$$Q_i \, \text{diag}(\boldsymbol{\zeta}_i) = P_i M \tag{16.10}$$

which is analogous to (5.21) already solved in Sect. 5.3.2.

An iterative scheme that is often adopted in cases like this is to alternate the solution of two problems: in one step we estimate ζ_i^j given P and M; in the next step, we estimate P and M given ζ_i^j and iterate.

To avoid convergence to the trivial solution $Z = 0$, the matrix Z must be normalised in some way at each iteration, for example, by fixing the norm of its rows or columns, or other subsets of its entries. Nasihatkon et al. (2015) show that there are other trivial solutions besides $Z = 0$ and that the choice of normalisation criterion is crucial to exclude all or part of them.

In the implementation shown in Listing 16.1, we chose to normalise the rows of Z, a simple and widely used solution, but one that actually leaves open the possibility of convergence to trivial solutions in which an entire column vanishes (see Nasihatkon et al. 2015 for details).

Note also that preconditioning is applied to the coordinates of the input image points, as in the fundamental matrix computation (Sect. 6.4): experimentally it is observed that this step is crucial to obtain a good result.

Listing 16.1 Sturm-Triggs projective reconstruction

```
function [PPM, M] = prec(m)
%PREC Sturm-Triggs' projective reconstruction

    n_imm = length(m);
    for j = 1:n_imm   % preconditioning
        [T{j},m{j}] = precond(m{j});
    end

    MaxIterations = 400;
    FunctionTol = 1e-15;
    StepTol = 1e-15;

    Q = reshape(cell2mat(m(:)),2*length(m),[]);
    tmp=ones(3*n_imm,size(Q,2));
    for i=1:n_imm
        tmp((2*i+i-2):(2*i+i-1),:) = Q((2*(i-1)+1):2*i,:);
    end
    Q=tmp;  % converted to homogeneous

    % initialization
    Z = ones(n_imm,size(Q,2));  res = Inf;

    for iter = 1: MaxIterations

        [U,D,V] = svd(Q.*kron(Z,ones(3,1))); % estimate P and M
        P = U(:,1:4)*D(1:4,1:4); M = V(:,1:4)';

        Z = [];  % estimate depth
        for i = 1:n_imm
            B = P(3*i-2:3*i,:) * M;
            zi = diagc(Q(3*i-2:3*i,:))\B(:);
            Z = [Z; zi'/norm(zi)]; % normalization
        end

        prevres = res;
        res = norm(Q.*kron(Z,ones(3,1)) - P*M,'fro');
        if norm(res-prevres)<StepTol || norm(res)<FunctionTol
            break; end
    end
    M = M./repmat(M(end,:),size(M,1),1);  M = M(1:end-1, :);

    for j = 1:n_imm  %  postconditioning
        PPM{j} = T{j} \  P(3*j-2:3*j,:);
    end
end
```

16.2 Autocalibration

Although some useful information can be derived from a projective reconstruction (Robert and Faugeras 1995), what we would ultimately like to obtain is at least a *Euclidean reconstruction* of the model, which differs from the true model by a similitude. The latter consists of a direct isometry (due to the arbitrary choice of world reference frame) plus a uniform change of scale (due to the well-known depth-speed ambiguity).

Fortunately, if some reasonable conditions are met (e.g. at least three cameras with constant intrinsic parameters), it is possible to switch to the Euclidean reconstruction, that is, the one we would get in the calibrated case. This process is called *autocalibration*, and can be achieved through two different ways: we can start from the projective reconstruction and then upgrade it to Euclidean (Sect. 16.2.1), or recover the intrinsic parameters from the fundamental matrices and fall back to the calibrated case (Sect. 16.2.2).

In practice, however, the camera parameters are not completely unknown, since an approximation of them can be inferred from EXIF (Exchangeable Image File Format) data of the image. EXIF is a format for specifying metadata implemented by the major image file formats (including JPEG). Metadata includes static information such as the model and manufacturer of the camera, and information about the shot, such as dates and times, geographical location (if available), aperture, speed, focal length, white balance, etc.

16.2.1 Absolute Quadric Constraint

In this section we will follow the first of the two paths outlined above, namely, performing a projective reconstruction followed by Euclidean upgrade.

The projective reconstruction differs from the Euclidean reconstruction by an unknown projectivity T, which can be interpreted as an appropriate change of projective basis.

The following passages replicate those of Sect. 8.6. From the projective reconstruction, we obtain a set of perspective projection matrices $\{P_i^{\mathrm{p}}\}$. Without loss of generality, we assume a particular expression for the first of them, which arbitrarily fixes the projective basis:

$$P_0^{\mathrm{p}} = [I, \mathbf{0}]; \quad P_i^{\mathrm{p}} = [A_i, \mathbf{e}_i] \quad \forall i > 0. \tag{16.11}$$

We want to arrive at a Euclidean reconstruction, which, without loss of generality, can be written as

$$P_0^{\mathrm{e}} = K_0[I, \mathbf{0}] \quad P_i^{\mathrm{e}} = K_i[R_i, \mathbf{t}_i] \quad \forall i > 0, \tag{16.12}$$

via estimation of the non-singular 4×4 matrix T, which transforms the projective reconstruction into the Euclidean one:

$$P_i^{\mathrm{e}} \simeq P_i^{\mathrm{p}} T. \tag{16.13}$$

As we have already observed in Sect. 8.6, the assumptions about the specific form of the first Perspective Projection Matrix (PPM) imply that T should be written as:

$$T = \begin{bmatrix} K_0 & \mathbf{0} \\ \mathbf{v}^{\mathsf{T}} & s \end{bmatrix} \tag{16.14}$$

where $\mathbf{v}^\mathsf{T}$ is an arbitrary vector of three elements and s is a scalar. With this parameterisation T is clearly non-singular, and, being defined up to a scaling factor, it depends on eight parameters (if we fix, e.g. $s = 1$). Proceeding as in Sect. 8.6, we obtain:

$$K_i R_i \simeq A_i K_0 + \mathbf{e}_i \mathbf{v}^\mathsf{T} = P_i^\mathrm{p} \begin{bmatrix} K_0 \\ \mathbf{v}^\mathsf{T} \end{bmatrix} \quad \forall i > 0. \tag{16.15}$$

This is the basic equation, which links the unknowns R_i, K_i and $\mathbf{v}$ to the available data A_i and $\mathbf{e}_i$. Since $R_i R_i{}^\mathsf{T} = I$ we can eliminate the latter, obtaining

$$\underbrace{K_i K_i^\mathsf{T}}_{C_i} = K_i R_i R_i{}^\mathsf{T} K_i^\mathsf{T} \simeq P_i^\mathrm{p} \begin{bmatrix} K_0 \\ \mathbf{v}^\mathsf{T} \end{bmatrix} \begin{bmatrix} K_0 \\ \mathbf{v}^\mathsf{T} \end{bmatrix}^\mathsf{T} P_i^{\mathrm{p}\mathsf{T}} = P_i^\mathrm{p} \underbrace{\begin{bmatrix} K_0 K_0^\mathsf{T} & K_0 \mathbf{v} \\ \mathbf{v}^\mathsf{T} K_0^\mathsf{T} & \mathbf{v}^\mathsf{T} \mathbf{v} \end{bmatrix}}_{\Omega} P_i^{\mathrm{p}\mathsf{T}} \tag{16.16}$$

Note that (16.16) contains five equations, because the matrices of both members are symmetric, and the homogeneity reduces the number of equations by one.

This is the basic equation for autocalibration, called *absolute quadric constraint* (Triggs 1997), relating the unknowns C_i ($i = 1 \ldots m$) and Ω to the available data P_i. The name comes from its geometrical interpretation, where C_i is special conic called *dual image of the absolute conic* and Ω is a quadric, the *dual absolute quadric*. The interested reader can look up the details, e.g. on (Hartley and Zisserman 2003).

16.2.1.1 Solution Strategies

We are thus required to solve (16.16), with respect to Ω and C_i. The collineation T that upgrades cameras from projective to Euclidean is obtained by decomposing Ω as

$$\Omega = T \,\mathrm{diag}(1, 1, 1, 0) T^\mathsf{T}. \tag{16.17}$$

Ω is a 4×4 symmetric matrix that is also singular and defined up to a scale factor, which results in 8 d.o.f., the same of T.

Solving (16.16) entails getting rid of the unknown scale factor contained in the $\simeq$ symbol. Several strategies have been proposed:

- Introduce the scale factor explicitly as an additional unknown (Heyden and Åström 1996):

$$C_i - \lambda_i P_i \Omega P_i^\mathsf{T} = \mathbf{0} \tag{16.18}$$

This gives six equations but introduces one additional unknown (the net sum is 5).

- Eliminate it by using the same idea of the cross product for 3D vectors (Triggs 1997):

$$\text{vech}(C_i) \simeq \text{vech}(P_i \Omega P_i^\mathsf{T}) \iff \underbrace{\text{r}\,[\text{vech}(C_i) | \,\text{vech}(P_i \Omega P_i^\mathsf{T})]}_{B} = 1$$

  where vech is the column-wise vectorisation with the upper portion excluded, as matrices in (16.16) are symmetric.

  This is tantamount to say that every 2×2 minor of B is zero. There are 15 different order-2 minors of a 6×2 matrix, but only 5 equations are independent.
- Normalise both sides of the equation (Pollefeys et al. 1998):

$$\frac{C_i}{||C_i||_F} - \frac{P_i \Omega P_i^\mathsf{T}}{||P_i \Omega P_i^\mathsf{T}||_F} = \mathbf{0} \tag{16.19}$$

In any case, a non-linear least-squares problem has to be solved. Available numerical techniques (based on the Gauss-Newton method) are iterative, and require an estimate of the solution to start. This can be obtained by doing an educated guess about skew angle, principal point and aspect ratio, and solving the linear problem that results.

Linear Solution

Linear equations on Ω are generated from zero entries of C_i, because this eliminates the scale factor (Pollefeys et al. 2002):

$$C_i(k, \ell) = 0 \;\Rightarrow\; \mathbf{p}_{i,k}^\mathsf{T} \Omega \mathbf{p}_{i,\ell} = 0$$

where $\mathbf{p}_{i,k}^\mathsf{T}$ is the k-th row of P_i.

If the principal point is known, $C_i(1, 3) = 0 = C_i(2, 3)$ and this gives two linear constraints. If, in addition, there is no skewing ($\theta = \pi/2$), we have $C_i(1, 2) = 0$. Known aspect ratio τ provides a further constraint: $\tau C_i(1, 1) = C_i(2, 2)$.

Likewise, linear constraints on Ω can be obtained from the equality of elements in the upper (or lower) triangular part of C_i (because C_i is symmetric).

In order to be able to solve linearly for Ω, at least ten linear equations must be stacked up, to form a homogeneous linear system, which can be solved as usual (via SVD). Singularity of Ω can be enforced a posteriori by forcing the smallest singular value to zero.

Constant Intrinsic Parameters

If all the cameras have the same intrinsic parameters, so $K_i = K$, then (16.16) becomes:

$$K K^{\mathsf{T}} \simeq P_i \begin{bmatrix} K K^{\mathsf{T}} & K\mathbf{r} \\ \mathbf{r}^{\mathsf{T}} K^{\mathsf{T}} & \mathbf{r}^{\mathsf{T}}\mathbf{r} \end{bmatrix} P_i^{\mathsf{T}} \tag{16.20}$$

We observe that (16.20) contains five equations, because the matrices of both members are 3×3 symmetric and the arbitrary scaling factor reduces the number of equations by one. Thus, each camera matrix, excluding the first one, gives five equations in the eight unknowns given by the five intrinsic parameters plus the three entries of $\mathbf{v}$. A single solution is determined as soon as three cameras are available. The constraints expressed in (16.20) are also known as *Kruppa constraints*. Although interesting from a theoretical point of view, solving (16.20) it was found experimentally to be numerically unstable, and so it is of little practical use.

Given a projective reconstruction: $\left\{P_i; \mathbf{M}^j\right\}\ i = 0 \ldots n;\ \ j = 0 \ldots m$, if the intrinsic parameters of the first two cameras K_0 and K_1 were known, it would be possible to compute the projective transformation T that promotes this to a Euclidean reconstruction, as seen in Sect. 8.6. Gherardi and Fusiello (2010) observed that this T work for all other cameras as well, so a pair (K_0, K_1) determines a solution, to which a cost $\mathcal{C}$ can be associated, which depends on the plausibility of K_ℓ ($\ell = 2 \ldots m$) where K_ℓ is the intrinsic parameter matrix of the ℓ-th camera after the Euclidean promotion determined by K_0 and K_1. We expect no skewing ($\theta \approx \pi/2$), unit aspect ratio ($\tau \approx 1$) and principal point at the center of the image; deviations from these assumptions are penalised. The cost function is then optimised with respect to K_0 and K_1:

$$\{K_0, K_1\} = \arg \min_{K_0, K_1} \sum_{\ell=2}^{m} \mathcal{C}^2(K_\ell) \tag{16.21}$$

16.2.2 Mendonça-Cipolla Method

The second route to Euclidean reconstruction is through the explicit computation of the intrinsic parameters, which then leads us back to the calibrated case.

Given the correspondences between homologous points, without knowing anything about the camera, it is possible to derive the fundamental matrix F.

We know that the fundamental matrix F has seven degrees of freedom, while the essential matrix E has five, since it must satisfy two more constraints, which are those arising from the equality of the two singular values (Theorem 7.1). Moreover, we know that E and F are related via the intrinsic parameters K by (7.4). This means that for every fundamental matrix F, there exist two intrinsic parameter matrices K and K' and a rigid motion represented by $\mathbf{t}$ and R such that $F = K'^{-\mathsf{T}}([\mathbf{t}]_\times R)K^{-1}$. This translates into two constraints available for computing the intrinsic parameters for every image pair.

With only two images available, it is not possible to derive all the intrinsic parameters, but only two of them: Hartley (1992), for example, shows how to derive the focal lengths of the two cameras. More than two images are needed, to accumulate constraints. If the intrinsic parameters are constant (i.e. the same for all the cameras), it is shown that only three images are needed. In fact, there are five unknowns, each pair of images gives two equations, and there are three image pairs: 1–2, 1–3 and 2–3.

The way of writing the constraints may vary. Mendonça and Cipolla (1999), who first proposed the method, directly used Theorem 7.1. An alternative formulation, which avoids SVD , is based on the following:

Proposition 16.1 (Huang and Faugeras 1989) *The condition that the matrix E has a singular value of zero and two nonzero singular values of equal value is equivalent to*

$$\det(E) = 0 \quad \textit{and} \quad 2\,\mathrm{tr}((E^{\mathsf{T}}E)^2) - \mathrm{tr}(E^{\mathsf{T}}E)^2 = 0. \tag{16.22}$$

The second condition, in particular, is equivalent to the equality of the two singular values.

The cost function takes the (unknown) intrinsic parameters as arguments, the fundamental matrices (computed from point correspondences) as parameters, and returns a positive value proportional to the violation of the constraint (16.22). The sought intrinsic parameters are those that satisfy the constraint or that minimise the violation. In formulae, let F_{ij} be the fundamental matrix relating images i and j, and let K be the matrix of intrinsic parameters, which we assume for simplicity to be constant, although the method has a general formulation. The least-squares cost function is

$$\chi(K) = \sum_{i=1}^{m} \sum_{j=i+1}^{m} \left(\frac{2\,\mathrm{tr}((E_{ij}^{\mathsf{T}} E_{ij})^2)}{\mathrm{tr}(E_{ij}^{\mathsf{T}} E_{ij})^2} - 1 \right)^2 \Bigg|_{E_{ij} = K^{\mathsf{T}} F_{ij} K}. \tag{16.23}$$

This is minimised using an iterative algorithm such as Gauss-Newton, for which it is necessary to calculate the derivative. We rewrite the single term of the cost function (we omit the indices for simplicity) as a composition of functions:

$$\frac{2\,\mathrm{tr}((E(K)^\mathsf{T}E(K))^2)}{\mathrm{tr}(E(K)^\mathsf{T}E(K))^2} - 1 = H(S(E(K))), \tag{16.24}$$

where

$$H : \mathbb{R}^{2\times 1} \to \mathbb{R},\ H(C) = \left(\tfrac{c_1}{c2} - 1\right),\ C = \begin{bmatrix} c_1 \\ c_2 \end{bmatrix} \tag{16.25}$$

$$S : \mathbb{R}^{3\times 3} \to \mathbb{R}^{2\times 1},\ S(E) = \begin{bmatrix} 2\,\mathrm{tr}((E^\mathsf{T}E)^2) \\ \mathrm{tr}\left(E^\mathsf{T}E\right)^2 \end{bmatrix} \tag{16.26}$$

$$E : \mathbb{R}^{3\times 3} \to \mathbb{R}^{3\times 3},\ E(K) = K^\mathsf{T}FK \tag{16.27}$$

and $\mathbb{R}^{n\times m}$ denotes the set of real matrices $n \times m$. Since we are dealing with matrix functions, we employ the definition of the Jacobian given by Magnus-Neudecker (Sect. B.1), which allows to apply the chain rule (Theorem B.1) to the compound function $(H \circ S \circ E)$, obtaining

$$\mathsf{D}\, H(S(E(K))) = \mathsf{D}\, H(C)|_{C=S(E)}\, \mathsf{D}\, S(E)|_{E=E(K)}\, \mathsf{D}\, E(K). \tag{16.28}$$

The Jacobian $\mathsf{D}\, E(K)$ is immediately derived from (B.26), while as for $\mathsf{D}\, S(E)$, using the chain rule and formulae (B.27) and (B.4), we arrive at

$$\mathsf{D}\, S(E) = \begin{bmatrix} 4(\mathrm{vec}(E^\mathsf{T}E))^\mathsf{T}(I_9 + K_3)(I_3 \otimes E^\mathsf{T}) \\ 4\,\mathrm{tr}\left(E^\mathsf{T}E\right)(\mathrm{vec}\, E)^\mathsf{T} \end{bmatrix}. \tag{16.29}$$

Finally, it is immediate to verify that

$$\mathsf{D}\, H(C) = \begin{bmatrix} -\frac{c_1}{c_2^2} & \frac{1}{c_2} \end{bmatrix}. \tag{16.30}$$

Since K is parameterised through the intrinsic parameters, it will be necessary to multiply the Jacobian (16.28) to the right by the derivative of K with respect to its parameters.

The Mendonça-Cipolla autocalibration algorithm is implemented in Listing 16.2, where we assume that only three intrinsic parameters are unknown ($\tau = 1, \theta = \pi/2$).

Gauss-Newton convergence is not guaranteed for any initial value; however, in practice the algorithm often converges to the correct solution from a reasonable initial guess. An algorithm that minimises (16.23) with *global* convergence has been proposed by Fusiello et al. (2004).

Listing 16.2 Mendonça-Cipolla self-calibration

```
function K_out = autocalMC(F,K0)
    %AUTOCALMC Autocalibration from fundamental matrices with Mendonca-Cipolla

k0 = K2par(K0,3);  % aspect ratio=1, skew=0

k_out = lsq_nonlin(@(x)fobj(x,F),k0);

K_out = par2K(k_out);

end

function [res,J]  = fobj(k,F)

J =[]; res=[];
for i=1:size(F,1)
    for j=1:size(F,2)
        if ~isempty(F{i,j})
            [r, D] = jacobianHF(k,F{i,j});
            res = [res; r];
            J = [J; D];
        end
    end
end
end


function [r,J] = jacobianHF(k,F)
%JACOBIANHF  Jacobian of the Huang-Faugeras residual

K3 = commutation(3,3);
[K,JK] = par2K(k);

E = K'*F*K; EE = E'*E;
c1 = 2*trace(EE*EE);
c2 = (trace(EE))^2;

r = c1/c2-1; % objective f.

JH = [ 1/c2, -c1/(c2^2)];
JS = 4* [ EE(:)' * (eye(9)+K3) * kron(eye(3),E')
    trace(EE)*(E(:))'];
JE = K3*kron(eye(3),K'*F') + kron(eye(3),K'*F);
J = JH * JS * JE * JK; % jacobian
end
```

16.3 Autocalibration via H_∞

This calibration method, which we will denote by LVH, since it was invented independently by Luong and Viéville (1996) and Hartley (1997), is based on the knowledge of the homography of the plane at infinity, H_∞, so it is less general than the previous ones that assumed no knowledge on the camera or the scene.

Consider two cameras with identical and unknown intrinsic parameters (or, equivalently, the same camera in motion) and suppose that we can somehow arrive at the knowledge of the homography induced by the plane at infinity H_∞. Observe that H_∞ does not carry any scale ambiguity; in fact it must have unit determinant being similar to a rotation matrix.

If $K' = K$, from $H_\infty = KRK^{-1}$ we get $R = K^{-1}H_\infty K$, and, since $RR^\mathsf{T} = I$, we have:

$$H_\infty K K^\mathsf{T} H_\infty^\mathsf{T} = K K^\mathsf{T}. \tag{16.31}$$

Elaborating with Kronecker's product, we arrive at

$$(I - H_\infty \otimes H_\infty) D_3 \operatorname{vech} C = 0 \tag{16.32}$$

where $C = KK^\mathsf{T}$ and D_3 is the duplication matrix. This is a homogeneous linear system of nine equations in six unknowns. However, since (16.31) is an equality between symmetric 3×3 matrices, only six equations are unique, and of these Luong and Viéville (1996) showed that only four are independent. The solution of the homogeneous system requires a matrix of rank five, so at least three images are needed, with the same intrinsic parameters.

The implementation is given in Listing 16.3.

Listing 16.3 LVH calibration

```
function K  = calibLVH( H )
%CALIBLVH Camera calibration from infnity homographies
% Luong-Vieville-Hartley

    numV = length(H);
    S = duplication(3);

    L = [];                 % build linear system
    for i =1:numV
        L = [L; (eye(9)-kron(H{i},H{i}))*S];
    end

    [~,~,V] = svd(L);        % solve
    B = inv(reshape(S*V(:,end),3,[]));

     if trace(B)<0
        B=-B;% either B or -B is p.d.
    end
    iK = diag([-1,-1,1])*chol(B);

    K = inv(iK); K = K./K(3,3);
end
```

Regarding the estimation of H_∞, we briefly mention three different approaches:

- use three vanishing points plus the epipole (see Problem 6.7) in the DLT;
- use any plane sufficiently far from the camera (Viéville et al. 1996);
- rotate the camera around the Centre of Projection (COP) (Hartley 1997).

16.4 Tomasi-Kanade's Factorisation

We consider here a different camera model than the perspective camera; in fact this method assumes that the *affine camera* approximation applies, which realises a parallel projection instead of a central (perspective) one. In this case one can directly obtain the reconstruction by the SVD of a matrix containing the coordinates of the tie points in the different images. If the intrinsic parameters of the camera

are unknown, the reconstruction is affine. If, on the other hand, the intrinsic parameters are known, as in the original work of Tomasi and Kanade (1992), a metric reconstruction can be obtained.

16.4.1 Affine Camera

The most general affine camera is represented by a full rank 3×4 matrix P_A that has the last row equal to [0 0 0 1]. It is easy to verity that such camera projects points at infinity to infinity and points of the affine space to the affine plane, hence the name.

Let us write $P_A = [\widetilde{P}_A, \mathbf{p}_4]$, as we did for the PPM. In this case $\widetilde{P}_A$ is clearly singular, so the COP cannot be computed with (3.29). Instead, let $\mathbf{d} \in \ker(\widetilde{P}_A)$, than

$$[\widetilde{P}_A, \mathbf{p}_4] \begin{bmatrix} \mathbf{d} \\ 0 \end{bmatrix} = \mathbf{0} \tag{16.33}$$

hence $\mathbf{C} = [\mathbf{d}, 0]^\top$. The COP of an affine camera is at infinity, and P_A realises a *parallel projection*. As a matter of fact, P_A may be thought of as the parallel projection version of the projective camera.

The most general affine camera matrix has 8 d.o.f. and can be factored, similarly to the PPM, as

$$P_A = \begin{bmatrix} \alpha_u & s & 0 \\ 0 & \alpha_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \alpha_u & s & 0 \\ 0 & \alpha_v & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \mathbf{r}_1 & t_1 \\ \mathbf{r}_2 & t_2 \\ \mathbf{0} & 1 \end{bmatrix} \tag{16.34}$$

The central matrix represents an orthographic projection (instead of perspective), the rightmost matrix represents the transformation from the word reference frame to the camera reference frame (as in the PPM), and the leftmost matrix $K_A$is an affine transformation of the image plane that plays the same role as K in the PPM, with the difference that the principal point is not defined. Indeed, in a parallel projection, a translation of the image and a translation of the scene yield the same effect.

16.4.2 The Factorisation Method for Affine Camera

Let us write the affine camera matrix as

$$P_A = \begin{bmatrix} A_{2\times 3} & \mathbf{s}_{2\times 1} \\ \mathbf{0} & 1 \end{bmatrix}. \tag{16.35}$$

with $\mathrm{r}(P_A) = 3$ and $\mathrm{r}(A_{2\times 3}) = 2$

In Cartesian coordinates, the affine projection is written:[1]

$$\mathbf{m} = A\mathbf{M} + \mathbf{s}. \tag{16.36}$$

The point $\mathbf{s}$ is the image of the origin of the world reference frame, indeed $P_A[0\ 0\ 0\ 1]^\mathsf{T} = \mathbf{s}$.

Consider a set of n 3D points, viewed by m affine cameras with matrices $\{P_i\}_{i=1\ldots m}$. Let $\mathbf{m}_i^j$ be the Cartesian coordinates of the projection of the j-th point in the i-th camera. The goal of the reconstruction is to estimate the PPM of the cameras, that is, $\{A_i, \mathbf{s}_i\}$, and the 3D points $\mathbf{M}^j$ such that

$$\mathbf{m}_i^j = A_i\mathbf{M}^j + \mathbf{s}_i. \tag{16.37}$$

We consider the "centralised" coordinates obtained by subtracting the centroid $\langle \mathbf{m}_i \rangle = \frac{1}{n}\sum_{j=1}^n \mathbf{m}_i^j$ in each image:

$$\bar{\mathbf{m}}_i^j = \mathbf{m}_i^j - \langle \mathbf{m}_i \rangle \tag{16.38}$$

(this step implies that all n points are visible in all m images).

We choose the 3D reference frame in the centroid of the points, so that $\langle \mathbf{M}^j \rangle = 0$. Taking this into account, we substitute (16.37) into (16.38), thus obtaining to eliminate the vector $\mathbf{s}_i$ in (16.38), which becomes

$$\bar{\mathbf{m}}_i^j = A_i\bar{\mathbf{M}}^j. \tag{16.39}$$

In this way we immediately get that $\mathbf{s}_i = \langle \mathbf{m}_i \rangle$. We rewrite the equations in matrix form:

$$\underbrace{\begin{bmatrix} \bar{\mathbf{m}}_1^1 & \bar{\mathbf{m}}_1^2 & \ldots & \bar{\mathbf{m}}_1^n \\ \bar{\mathbf{m}}_2^1 & \bar{\mathbf{m}}_2^2 & \ldots & \bar{\mathbf{m}}_2^n \\ \vdots & \vdots & \ddots & \vdots \\ \bar{\mathbf{m}}_m^1 & \bar{\mathbf{m}}_m^2 & \ldots & \bar{\mathbf{m}}_m^n \end{bmatrix}}_{\text{measures } W} = \underbrace{\begin{bmatrix} A_1, \\ A_2, \\ \vdots \\ A_m \end{bmatrix}}_{\text{motion } A} \underbrace{\begin{bmatrix} \bar{\mathbf{M}}^1 & \bar{\mathbf{M}}^2 & \ldots & \bar{\mathbf{M}}^n \end{bmatrix}}_{\text{structure } M}. \tag{16.40}$$

The matrix of W measurements represents the location of n points tracked along m images. It is a $2m \times n$ matrix that has rank at most 3, being the product of a $2m \times 3$ matrix (motion) and a $3 \times n$ matrix (structure). This observation is the key to Tomasi and Kanade's factorisation method. In fact, to recover A and M from W, we factorise the latter into the product of two matrices of rank three using the SVD.

[1] In this paragraph, in order not to clutter the notation, the Cartesian coordinates of the points are denoted without the ~.

Let

$$W_{2m\times n} = U_{2m\times 2m} D_{2m\times n} V^{\mathsf{T}}_{n\times n} \tag{16.41}$$

the SVD of W. In the absence of noise, only the first three singular values would be different from zero; in practice, however, the rank of W is only approximately three. We then consider only the first three singular values, getting

$$\widehat{W}_{2m\times n} = U_{2m\times 3} D_{3\times 3} V^{\mathsf{T}}_{n\times 3} \tag{16.42}$$

where $\widehat{W}$ is the best (in Frobenius norm) rank three approximation of W, by the Eckart-Young theorem. The factorisation sought is obtained by setting

$$A = U_{2m\times 3} \quad M = D_{3\times 3} V^{\mathsf{T}}_{n\times 3}. \tag{16.43}$$

The factorisation is not unique: the matrix $D_{3\times 3}$ could be arbitrarily distributed between the two matrices; in fact one can insert an arbitrary matrix T of rank three into the factorisation without altering the result:

$$\widehat{W} = (AT)(T^{-1}M). \tag{16.44}$$

Thus, there is an affine ambiguity on the reconstruction.

If instead the intrinsic parameters are known, working in normalised coordinates yields:

$$P_A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{r}_1 & t_1 \\ \mathbf{r}_2 & t_2 \\ \mathbf{0} & 1 \end{bmatrix}. \tag{16.45}$$

So the centroid $\langle \mathbf{m}_i \rangle$ provides the translation component (in X and Y), and the rows of A are two orthogonal vectors, a condition that can be exploited to fix the ambiguity. Note, however, that the translation of each camera along its optical axis Z cannot be derived.

Problems

16.1 Adapt (i) AIM, (ii) incremental reconstruction and (iii) bundle adjustment to the uncalibrated case.

References

A. Fusiello, A. Benedetti, M. Farenzena, and A. Busti. Globally convergent autocalibration using interval analysis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26 (12): 1633–1638, December 2004.

Riccardo Gherardi and Andrea Fusiello. Practical autocalibration. In *Proceedings of the European Conference on Computer Vision*, Lecture Notes in Computer Science, pages 790–801. Springer, Berlin, 2010.

R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2nd edition, 2003.

R. I. Hartley. Estimation of relative camera position for uncalibrated cameras. In *Proceedings of the European Conference on Computer Vision*, pages 579–587, Santa Margherita L., 1992.

Richard Hartley. Self-calibration of stationary cameras. *International Journal of Computer Vision*, 22 (1): 5–24, February 1997.

A. Heyden and K. Åström. Euclidean reconstruction from constant intrinsic parameters. In *Proceedings of the International Conference on Pattern Recognition*, pages 339–343, Vienna, 1996.

T.S. Huang and O.D. Faugeras. Some properties of the E matrix in two-view motion estimation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 11 (12): 1310–1312, December 1989.

Q.-T. Luong and T. Viéville. Canonical representations for the geometries of multiple projective views. *Computer Vision and Image Understanding*, 64 (2): 193–229, 1996.

P.R.S. Mendonça and R. Cipolla. A simple technique for self-calibration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 500–505, 1999.

Behrooz Nasihatkon, Richard Hartley, and Jochen Trumpf. A generalized projective reconstruction theorem and depth constraints for projective factorization. *International Journal of Computer Vision*, 115 (2): 87–114, Nov 2015.

M. Pollefeys, R. Koch, and L. Van Gool. Self-calibration and metric reconstruction in spite of varying and unknown internal camera parameters. pages 90–95, Bombay, 1998.

M. Pollefeys, F. Verbiest, and L. Van Gool. Surviving dominant planes in uncalibrated structure and motion recovery. pages 837–851, 2002.

L. Robert and O. Faugeras. Relative 3-D positioning and 3-D convex hull computation from a weakly calibrated stereo pair. *Image and Vision Computing*, 13 (3): 189–197, 1995.

P. Sturm and B. Triggs. A factorization based algorithm for multi-image projective structure and motion. In *Proceedings of the European Conference on Computer Vision*, pages 709–720. Springer, Berlin, 1996.

C. Tomasi and T. Kanade. Shape and motion from image streams under orthography—a factorization method. *International Journal of Computer Vision*, 9 (2): 137–154, November 1992.

B. Triggs. Autocalibration and the absolute quadric. pages 609–614, Puerto Rico, 1997.

T. Viéville, C. Zeller, and L. Robert. Using collineations to compute motion and structure in an uncalibrated image sequence. *International Journal of Computer Vision*, 20 (3): 213–242, 1996.

Chapter 17
Multi-view Stereo Reconstruction



17.1 Introduction

We treat in this chapter the problem of multi-view stereo reconstruction,[1] that is, recovering the surface of an object from many ($\gg 2$) images.

The key notion to understand multi-view stereo reconstruction algorithms is that of *photo-consistency*. A point in the scene $\mathbf{x}$ is defined as *photo-consistent* in a set of images if, for each image I_k in which it is visible, its radiance $L(\mathbf{x}, \mathbf{v})$ in the direction of the pixel $\mathbf{m}$ is equal to the intensity of the corresponding pixel $I_k(\mathbf{m})$. The surface reconstruction problem then reduces to that of determining the photo-consistency of the points in space: the photo-consistent ones belong to the surface, and the others correspond to empty space.

If the surface is Lambertian, the radiance is the same in all directions, and therefore for a photo-consistent point all values $I_k(\mathbf{m})$ are equal $\forall k$. In fact the equality of the colour (modulo radiometric and geometric perturbations) of the projections of the point in the images in which it is visible is commonly taken as a test of photo-consistency (Fig. 17.1), usually including a small region around it (a.k.a. *window*). Binocular stereopsis (Chap. 12) can be interpreted as the determination of a photo-consistent surface from two images.

An aspect that in the binocular case can be ignored, as it reduces to occlusions handling, but in multi-view stereo becomes crucial is that of *visibility*. In fact, the problem of determining photo-consistency is closely related to that of visibility. In order to correctly evaluate the photo-consistency of a point, it is necessary to consider only the images in which it is visible; however determining in which images a point is visible implies the knowledge of the shape of the surface, as it is made clear in Fig. 17.2.

[1] In this chapter the term reconstruction is used with a different meaning from the one defined in Chap. 16.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_17

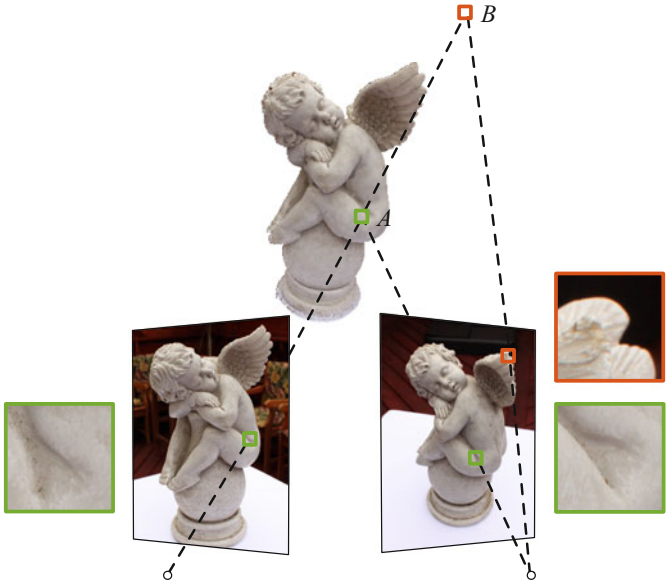


Fig. 17.1 Illustration of the concept of photo-consistency. The patch A is photo-consistent, and its two projections are similar, whereas patch B is not, and in fact its two projections are very different

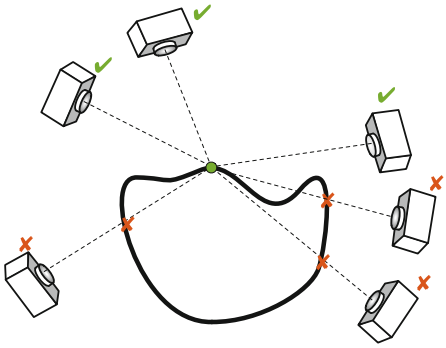


Fig. 17.2 The visibility problem. In order to recover the surface by photo-consistency, the visibility is needed. At the same time, to calculate the visibility the surface is needed. Adapted from Hernández and Vogiatzis (2010)

Multi-view stereo algorithms can be grouped into four classes.

Volumetric stereo approaches (on which we will focus in this chapter) assume that there exists a known, finite volume within which the objects of interest lie. This volume can be viewed as a parallelepiped surrounding the scene and is represented by a grid of "cubes" called *voxels*, the 3D equivalent of pixels. The goal is to assign a binary occupancy label (or, possibly, a colour) to each element in the working volume. The desired final representation is a polygonal mesh representing the surface of the object. To step from a volumetric representation to a polygonal mesh, marching cubes (Sect. 17.4) is the standard algorithm.

Within these approaches, we have *object-space* techniques (see Dyer 2001 for an excellent review), which proceed by scanning the working volume and assigning an occupancy label to each voxel according to its photo-consistency (Sect. 17.2), and *image-space* techniques, which start by calculating correlation (or any other matching cost) between windows in the images (similarly to binocular stereo), and only eventually assign a photo-consistency value to the voxel (Sect. 17.3).

The second class of techniques start directly from a polygonal mesh that roughly represents the object and evolve it in the direction of optimising its photo-consistency (Faugeras and Keriven 1998; Delaunoy et al. 2008). This class can also be considered as a post-processing for other methods.

The third class consists of algorithms that compute a set of depth maps from binocular stereo and merge them into a 3D model.[2] These methods immediately generalise binocular stereo and can exploit fusion methods originally conceived for range images (Goesele et al. 2006).

The last class starts from photo-consistent points that definitely belong to the surface and expand them by generating a "cloud" of planar elements or *patches* oriented in space (Furukawa and Ponce 2010).

The last three classes will not be covered in this chapter. The reader is referred to Seitz et al. (2006) for a comprehensive review

To obtain a number of images of an object in a controlled way, one can use a fixed calibrated camera and a turntable on which the object is placed. For each rotation step of the plate, one image is taken. The situation is equivalent to a static object and a camera moving around, but the rotation of the plate is easily measurable, while the freehand motion of the camera is more complex to gauge.

17.2 Volumetric Stereo in Object-Space

Object-space methods scan the working volume voxel by voxel and assign its occupancy value. The implementation can start from a completely empty volume as in Seitz and Dyer (1999) and mark as opaque the voxels that pass the photo-consistency test or it can start from a full space (Kutulakos and Seitz 2000) and selectively remove the voxels that do not pass the test. In the end a colour is assigned to each opaque voxel taken from the images on which it is projected (e.g., by averaging).

[2] If the fusion algorithm is volumetric, the boundary with image-space volumetric stereo becomes blurred.

Compared to image-space techniques, volumetric stereo in object space offers some advantages: it avoids the difficult problem of finding correspondences (it also works for non-textured surfaces), it allows explicit treatment of occlusions, and it yields directly a three-dimensional model of the object by simultaneously integrating all views, instead of aligning portions of the model.

On the other hand, deciding the photo-consistency of a voxel is a non-trivial task, and to evaluate it correctly would require—in principle—information about the geometry of the objects, the reflectance properties of the surfaces, and the direction of illumination. In image-space, instead, one is confronted with the simpler task of choosing the best photo-consistency value in a given set.

17.2.1 Shape from Silhouette

A special case of volumetric stereo is the reconstruction from binary silhouettes, a.k.a. *shape from silhouette* (Martin and Aggarwal 1983). Intuitively, a silhouette is the outline of an object, including its internal part. More precisely, we define *silhouette* as a binary image whose value at some point (u, v) indicates whether or not the optical ray passing through the pixel (u, v) intersects the surface of an object in the scene.

Each white point on the silhouette therefore identifies an optical ray that intersects the object. The union of all white optical rays defines a generalised cone that contains the object. The intersection of the generalised cones associated with all the frames identifies a volume within which the object lies (Fig. 17.3).

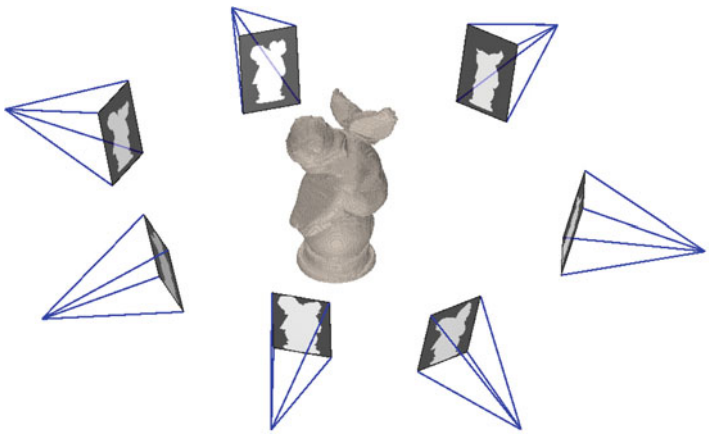


Fig. 17.3 Illustration of reconstruction from silhouettes

Listing 17.1 shows a (not particularly efficient) implementation of shape from silhouette. It starts with a cubic lattice of points sampling the working volume (corresponding to the voxel centres), passed to the function via the variable `M`. Each point is processed once, its projections in the images are determined, and it is defined as opaque if all projections fall within the silhouette, transparent otherwise.

Listing 17.1 Reconstruction from silhouettes

```
function V = shape_from_silhouettes(M, Im, PPM, bgnd)
%SHAPE_FROM_SILHOUETTES Shape from silhouettes

    V = true(size(M,2),1); % all opaque
    S = zeros(2,size(PPM,2));

    for j = 1: size(M,2) % for every voxel
        for i = 1: size(PPM,2) % for every image
            S(:,i) = htx(PPM{i}, M(:,j));
        end
        % S contains pixels corresponding to voxel j
        C = get_colors(Im, S, bgnd);  % get the colors of S
        V(j) = ~any(C == bgnd);  % set opacity
    end
end  % output: V is true for opaque, false for transparent

function C = get_colors(Im,x,bgnd)
    % return the color of pixels x
    C = zeros(size(Im,2),1);
    for i = 1: size(Im,2)
        if   x(1,i)<=0 || x(2,i)<=0 ||...
                x(1,i)>size(Im{i},2) || x(2,i)>size(Im{i},1)
            C(i) = bgnd;
        else
            C(i) = Im{i}(ceil(x(2,i)),ceil(x(1,i)));
            % would be better to interpolate
        end
    end
end
```

The resulting volume is taken as an approximation of the object (Fig. 17.4). More precisely, it is an outer approximation of its *visual hull*, defined by Laurentini (1994) as the maximal volume that returns the same silhouettes as the real object for all views outside the convex hull of the object. In practice, since only a finite

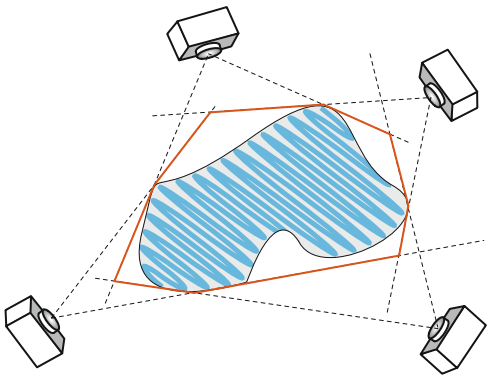


Fig. 17.4 The object and the approximation of its visual hull computed from four frames



Fig. 17.5 Volumetric reconstruction of the cherub obtained from 65 silhouettes, 6 of which are shown as examples. The image was obtained from the volume after surface extraction with marching cubes (Sect. 17.4)

number of silhouettes are available, only an outer approximation of the visual hull is achievable.

The shape from silhouettes is a proven technique that produces stable results, and often the resulting 3D model is a good approximation of the real object and is an excellent initialisation for other methods. An example result is shown in Fig. 17.5.

17.2.2 Szeliski's Algorithm

In order to save memory and computational time, Szeliski (1993) exploits an octonary tree (*octree*) data structure to store voxel labels. These are eight-way trees in which each node represents a cubic region of space and the child nodes are the eight equispaced subdivisions of that region (octants). The octree is a more space-efficient representation when the scene contains large empty areas.

The octree is constructed recursively by dividing each cube into eight cubes, starting with the root node representing the working volume. Each node represents a cube, to which a label is associated:

White: leaf representing an occupied volume
Black: leaf representing an empty volume
Grey: internal node whose classification is still undecided

For each octant, the algorithm checks whether its projection onto the i-th image is entirely contained in the silhouette. If this is the case for all N silhouettes, the octant is marked as "white". If, on the other hand, the octant projection is entirely contained in the background, *even for a single camera*, the octant is marked as "black". If either of these cases occurs, the octant becomes a leaf of the octree and

is no longer processed. Otherwise, the octant is classified as "grey" and is in turn subdivided into eight children. To limit the size of the tree, the "grey" octants of minimum size are marked as "white". In the end an octree that approximates the visual hull of the object is obtained.

The shape from silhouette can also be implemented efficiently on the GPU with a plane sweep method (Li et al. 2003).

The shape from silhouette was one of the first 3D modelling approaches to be invented: around 1857, Francois Willeme developed a method, dubbed photosculpture, which is nothing more than a mechanical version of the visual hull reconstruction. Willeme captured images in a circle around the subject and then projected them sequentially onto a screen. A block of clay was placed behind the screen, which was subsequently rotated to reproduce the angle under which the current image had been acquired. Then, using a pantograph, the sculptor followed the silhouette of the object on the projector screen, while at the other end of the device a tool carved out the parts of the clay block that were outside the silhouette. Repeating this procedure for all images, the result is the visual hull of the subject in clay. The sculpture was then finished by hand to round off the edges and add detail.

This note is taken from Sturm (2011), which contains much more interesting information about the very early versions of computer vision algorithms.

17.2.3 Voxel Colouring

When the input images are not simple binary silhouette but are grey-level or colour images, additional radiometric information can be exploited to improve the reconstruction.

The order in which the voxels are visited is not irrelevant here. In fact, when testing the photo-consistency of a voxel, it is necessary to know on which images that voxel projects, otherwise it could be mistakenly considered not photo-consistent, as would happen to voxel B in Fig. 17.6 if one ignored that it is occluded by voxel A in the right image. In this case, the photo-consistency of voxel A must be evaluated before B. In general, the photo-consistency of a voxel should only be evaluated after all voxels that could affect its visibility have been evaluated (i.e. have a final label).

To simplify the calculation of voxel visibility so that a scene can be reconstructed in a single voxel scan, Seitz and Dyer (1999) introduced the *ordinal visibility constraint* on camera positions. It requires that there exists an ordering of voxels

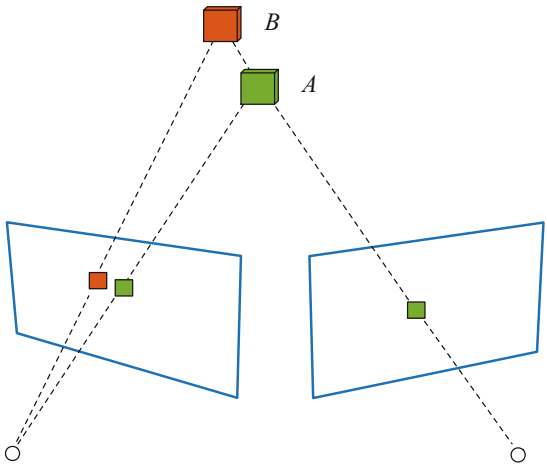


Fig. 17.6 Photo-consistency and visibility. If we ignored that voxel B is not visible in the right image, we could mistakenly consider it not photo-consistent

such that the visibility of a certain voxel can only be affected by the voxels preceding it in the ordering.

Evidently, if the constraint is satisfied, it becomes possible to implement an algorithm that verifies photo-consistency with a single pass. This happens, for example, when the cameras are all on the same side of a plane. In such a case the voxels can be visited by sliding that plane front to back, that is, from the closest voxels to the voxels farthest from the COP of the cameras.

The Voxel Colouring (VC) is a one-pass algorithm that, starting from empty space, marks voxels that pass the photo-consistency test as opaque. The occlusions are stored in a simple binary map for each of the input images. When a voxel is found to be consistent (and therefore opaque), the occlusion bits corresponding to the projections of the voxel on the various images are set to 1. At any given time, the visibility set of a voxel is given by the pixels onto which the voxel projects whose occlusion bits are 0.

More generally, the ordinal visibility constraint is satisfied when no point in the scene falls inside the convex hull formed by the COPs. This generalisation allows the algorithm to be implemented by sliding not a simple plane, but a front of surfaces at an increasing distance from the convex hull.

VC is an elegant and efficient technique, but the ordinal visibility constraint poses a significant limitation: cameras cannot surround the scene.

In the case of semitransparent objects, an opacity value should be added to the voxel, besides to the colour. There is an interesting analogy with computer tomography, which has been described in Gering and Wells (1999), Dachille et al. (2000).

17.2.4 Space Carving

The typical algorithm of this class starts with a volume composed of initially opaque voxels that enclose the scene to be reconstructed. As the algorithm proceeds, the photo-consistency of the opaque voxels is evaluated, and those that are found to be inconsistent are rendered transparent, that is, *carved*. The algorithm stops when all remaining opaque voxels are photo-consistent. As already observed, for generic camera configurations there is no ordering of voxels that guarantee that the visibility of a voxel does not change after its photo-consistency has been evaluated. Therefore, multiple passes are needed.

Listing 17.2 describes the general approach for Space Carving (SC) algorithms. In the inner loop, the visibility of voxels is determined, their photo-consistency is checked, and they are finally carved if they are found to be inconsistent. When a voxel is carved, the visibility of other voxels potentially changes, invalidating any consistency tests they may have passed. Thus, there is an outer loop that repeats the consistency check until no carving occurs in the inner loop. In this way, the final set of opaque voxels is guaranteed to be photo-consistent.

Listing 17.2 General scheme (stub) of space carving

```
function V = space_carve(M, Im, PPM)
%SPACE_CARVE Space carving algorithm

    V = ones(size(M,2),1); % all opaque

    while true % loop
        allVoxelsConsistent =  true;

        for j = 1: size(M,2) % for every voxel
            if ~isnan(V(j)) % process only opaque voxels

                % S contains the pixels corresponding to voxel j
                S = find_pixels(PPM,M(:,j));

                % get the colors of S
                C = get_colors(Im,S) ;

                % set voxel opacity/color
                if is_consistent(C)
                    V(j) = mean(C); % mean color
                else
                    V(j) = NaN; % transparent
                    allVoxelsConsistent =  false;
                end
            end
        end
        if allVoxelsConsistent; break; end
        % until allVoxelsConsistent
    end
end  % output: V has color of the voxel or NaN for transparent
```

At first, the reconstruction bears little resemblance to the real object; however, the SC algorithm calculates the visibility for voxels and sculpts those deemed inconsistent, based on this reconstruction. It is reasonable to ask, then, whether the algorithm might fail by sculpting voxels too early that should then have been found photo-consistent eventually. Seitz and Dyer (1999) have shown that, in fact, this

cannot happen if the consistency measure is *monotonic*, that is, if a certain set of pixels is inconsistent, then every superset of those pixels will also be inconsistent.

Since the algorithm only changes the assignment of voxels from opaque to transparent and never vice versa, the remaining opaque voxels can only become visible as the algorithm continues, and the pixels that see a voxel at a certain instant of time will necessarily be a subset of those that see the voxel at any later time. Thus, if the monotonic photo-consistency measure results in the inconsistency of a voxel, the voxel will also be inconsistent in the final reconstruction. Therefore, the algorithm is *conservative*, that is, it never carves a voxel that it should not (i.e. a truly opaque voxel) and thus correctness is guaranteed.

Kutulakos and Seitz (2000) proposed a variant of SC that evaluates voxels according to a sweeping scheme: at each iteration the scene is swept with a different plane (for simplicity, the three planes parallel to the coordinate axes are chosen). The visibility of a voxel is evaluated by considering only the cameras that are behind the current plane. Thus, when a voxel is evaluated, the transparency of other voxels that might occlude it from the cameras currently in use is already known.

Note that in the photo-consistency evaluation not all cameras are used but only those behind the current plane, even if the voxel is visible from other cameras. This introduces an approximation of visibility that manifests itself in a tendency to retain more voxels than necessary. An additional photo-consistency test is applied at the end of each iteration with respect to *all* the cameras to remove any excess voxels.

17.3 Volumetric Stereo in Image-Space

We will now describe a multi-view (volumetric) stereo method inspired by the work of Hernández and Vogiatzis (2010).

We assume that we start with a set of oriented $I_1 \ldots I_m$ images: typically, before this step, the PPM and a sparse model have been computed, as described in Chap. 14.

The goal is to obtain a three-dimensional photo-consistency map $\varphi(\mathbf{x})$ that indicates how photo-consistent a particular point $\mathbf{x} \in \mathbb{R}^3$ is in a given set of images in which it is visible and then recover the surface of the object.

The approach of Hernández and Vogiatzis (2010) is to compute, for each image, a depth map by aggregating information from a number of neighbouring images (generalising binocular stereo). Then the individual depth maps are combined in object space to obtain the photo-consistency map $\varphi(\mathbf{x})$.

This algorithm does not explicitly treat visibility. However, due to the locality of the images used, if the correlation measure is robust, it is possible to ignore the visibility problem by regarding occlusions as disturbances.

Let us now see how a depth map is generated for each image. Fixed a reference image I_i, for each point $\mathbf{m}$ of I_i we compute correlation functions along the point's epipolar lines in neighbouring images. More precisely, let $\mathcal{N}(i)$ be the indices of the k images closest to I_i and march along the epipolar line of $\mathbf{m}$ in each of them,

computing (for example) the normalised cross-correlation (NCC) over a window of fixed size. We thus have k correlation profiles[3] $c_j(\zeta)$, $j \in \mathcal{N}(i)$, which must be aggregated into one $c_a(\zeta)$.

Note that in order to compare these functions with each other, it is necessary to find a common parameter to describe the epipolar lines, which, as we will see below, is precisely the depth ζ.

Regarding aggregation, the immediate (but not necessarily the best) choice falls on the average:

$$c_a(\zeta) = \frac{1}{k} \sum_{j \in \mathcal{N}(i)} c_j(\zeta). \tag{17.1}$$

The depth $z_i(\mathbf{m})$ to be associated with the point $\mathbf{m}$ in the constructing map is the value for which $c_a(\zeta)$ reaches the maximum.

The merging of the z_i maps into 3D is done by a voting mechanism in the discretised space, divided into *voxels*: each point in each depth map votes for a specific voxel of centre $\mathbf{x}$ with its aggregate correlation value $c_a(\zeta)$.

The procedure for calculating the photo-consistency map is summarised in Algorithm 17.1.

Algorithm 17.1 MULTI-VIEW STEREOPSIS

Input: m oriented images $I_1 \ldots I_m$
Output: photo-consistency map $\varphi(\mathbf{x})$

1. Initialise $\varphi(\mathbf{x}) = 0$
2. For each image I_i, construct the depth map z_i as follows:
   (a) for each point $\mathbf{m} \in I_i$,
      i. for every I_j with $j \in \mathcal{N}(i)$ (neighbourhood of I_i)
      ii. computes $c_j(\zeta)$, the NCC of $\mathbf{m}$ along its epipolar line in I_j.
   (b) computes the aggregate correlation of $\mathbf{m}$, $c_a(\zeta)$,
   (c) computes the depth of $\mathbf{m}$, $z_i(\mathbf{m}) = \arg\max_\zeta c_a(\zeta)$, ...
   (d) and the correlation value $\widehat{c}_i(\mathbf{m}) = \max_\zeta c_a(\zeta) = c_a(z_i(\mathbf{m}))$.
3. For each depth map z_i, raise it in space as follows:
   (a) for each point $\mathbf{m} \in z_i$,
      i. $\varphi(\mathbf{x}) = \varphi(\mathbf{x}) + \widehat{c}_i(\mathbf{m})$ where $\mathbf{x}$ is the point of depth $z_i(\mathbf{m})$ along the optical radius of point $\mathbf{m}$ in I_i.

[3] Note that correlation is a measure of photo-consistency, so it is the opposite of the matching cost introduced in Chap. 12.

Alignment of Epipolar Lines

Consider a pair of images I and I', and let $\mathbf{m}$ be a point in the first image. Let us write the epipolar line of $\mathbf{m}$ parameterised with the depth of the point, ζ (see Problem 6.1):

$$\zeta' \mathbf{m}' = \zeta \widetilde{P}' \widetilde{P}^{-1} \mathbf{m} + \mathbf{e}'. \tag{17.2}$$

Using Proposition A.18 to solve (17.2), we can derive the ζ' of a generic point $\mathbf{m}'$ of the epipolar line of $\mathbf{m}$:

$$\zeta' = \frac{(\mathbf{e}' \times \mathbf{m}'_\infty)^\mathsf{T} (\mathbf{m}' \times \mathbf{m}'_\infty)}{\|\mathbf{m}' \times \mathbf{m}'_\infty\|^2}, \tag{17.3}$$

where $\mathbf{m}'_\infty = \widetilde{P}' \widetilde{P}^{-1} \mathbf{m}$. This relation allows us to align (via the depth ζ) the epipolar lines of $\mathbf{m}$ in different images.

Aggregation

The choice of the mean as an aggregator proves to be not very robust in dealing with occlusions, specular reflections, and textureless areas. Hernández and Vogiatzis (2010) observe that although the global maximum of the correlation curves (and their mean) may be wrong, it is seldom true that the individual correlation curves present a local maximum at the correct depth, if the point is visible in that image (Fig. 17.7). They therefore propose to aggregate the local maxima of the individual correlation curves with a voting scheme. A triangular or Gaussian Parzen window, W, is employed to compute an aggregate correlation that accounts for local maxima with their value and reinforces the presence of nearby local maxima in different correlation curves. In formulae, let ζ_ℓ be the local maxima of $c_j(\zeta)$ above a certain threshold (e.g., 0.6 out of 1):

$$c_a(\zeta) = 1/k \sum_{j \in \mathcal{N}(i)} \sum_{\ell} c_j(\zeta_\ell) W(\zeta - \zeta_\ell). \tag{17.4}$$

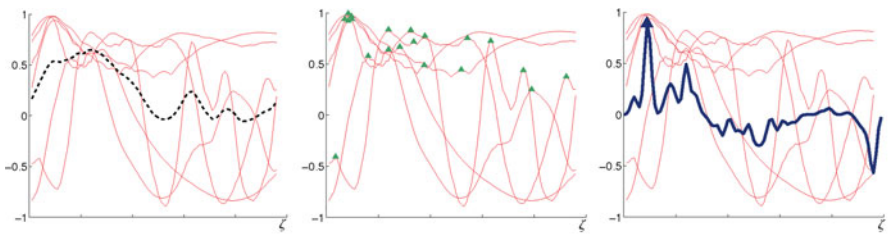


Fig. 17.7 On the left are the k correlation curves $c_j(\zeta)$ and their mean (dashed), in the centre are highlighted with a triangle the local maxima of each curve, and on the right is the aggregate correlation curve with the Parzen window and its maximum (triangle), which corresponds to the true depth, different from the maximum of the mean. Images courtesy of G. Vogiatzis



Fig. 17.8 Graphic rendering of the 3D model of the cherub. This is a polygonal mesh with colours associated with the vertices. The detail on the right shows the structure of the mesh

The depth map calculation step can be further improved (Hernández and Vogiatzis 2010) by relaxing the approach that considers the maximum of the aggregate correlation function with a Markov random field in which the best local maxima of the correlation are all candidates for the depth of a point.

Surface Recovery
At this point one has a photo-consistency map $\varphi(\mathbf{x})$ defined on a volume and wants to determine from it a surface that is photo-consistent and regular (Fig. 17.8). The problem can be seen as a segmentation of the volume into object-background, the result of which is precisely the surface of the object. The solution can be obtained by applying the minimum-cut algorithm (dual of the maximum flow) on a suitably constructed graph, where the nodes are the voxel centres and the edge connecting two voxels $\mathbf{x}_i$ and $\mathbf{x}_j$ has a weight equal to $\varphi(\frac{\mathbf{x}_i+\mathbf{x}_j}{2})$. The rational is that the cost of separating two adjacent voxels (i.e., considering them object and background) is proportional to the photo-consistency of the two.

Visibility is implicitly taken into account by the photo-consistency measure, which is robust to occlusions. However, the accuracy of the reconstruction improves if visibility is taken into account in the formulation of the minimum cut problem.

Alternatively, one can instead obtain a photo-consistent point cloud by thresholding on $\varphi(\mathbf{x})$ and fall back to the problem of reconstructing a surface from scattered points, which we will mention at the end of the chapter.

17.4 Marching Cubes

The marching cubes algorithm, introduced by Lorensen and Cline (1987), is a well-known technique for extracting a surface from a volume. More precisely, given a 3D

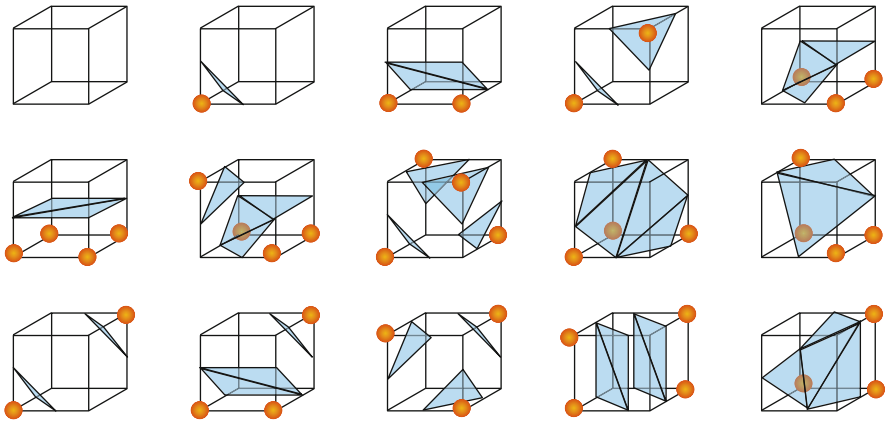


Fig. 17.9 The 15 possible configurations of the vertices of a cube of the lattice, modulo symmetries. From https://commons.wikimedia.org/w/index.php?curid=1282165

scalar field $f(x, y, z) : \mathbb{R}^3 \rightarrow \mathbb{R}$ sampled on a cubic lattice, one wants to extract the isosurface corresponding to a given value c.

The isosurface divides the space into two regions, the one for which $f(x, y, z) > c$ and the one for which $f(x, y, z) \leq c$. We mark all points in the cubic lattice that belong to the region $f(x, y, z) > c$. Assuming that the underlying function $f(x, y, z)$ is reasonably regular, it takes the value c once on each side connecting a marked vertex with an unmarked one.

Given any cube in the lattice, it is then possible to determine the shape that the isosurface must take relative to that single cube. There are 256 cases, but they are reduced to 15 by symmetry and can be tabulated (Fig. 17.9). It is seen that in this way the cubes are separately processable, and the algorithm is implemented in the function `isosurface` of MATLAB.

As a first approximation, we decide to place the point where $f(x, y, z) = c$ exactly in the middle of edge, as in Fig. 17.9. However, it is possible to do better by determining the point of intersection of the isosurface with the edge of the cube by a linear interpolation of the values of f at the endpoints of the edge.

The marching cubes algorithm also finds use as the last stage of surface extraction from *point clouds* (Hoppe et al. 1992; Kazhdan et al. 2006). Unlike the previous case, where the lattice points can be seen as sampling a 3D scalar field, the point cloud does not have an associated scalar value and this must be constructed as part of the procedure.

We then define the field function $f(x, y, z)$ as the distance from (x, y, z) to the nearest point on the surface of the object and approximate it through the data points, which are on the surface by definition. In this way, the surface sought is the zero-level set of the scalar field: $f(x, y, z) = 0$. To finally move from this implicit representation of the surface to a triangular mesh, the *marching cubes* algorithm is used.

References

Frank Dachille, Klaus Mueller, and Arie Kaufman. Volumetric backprojection. In *Proceedings of the 2000 IEEE symposium on Volume visualization*, pages 109–117. ACM Press, 2000.

Amael Delaunoy, Emmanuel Prados, Pau Gargallo, Jean-Philippe Pons, and Peter Sturm. Minimizing the multi-view stereo reprojection error for triangular surface meshes. *British Machine Vision Conference*, 2008.

C.R. Dyer. Volumetric scene reconstruction from multiple views. In L. S. Davis, editor, *Foundations of Image Understanding*, chapter 16. Kluwer, Boston, 2001.

O. Faugeras and R. Keriven. Variational principles, surface evolution, PDEs, level set methods, and the stereo problem. *IEEE Transactions on Image Processing*, 7(3):336–344, March 1998.

Yasutaka Furukawa and Jean Ponce. Accurate, dense, and robust multiview stereopsis. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 32(8):1362–1376, August 2010.

D. T. Gering and W. M. Wells. Object modeling using tomography and photography. In *Proceedings of the IEEE Workshop on Multi-View Modeling and Analysis of Visual Scenes*, pages 11–18, 1999.

Michael Goesele, Brian Curless, and Steven M. Seitz. Multi-view stereo revisited. pages 2402–2409, Washington, DC, USA, 2006. IEEE Computer Society. ISBN 0-7695-2597-0. doi: 10.1109/CVPR.2006.199. URL http://dx.doi.org/10.1109/CVPR.2006.199.

Carlos Hernández and George Vogiatzis. Shape from photographs: A multi-view stereo pipeline. In Roberto Cipolla, Sebastiano Battiato, and Giovanni Maria Farinella, editors, *Computer Vision: Detection, Recognition and Reconstruction*, volume 285 of *Studies in Computational Intelligence*, pages 281–311. Springer, Berlin, 2010.

Hugues Hoppe, Tony DeRose, Tom Duchamp, John McDonald, and Werner Stuetzle. Surface reconstruction from unorganized points. *Computer Graphics*, 26(2):71–78, July 1992.

Michael Kazhdan, Matthew Bolitho, and Hugues Hoppe. Poisson surface reconstruction. In *Proceedings of the fourth Eurographics symposium on Geometry processing*, pages 61–70. Eurographics Association, 2006.

K. N. Kutulakos and S. M. Seitz. A theory of shape by space carving. *International Journal of Computer Vision*, 38(3):199–218, 2000.

A. Laurentini. The visual hull concept for silhouette-based image understanding. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 16(2):150–162, 1994.

Ming Li, Marcus Magnor, and Hans Peter Seidel. Hardware-accelerated visual hull reconstruction and rendering. In *In Graphics Interface 2003*, pages 65–71, 2003.

W.E. Lorensen and H.E. Cline. Marching cubes: a high resolution 3-D surface construction algorithm. In M.C. Stone, editor, *SIGGRAPH: International Conference on Computer Graphics and Interactive Techniques*, pages 163–170, Anaheim, CA, July 1987.

W. N. Martin and J. K. Aggarwal. Volumetric descriptions of objects from multiple views. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 5(2):150–158, March 1983.

S. M. Seitz and C. R. Dyer. Photorealistic scene reconstruction by voxel coloring. *International Journal of Computer Vision*, 35(2):151–173, 1999.

Steven M. Seitz, Brian Curless, James Diebel, Daniel Scharstein, and Richard Szeliski. A comparison and evaluation of multi-view stereo reconstruction algorithms. In *IEEE Conference on Computer Vision and Pattern Recognition*, pages 519–528, 2006.

Peter Sturm. A historical survey of geometric computer vision. In *Computer Analysis of Images and Patterns*, pages 1–8. Springer, Berlin, 2011.

R. Szeliski. Rapid octree construction from image sequences. *CVGIP: Image Understanding*, 58 (1):23–32, 1993.

Chapter 18
Image-Based Rendering



18.1 Introduction

In this last chapter, techniques are presented that have in common the fact to synthesise—or to *render*—an image obtained from the geometric transformation of one or more input images; we speak also of image-based rendering. Synthetic images can be interpreted as obtained from a virtual camera that differs from the real one for some aspects such as a greater field of view, or a different orientation.

Image synthesis from other images (image-based rendering), as opposed to image synthesis from three-dimensional models (model-based rendering), originates from the idea that a scene can be represented by a collection of its images. Those that are missing can be synthesised from existing ones. It is also referred to as image interpolation.

18.2 Parametric Transformations

There are several types of geometric image transformations: affine, projective, piecewise affine and non-parametric transformations, as in the case of a disparity or parallax field.

The simplest case of synthesis is when an image is transformed through a homography. We will describe in this section some applications based on projective transformations.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4_18

18.2.1 Mosaics

Image *mosaicing* is the alignment (or registration) of multiple images into larger aggregates through the application (in general) of homographies. Mosaicing synthesises an image taken from a virtual camera with a larger field of view. Since there are two cases where images of a scene are linked by homographies, there are correspondingly two types of mosaics:

panoramic mosaics: images are taken from a camera rotating on its COP, as in the case of Fig. 18.1;

planar mosaics: different images of a planar scene, or approximately so, are aggregated, such as the terrain seen from the zenith (Fig. 18.2).

The construction of a mosaic is normally accomplished by aligning the input images with respect to a common reference frame and then blending them into a single composite image. Thus, it proceeds through two stages:

alignment: for each image calculate and apply the homography that aligns it with the reference image;

blending: assign a colour to each pixel of the mosaic, based on the colours of the various images that insist on it.

An excellent reference to delve into mosaicing is Szeliski (2006).

When a panoramic mosaic is projected onto a plane, as one moves away from the reference frame, the images are "stretched" horizontally (obvious) and vertically (less obvious), as seen in Fig. 18.1. To cope with large rotations and overcome the 180-degree limit, one can switch to cylindrical or spherical coordinates.



Fig. 18.1 Panoramic mosaic from three images. Note the stretching both horizontally and vertically as one moves away from the central reference image



Fig. 18.2 Planar mosaic of a terrain taken from a helicopter. Courtesy of Helica srl

18.2.1.1 Alignment

Alignment of images can be approached by methods analogous to those seen for 3D model registration in Chap. 15: what changes is the nature of the data, from 3D point clouds to 2D images, and the type of transformation, homographies of $\mathbb{P}^2$ in place of isometries (or similitudes) of $\mathbb{R}^3$.

Therefore, in the wake of ICP, one can devise correspondence-less methods that, use all pixels to compute the best homography that aligns the two images (such as Szeliski 1996). This approach would become impractical if extended to many images, where feature extraction and matching is to be preferred, as in Brown and Lowe (2007). Another approach would be to compute homographies between pairs of images with any method and then *synchronise* them, as we did for rotations in Chap. 14 and Sect. 15.2.3.

Synchronisation of Homographies
The synchronisation that we have seen at work in the case of rotations (Sect. 14.3.1) can be employed in the same way for homographies (Fig. 18.3). The homogeneous scaling factor, however, must be eliminated in order for the synchronisation to work: for this purpose we force the homography matrices to have unit determinant, and exploit the fact that they constitute a group, since the product of two matrices with unit determinant is also a matrix with unit determinant.

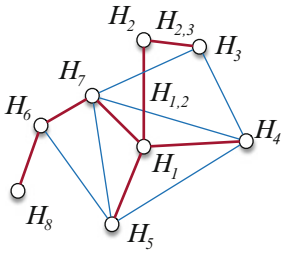


Fig. 18.3 Putative graph representing available homographies between a set of eight images. The spanning tree (thick line) provides a unique path from each node to the root (the reference image), along which to concatenate the homographies. Synchronisation, on the other hand, uses all available edges to compensate for error

The known homographies are packed into a matrix Z_A; the first three eigenvectors of $(D \otimes I_3)^{-1} Z_A$ are computed and stacked into the matrix U. The 3×3 blocks of this matrix correspond to the homographies to be applied to each image to realise the alignment. See Listing 18.1.

Listing 18.1 Homography synchronisation

```
function H = hom_synch(Z,A)
%HOM_SYNCH Homography syncronization

    n = size(A,1);

    iD = diag(1./sum(A,2));              % inverse degree matrix
    [Q,~]=eigs( kron(iD,eye(3))*Z,3,'lr'); % top 3 eigenvectors

    Q = real(Q/(Q(1:3, 1:3))); % normalize first homography to I
    % and guard against spurious complex valuess from roundoff

    H=cell(1,n);
    for i=1:n  % Force det = 1
        H{i} = Q(3*i-2:3*i,:)./nthroot(det(Q(3*i-2:3*i,:)),3);
    end
end
```

The benefit of synchronisation can be appreciated in Fig. 18.4, where two solutions are compared in the case of a sequence of homographies affected by small random errors; the first is obtained from the concatenation of consecutive transformations, creating a chain from the first to the last image; the second also considers the transformation that links the last image to the first, thus creating a loop on which synchronisation is performed. While in the first solution is clearly visible a drift that leads the solution to diverge from the true, the synchronisation exploits the presence of the loop to compensate for the error (a.k.a. *loop closing*), producing a solution much closer to the ground truth.

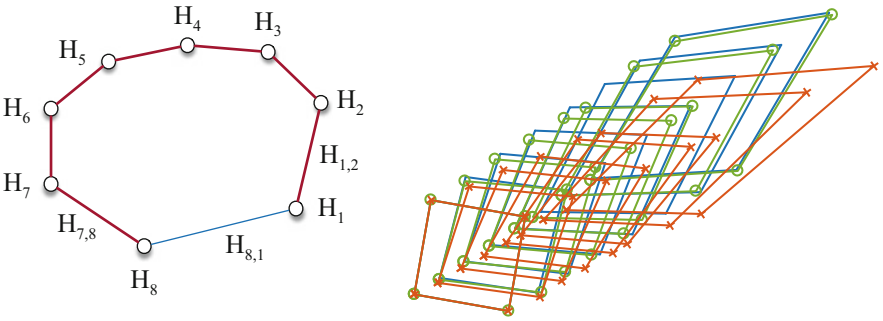


Fig. 18.4 Synchronisation of a single loop. The left figure shows the graph and its spanning tree (thick line). The two differ only in the presence of the edge $H_{8,1}$ that closes the loop. On the right, the transformation applied to a square is used to display the sequence of homographies $H_1 \dots H_8$. The vertices marked with a circle (green) represent the outcome of synchronisation, while the cross (red) represents simple concatenation. The ground truth is drawn in blue

18.2.1.2 Blending

After alignment, on a single pixel in the mosaic generally insist more than one image, which raises the question of how to assign the colour to the mosaic.

The most commonly used mixing operators are the average, the median and the selection of a single image as the colour source according to appropriate criteria. The feathering operator performs a weighted average of the colours, with the weight decreasing according to the distance of the pixel from the image edge. This remedies the vignetting effect which results in brighter images in the centre than in the periphery. Brown and Lowe (2007) show very good result with multi-band blending.

In the presence of misalignments due to residual alignment errors, parallax (deviations from flatness) or moving objects, the techniques just listed are nevertheless bound to produce visible artefacts. Blending cannot remedy problems inherently present the data; however, it can mask them as much as possible. This is the approach that informs methods that attempt to compose the mosaic with tiles each cut from a single image, so that the transitions between them (or "seams") are as little visible as possible.

The first criterion for reducing the impact of seams is to minimise their total length, so a tessellation of the plane with minimal perimeter is sought. Consider the centroidal *Voronoi tessellation* induced by the image centres projected on the mosaic reference plane. Recall that, given a set of centres, the *Voronoi tessellation* assigns to each centre a convex cell, to which belong the points in the plane closer to that centre than to any other. If the centers are themselves the centroids of those regions such a tessellation is called a *centroidal* Voronoi tessellation.

Two facts are relevant here: (i) as the number of centroidal Voronoi cells in a bounded region increases, each Voronoi cell approaches a hexagon (Du et al. 1999); (ii) the "honeycomb conjecture" (Hales 2001) states that the tessellation that divides the plane into regions of equal area with the smallest total perimeter is one with

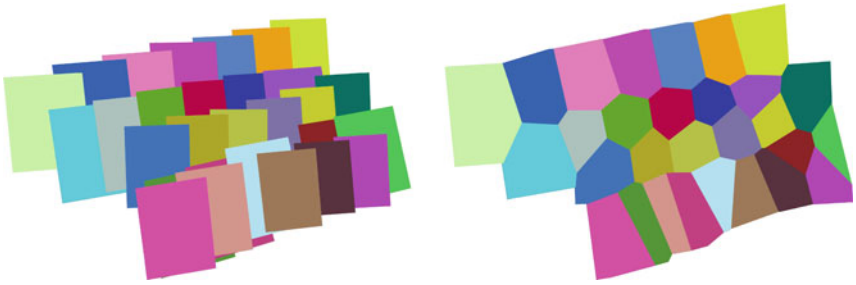


Fig. 18.5 Representation of the frames of the images transferred into the mosaic reference plane and Voronoi tessellation. Images taken from Santellani et al. (2018)

regular hexagons. For these reasons, the centroidal Voronoi tessellation of the image centres can be considered a cost-effective approximation of the lowest-perimeter tessellation.

Figure 18.5 shows the area covered by each image after its transformation into the mosaic reference plane (left) and the area assigned to that image after decomposition into Voronoi cells (right).

Boundaries between tiles are straight segments that do not take into account the content of the image; they should be refined by evolving them into paths connecting two vertices through pixels where colour differences between adjacent cells are minimal, as suggested by Davis (1998). In this way, if the original cell boundary crosses, for example, a moving object, the refined path will avoid it, since there are likely to be large differences in the affected pixels due to object-background contrast.

Figure 18.6 shows an example of the seams before and after optimisation. One can see how the cell boundary follows the space between the trees and avoids crossing them, since the canopies are more textured than the ground and affected by parallax.

18.2.2 Image Stabilisation

Given a sequence of images, the purpose of stabilisation is to compensate for camera motion, that is, to geometrically deform each image so that a given plane Π of the scene appears motionless, despite the camera being in motion. This can be easily accomplished by transforming each image with respect to a reference one (arbitrarily chosen) with the homography of the plane Π. This is the same procedure used in mosaics, but without blending: only the last transformed image is displayed, as in Fig. 18.7. If the scene is planar or the motion is rotational (in this case the plane Π is the one at infinity), the sequence will be completely stabilised; otherwise, the plane Π will appear motionless, and the rest of the scene in motion.

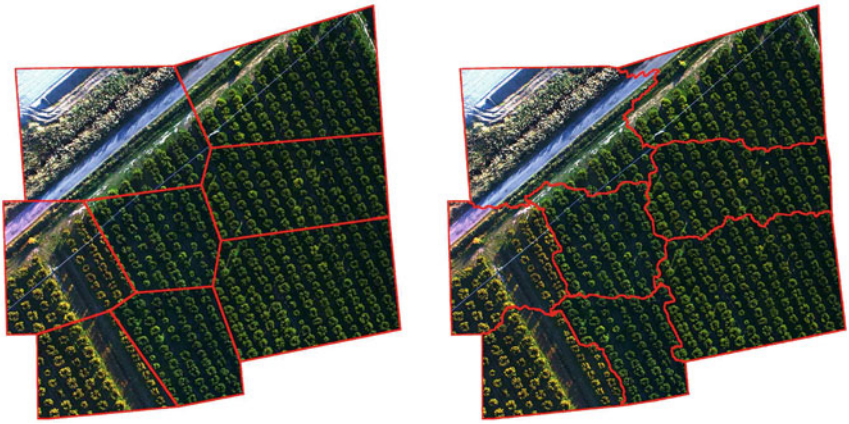


Fig. 18.6 Detail of the seams produced by Voronoi tessellation and optimised seams. Images taken from Santellani et al. (2018)

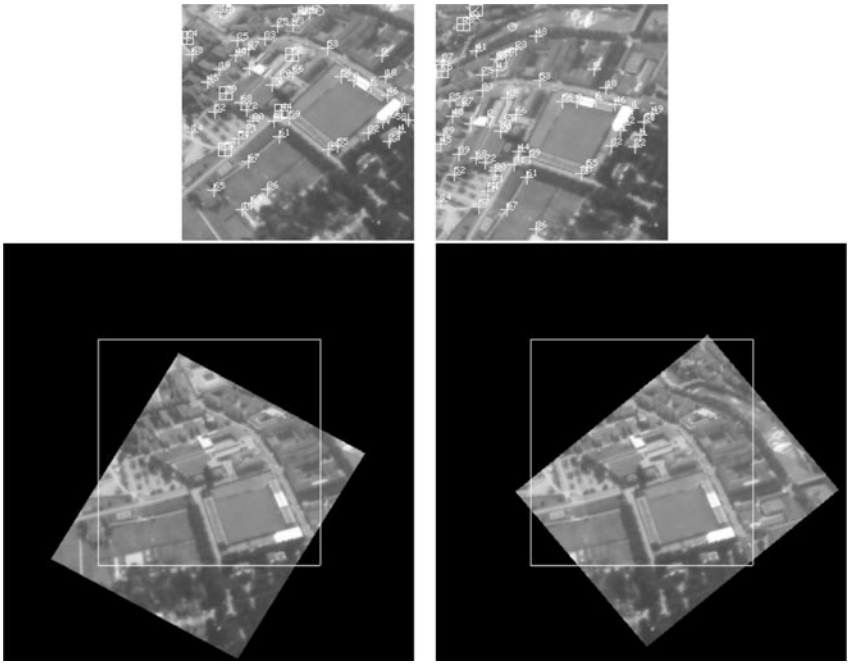


Fig. 18.7 Top row: two images from an aerial sequence. Bottom row: the same images stabilised with respect to the ground plane. In white the frame of the reference image. Images taken from Censi et al. (1999)

18.2.3 Perspective Rectification

There are cases where one would like to use an image as if was a map, a blueprint or an elevation, that is, drawings that has a constant scale of representation, thus allowing to measure real distances on them.

Photographs result from a perspective projection, and the scale is not constant across the image, unless in a special case where framed object is planar and parallel to the focal plane, as noted in Problem 3.3. In general, however, to have a constant scale, a scaled orthographic projection is required, which must be synthesised from the actual images and the depth of the points.

Let us first concentrate on the special case of approximately planar objects such as portions of the territory without great orographic discontinuity or architectural items such as paving or facades. Let the plane Π be our proxy of such object, then the image can be transformed or *rectified* so that the virtual focal plane is parallel to Π. The transformation between Π and its perspective image is a homography, which is completely defined by four points whose coordinates in Π are known. Once such a homography is determined, the image can be projected backwards onto Π. This is equivalent to synthesising an image taken by a virtual camera whose optical axis is perpendicular to Π. Such an image is also called a *photoplane*. An example is shown in Fig. 18.8. The photoplane can be obtained from a single photo or from a mosaic (in this case it is also called a *controlled mosaic*).

With reference to the photoplane, more complex methods that exploit information other than the four points are described by Liebowitz and Zisserman (1998).



Fig. 18.8 To the right a photograph of Porta Udine (Palmanova, IT). On the left the photoplane of the facade. The four points used in the rectification are the vertices of the yellow quadrilateral, which in the photoplane becomes a rectangle

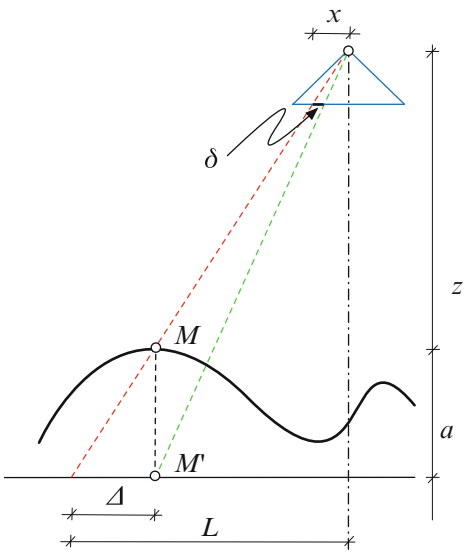


Fig. 18.9 The relief displacement. The actual object point is M, with relief a. Ignoring a it is tantamount to confusing M with M' that lies on the reference plane

The photoplane is correct only if applied to perfectly flat objects or with such depth variations that generates negligible errors at the scale of representation chosen. When the object to be represented is not exactly a plane, we can evaluate the extent of the error that is made. With reference to Fig. 18.9, consider a generic point M of depth z, which projects in M' onto the reference plane Π, and let a be the elevation of M with respect to that plane, or *relief*. The difference δ between the image of M, which is x away from the principal point, and that of M' represents the error in the image plane. From $\Delta/L = \delta/x$ and $\Delta/L = a/(z+a)$, one obtains:

$$\delta = x\frac{a}{d} \tag{18.1}$$

where $d = z + a$ does not depend on the point and is equal to the distance of the plane from the camera.

In the photogrammetric literature, δ is called *relief displacement*, since it manifests as a (radial) displacement of the image point caused by the relief; it is in fact a form of planar parallax.

We observe that δ is small when:

- the height a is small compared to the distance from the plane d;
- the distance x of the projected point from the principal point is small.

If the maximum parallax δ falls within a tolerance threshold,[1] then the photoplane can be considered an orthographic projection. Since parallax increases with the

[1] In photogrammetric survey this threshold is given by the graphical error, equal to the thickness of the mark on the paper, conventionally fixed at 0.2 mm.

distance of the image point from the principal point, using only the central part of the image in the composition of a mosaic allows to keep parallax under control.

18.3 Non-parametric Transformations

As long as the COP does not vary (or the scene is a plane), a homography is sufficient to synthesise new images, as we have seen (Fig. 10.4). If the COP changes, however, the structure of the scene comes into play in establishing the (non-parametric) function that maps the given images into the synthetic one.

18.3.1 Transfer with Depth

Consider the following two PPM:

$$P_1 = K_1[I, \mathbf{0}] = [K_1, \mathbf{0}] \quad \text{and} \quad P_2 = K_2[R_{12}, \mathbf{t}_{12}]. \tag{18.2}$$

Substituting these PPMs into the equation of the epipolar line of $\mathbf{m}$ with the explicit depths (6.30), we get:

$$\zeta_2 \mathbf{m}_2 = \zeta K_2 R_{12} K_1^{-1} \mathbf{m} + K_2 \mathbf{t}_{12}. \tag{18.3}$$

Thus, if the depth ζ of a point $\mathbf{m}$ in the reference image is known, its position $\mathbf{m}_3$ in the synthetic image is obtained by

$$\mathbf{m}_3 \simeq \zeta K_3 R_{13} K_1^{-1} \mathbf{m} + K_3 \mathbf{t}_{13} \tag{18.4}$$

where R_{13} and $\mathbf{t}_{13}$ specify the orientation of the virtual camera with respect to the reference (real) camera. K_3 are the intrinsic parameters of the virtual camera, and K_1 are those of the reference camera (which is therefore calibrated).

18.3.2 Transfer with Disparity

Ultimately, the depth is not really needed to transfer points onto the synthetic image. Also disparity, being a depth proxy, would do.

Assume that *two* calibrated reference images I_1, I_2 with a dense map of correspondences are given. One can exploit epipolar geometry to predict the position of a point in the third (synthetic) image I_3. The COP of the virtual camera is constrained to lie on the baseline segment between the COPs of the two reference cameras: hence, this is an *interpolation*.

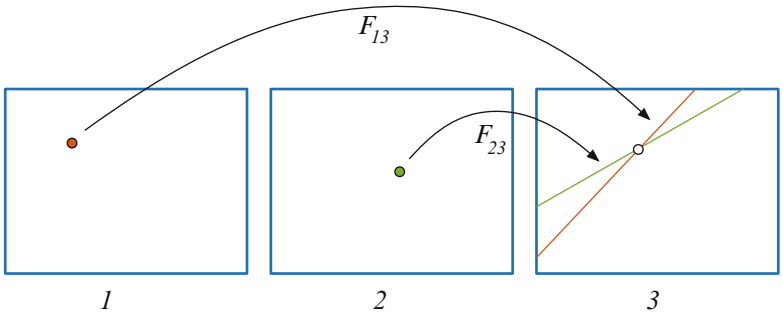


Fig. 18.10 The point in the third image is determined by the intersection of the epipolar lines of the other two

We assume that the cameras are in normal configuration, that is, the images are rectified. In this case, (18.3) specialises to:

$$\mathbf{m}_2 = \mathbf{m}_1 + \frac{1}{\zeta} K_2 [t_x, 0, 0]^\mathsf{T} \tag{18.5}$$

where t_x is the first component of $\mathbf{t}_{13}$. The difference $\mathbf{m}_2 - \mathbf{m}_1$ is the *disparity*. It is a vector, but since only the first component is nonzero, we normally identify the disparity with it. The transfer equation in the virtual image I_3 is

$$\mathbf{m}_3 = \mathbf{m}_1 + [\alpha d, 0, 0]^\mathsf{T} \quad \alpha \in [0, 1]. \tag{18.6}$$

It is easily verified that interpolating the disparity in this case is equivalent to moving the camera to intermediate positions along the baseline, from 0 to t_x. The algorithm was introduced by Chen and Williams (1993) and extended to unrectified images by Seitz and Dyer (1996) under the name of view morphing. The latter simply rectifies the two images, performs interpolation and eventually de-rectifies the resulting image.

18.3.3 Epipolar Transfer

Transfer is possible even in the uncalibrated case, that is, without knowing the intrinsic parameters. In fact all that is needed are correspondences and epipolar geometry. Given two reference images I_1, I_2 with a dense map of correspondences, one can exploit epipolar geometry to predict the position of a point in the third (synthetic) image I_3.

We assume that the fundamental matrices F_{21}, F_{31}, F_{32} have been computed. Given two homologous points in the two reference images, $\mathbf{m}_2$ and $\mathbf{m}_1$, the position of their homologous point in the third image $\mathbf{m}_3$ is determined by the constraint of

simultaneously belonging to the epipolar line of $\mathbf{m}_2$ and that of $\mathbf{m}_1$, as illustrated in Fig. 18.10. In formula:

$$\mathbf{m}_3 \simeq F_{31}\mathbf{m}_1 \times F_{32}\mathbf{m}_2. \tag{18.7}$$

This method fails when the three optical rays are coplanar (the two lines in the third image become coincident).

18.3.4 Transfer with Parallax

In the method of Sect. 18.3.2 disparity is used as a substitute for unknown depth. The planar parallax can also serve the same purpose. Recall that the relation between two images via a reference plane is given by (6.29), which we rewrite here as

$$\mathbf{m}_2 \simeq H_{12}\mathbf{m} + \gamma\mathbf{e}_2. \tag{18.8}$$

Given a number (>6) of homologous points, the homography H_{12} and the epipole $\mathbf{e}_2$ can be easily computed (see Ex. 18.1). The parallax at each point on the reference image is also calculated (Proposition A.18) by solving with respect to γ in (18.8):

$$\gamma = \frac{(H_{12}\mathbf{m}_1 \times \mathbf{m}_2)^\mathsf{T}(\mathbf{m}_2 \times \mathbf{e}_2)}{\|\mathbf{m}_2 \times \mathbf{e}_2\|^2}. \tag{18.9}$$

Beware that H_{12} and $\mathbf{e}_2$ are only known up to a scaling factor, so the modulus of the parallax γ also contains an unknown scaling factor.

Since the parallax γ does not depend on the second image, this can be replaced by a third one (Shashua and Navab 1996) and (18.8) can therefore be used for transferring points into the synthetic image I_3:

$$\mathbf{m}_3 \simeq H_{13}\mathbf{m}_1 + \gamma\mathbf{e}_3 \tag{18.10}$$

where the homography H_{13} induced by Π between the first and third images and the epipole $\mathbf{e}_3$ in the third image specify—indirectly—the orientation of the virtual camera.

Note the similarity of this technique to transfer with depth. The equation is similar, but in one case we employ "calibrated" quantities such as ζ, R and $\mathbf{t}$, while in the other "uncalibrated" quantities such as γ, H and $\mathbf{e}$, respectively. In other words, in the former case, we work in a Euclidean frame, while in the latter in a projective frame. The second requires less input information (no calibration is needed), but specifying the orientation of the virtual frame is problematic (Fusiello 2007), whereas in the Euclidean case, it is immediate.

An example of synthesis with parallax is shown in Fig. 18.11.

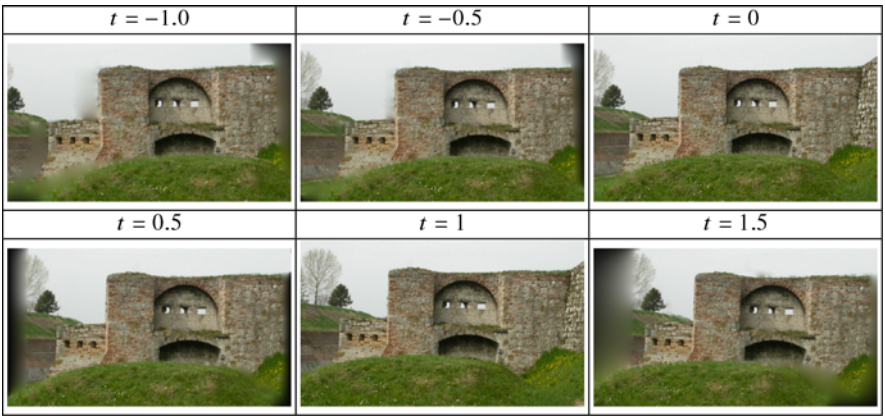


Fig. 18.11 Some frames of a sequence (Palmanova, IT) synthesised using parallax. The values $t = 0$ and $t = 1$ correspond to the reference images. For values between 0 and 1, there is interpolation; for values outside the range, there is extrapolation

18.3.5 Ortho-Projection

An *orthophoto* is an orthographic projection of the object along a direction perpendicular to a reference plane, such as the ground or a facade.

Contrary to the creation of the photoplane, where it is sufficient to identify the reference plane with at least four points, for rendering the orthophoto (a.k.a. *ortho-projection*), it is necessary to know the structure of the scene in order to define a non-parametric transformation of the original image. Hence, the orthophoto is synthesised in a similar way to the transfer with depth (Sect. 18.3.1), with the difference that now the virtual camera is affine. Consider then the following two projection matrices, the first perspective, the second affine:

$$P_1 = [K_1, \mathbf{0}] \quad \text{and} \quad P_3 = \begin{bmatrix} K_A & 0 \\ \mathbf{0} & 1 \end{bmatrix} \begin{bmatrix} \hat{R}_{13} & \hat{\mathbf{t}}_{13} \\ \mathbf{0} & 1 \end{bmatrix} = \begin{bmatrix} K_A \hat{R}_{13} & K_A \hat{\mathbf{t}}_{13} \\ \mathbf{0} & 1 \end{bmatrix} \tag{18.11}$$

where R_{13} and $\mathbf{t}_{13}$ specify the orientation of the virtual camera relative to the reference (real) camera and K_A contains the affine intrinsic parameters; it is a 2×2 matrix that corresponds to the principal submatrix of the usual intrinsic parameters matrix. $\hat{R}_{13}$ and $\hat{\mathbf{t}}_{13}$ are R_{13} and $\mathbf{t}_{13}$ deprived of the third row, respectively.

To obtain the expression of the epipolar line, which is used for the transfer, we write the optical ray of a point $\mathbf{m}$ in the first camera and project it (orthographically) into the second in $\mathbf{m}_3$, obtaining after a few steps:

$$\mathbf{m}_3 = \zeta \begin{bmatrix} K_A \hat{R}_{13} K_1^{-1} \mathbf{m}_1 \\ 0 \end{bmatrix} + \begin{bmatrix} K_A \hat{\mathbf{t}}_{13} \\ 1 \end{bmatrix} \tag{18.12}$$

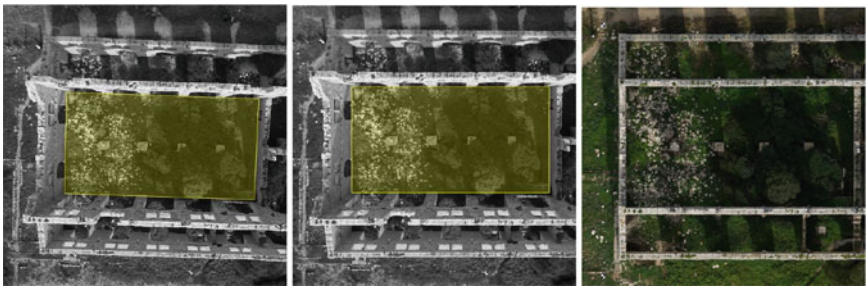


Fig. 18.12 From left: image taken from above (nadiral), photoplane relative to the ground, and orthophoto. The (vertical) walls in the orthophoto are not visible, while in the nadiral photo, they are visible, as well as in the photoplane, which does not change the visibility of the object points. Images taken from the "Benchmark SIFT2017", processed by E. Maset with 3DF Zephyr

In this case the structure of the scene is assigned through a depth map (ζ) referred to the original image, but usually it is given equivalently through a Digital Elevation Map (DEM) that for each point of the object shows the distance from the reference plane.

In the generation of an orthophoto, you can use a DEM relative to the ground (Digital Terrain Model, DTM) or a DEM including whatever is above the ground (Digital Surface Model, DSM). In the first case, the orthophoto is correct only for the points that are on the ground and has parallax errors for buildings and vegetation. To distinguish between the two cases, when a DSM is used, it is called a *true orthophoto*.

Often the photoplane is confused with the *orthophoto*, but as we said, the photoplane is only correct for a particular plane, so points outside the plane are affected by parallax, while in the orthophoto the parallax is removed. Figure 18.12 allows to appreciate the differences between a photoplane (in a situation where it is clearly not applicable) and an orthophoto.

The generation of the orthophotos can also be done differently as illustrated. For example, one can produce a textured model in the form of a triangular mesh with texture and render it with orthographic projection through a rasterisation pipeline. Or, in order to avoid mesh construction, one can start from a dense coloured point cloud and render it with *splatting* (Westover 1991; Zwicker et al. 2001).

18.4 Geometric Image Transformation

Geometric image transformation is an operation that alters the spatial arrangement of pixels. It is also referred to in the literature as image warping. It shares problems and solutions with texture mapping in computer graphics.

To perform a geometric transformation of an image, we need the application that associates corresponding points between the source image and the target image. If (u, v) are the source coordinates and (s, t) are the target coordinates, one can be given the *direct transformation*: $(s, t) = T(u, v)$, or the *inverse transformation*: $(u, v) = T^{-1}(s, t)$.

The forward mapping uses the direct transformation and scans the source image, according to the following scheme:

```
for v = vmin:vmax
  for u = umin:umax
     (s,t) = T(u,v)
     copy pixel at source(u,v) to dest(s,t)
```

The geometric transformation of an image requires a resampling step, that is, the conversion of a digital (2D) signal from one sampling grid to another. If, by literally following the previous pseudo-code, one simply copies the pixels, the results will be unsatisfactory. In particular:

- if the transformation enlarges the image, the operation results in oversampling. If not done well, it results in the repetition of pixels;
- if the transformation shrinks the image, the operation results in undersampling. If it is not done well, it results in aliasing.

In addition, for non-invertible transformations, there are artefacts due to violation of injeictivity or surjectivity, namely (Fig. 18.13):

folding: occurs when two or pixels of the source image are associated with the same pixel of the target image;

holes: occurs when points not visible in the source image are visible in the target image;

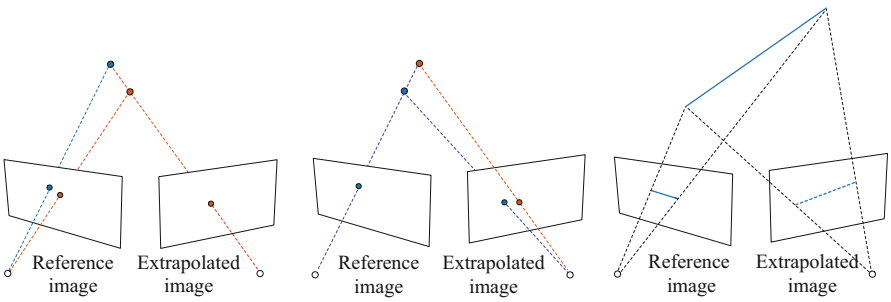


Fig. 18.13 Artefacts in image synthesis. From left to right: folding, holes and magnification

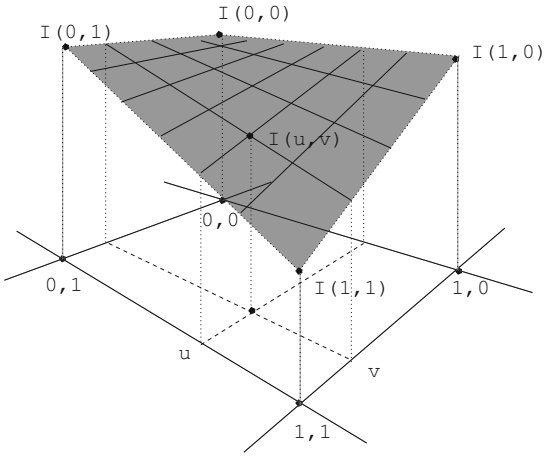


Fig. 18.14 Bilinear interpolation

Folding can be avoided by following an appropriate pixel evaluation order (McMillan and Bishop 1995), which ensures that the points closest to the camera are transformed last, so that they eventually overwrite the others. For example, if the synthetic camera is shifted to the left of the reference camera, the pixels should be processed from right to left.

Holes are difficult to fix because the information needed to fill them is missing. Interpolation or inpainting techniques can be used (Criminisi et al. 2004).

Magnification artefacts are resolved by *splatting*, that is, drawing a footprint of the transferred point that are larger than the pixel itself (Westover 1991; Zwicker et al. 2001).

When the inverse of the transformation is available (as in the case of homographies), one can apply the more effective technique of backward mapping. It proceeds by scanning the target image, according to the following scheme:

```
for y = ymin:ymax
  for x = xmin:xmax
      (u,v) = T^-1(s,t)
      copy pixel at source(u,v) to dest(s,t)
```

With backward mapping the problem of magnification/minification has an effective solution through interpolation.

Bilinear Interpolation

A good resampling technique that works for a scale factor of up to two can be achieved with bilinear interpolation. This is a continuous function that is inexpensive to compute and interpolates the data on a square grid.

Consider, for simplicity, the square of vertices $(0, 0)$, $(1, 0)$, $(0, 1)$ and $(1, 1)$ in the source image I (Fig. 18.14). The interpolated value at the point (u, v), included in the square, is calculated by

$$I(u, v) = (1-u)(1-v)I(0, 0) + u(1-v)I(1, 0) + (1-u)vI(0, 1) + uvI(1, 1).$$

The implementation of backward mapping, shown in Listing 18.2, is based on the `interp2` function of MATLAB and assumes a monochrome image. When working with RGB images, the same operation is performed on each of the three channels, independently.

Listing 18.2 Image transform with backward mapping

```
function [Iout, mask]  = imwarp(I,f,bb)
%IMWARP  Image warp
% Apply the projective transformation specified by the _inverse_ of f
% The bounding box is specified with [minx; miny; maxx; maxy];
% mask is 1 for valid pixels and 0 otherwise

[x,y] = meshgrid(bb(1):bb(3),bb(2):bb(4));
% pp = htx(inv(H),[x(:),y(:)]');
pp = f([x(:),y(:)]');

xi=reshape(pp(1,:),size(x,1),[]);
yi=reshape(pp(2,:),size(y,1),[]);
Iout=interp2(1:size(I,2),1:size(I,1),double(I),xi,yi,'linear',NaN);

mask = (~isnan(Iout));
Iout(~mask) = median(I(:));  % the background value is the median of the image

cast(Iout,class(I)); % cast I2 to whatever was I

% implements backward mapping, so f is the inverse of the function
% that is being applied to I
```

In bilinear interpolation, to save on multiplications, one can apply the following formulae that uses three multiplications instead of eight:

$$\begin{aligned} a &= I(0,0) + u(I(1,0) - I(0,0)) \\ b &= I(0,1) + u(I(1,1) - I(0,1)) \\ I(u,v) &= a + v(b-a). \end{aligned} \tag{18.13}$$

Problems

18.1 How to calculate the epipole given six homologous points, four of which are coplanar?

18.2 In an autonomous driving application (e.g. of a car), if one fixes the road plane as reference, all points for which the parallax is significantly different from zero are

immediately detected as obstacles. Expand this idea to build an obstacle avoidance algorithm.

18.3 In the game of soccer, the signed parallax of the ball with respect to the plane formed by the posts and the crossbar can detect the scoring. Expand this idea to build an automatic score detection algorithm.

References

M. Brown and D. G. Lowe. Automatic panoramic image stitching using invariant features. *International Journal of Computer Vision*, 74(1):59–73, 2007.

A. Censi, A. Fusiello, and V. Roberto. Image stabilization by features tracking. In *Proceedings of the 10th International Conference on Image Analysis and Processing*, pages 665–667, Venice, Italy, 27–29 September 1999.

Shenchang Eric Chen and Lance Williams. View interpolation for image synthesis. In James T. Kajiya, editor, *SIGGRAPH: International Conference on Computer Graphics and Interactive Techniques*, volume 27, pages 279–288, August 1993.

A. Criminisi, P. Perez, and K. Toyama. Region filling and object removal by exemplar-based image inpainting. *Image Processing, IEEE Transactions on*, 13(9):1200–1212, 2004.

James Davis. Mosaics of scenes with moving objects. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 354–360, 1998.

Qiang Du, Vance Faber, and Max Gunzburger. Centroidal voronoi tessellations: Applications and algorithms. *SIAM Review*, 41(4):637–676, 1999. https://doi.org/10.1137/S0036144599352836.

A. Fusiello. Specifying virtual cameras in uncalibrated view synthesis. *IEEE Transactions on Circuits and Systems for Video Technology*, 17(5):604–611, May 2007.

T. C. Hales. The honeycomb conjecture. *Discrete Comput. Geom.*, 25(1):1–22, jan 2001. ISSN 0179-5376. https://doi.org/10.1007/s004540010071.

D. Liebowitz and A. Zisserman. Metric rectification for perspective images of planes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 482–488, 1998.

L. McMillan and G. Bishop. Head-tracked stereo display using image warping. In *Stereoscopic Displays and Virtual Reality Systems II*, number 2409 in SPIE Proceedings, pages 21–30, San Jose, CA, 1995.

E. Santellani, E. Maset, and A. Fusiello. Seamless image mosaicking via synchronization. *ISPRS Annals of Photogrammetry, Remote Sensing and Spatial Information Sciences*, IV-2:247–254, 2018.

Steven M. Seitz and Charles R. Dyer. View morphing: Synthesizing 3D metamorphoses using image transforms. In Holly Rushmeier, editor, *SIGGRAPH: International Conference on Computer Graphics and Interactive Techniques*, pages 21–30, New Orleans, Louisiana, August 1996.

A. Shashua and N. Navab. Relative affine structure: Canonical model for 3D from 2D geometry and applications. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(9): 873–883, September 1996.

R. Szeliski. Video mosaics for virtual environments. *IEEE Computer Graphics and Applications*, 16(2):22–30, March 1996.

Richard Szeliski. Image alignment and stitching: a tutorial. *Foundations and Trends in Computer Graphics and Vision*, 2(1):1–104, 2006.

Lee Alan Westover. Splatting: A parallel, feed-forward volume rendering algorithm. Technical Report TR91-029, University of North Carolina at Chapel Hill, USA, 1991.

Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus H. Gross. Surface splatting. In *SIGGRAPH '01*, 2001.

Appendix A
Notions of Linear Algebra

«If you were to be stranded on a deserted island, make sure to bring SVD with you.» [Self-citation]

A.1 Introduction

This appendix reports a set of linear algebra results useful for understanding the methods and techniques outlined in the previous chapters. For a systematic treatment, see Strang (1988).

A.2 Scalar Product

Definition A.1 (Scalar Product) The scalar product of two vectors $\mathbf{x}$ and $\mathbf{y}$ of $\mathbb{R}^n$ is defined as

$$\langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i=1}^{n} x_i \, y_i . \tag{A.1}$$

The scalar product is commutative, that is, $\langle \mathbf{x}, \mathbf{y} \rangle = \langle \mathbf{y}, \mathbf{x} \rangle$. The scalar product induces the Euclidean norm:

$$\|\mathbf{x}\|_2 = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}. \tag{A.2}$$

Geometrically, the norm represents the *length* of the vector. If $\mathbf{y}$ is *unitary*, that is, $\|\mathbf{y}\|_2 = 1$, then $\langle \mathbf{x}, \mathbf{y} \rangle$ represents the length of the projection of $\mathbf{x}$ along the direction of $\mathbf{y}$. The angle ϑ between two vectors $\mathbf{x}$ and $\mathbf{y}$ is defined as

$$\vartheta = \arccos \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\|_2 \|\mathbf{y}\|_2}. \tag{A.3}$$

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

Definition A.2 (Orthogonality) Two vectors $\mathbf{x}$ and $\mathbf{y}$ of $\mathbb{R}^n$ are orthogonal if

$$\langle \mathbf{x}, \mathbf{y} \rangle = 0. \tag{A.4}$$

Considering the vectors as matrices formed by a single column, we can write the scalar product using the matrix product and transposition (superscript $^\mathsf{T}$):

$$\langle \mathbf{x}, \mathbf{y} \rangle = \mathbf{x}^\mathsf{T}\mathbf{y} = \mathbf{y}^\mathsf{T}\mathbf{x}. \tag{A.5}$$

A.3 Matrix Norm

The Euclidean vector norm naturally induces a Euclidean matrix norm in the following way (applies in general to any norm):

$$\|A\|_2 = \sup_{\|\mathbf{x}\|_2 \leq 1} \frac{\|A\mathbf{x}\|_2}{\|\mathbf{x}\|_2}. \tag{A.6}$$

We will see later a property that allows to compute this norm from the *eigenvalues*.

There is another way to generalise the Euclidean norm to matrices, which starts from a scalar product on matrices that generalises the one defined for vectors.

Define the following scalar product between two matrices $A = [a_{ij}]$ and $B = [b_{ij}]$:

$$\langle A, B \rangle = \mathrm{tr}(A^\mathsf{T}B) = \sum_{i,j} a_{ij} b_{ij} \tag{A.7}$$

which reduces to the scalar product between vectors previously defined in the case where the two matrices consist of only one column. The corresponding induced norm is called the *Frobenius* norm:

Definition A.3 (Frobenius Norm) Given a matrix A, its Frobenius norm is defined as:

$$\|A\|_F = \sqrt{\mathrm{tr}(A^\mathsf{T}A)} = \sqrt{\sum_{i,j} a_{i,j}^2}. \tag{A.8}$$

In the special case where the matrix is a vector, the two norms coincide:

$$\|\mathbf{x}\|_F = \sqrt{\sum_i x_i^2} = \|\mathbf{x}\|_2. \tag{A.9}$$

A.4 Inverse Matrix

Definition A.4 (Square Matrix) A matrix A is said to be square if it has the same number of rows and columns.

A square matrix in which all elements below (or above) the diagonal are null is said to be *triangular*.

A square matrix in which all elements other than the diagonal are null is said to be *diagonal*:

$$\text{diag}(a_1, \dots a_n) = \begin{bmatrix} a_1 & 0 & \dots & 0 \\ 0 & a_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \dots & a_n \end{bmatrix}. \tag{A.10}$$

A particular square matrix is the *identity*: $I = \text{diag}(1, \dots, 1)$. The identity matrix is the neutral element for the matrix product: $IA = AI = A$.

Definition A.5 (Inverse Matrix) Let A be a square matrix. If there exists a matrix B such that $AB = BA = I$, then B is called the inverse of A and is denoted by A^{-1}.

Properties:

1. The inverse, if it exists, is unique
2. $(A^{-1})^\mathsf{T} = (A^\mathsf{T})^{-1} = A^{-\mathsf{T}}$
3. $(AB)^{-1} = B^{-1}A^{-1}$.

Given a linear system of n equations in n unknowns $A\mathbf{x} = \mathbf{b}$, we can write $\mathbf{x} = A^{-1}\mathbf{b}$, but recall that to solve a linear system numerically, one should *not* compute the inverse (Strang 1988).

We will see ahead how the notion of inverse generalises to non-square matrices with the *Moore-Penrose* inverse.

A.5 Determinant

Definition A.6 (Determinant) Let A be a square matrix $n \times n$. The determinant of A is defined (recursively) as follows, for a fixed i:

$$\begin{cases} \det(A) = \sum_{j=1}^{n} a_{ij}(-1)^{i+j} \det(A_{ij}) \sum_{j=1}^{n} a_{ij}(-1)^{i+j} \\ \det(a) = a \end{cases} \tag{A.11}$$

where A_{ij} is the submatrix of A obtained by suppressing row i and column j.

This definition is also named *Laplace expansion* along row i. It could be defined analogously along a column. The choice of row (or column) is arbitrary.

For a 2×2 matrix:

$$\det\left(\begin{bmatrix} a_{1,1} & a_{1,2} \\ a_{2,1} & a_{2,2} \end{bmatrix}\right) = a_{1,1}a_{2,2} - a_{1,2}a_{2,1}. \tag{A.12}$$

Remark A.1 The determinant of a triangular matrix is the product of the diagonal elements.

Properties:

1. $\det(AB) = \det(A)\det(B)$
2. $\det(\alpha A) = \alpha^n \det(A)$ with $\alpha \in \mathbb{R}$
3. $\det(I) = 1$
4. $\det(A^{-1}) = \frac{1}{\det(A)}$
5. A is invertible $\iff$ $\det(A) \neq 0$
6. Swapping two rows (or columns) of A changes the sign of $\det(A)$.

Definition A.7 (Minor of Order q) Let A be a matrix $m \times n$. We call the minor of order q extracted from A the determinant of a $q \times q$ submatrix of A.

Definition A.8 (Adjoint Matrix) The adjoint matrix of a square matrix A is defined as

$$[\text{adj}(A)]_{ij} = (-1)^{i+j} \det(A_{ji}) \tag{A.13}$$

where A_{ji} is the submatrix of A obtained by suppressing row j and column i. The entries of $\text{adj}(A)$ are also called *cofactors* of A and are in fact minors of A, up to the sign.

The adjoint matrix is closely related to the inverse via the following relationship:

$$\text{adj}(A) = \det(A)A^{-1}. \tag{A.14}$$

Definition A.9 (Singular Matrix) A matrix A such that $\det(A) = 0$ is said to be singular.

By the Property 5, singular is synonymous with non-invertible.

The determinant is related to the notion of linear independence of a set of vectors.

Definition A.10 (Linear Independence) The vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$ of $\mathbb{R}^n$ are said to be linearly independent if

$$\sum_{i=1}^{m} \alpha_i \mathbf{x}_i = \mathbf{0} \implies \forall i : \alpha_i = 0. \tag{A.15}$$

If the vectors are not linearly independent, then (at least) one of them is a linear combination of the others.

Proposition A.1 *The vectors* $\mathbf{x}_1, \ldots, \mathbf{x}_n$ *of* $\mathbb{R}^n$ *are linearly independent if and only if*

$$\det([\mathbf{x}_1, \ldots, \mathbf{x}_n]) \neq 0. \tag{A.16}$$

A.6 Orthogonal Matrices

The vectors $\mathbf{x}_1, \ldots, \mathbf{x}_m$ of $\mathbb{R}^n$ are said to be *mutually orthogonal* if each is orthogonal to each other. If the vectors are all unitary, then they are said to be *orthonormal*.

Proposition A.2 *If the vectors* $\mathbf{x}_1, \ldots, \mathbf{x}_m$ *are mutually orthogonal, then they are linearly independent.*

Definition A.11 (Orthogonal Matrix) A square real matrix A is said to be orthogonal if

$$AA^\mathsf{T} = A^\mathsf{T}A = I. \tag{A.17}$$

So for an orthogonal matrix, the transpose coincides with the inverse. It also follows from the definition that if A is orthogonal, its columns (or rows) are mutually orthogonal unit vectors (i.e. orthonormal).

Remark A.2 A rectangular matrix B can satisfy $B^\mathsf{T}B = I$ or $BB^\mathsf{T} = I$ (but not both). It means that its columns or rows are orthonormal vectors. It is called in such a case *semi-orthogonal*.

Properties: if A is orthogonal, then:

1. $\det(A) = \pm 1$
2. $\|Ax\| = \|x\|$.

The second property means that transforming a vector with an orthogonal matrix does not alter its length. In fact, orthogonal matrices are the all and only linear rigid transformations. In particular, those with a positive determinant are a pure rotation, while if the determinant is negative, they contain a specular reflection.

Definition A.12 (Group) A group is an algebraic structure formed by a nonempty set endowed with an internal binary operation, satisfying the properties of associativity, existence of the neutral element and existence of the inverse of each element. The operation is *internal* if it always returns an element of the set itself.

Orthogonal matrices form a *group*. In particular, the group of orthogonal matrices in $\mathbb{R}^3$ is called the orthogonal group and is denoted by $O(3)$.

The rotation matrices also form a group, called the *special* orthogonal group and denoted (in $\mathbb{R}^3$) by $SO(3)$. Clearly, $SO(3) \subset O(3)$.

A.7 Linear and Quadratic Forms

Definition A.13 (Symmetric and Antisymmetric Matrix) A square matrix A is said to be symmetric if $A = A^\mathsf{T}$, that is, $a_{ij} = a_{ji}$. It is said to be antisymmetric (or skew-symmetric) if $A = -A^\mathsf{T}$.

Properties: let $\mathbf{a} \in \mathbb{R}^n$ and A $n \times n$:

1. $f(\mathbf{x}) = \mathbf{a}^\mathsf{T}\mathbf{x}$ is a *linear form* in $\mathbf{x}$
2. $f(\mathbf{x}) = \mathbf{x}^\mathsf{T}A\mathbf{x}$ is a *square form* in $\mathbf{x}$
3. $f(\mathbf{x}, \mathbf{y}) = \mathbf{x}^\mathsf{T}A\mathbf{y}$ is a *bilinear form* in $\mathbf{x}$ and $\mathbf{y}$.

Remark A.3 In a quadratic form, one can replace A with $\frac{A+A^\mathsf{T}}{2}$ (symmetric by construction) without changing the result, so it can always be assumed that the matrix A in a quadratic form is symmetric.

Definition A.14 (Positive Definite Matrix) A symmetric matrix A is said to be positive definite if $\forall x \neq 0 : \mathbf{x}^\mathsf{T}A\mathbf{x} > 0$. It is said to be positive semidefinite if $\forall x : \mathbf{x}^\mathsf{T}A\mathbf{x} \geq 0$.

Properties: let A $n \times n$ and B $n \times m$

1. BB^T and $B^\mathsf{T}B$ are positive semidefinite by construction
2. $\forall x : B\mathbf{x} = \mathbf{0} \iff B = 0$
3. $\forall x : \mathbf{x}^\mathsf{T}A\mathbf{x} = \mathbf{0} \iff A$ is antisymmetric
4. $\forall x : \mathbf{x}^\mathsf{T}A\mathbf{x} = \mathbf{0} \wedge A$ is symmetric. $\iff A = 0$.

Property 4 derives from the fact that the null matrix is the only one simultaneously symmetric and antisymmetric.

Theorem A.1 (Cholesky) *A positive semidefinite matrix A can be uniquely decomposed as*

$$A = K^\mathsf{T}K \tag{A.18}$$

where K is an upper triangular matrix with positive diagonal.

A.8 Rank

Definition A.15 (Rank of a Matrix) The rank of a $m \times n$ matrix A is the maximum order of the nonzero minors extracted from A, and is denoted by $\mathrm{r}(A)$.

Proposition A.3 *Let A be a $m \times n$ matrix. Its rank is equal to the number of linearly independent columns or rows it contains.*

Remark A.4 $\mathrm{r}(A) \leq \min(m, n)$. If $\mathrm{r}(A) = \min(m, n)$, it is said that A has *full rank*.

Properties:

1. A square has full rank $\iff \det(A) \neq 0$
2. $\mathrm{r}(A) = \mathrm{r}(A^\mathsf{T}) = \mathrm{r}(AA^\mathsf{T}) = \mathrm{r}(A^\mathsf{T}A)$
3. The rank does not change after swapping two rows (or columns)
4. $\mathrm{r}(AB) \leq \min(\mathrm{r}(A), \mathrm{r}(B))$.
5. If B has full rank equal to the number of rows: $\mathrm{r}(AB) = \mathrm{r}(A)$
6. If B has full rank equal to the number of columns: $\mathrm{r}(BA) = \mathrm{r}(A)$

Definition A.16 (Null Space of a Matrix) Let A be a $m \times n$ matrix. Its null space (or kernel), denoted by $\ker(A)$, is the subspace of $\mathbb{R}^n$ defined as follows:

$$\ker(A) = \{\mathbf{x} \in \mathbb{R}^n \mid A\mathbf{x} = \mathbf{0}\}. \tag{A.19}$$

The null space $\ker(A)$ is a subspace because:

- it always contains the null vector: $\forall A : \mathbf{0} \in \ker(A)$
- it is closed by linear combination: $\mathbf{x}, \mathbf{y} \in \ker(A) \implies \forall \alpha, \beta \in \mathbb{R} : \alpha\mathbf{x} + \beta\mathbf{y} \in \ker(A)$.

Definition A.17 (Image of a Matrix) Let A be a $m \times n$ matrix. Its image, denoted by $\mathrm{im}(A)$, is the subspace of $\mathbb{R}^m$ defined as follows:

$$\mathrm{im}(A) = \{\mathbf{y} \in \mathbb{R}^m \; \exists \mathbf{x} \in \mathbb{R}^n : A\mathbf{x} = \mathbf{y}\}. \tag{A.20}$$

Remark A.5 The image of A is the subspace generated by the columns of A, that is, the vectors of $\mathrm{im}(A)$ are linear combination of the columns of A. For this reason it is also called *column space* of A.

This is easily seen, thanks to the block matrix product:

$$A\mathbf{x} = \begin{bmatrix} \mathbf{a}_1, \ldots, \mathbf{a}_n \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = x_1\mathbf{a}_1 + \ldots + x_n\mathbf{a}_n \tag{A.21}$$

where by $\mathbf{a}_1, \ldots, \mathbf{a}_n$, we have denoted the columns of A.

Properties:

1. $\dim(\mathrm{im}(A)) = \mathrm{r}(A)$. It follows from the previous observation and from Prop. A.3
2. $\dim(A) = \mathrm{im}(AA^\mathsf{T})$.

Theorem A.2 (Rank Nullity) *Let A be a matrix $m \times n$, then*

$$n = \underbrace{\dim(\mathrm{im}(A))}_{\mathrm{r}(A)} + \underbrace{\dim(\ker(A))}_{\textit{nullity of } A}. \tag{A.22}$$

Corollary A.1 *A matrix A has full rank* $\iff$ $\ker(A) = \{\mathbf{0}\}$.

A.9 QR Decomposition

Theorem A.3 (QR Decomposition) *Let A $m \times n$, then there exists an $m \times m$ orthogonal matrix Q and a $m \times n$ upper triangular*[1] *matrix R such that*

$$A = QR \tag{A.23}$$

Note that, because Q is nonsingular, $\mathrm{r}(R) = \mathrm{r}(A)$.

Remark A.6 If $m > n$, the last $m - n$ rows of R are zero, and so we can write the *reduced QR decomposition*:

$$A = QR = \begin{bmatrix} Q_1 & Q_2 \end{bmatrix} \begin{bmatrix} R_1 \\ 0 \end{bmatrix} = Q_1 R_1 \tag{A.24}$$

where R_1 is $n \times n$ upper triangular and Q_1 is $m \times n$ semi-orthogonal, that is, $Q_1^{\mathsf{T}} Q_1 = I_n$.

Remark A.7 If $m \leq n$ and A has full rank, the QR decomposition is unique as soon as R is required to have positive diagonal elements.

A.10 Eigenvalues and Eigenvectors

Definition A.18 (Eigenvalues) Let A be an $n \times n$ matrix. The eigenvalues of A are the roots of the polynomial in λ (characteristic polynomial):

$$\det(\lambda I - A) = 0. \tag{A.25}$$

Let λ be an eigenvalue of A, then $\det(\lambda I - A) = 0 \iff \ker(\lambda I - A) \neq \{\mathbf{0}\}. \iff \exists \mathbf{x} \neq \mathbf{0} : (\lambda I - A)\mathbf{x} = \mathbf{0}$ or, equivalently

$$A\mathbf{x} = \lambda \mathbf{x}. \tag{A.26}$$

[1] in the sense that $r_{ij} = 0$ for $i > j$.

Such a vector $\mathbf{x}$ is called *eigenvector* of A.

Let $\lambda_1, \ldots, \lambda_n$ be the eigenvalues of the matrix A $n \times n$. Then:

$$\mathrm{tr}(A) = \sum_{i=1}^{n} \lambda_i \tag{A.27}$$

$$\det(A) = \prod_{i=1}^{n} \lambda_i . \tag{A.28}$$

Remark A.8 A singular matrix possesses at least one null eigenvalue. In fact $\det(A) = 0 \iff \det(\lambda I - A) = 0$ with $\lambda = 0$.

Proposition A.4 *A symmetric real matrix has real eigenvalues.*

Proposition A.5 *A symmetric matrix is defined to be positive* $\iff$ *all eigenvalues are positive.*

Proposition A.6 *An antisymmetric real matrix of dimension* $2n \times 2n$ *(even) has purely imaginary eigenvalues of the form* $\pm i\lambda_1, \ldots, \pm i\lambda_n$ *with* $\lambda_i \in \mathbb{R}$*. In the case of odd dimension, a single null eigenvalue appears.*

Proposition A.7 *An orthogonal matrix has eigenvalues of unit modulus.*

Theorem A.4 (Schur's Real Canonical Form) *If A is real, there exists an orthogonal real matrix V such that $V^{\mathsf{T}} A V = T$ is upper block triangular, with blocks of size 1 or 2 on the diagonal. The eigenvalues of A coincide with the eigenvalues of the diagonal blocks, where blocks of dimension 1 correspond to real eigenvalues and blocks of dimension 2 to conjugate pairs of complex eigenvalues.*

Observe that in general V *do not* contain the eigenvectors of A.

Two remarkable corollaries follow from this theorem, of which the first is the spectral theorem for symmetric real matrices.

Corollary A.2 (Schur's Diagonalisation) *Let A be a $n \times n$ real symmetric matrix with eigenvalues $\lambda_1, \ldots, \lambda_n$. Then, there exists a real diagonal matrix $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_n)$ and a real orthogonal matrix V $n \times n$ whose columns are the eigenvectors of A, such that:*

$$V^{\mathsf{T}} A V = \Lambda. \tag{A.29}$$

The proof is simple: since A is symmetric, the eigenvalues are all real, so T is upper triangular, but since it must also be symmetric, it can only be diagonal.

For antisymmetric matrices, instead, the following applies:

Corollary A.3 *Let A be a $2n \times 2n$ real antisymmetric matrix with eigenvalues $\pm i\lambda_1, \ldots, \pm i\lambda_n$. Then there exists a block diagonal matrix $S = \mathrm{diag}(\lambda_1 Z, \ldots, \lambda_n Z)$ with $Z = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ and an orthogonal matrix V $2n \times 2n$,*

such that:

$$V^{\mathsf{T}} A V = S. \tag{A.30}$$

Proof: since A is antisymmetric, T is upper block triangular, but since it must also be antisymmetric it can only have the structure of the statement.

Remark A.9 If the order of A is odd, then one eigenvalue of A is null and consequently the last element of the diagonal of S is zero.

Remark A.10 Antisymmetric matrices have even rank.

Proposition A.8 (Rayleigh Quotient) *Let A be a $n \times n$ real symmetric matrix with eigenvalues $\lambda_1, \ldots, \lambda_n$ ordered from largest to smallest. Then:*

$$\lambda_n \leq \frac{\mathbf{x}^{\mathsf{T}} A \mathbf{x}}{\mathbf{x}^{\mathsf{T}} \mathbf{x}} \leq \lambda_1. \tag{A.31}$$

This is easily proved by Schur's diagonalisation. From this it also follows that

$$\|A\|_2 = \sqrt{\lambda_1(A^{\mathsf{T}} A)}. \tag{A.32}$$

Proposition A.9 (Similar Matrices) *Let A be a $n \times n$ matrix and G be a $n \times n$ nonsingular matrix, then $G^{-1}AG$ has the same eigenvalues as A. A and $G^{-1}AG$ are said to be similar.*

A.11 Singular Value Decomposition

Theorem A.5 (Singular Values Decomposition) *Let A be a $m \times n$ matrix. There exists a $m \times n$ matrix D with positive diagonal elements $\sigma_1 \geq \sigma_2 \geq \ldots \sigma_n \geq 0$ and null elsewhere, an orthogonal $m \times m$ matrix U and an $n \times n$ orthogonal matrix V such that*

$$U^{\mathsf{T}} A V = D. \tag{A.33}$$

The diagonal elements of D are called singular values*. The columns of U are the left* singular vectors *; the columns of V are the right* singular vectors*.*

As we will make clearer later, the Singular Value Decomposition (SVD) is related to the spectral decomposition, and singular values are related to eigenvalues.

The following proposition justifies the importance of SVD.

Proposition A.10 (Null Space and Image of the Matrix) *Let A be a $m \times n$ matrix and let $U^{\mathsf{T}} A V = D$ be its singular value decomposition. The columns of U*

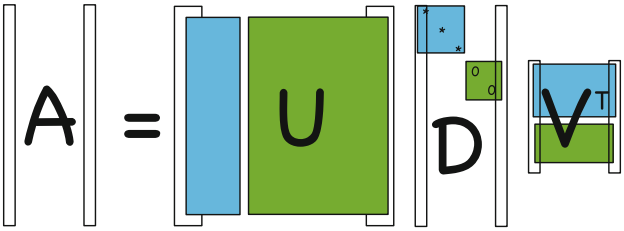


Fig. A.1 Graphical representation of the SVD, in which the partitions corresponding to the null and non-null singular values are highlighted

corresponding to nonzero singular values are a basis for $\mathrm{im}(A)$*, while the columns of* V *corresponding to null singular values are a basis for* $\ker(A)$.

Compare this result with Theorem A.2. Figure A.1 gives a graphical representation of the decomposition.

The previous proposition provides, via SVD, an algorithm for determining basis for $\ker(A)$ and $\mathrm{im}(A)$.

Remark A.11 The number of nonzero singular values is equal to the rank of the matrix.

Remark A.12 If A is a square matrix, $\det(A) \neq 0 \iff \sigma_1, \ldots, \sigma_n \neq 0$.

More specifically, the *condition number* $c = \frac{\sigma_1}{\sigma_n}$ indicates the extent to which the matrix is close to singularity. If σ_n is close to zero, then c is large and the matrix A is *ill-conditioned*, that is, it is singular to practical effects.

The following proposition specifies the relationship with eigenvalues.

Proposition A.11 . *Let* A *be a* $m \times n$ *matrix and let* $\sigma_1, \ldots, \sigma_r$ *be its nonzero singular values. Then* $\sigma_1^2, \ldots, \sigma_r^2$ *are the nonzero eigenvalues of* $A^\mathsf{T}A$ *and* AA^T*. The columns of* U *are the eigenvectors of* AA^T*; the columns of* V *are the eigenvectors of* $A^\mathsf{T}A$.

Proof $A^\mathsf{T}A = VD^\mathsf{T}UDV^\mathsf{T} = V(D^\mathsf{T}D)V^\mathsf{T}$. This is the Schur diagonalisation of $A^\mathsf{T}A$ (it is symmetric by construction, so it satisfies the assumptions of Theorem A.2.) So the diagonal matrix $(D^\mathsf{T}D)$ contains the eigenvalues of $A^\mathsf{T}A$, and the columns of V are its eigenvectors.

Remark A.13 $\|A\|_F = \sqrt{\sum_i \sigma_i^2}$ e $\|A\|_2 = \sigma_1$

Proposition A.12 (Compact SVD) *Let* A *be a* $m \times n$ *matrix and let* $U^\mathsf{T}AV = D$ *be its singular value decomposition. Let* $\sigma_1, \ldots, \sigma_r$ *be its nonzero singular values. Then:*

$$A = U_r D_r V_r^\mathsf{T} \tag{A.34}$$

where $D_r = \mathrm{diag}(\sigma_1, \ldots, \sigma_r)$, U_r is the $m \times r$ matrix composed of the first r columns of U and V_r is the $n \times r$ matrix composed of the first r columns of V.

The following theorem exploits SVD to determine the lower-rank approximation of a matrix.

Theorem A.6 (Eckart-Young) *Let A be a matrix $m \times n$ of rank r and let $A = U_r D_r V_r^\mathsf{T}$ be its compact SVD. The matrix of rank $k < r$ closest to A in Frobenius norm is the matrix A_k defined as*

$$A_k = U_r D_k V_r^\mathsf{T} \tag{A.35}$$

where $D_k = \mathrm{diag}(\sigma_1, \ldots, \sigma_k, \underbrace{0, \ldots, 0}_{r-k})$. Furthermore, $\|A - A_k\|_F = \sigma_{k+1}$.

Proof *(Sketch)* We can rewrite: $A = U_r D_r V_r^\mathsf{T} = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$ where $\mathbf{u}_i$ and $\mathbf{v}_i$ are the columns of U and V, respectively. Each of the terms of the summation is a matrix of rank one, so we write A –which has rank r– as a weighted sum of r matrices of rank one, with weights equal to the corresponding singular values. If a lower-rank approximation is sought, some terms must be cancelled, and the cost to pay, in terms of Frobenius' norm (by remark A.13), is the corresponding singular value. So the least-cost solution is to delete the terms starting from the end: $A_k = \sum_{i=1}^{k} \sigma_i \mathbf{u}_i \mathbf{v}_i^\mathsf{T}$.

A primary use for the SVD in this text is the solution of homogeneous systems of linear equations. Consider the system $A\mathbf{x} = \mathbf{0}$. Two cases arise:

- $\mathrm{r}(A)$ is full, so $\ker(A) = \{\mathbf{0}\}$, that is, the system has only the trivial solution $\mathbf{x} = \mathbf{0}$
- $\mathrm{r}(A)$ is not full, so $\dim(\ker(A)) > 0$, that is, the system has non-trivial solutions. Note that if $\mathbf{x}$ is a solution, so is $\alpha\mathbf{x}\ \forall \alpha \in \mathbb{R}$.

In practice the case often arises where A has full rank, so strictly speaking there are no meaningful solutions, but this is actually due to measurement errors or noise. In such cases we then look for a non-trivial *approximate* solution, that is, we solve the *least-squares* problem:

$$\min_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|^2 \tag{A.36}$$

The constraint is placed to avoid the solution $\mathbf{x} = \mathbf{0}$ and the value 1 is completely arbitrary.

Proposition A.13 (Least-Squares Solution to a Linear System of Homogeneous Equations) *Let A be a $m \times n$ matrix. The solution of the least-squares problem*

$$\min_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|^2 \tag{A.37}$$

is $\mathbf{x} = \mathbf{v}_n$ where $\mathbf{v}_n$ is the last column of V in the singular value decomposition of A: $U^\mathsf{T} A V = D$.

Proof $\min_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|^2 = \min_{\|\mathbf{x}\|=1} \|UDV^\mathsf{T}\mathbf{x}\|^2 = \min_{\|\mathbf{x}\|=1} \|DV^\mathsf{T}\mathbf{x}\|^2$. After making the variable change $\mathbf{y} = V^\mathsf{T}\mathbf{x}$ (recall that V is orthogonal), we get:

$$\min_{\|\mathbf{x}\|=1} \|A\mathbf{x}\|^2 = \min_{\|\mathbf{y}\|=1} \|D\mathbf{y}\|^2 = \min_{\|\mathbf{y}\|=1} \sum (\sigma_i y_i)^2. \tag{A.38}$$

The constraint prevents us from choosing $\mathbf{y} = \mathbf{0}$, and since the singular values in D are ordered, the least-cost solution is $\mathbf{y} = [0, \ldots, 0, 1]^\mathsf{T}$, so $\mathbf{x} = \mathbf{v}_n$.

We will refer to $\mathbf{v}_n$ as the *least right singular vector* of A.

Proposition A.14 (Orthogonal Procrustian Problem) *Given two matrices A and B, the solution of the problem*

$$\min_{Q^\mathsf{T}Q=I} \|A - QB\|_F^2 \tag{A.39}$$

is $Q = UV^\mathsf{T}$ where UDV^T is the SVD of AB^T: $AB^\mathsf{T} = UDV^\mathsf{T}$.

Proof $\|A - QB\|_F^2 = \mathrm{tr}(A^\mathsf{T}A - 2AB^\mathsf{T}Q^\mathsf{T} + BB^\mathsf{T})$; therefore, the problem becomes to maximise $\mathrm{tr}(AB^\mathsf{T}Q^\mathsf{T})$. Let $AB^\mathsf{T} = UDV^\mathsf{T}$, then

$$\mathrm{tr}(AB^\mathsf{T}Q^\mathsf{T}) = \mathrm{tr}(UDV^\mathsf{T}Q^\mathsf{T}) = \mathrm{tr}(DV^\mathsf{T}Q^\mathsf{T}U) = \mathrm{tr}(DZ) = \sum \sigma_i Z_{ii} \tag{A.40}$$

Since Z is orthogonal, we have $|Z_{ii}| \leq 1$ and therefore the maximum is reached with $Z = V^\mathsf{T}Q^\mathsf{T}U = I$, hence the thesis.

In the case where the constraint is added that Q is a rotation, that is, $\det(Q) = 1$, the solution is modified as follows:

$$Q = U \begin{bmatrix} 1 & 0 & & 0 \\ 0 & 1 & & 0 \\ & & \ddots & \\ 0 & 0 & & \det(VU^\mathsf{T}) \end{bmatrix} V^\mathsf{T}. \tag{A.41}$$

Indeed, $\det(Z) = \det(V^\mathsf{T}U)\det(Q) = \det(V^\mathsf{T}U)$. So, if $\det(V^\mathsf{T}U) = 1$, we take $Z = I$ as before, while if $\det(V^\mathsf{T}U) = -1$, we are forced to take a Z with determinant -1. Since the singular values σ_i are ordered, the least-cost choice is to take $Z = \mathrm{diag}(1, 1, 1, \ldots, -1)$.

A special case of this problem occurs when $B = I$. In that case it is a matter of finding the closest orthogonal matrix Q to the given matrix A, that is:

$$\min_{Q^\mathsf{T}Q=I} \|A - Q\|_F^2. \tag{A.42}$$

The solution reduces to replacing the matrix D with the identity in the SVD of A. Similarly we proceed in the case of a rotation matrix.

On the Procrustean problem, see also Goodall (1991).

A.12 Pseudoinverse

The pseudoinverse, also known as the Moore-Penrose inverse, is a generalisation of the inverse from square matrices to non-square matrices. It is defined as a matrix that satisfies certain properties resembling those of the inverse of a non-square matrix. Let $A \in \mathbb{R}^{m \times n}$ be a matrix. The Moore-Penrose inverse of A, denoted by A^+, satisfies the following properties:

- $AA^+A = A$
- $A^+AA^+ = A^+$
- $(AA^+)^\mathsf{T} = AA^+$
- $(A^+A)^\mathsf{T} = A^+A$

It can can be shown that for each A, the pseudoinverse exists and is unique. Some properties derived from the defining equations:

- $A^+ = A^{-1}$ for nonsingular A
- $A^+ = (A^\mathsf{T}A)^+A^\mathsf{T} = A^\mathsf{T}(AA^\mathsf{T})^+$
- $A^+ = (A^\mathsf{T}A)^{-1}A^\mathsf{T}$ if A has full column rank
- $A^+ = A^\mathsf{T}(AA^\mathsf{T})^{-1}$ if A has full row rank

Please note that the last two properties follow from the first two. They imply that

- if A has full column rank, then A^+ is a left inverse of A, that is, $A^+A = I$;
- if A has full row rank, then A^+ is a right inverse of A, that is, $AA^+ = I$.

Proposition A.15 (Least-Squares Solution to a Linear System of Equations) *Let A be a matrix $m \times n$ and $b \in \mathbb{R}^m$. If $\hat{\mathbf{x}}$ is a solution of the system of equations (called* normal equations*):*

$$A^\mathsf{T}A\mathbf{x} = A^\mathsf{T}\mathbf{b} \tag{A.43}$$

then $\hat{\mathbf{x}}$ also solves the least-squares problem

$$\min \|A\mathbf{x} - b\|^2. \tag{A.44}$$

Normal equations always have solutions. If A has full column rank, then $(A^\mathsf{T}A)$ is non-singular and therefore the unique solution writes[2]

$$\mathbf{x} = (A^\mathsf{T}A)^{-1}A^\mathsf{T}\mathbf{b} \tag{A.45}$$

Otherwise, there are many solutions, and one is arbitrarily chosen as the representative.

[2] Nobody would *really* invert $(A^\mathsf{T}A)$, though. MATLAB, for example, uses QR decomposition.

The pseudoinverse can be used to find a least-squares solution to a linear system of equations with

$$\mathbf{x} = A^{+}\mathbf{b}. \tag{A.46}$$

When $\mathrm{r}(A) = n$, the unique solution is exactly (A.45); otherwise, it is the one with minimum Euclidean norm.

In general, the pseudoinverse of A can be calculated using the SVD as

$$A^{+} = V D^{+} U^{\mathsf{T}} \tag{A.47}$$

where D^{+} is the diagonal matrix with the reciprocals of the nonzero singular values on the diagonal, and zeroes elsewhere.

Ultimately, also the least-squares solution of inhomogeneous systems reduces to the SVD.

A.13 Cross Product

This is a product between two vectors that returns a vector, and is defined only in $\mathbb{R}^3$.

Definition A.19 (Cross Product) The cross product of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$ is defined as the vector:

$$\mathbf{a} \times \mathbf{b} = \begin{bmatrix} \det \begin{bmatrix} a_2 & a_3 \\ b_2 & b_3 \end{bmatrix} \\ -\det \begin{bmatrix} a_1 & a_3 \\ b_1 & b_3 \end{bmatrix} \\ \det \begin{bmatrix} a_1 & a_2 \\ b_1 & b_2 \end{bmatrix} \end{bmatrix}. \tag{A.48}$$

The cross product is used (among other things) to check whether two vectors differ only by a multiplicative constant.

Remark A.14 Given two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^3$, $\mathbf{a}, \mathbf{b}, \neq \mathbf{0}$, we have: $\mathbf{a} \times \mathbf{b} = \mathbf{0} \iff \mathbf{a} = \lambda \mathbf{b}, \lambda \in \mathbb{R}$.

In fact, the hypothesis is equivalent to stating that all determinants of order two extracted from $[\mathbf{a}, \mathbf{b}]$ are null, so it has rank one, from which the thesis follows, and vice versa.

The cross product is naturally associated with an antisymmetric matrix:

Remark A.15 Given a vector $\mathbf{a} \in \mathbb{R}^3$, the matrix

$$[\mathbf{a}]_\times = \begin{bmatrix} 0 & -a_3 & a_2 \\ a_3 & 0 & -a_1 \\ -a_2 & a_1 & 0 \end{bmatrix} \tag{A.49}$$

acts as the cross product for $\mathbf{a}$, that is, $[\mathbf{b}]_\times \mathbf{b} = \mathbf{a} \times \mathbf{b}$.

The matrix $[\mathbf{a}]_\times$ is antisymmetric, and $\ker([\mathbf{a}]_\times) = \mathbf{a}$, since $[\mathbf{a}]_\times \mathbf{a} = \mathbf{a} \times \mathbf{a} = \mathbf{0}$. It is therefore singular, and in particular it has rank two, since the rank must be even (Remark A.10).

Remark A.16 Given a vector $\mathbf{a} \in \mathbb{R}^3$

$$[\mathbf{a}]_\times^2 = \mathbf{a}\mathbf{a}^\mathsf{T} - \mathbf{a}^\mathsf{T}\mathbf{a} I_3 \quad \text{and} \quad \operatorname{tr}([\mathbf{a}]_\times^2) = -2\|\mathbf{a}\|^2. \tag{A.50}$$

Proposition A.16 (Triple Product) *Given three vectors* $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$*, we have:*

$$\det([\mathbf{a}, \mathbf{b}, \mathbf{c}]) = \mathbf{a}^\mathsf{T}(\mathbf{b} \times \mathbf{c}) = (\mathbf{a} \times \mathbf{b})^\mathsf{T}\mathbf{c}. \tag{A.51}$$

This quantity also takes the name of the triple product of $\mathbf{a}, \mathbf{b}, \mathbf{c}$*.*

Demonstration follows immediately using the Laplace expansion and the definition of the cross product.

The sign of the triple product depends on the order of the arguments, but is invariant for a cyclic permutation.

The absolute value of the triple product is equal to the volume of the parallelepiped constructed on the three vectors. Consequently, assuming that none of the three vectors is zero, the triple product is zero if and only if the vectors are coplanar. In other words, since the triple product is a determinant, it can be used to check if three vectors are linearly dependent.

Proposition A.17 (Lagrange's Formula) *Given four vectors* $\mathbf{a}, \mathbf{b}, \mathbf{c}, \mathbf{d} \in \mathbb{R}^3$*, we have*

$$(\mathbf{a} \times \mathbf{b})^\mathsf{T}(\mathbf{c} \times \mathbf{d}) = (\mathbf{c}^\mathsf{T}\mathbf{a})(\mathbf{b}^\mathsf{T}\mathbf{d}) - (\mathbf{d}^\mathsf{T}\mathbf{a})(\mathbf{b}^\mathsf{T}\mathbf{c}). \tag{A.52}$$

Proposition A.18 *Given three vectors* $\mathbf{a}, \mathbf{b}, \mathbf{c} \in \mathbb{R}^3$ *such that* $\mathbf{a}^\mathsf{T}(\mathbf{b} \times \mathbf{c}) = 0$ *the two scalars* μ, λ *such that*

$$\mathbf{c} = \mu\mathbf{a} - \lambda\mathbf{b} \tag{A.53}$$

are given by

$$\mu = \frac{(\mathbf{c} \times \mathbf{b})^\mathsf{T}(\mathbf{a} \times \mathbf{b})}{\|\mathbf{a} \times \mathbf{b}\|^2}, \tag{A.54}$$

and likewise for λ*.*

Proof The hypothesis $\mathbf{a}^\mathsf{T}(\mathbf{b} \times \mathbf{c}) = 0$ guarantees the existence of the solution, since one of the vectors is linear combination of the other two. Regarding the formula (A.54), we have: $\mathbf{c} \times \mathbf{b} = \mu\mathbf{a} \times \mathbf{b} - \lambda\mathbf{b} \times \mathbf{b} = \mu\mathbf{a} \times \mathbf{b} \implies (\mathbf{c} \times \mathbf{b})^\mathsf{T}(\mathbf{a} \times \mathbf{b}) = \mu(\mathbf{a} \times \mathbf{b})^\mathsf{T}(\mathbf{a} \times \mathbf{b}) \implies$ thesis.

It is shown (Kanatani 1993) that the formula (A.54) also solves the least-squares problem:

$$\min \|\mu\mathbf{a} - \lambda\mathbf{b} - \mathbf{c}\|^2 \tag{A.55}$$

when $\mathbf{a}^\mathsf{T}(\mathbf{b} \times \mathbf{c}) \neq 0$.

A.14 Kronecker's Product

Definition A.20 (Kronecker's Product) Let A be a $m \times n$ matrix and B a $p \times q$ matrix. Kronecker's product[3] of A and B is the matrix $mp \times nq$ defined by

$$A \otimes B = \begin{bmatrix} a_{11}B & \dots & a_{1n}B \\ \vdots & & \vdots \\ a_{m1}B & \dots & a_{mn}B \end{bmatrix}. \tag{A.56}$$

Note that Kronecker's product is defined for any pair of matrices. Properties:

1. $(A \otimes B) \otimes C = A \otimes (B \otimes C)$
2. $(A \otimes B)(C \otimes D) = AC \otimes BD$ if the dimensions are compatible
3. $A \otimes B \neq B \otimes A$
4. $(A \otimes B)^\mathsf{T} = A^\mathsf{T} \otimes B^\mathsf{T}$
5. $(A \otimes B)^{-1} = A^{-1} \otimes B^{-1}$.

Proposition A.19 *The eigenvalues of $A \otimes B$ are Kronecker's product of the eigenvalues of A by the eigenvalues of B.*

Proof Let $A = SLS^{-1}$ and $B = TMT^{-1}$ be the Schur decompositions of A and B, respectively, where L and M are upper triangular matrices whose diagonal elements are the eigenvalues of A and B, respectively:

$$A \otimes B = (SLS^{-1}) \otimes (TMT^{-1}) = (S \otimes T)(L \otimes M)(S^{-1} \otimes T^{-1}). \tag{A.57}$$

Since $A \otimes B$ and $L \otimes M$ are similar, they have the same eigenvalues, and since $L \otimes M$ is upper triangular, the eigenvalues of $A \otimes B$ are the diagonal elements of $L \otimes M$.

[3] Implemented by the function `kron` in MATLAB.

It follows immediately from the proposition that:

Corollary A.4 (Kronecker's Product Rank)

$$\mathrm{r}(A \otimes B) = \mathrm{r}(A)\,\mathrm{r}(B). \tag{A.58}$$

The *vectorisation* of a matrix is a linear transformation that converts the matrix to a vector (column).

Definition A.21 (Vectorisation) The vectorisation of a $m \times n$ matrix A, denoted by $\mathrm{vec}\,A$, is the vector $mn \times 1$ that is obtained by stacking the columns of A one below the other.[4]

The connection between the Kronecker product and vectorisation is given by the following relation, also known as the *vec-trick*:

$$\mathrm{vec}(AXB) = (B^{\mathsf{T}} \otimes A)\,\mathrm{vec}\,X \tag{A.59}$$

for matrices A, B, X of compatible dimensions. This will be very useful for extracting the unknown X from a matrix equation.

When dealing with symmetric matrices, the vectorisation must consider only the non-repeated elements. For this purpose we define the following.

Definition A.22 (Half-Vectorisation) The *half-vectorisation* of a symmetric $n \times n$ matrix – denoted by $\mathrm{vech}\,A$ – is the vector $n(n+1)/2 \times 1$ obtained by vectorising only the lower triangular part of A.

To go from $\mathrm{vech}\,A$ to $\mathrm{vec}\,A$, there exists an appropriate $n^2 \times n(n+1)/2$ matrix D_n, called *duplication matrix*, such that $D_n\,\mathrm{vech}\,A = \mathrm{vec}\,A$.

In this way we can therefore write:

$$\mathrm{vec}(AXB) = (B^{\mathsf{T}} \otimes A) D_n\,\mathrm{vech}\,X \tag{A.60}$$

Similarly, we can treat the case where X is diagonal. Let us define $\mathrm{diag}^{-1}(\cdot)$ as the operator that consumes a square matrix and returns the vector containing the diagonal of its argument, while the operator $\mathrm{diag}(\cdot)$ consumes a vector and constructs a diagonal matrix.[5] We can therefore write:

$$\mathrm{vec}(AXB) = (B^{\mathsf{T}} \otimes A) D_n\,\mathrm{diag}^{-1}(X) \tag{A.61}$$

for an suitable $n^2 \times n$ matrix D_n (different from the previous one) such that $\mathrm{vec}\,X = D_n\,\mathrm{diag}^{-1}(X)$.

[4] In MATLAB we obtain $\mathrm{vec}\,\mathbf{x}$ with `x(:)`.

[5] In MATLAB both operations are performed by the overloaded function `diag`.

However, for computational efficiency reasons, it is preferable to avoid forming the matrix $(B^{\mathsf{T}} \otimes A)$ which has n^2 columns, when the unknowns are only n.

Proposition A.20 *Let A be an $m \times n$ matrix, B be an $n \times q$ matrix and X be an $n \times n$ diagonal matrix. Then, the following relation holds:*

$$\mathrm{vec}(AXB) = \left((B^{\mathsf{T}} \otimes \mathbf{1}_m) \circ (\mathbf{1}_q \otimes A)\right) \mathrm{diag}^{-1}(X) \tag{A.62}$$

where $\circ$ denotes the Hadamard product (element by element).

With $B = I_n$, we obtain:

$$\mathrm{vec}(AX) = ((I_n \otimes \mathbf{1}_m) \circ (\mathbf{1}_n \otimes A))\, \mathrm{diag}^{-1}(X) \tag{A.63}$$

It is easy to see that $((I_n \otimes \mathbf{1}_m) \circ (\mathbf{1}_n \otimes A))$ is nothing else but the diagonal block matrix which has the columns of A as blocks. This construction is implemented in the CV Toolkit by the MATLAB function `diagc`.

A.15 Rotations

The rotations in $\mathbb{R}^3$ are represented by 3×3 orthogonal matrices with positive determinant and form the special orthogonal group denoted by $SO(3)$.

According to a well-known Euler theorem, every rotation in $\mathbb{R}^3$ can be written as a rotation of some angle ϑ around an axis passing through the origin, identified by a versor $\mathbf{u}$.

The corresponding rotation matrix $R(\vartheta, \mathbf{u})$ that performs a rotation of ϑ around the axis identified by the versor $\mathbf{u}$ can be obtained from ϑ and $\mathbf{u}$ using Rodriguez's formula:

$$R(\vartheta, \mathbf{u}) = I_3 + \sin\vartheta\, [\mathbf{u}]_\times + (1 - \cos\vartheta)\, [\mathbf{u}]_\times^2 \tag{A.64}$$

where $[\mathbf{u}]_\times$ is the antisymmetric matrix associated with $\mathbf{u}$.

Note that it only takes three numbers to encode a rotation, since $\mathbf{u}$ is unitary. In fact, we can use the vector:

$$\mathbf{r} = \vartheta \mathbf{u}. \tag{A.65}$$

Euler also proved that every rotation in $\mathbb{R}^3$ around an axis passing through the origin can be considered as a sequence of three rotations around the Cartesian axes.

Let ω, φ, κ be the rotation angles along the Cartesian axes x, y and z, respectively, identified by the three unit vectors $\mathbf{e}_1, \mathbf{e}_2, \mathbf{e}_3$. Based on Rodriguez's formula, the three rotation matrices are

$$R(\omega, \mathbf{e}_1) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\omega & -\sin\omega \\ 0 & \sin\omega & \cos\omega \end{bmatrix} \tag{A.66}$$

$$R(\varphi, \mathbf{e}_2) = \begin{bmatrix} \cos\varphi & 0 & \sin\varphi \\ 0 & 1 & 0 \\ -\sin\varphi & 0 & \cos\varphi \end{bmatrix} \tag{A.67}$$

$$R(\kappa, \mathbf{e}_3) = \begin{bmatrix} \cos\kappa & -\sin\kappa & 0 \\ \sin\kappa & \cos\kappa & 0 \\ 0 & 0 & 1 \end{bmatrix}. \tag{A.68}$$

By convention we define the rotation matrix corresponding to the triplet of angles $(\omega, \varphi, \kappa)$ as follows:

$$R(\omega, \varphi, \kappa) = R(\kappa, \mathbf{e}_3) R(\varphi, \mathbf{e}_2) R(\omega, \mathbf{e}_1). \tag{A.69}$$

Beware that the order of composition of the matrices matters (since they do not commute). This representation of the rotation matrices is called the Euler system.[6] Like the representation with axis and angle, it has three degrees of freedom.

Euler system has singularities that go by the name of gimbal lock, at which the degrees of freedom are reduced. In particular, if $\varphi = \pm\pi/2$, the roles of ω and κ become interchangeable:

$$R(\kappa, \mathbf{e}_3) R(-\pi/2, \mathbf{e}_2) R(\omega, \mathbf{e}_1) = R(\omega, \mathbf{e}_3) R(-\pi/2, \mathbf{e}_2) R(\kappa, \mathbf{e}_1) \tag{A.70}$$

that is, there is only one degree of freedom.

See the MATLAB functions of the CV Toolkit `eul`, `ieul`, `rod` and `irod`.

A.16 Matrices Associated with Graphs

Let $G = (V, E)$ be a simple undirected graph with n nodes and m edges. The *adjacency matrix* of G is defined as the $n \times n$ matrix $A(G)$ where

$$[A(G)]_{ij} = \begin{cases} 1, & \text{if node } i \text{ and node } j \text{ are adjacent} \\ 0, & \text{otherwise} \end{cases}. \tag{A.71}$$

The *incidence matrix* of the simple directed graph $G = (V, E)$ with n nodes and m edges is defined as

[6] Other authors may follow different conventions. In fact, there are 12 different Euler angle conventions.

$$[B(G)]_{ij} = \begin{cases} 1, & \text{if } i \text{ is the head of edge } e_j \\ -1, & \text{if } i \text{ is the tail of edge } e_j \\ 0, & \text{otherwise} \end{cases} . \tag{A.72}$$

The rows of the incidence matrix correspond to the vertices of G and its columns to the edges of G.

A *spanning tree* of a connected graph is a tree that contains all the vertices of the graph and only the edges necessary to connect any pair of vertices with one and only one path. Removing any edge from the spanning tree disconnects it.

Remark A.17 Let G be a directed and connected graph. A square submatrix $n-1 \times n-1$ of $B(G)$ is non-singular if and only if its columns correspond to the set of edges of a spanning tree of G.

It follows that G is connected if and only if $\mathrm{r}(B) = n - 1$.

The *degree matrix* of a graph G is defined as

$$[D(G)]_{ij} = \begin{cases} \deg(v_i) = \sum_j [A(G)]_{ij}, & \text{if } i = j \\ 0, & \text{otherwise} \end{cases} \tag{A.73}$$

or, equivalently: $D = \mathrm{diag}(A\mathbf{1})$.

Regardless of the orientation assigned to the edges of G, the following relation holds:

$$D - A = BB^{\mathsf{T}}. \tag{A.74}$$

References

Colin Goodall. Procrustes Methods in the Statistical Analysis of Shape. *Journal of the Royal Statistical Society. Series B (Methodological)*, 53 285–339, 1991.

K. Kanatani. *Geometric Computation for Machine Vision*. Oxford University Press, Oxford, 1993.

Gilbert Strang. *Linear Algebra and Its Applications*. Harcourt Brace & Co., San Diego, CA, 1988.

Appendix B
Matrix Differential Calculation

B.1 Derivatives of Vector and Matrix Functions

The derivatives of functions involving vectors and matrices ultimately lead back to the partial derivatives of the individual components, and it is all about how to arrange these partial derivatives. There are several conventions, but the diriment feature for our purposes is to allow the calculation of derivatives through formal manipulations of matrix formulae.

In this book we will follow Magnus and Neudecker (1999) notation, and in this chapter we give a brief summary of it.

Definition B.1 Let $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$ a differentiable function. Its derivative is defined as follows:

$$[\mathsf{D}\,\mathbf{f}(x)]_{ij} = \frac{\partial f_i}{\partial x_j} \tag{B.1}$$

or

$$\mathsf{D}\,\mathbf{f}(\mathbf{x}) = \frac{\partial f}{\partial \mathbf{x}^\mathsf{T}}. \tag{B.2}$$

By this definition, $\mathsf{D}\,\mathbf{f}(\mathbf{x})$ coincides with the usual *Jacobian matrix* of $\mathbf{f}$.

Example B.1 The derivative of the vector function $\mathbf{f}(\mathbf{x}) = A\mathbf{x}$ is $\mathsf{D}\,\mathbf{f}(\mathbf{x}) = A$.

As special cases of the Definition B.1, we have:

1. $\mathbf{f}(x) : \mathbb{R} \to \mathbb{R}^m$, whose derivative $\mathsf{D}\,\mathbf{f}(x)$ is a column vector

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

2. $f(\mathbf{x}) : \mathbb{R}^n \to \mathbb{R}$, whose derivative $\mathsf{D}\, f(\mathbf{x})$ is a row vector,[1] which coincides with the gradient of f, also denoted as ∇f.

Example B.2 Consider the scalar vector function $f(\mathbf{x}) = \mathbf{x}^\mathsf{T} A\mathbf{x}$. Its derivative is $\mathsf{D}\,\mathbf{f}(\mathbf{x}) = \mathbf{x}^\mathsf{T}(A + A^\mathsf{T})$.

Definition B.1 is widely adopted, and it is a good definition, since it leads, e.g. to the possibility of applying the chain rule. However, when matrices come into play, things get complicated, as it is not obvious how to extend it to functions $F : \mathbb{R}^{n\times q} \to \mathbb{R}^{m\times p}$. In this case there are many ways of arranging the $mnpd$ partial derivatives; the notation of Magnus and Neudecker (1999) has the advantage of generalising the previous one if the matrices become vectors and, more importantly, of allowing the application of the chain rule.

Definition B.2 Let F be a differentiable function $F : \mathbb{R}^{n\times q} \to \mathbb{R}^{m\times p}$. The derivative of F in X is the (Jacobian) matrix $mp \times nq$:

$$\mathsf{D}\, F(X) = \frac{\partial \operatorname{vec} F(X)}{\partial (\operatorname{vec} X)^\mathsf{T}}. \tag{B.3}$$

Warning According to this notation, the derivatives of scalar functions of matrices and matrix functions of a scalar are vectors. Instead, the expressions $\frac{\partial F(x)}{\partial x}$ and $\frac{\partial f(X)}{\partial X}$ found elsewhere are matrices of the same dimension as $F(x)$ and X, respectively, whose entries are the same partial derivatives appearing $\mathsf{D}\, F(x)$ and $\mathsf{D}\, f(X)$ but differently arranged:

1. $F : \mathbb{R} \to \mathbb{R}^{m\times p} \quad \mathsf{D}\, F(x) = \operatorname{vec} \frac{\partial F(x)}{\partial x}$
2. $f : \mathbb{R}^{n\times q} \to \mathbb{R} \quad \mathsf{D}\, f(X) = \operatorname{vec} \left(\frac{\partial f(X)}{\partial X}\right)^\mathsf{T}$.

Example B.3 Consider the matrix scalar function $f(X) = \operatorname{tr}(X^\mathsf{T} X)$. It is immediately verified that

$$\frac{\partial f}{\partial X} = 2X$$

while its Jacobian matrix is, according to our definition:

$$\mathsf{D}\, f(X) = 2(\operatorname{vec} X)^\mathsf{T}. \tag{B.4}$$

Theorem B.1 (Chain Rule) *Let F be a function $F : \mathbb{R}^{n\times q} \to \mathbb{R}^{m\times p}$ and G a function $F : \mathbb{R}^{m\times p} \to \mathbb{R}^{r\times s}$. If the composite function $H(X) = G(F(X))$ is differentiable in X_0, its derivative is*

[1] Only exception in this text to the convention that vectors are arranged in columns.

$$\mathsf{D}\, H(X_0) = (\mathsf{D}\, G(Y_0))(\mathsf{D}\, F(X_0)) \tag{B.5}$$

where $Y_0 = F(X_0)$.

The following theorem is very useful, as it transforms the problem of finding the Jacobian of a matrix function, into the problem of finding its differential, which is usually easier.

Theorem B.2 (Identification) *Let* F *be a differentiable function* $F : \mathbb{R}^{n\times q} \to \mathbb{R}^{m\times p}$ *and let* d *be the differential. Then:*

$$\mathsf{d}\,\mathrm{vec}\, F(X) = A(X)\,\mathsf{d}\,\mathrm{vec}\, X \tag{B.6}$$

is equivalent to

$$\mathsf{D}\, F(X) = A(X). \tag{B.7}$$

Remark B.1 Given two differentiable matrix functions $U : \mathbb{R}^{n\times q} \to \mathbb{R}^{m\times r}$ and $V : \mathbb{R}^{n\times q} \to \mathbb{R}^{r\times p}$, let us compute the derivative of the product function:

$$F(X) = U(X)V(X). \tag{B.8}$$

$$\begin{aligned}
\mathsf{d}\, F(X) &= (\mathsf{d}\, U)V + U\,\mathsf{d}\, V \\
\mathsf{d}\,\mathrm{vec}\, F(X) &= \mathrm{vec}((\mathsf{d}\, U)V + U\,\mathsf{d}\, V) \\
&= (V^\mathsf{T} \otimes I_m)\,\mathrm{vec}(\mathsf{d}\, U) + (I_p \otimes U)\,\mathrm{vec}(\mathsf{d}\, V) \\
&= (V^\mathsf{T} \otimes I_m)\,\mathsf{D}\, U\,\mathsf{d}\,\mathrm{vec}\, X + (I_p \otimes U)\,\mathsf{D}\, V\,\mathsf{d}\,\mathrm{vec}\, X \\
&= \left((V^\mathsf{T} \otimes I_m)\,\mathsf{D}\, U + (I_p \otimes U)\,\mathsf{D}\, V\right)\mathsf{d}\,\mathrm{vec}\, X
\end{aligned} \tag{B.9}$$

therefore, by the identification theorem

$$\mathsf{D}\,(U(X)V(X)) = (V^\mathsf{T} \otimes I_m)\,\mathsf{D}\, U + (I_p \otimes U)\,\mathsf{D}\, V. \tag{B.10}$$

We now consider several cases of a matrix function $F(X)$ and compute its Jacobian $\mathsf{D}\, F(X)$ by exploiting the identification theorem.

Example B.4 Derivative of AXB.

$$\begin{aligned}
F(X) &= AXB \\
\mathsf{d}\,\mathrm{vec}(F(X)) &= \mathsf{d}\,\mathrm{vec}(AXB) = (B^\mathsf{T} \otimes A)\,\mathsf{d}\,\mathrm{vec}\, X.
\end{aligned} \tag{B.11}$$

Therefore, by the identification theorem

$$\mathsf{D}(AXB) = (B^\mathsf{T} \otimes A). \tag{B.12}$$

Example B.5 Derivative of X^T where X is $n \times q$.

$$\begin{aligned} F(X) &= X^\mathsf{T} \\ \mathsf{d}\, F(X) &= \mathsf{d}\, X^\mathsf{T} \\ \mathsf{d}\,\mathrm{vec}\, F(X) &= \mathsf{d}\,\mathrm{vec}(X^\mathsf{T}) \end{aligned} \tag{B.13}$$

We introduce the *commutation matrix* K, a specific permutation matrix that transforms vec A into $\mathrm{vec}(A^\mathsf{T})$. In particular, let A be an $n \times q$ matrix:

$$\mathrm{vec}(A^\mathsf{T}) = K_{nq}\,\mathrm{vec}\, A. \tag{B.14}$$

So

$$\mathsf{d}\,\mathrm{vec}\, F(X) = K_{nq}\,\mathsf{d}\,\mathrm{vec}\, X \tag{B.15}$$

and, by the identification theorem

$$\mathsf{D}(X^\mathsf{T}) = K_{nq}. \tag{B.16}$$

Example B.6 Derivative of the inverse. First let us work out the differential of the inverse:

$$0 = \mathsf{d}\, I = \mathsf{d}(X^{-1}X) = (\mathsf{d}\, X^{-1})X + X^{-1}\,\mathsf{d}\, X \tag{B.17}$$

and therefore

$$(\mathsf{d}\, X^{-1}) = -X^{-1}\,\mathsf{d}\, X X^{-1}. \tag{B.18}$$

Then:

$$\begin{aligned} F(X) &= X^{-1} \\ \mathsf{d}\, F(X) &= -X^{-1}(\mathsf{d}\, X)X^{-1} \\ \mathsf{d}\,\mathrm{vec}\, F(X) &= \mathrm{vec}(-X^{-1}(\mathsf{d}\, X)X^{-1}) = -(X^{-\mathsf{T}} \otimes X^{-1})\,\mathsf{d}\,\mathrm{vec}\, X \end{aligned} \tag{B.19}$$

hence, by the identification theorem

$$\mathsf{D}\left(X^{-1}\right) = -(X^{-\mathsf{T}} \otimes X^{-1}). \tag{B.20}$$

Example B.7 Derivative of the pseudoinverse where X is $n \times q$. The differential of $F(X)$ is derived in Golub and Pereyra (1973):

$$\mathrm{d}\,X^{+} = -X^{+}\,\mathrm{d}\,X X^{+} + X^{+}X^{+\mathsf{T}}\,\mathrm{d}\,X^{\mathsf{T}}\left(I_n - XX^{+}\right) + \left(I_q - X^{+}X\right)\mathrm{d}\,X^{\mathsf{T}}X^{+\mathsf{T}}X^{+}. \tag{B.21}$$

From the differential the derivative of $F(X) = X^{+}$is easily obtained:

$$\mathsf{D}\,X^{+} = -X^{+\mathsf{T}} \otimes X^{+} + \left((I_n - XX^{+}) \otimes X^{+}X^{+\mathsf{T}} + X^{+\mathsf{T}}X^{+} \otimes (I_q - X^{+}X)\right) K_{n,q}. \tag{B.22}$$

This is a general expression that makes no assumption on the rank or the shape of X. If X has full column rank, the term $(I_q - X^{+}X)$ vanishes, whereas if X has full row rank the term $(I_n - XX^{+})$ vanishes, by the properties of pseudoinverse.

Example B.8 Derivative of $X^{\mathsf{T}}AX$ where X is $n \times q$

$$\begin{aligned} F(X) &= X^{\mathsf{T}}AX \\ \mathrm{d}\,F(X) &= \mathrm{d}\,X^{\mathsf{T}}(AX) + X^{\mathsf{T}}A\,\mathrm{d}\,X. \end{aligned} \tag{B.23}$$

Thanks to the linearity of vec, we obtain:

$$\mathrm{d}\,\mathrm{vec}\,F(X) = \mathrm{vec}(\mathrm{d}\,X^{\mathsf{T}}(AX)) + \mathrm{vec}(X^{\mathsf{T}}A\,\mathrm{d}\,X). \tag{B.24}$$

So

$$\mathrm{d}\,\mathrm{vec}\,F(X) = K_{qq}\,\mathrm{vec}(X^{\mathsf{T}}A^{\mathsf{T}}\,\mathrm{d}\,X) + \mathrm{vec}(X^{\mathsf{T}}A\,\mathrm{d}\,X) \tag{B.25}$$

and by exploiting (A.59) and Theorem B.2, we arrive at

$$\mathsf{D}\,F(X) = K_{qq}(I_q \otimes X^{\mathsf{T}}A^{\mathsf{T}}) + (I_q \otimes X^{\mathsf{T}}A). \tag{B.26}$$

Example B.9 As a special case of the previous example with $A = I$, we get $F(X) = X^{\mathsf{T}}X$, whose derivative is

$$\mathsf{D}\,F(X) = (I_{q^2} + K_{q,q})(I_q \otimes X^{\mathsf{T}}). \tag{B.27}$$

B.2 Derivative of Rotations

B.2.1 Axis/Angle Representation

Let $\mathbf{u} = [u_1, u_2, u_3]^{\mathsf{T}}$ a versor representing the axis of rotation $\mathbf{u}$ and ϑ an angle. The corresponding rotation matrix is given by Rodriguez's formula:

$$R(\vartheta, \mathbf{u}) = (I + \sin\vartheta\,[\mathbf{u}]_{\times} + (1 - \cos\vartheta)\,[\mathbf{u}]_{\times}^{2}). \tag{B.28}$$

$$[\mathbf{u}]_\times^2 = \begin{bmatrix} -u_3^2 - u_2^2 & u_2 u_1 & u_3 u_1 \\ u_2 u_1 & -u_3^2 - u_1^2 & u_3 u_2 \\ u_3 u_1 & u_3 u_2 & -u_2^2 - u_1^2 \end{bmatrix} \tag{B.29}$$

The derivative is

$$\frac{\partial R(\vartheta, \widehat{\mathbf{u}})}{\partial \vartheta} = \frac{\mathrm{d} \sin \vartheta}{\mathrm{d}\vartheta} [\mathbf{u}]_\times - \frac{\mathrm{d} \cos \vartheta}{\mathrm{d}\vartheta} [\mathbf{u}]_\times^2 = \cos \vartheta \, [\mathbf{u}]_\times + \sin \vartheta \, [\mathbf{u}]_\times^2 \tag{B.30}$$

$$\frac{\partial R(\vartheta, \widehat{\mathbf{u}})}{\partial u_i} = \sin \vartheta \frac{\partial [\mathbf{u}]_\times}{\partial u_i} + (1 - \cos \vartheta) \frac{\partial [\mathbf{u}]_\times^2}{\partial u_i}. \tag{B.31}$$

For example, for $i = 1$, we have:

$$\frac{\partial [\mathbf{u}]_\times}{\partial u_1} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix} \quad \frac{\partial [\mathbf{u}]_\times^2}{\partial u_1} = \begin{bmatrix} 0 & u_2 & u_3 \\ u_2 & -2u_1 & 0 \\ u_3 & 0 & -2u_1 \end{bmatrix}. \tag{B.32}$$

B.2.2 Euler Representation

Consider an Euler representation of a rotation with angles ω, φ, κ defined as follows:

$$R(\omega, \varphi, \kappa) = R(\kappa, \mathbf{e}_3) R(\varphi, \mathbf{e}_2) R(\omega, \mathbf{e}_1) \tag{B.33}$$

where $\mathbf{e}_i$ denotes the versor of the i-th axis (e.g. $\mathbf{e}_1 = [1, 0, 0]^\mathsf{T}$)

For $i = 1$ (angle ω) the derivative is

$$\begin{aligned} \frac{\partial R(\omega, \varphi, \kappa)}{\partial \omega} &= R(\kappa, \mathbf{e}_3) R(\varphi, \mathbf{e}_2) \frac{\partial R(\omega, \mathbf{e}_1)}{\partial \omega} \\ &= R(\kappa, \mathbf{e}_3) R(\varphi, \mathbf{e}_2) \left(\cos \omega \, [\mathbf{e}_1]_\times + \sin \omega \, [\mathbf{e}_1]_\times^2 \right) \end{aligned} \tag{B.34}$$

and similarly one can proceed for $\dfrac{\partial R(\omega, \varphi, \kappa)}{\partial \varphi}$ and $\dfrac{\partial R(\omega, \varphi, \kappa)}{\partial \kappa}$. In the end:

$$\mathsf{D}\, R = \frac{\partial \operatorname{vec} R}{\partial [\omega, \varphi, \kappa]} = \left[\operatorname{vec} \frac{\partial R}{\partial \omega}, \operatorname{vec} \frac{\partial R}{\partial \varphi}, \operatorname{vec} \frac{\partial R}{\partial \kappa} \right]. \tag{B.35}$$

References

G. H. Golub and V. Pereyra. The differentiation of pseudo-inverses and nonlinear least-squares problems whose variables separate. *SIAM Journal on Numerical Analysis*, 10 413–432, 1973. doi: https://doi.org/10.1137/0710036.

J. R. Magnus and H. Neudecker. *Matrix Differential Calculus with Applications in Statistics and Econometrics*. John Wiley & Sons, New York, revised edition, 1999.

Appendix C
Regression

C.1 Introduction

Fitting of a model to noisy data, or regression, is an important statistical tool, frequently employed in computer vision for a wide variety of purposes.

The aim of regression analysis is to fit the parameters of a function (model) to the observations (data), which are obtained by *measuring* a dependent variable y_i in correspondence to m data values of n independent variables $\mathbf{t}_i = [t_{i1}, ..., t_{in}]^\mathsf{T}$, with $i = 1, \ldots, m$. The model function has the form $f(\mathbf{t}_i, \mathbf{x})$, where $\mathbf{x}$ contains the p parameters $[x_1, \ldots, x_p]^\mathsf{T}$.

The adequacy of the model in fitting to the data is measured through its *residual*, defined as the difference between the measured value of the dependent variable y_i and the value predicted by the model $f(\mathbf{t}_i, \mathbf{x})$ (at the same values of the independent variables):

$$r_i = f(\mathbf{t}_i, \mathbf{x}) - y_i. \tag{C.1}$$

The goal is to compute the parameter estimates $\hat{\mathbf{x}} = [\hat{x}_1, \ldots, \hat{x}_p]^\mathsf{T}$ that best explain the observations, in the sense that they minimise the residuals defined above.

C.2 Least-Squares

The Least-Squares (LS) method is the most common regression estimator. The regression traces back to the solution of a system of m equations in p unknowns:

$$r_i(\mathbf{x}) = f(\mathbf{t}_i, \mathbf{x}) - y_i = 0, \tag{C.2}$$

which can be rewritten in compact form as

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

$$\mathbf{r}(\mathbf{x}) = \mathbf{f}(\mathbf{t}, \mathbf{x}) - \mathbf{y} = \mathbf{0}, \quad \mathbf{r} : \mathbb{R}^p \to \mathbb{R}^m. \tag{C.3}$$

The cost function to be minimised is

$$\chi(\mathbf{x}) = \frac{1}{2}\sum_{i=1}^{m} r_i(\mathbf{x})^2 = \frac{1}{2}\mathbf{r}(\mathbf{x})^\mathsf{T}\mathbf{r}(\mathbf{x}) = \frac{1}{2}\|\mathbf{r}(\mathbf{x})\|_2^2. \tag{C.4}$$

It is statistically convenient to minimise this weighted cost function instead:[1]

$$\chi(\mathbf{x}) = \frac{1}{2}\sum_{i=1}^{m}\left(\frac{r_i(\mathbf{x})}{\sigma_i}\right)^2 = \frac{1}{2}\mathbf{r}(\mathbf{x})^\mathsf{T}\Sigma_y^{-1}\mathbf{r}(\mathbf{x}) \tag{C.5}$$

where $\Sigma_y = \text{diag}\,[\sigma_1^2, \ldots, \sigma_m^2])$ and σ_i^2 is the variance of the y_i measure, which also coincides with the variance of the corresponding residual.

Division by the standard deviation has the effect of properly weighting the contribution of each equation: residuals affected by less error weigh more than those with greater error. Also, if the residuals have different physical dimensions (and thus are inherently incommensurable), division by σ_i makes them dimensionless.

It can be shown that the minimum of (C.5) is also the maximum likelihood estimator, under the assumption that the error on y_i has Gaussian distribution with zero mean. In that case, the least-squares estimator is also statistically optimal, in the sense that it has minimum variance and its expected value coincides with the true value (*unbiasedness*).

When the Gaussian error assumption fails, the least-squares estimator can be completely misleading, as we will discuss later.

C.2.1 Linear Least-Squares

In *linear regression* we consider a linear function *of the parameters*, that is:

$$\mathbf{y} = A\mathbf{x} \tag{C.6}$$

where $\mathbf{y} = [y_1, \ldots, y_m]^\mathsf{T}$. The entries of A are functions $\phi_j : \mathbb{R}^n \to \mathbb{R}$ of the independent variables: $A_{ij} = \phi_j(\mathbf{t}_i)$.

Weighting the equations with the variance, we get:

$$\Sigma_y^{-\frac{1}{2}}\mathbf{y} = \Sigma_y^{-\frac{1}{2}}A\mathbf{x} \tag{C.7}$$

[1] This is also a norm, which is called "Mahalanobis" norm.

Following Proposition A.15, the least-squares regression parameters $\mathbf{x}$ can be obtained by solving the *normal equation*:

$$A^\top \Sigma_y^{-1} A\mathbf{x} = A^\top \Sigma_y^{-1} \mathbf{y}. \tag{C.8}$$

C.2.2 Non-linear Least-Squares

If the function $\mathbf{f}$ is nonlinear in the parameters, we have the generic relation:

$$\mathbf{y} = \mathbf{f}(\mathbf{x}) \tag{C.9}$$

from which

$$\mathbf{r} = \mathbf{f}(\mathbf{x}) - \mathbf{y}. \tag{C.10}$$

In the following, to lighten the notation, we neglect the weighting with Σ_y^{-1}.

C.2.2.1 Gauss-Newton Method

We therefore consider the minimisation of (C.4). Starting from a suboptimal current solution $\mathbf{x}$, we want to determine the increment $\Delta\mathbf{x}$ that minimises the cost function. We approximate $\mathbf{r}$ with the Taylor series truncated to first order:

$$r(\mathbf{x} + \Delta\mathbf{x}) = \mathbf{r}(\mathbf{x}) + \mathsf{D}\,\mathbf{r}(\mathbf{x})\Delta\mathbf{x}. \tag{C.11}$$

Observe that since $\mathbf{y}$ is constant, $\mathsf{D}\,\mathbf{r}(\mathbf{x}) = \mathsf{D}\,\mathbf{f}(\mathbf{x})$; let us call this matrix J.
We substitute in the formula of the cost function $\chi(\mathbf{x}) = \frac{1}{2}\mathbf{r}(\mathbf{x})^\top \mathbf{r}(\mathbf{x})$, obtaining

$$\chi(\mathbf{x} + \Delta\mathbf{x}) = \chi(\mathbf{x}) + \mathbf{r}(\mathbf{x})^\top J\Delta\mathbf{x} + \frac{1}{2}\Delta\mathbf{x}^\top J^\top J\Delta\mathbf{x}. \tag{C.12}$$

The increment $\Delta\mathbf{x}$ that minimises the equation (C.12) is obtained by setting to zero its derivative (with respect to $\Delta\mathbf{x}$), from which we get:

$$J^\top \mathbf{r}(\mathbf{x}) = -J^\top J\Delta\mathbf{x}. \tag{C.13}$$

This is the *normal equation* corresponding to the least-squares solution of the equation:

$$\mathbf{r}(\mathbf{x}) = -J(\mathbf{x})\Delta\mathbf{x}. \tag{C.14}$$

Because of the first-order approximation in the (C.11), $(\mathbf{x} + \Delta\mathbf{x})$ does not minimise the cost function, but, under appropriate conditions on $\mathbf{r}$, it is a better approximation of the solution, so the iteration $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$ produces a sequence of values converging to the minimum of (C.4).

This iteration is known as the Gauss-Newton method, and an implementation is shown in Listing C.1.

Listing C.1 Gauss-Newton method

```
function x = simpleGN(res,x0)
    %SIMPLEGN  Gauss-Newton method for non-linear LS

    % constants
    MaxIterations = 100;
    FunctionTol = 1e-10;
    StepTol = 1e-6;

    % initialization
    x = x0(:); dx = 0;
    y = Inf;

    for iter = 1: MaxIterations
        %% Step 1: Save previous values and take a (tentative) move
        x_prev = x; y_prev = y;
        x = x + dx;
        [y,J]   = res(x);
        fprintf('\tsimpleGN: residual norm %0.5g \n', norm(y));

        %% Step 2: Test the move: does it increase error?
        if  norm(y) < norm(y_prev)
            if norm(y-y_prev)<StepTol || norm(y)<FunctionTol
                break; % convergence
            end
        else
            % restore old values
            x = x_prev; y = y_prev;
            disp('increase')
            break % GN does not accept error incraese
        end

        %% Step 3: Compute a tentative update
        dx = -(J'*J) \ J' * y; % normal equations
        % use pinv for regularizing rank-deficient jacobians
        % dx = -pinv(J) * y;
    end
end
```

In the case where the Jacobian matrix of $\mathbf{f}$ does not have full rank, one can generalise the procedure by computing the increment with the pseudoinverse:

$$\Delta\mathbf{x} = -J(\mathbf{x})^{+}\mathbf{r}(\mathbf{x}). \tag{C.15}$$

Finally, remember that we dropped the variance weighting, so the normal equation (C.13) should write:

$$J^{\mathsf{T}}\Sigma_y^{-1}\mathbf{r}(\mathbf{x}) = -J^{\mathsf{T}}\Sigma_y^{-1}J\Delta\mathbf{x}. \tag{C.16}$$

C.2.3 The Levenberg-Marquardt Method

Since $J^\mathsf{T} J$ is an approximation of the Hessian matrix, which in turn is a quadratic approximation of the function at a point, the Gauss-Newton step $\Delta\mathbf{x}$ can be interpreted as the one leading on the minimum of the paraboloid that locally approximates the cost function.

This approximation is all the more verified the closer we are to the minimum. On the other hand, when the quadratic approximation does not hold, Gauss-Newton does not reduce the error quickly enough, or not at all. In these cases it would be better to use the gradient descent technique, which always guarantees (under mild conditions) convergence to a stationary point, however slowly.

Consider again (C.4); if we wanted to find its minimum with gradient descent, we would have to calculate its gradient:

$$\nabla\chi = \mathbf{r}(\mathbf{x})^\mathsf{T} \mathsf{D}\, \mathbf{r}(\mathbf{x}) = \mathbf{r}(\mathbf{x})^\mathsf{T} J \tag{C.17}$$

and iterate $\mathbf{x} \leftarrow \mathbf{x} + \Delta\mathbf{x}$ with

$$\Delta\mathbf{x} = -\alpha\, (\nabla\chi)^\mathsf{T} = -\alpha\, J^\mathsf{T}\mathbf{r}(\mathbf{x}) \tag{C.18}$$

where α is the length of the step. Gradient descent has a somewhat complementary behavior to Gauss-Newton; in fact it has a very slow convergence rate right near the minimum when contour lines are very stretched in one direction, just where Gauss-Newton works best. Levenberg's idea was to blend the two methods with a strategy to choose which of the two to weigh more depending on local conditions.

Combining (C.18) and (C.13) gives:

$$\Delta\mathbf{x} = -(J^\mathsf{T} J + \lambda I)^{-1} J^\mathsf{T}\mathbf{r}(\mathbf{x})). \tag{C.19}$$

For $\lambda \to 0$, one gets Gauss-Newton, while for $\lambda \to \infty$ one obtains $\Delta\mathbf{x} = -\frac{1}{\lambda} J^\mathsf{T}\mathbf{r}(\mathbf{x})$ which is a gradient descent. The strategy to modulate the mixing factor λ is as follows: when the error decreases, then Gauss-Newton is working and we make it the prevailing strategy by decreasing λ (e.g. divide λ by 10). When, on the contrary, the error is increasing, we drift towards gradient descent by increasing λ (e.g. multiply λ by 10).

Marquardt observed that even when the strategy inclines towards gradient descent, we can still exploit the local curvature information by scaling the components of the diagonal matrix to make longer steps where the gradient is small. So the formula for calculating the Levenberg-Marquardt (LM) step becomes:

$$\Delta\mathbf{x} = -(J^\mathsf{T} J + \lambda\, \mathrm{diag}(J^\mathsf{T} J))^{-1} J^\mathsf{T}\mathbf{r}(\mathbf{x}). \tag{C.20}$$

Let it be clear that $\Delta\mathbf{x}$ is not actually computed by inverting a matrix but by solving a linear system of equations.

The LM method can be seen as an instance of the more general class of *damped* Gauss-Newton methods, where a diagonal matrix, called *damping term*, is summed to $J^{\mathsf{T}}J$. The damping term also solves the problem of rank-deficient Jacobian matrices.

The `lsq_nonlin` function provided in the CV Toolkit, not listed here for space reasons, is an instructional implementation of the Levenberg-Marquardt method

C.3 Robust Regression

In this section we will introduce some concepts concerning robust statistics and briefly describe some robust estimators. The interested reader might want to expand on these topics in Meer et al. (1991) and Stewart (1999).

C.3.1 Outliers and Robustness

The most common regression technique is least-squares, which is only optimal in the case where the errors that affect the data are Gaussian distributed. In the case where the distribution is contaminated by *outliers*, that is, observations that do not follow the general trend of the data, these can corrupt the least-squares estimate in an arbitrarily large way (see Fig. C.1). The outliers arise in most cases from gross measurement errors or impulsive noise that plagues the data.

In order to remedy these drawbacks, *robust* regression techniques have been developed that provide reliable results even in the presence of contaminated data.

Three indicators are used to evaluate a regression method: relative efficiency, breakpoint and computational complexity.

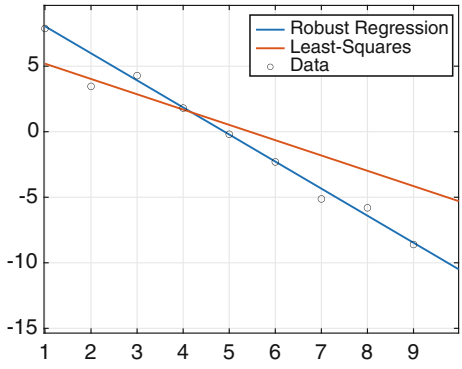


Fig. C.1 Example of corrupted data by an outlying sample: least-squares regression line and robust regression line

The *relative efficiency* of a regression method is defined as the ratio of the minimum achievable variance for the estimated parameters to the actual variance provided by the given method. The efficiency also depends on the noise distribution.

The breakpoint of a regression method is the minimum fraction of outlier samples that can arbitrarily cause the estimator to diverge from the true value. For example, least-squares has breakpoint equal to $\frac{1}{m}$, where m is the sample size, since it only takes one outlier sample out of m to arbitrarily corrupt the estimate. Asymptotically (for $m \to \infty$) the breakpoint is 0.

C.3.2 M-Estimators

The *M-estimators* are one of the most widely used robust regression methods. The idea is to replace the squares of the residuals in the traditional least-squares method with a function of the residuals:

$$\min_{vx} \sum_i \rho(r_i(\mathbf{x})/\sigma), \tag{C.21}$$

where σ is the standard deviation of the residuals, r_i and ρ is a symmetric, sub-quadratic function having a unique minimum in zero, called loss function. Examples of loss functions and their weights are shown in Fig. C.2 under the names "Bisquare", "Cauchy" and "Huber".

M-estimators have reduced sensitivity to peripheral samples due to the sub-quadratic loss function. If the function becomes flat after a certain point, then the

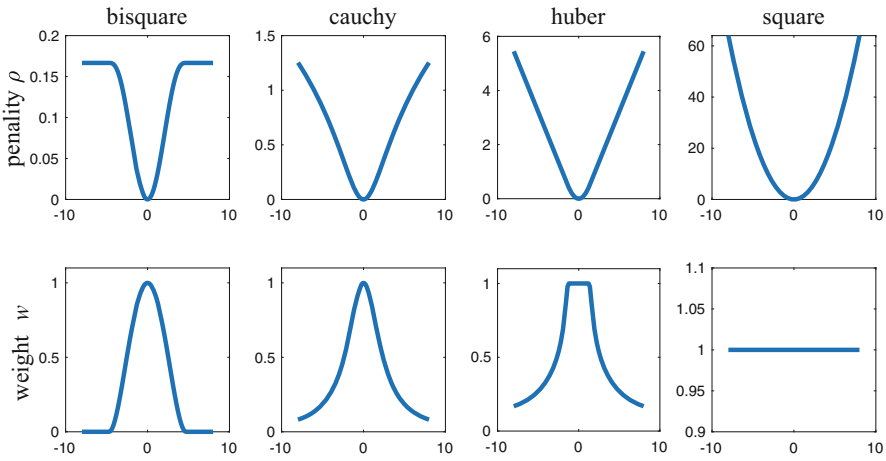


Fig. C.2 Loss functions "Bisquare"," Cauchy" and "Huber" (first line) and related weight functions (second line). The square function is also drawn for comparison

M-estimator can reach a breakpoint of 50%. Otherwise, it benefits from the reduced outliers influence, but the asymptotic breakpoint remains zero.

The σ in (C.21) is the standard deviation of the Gaussian noise that plagues the measurements, without considering the outliers. If it is not specified a priori, it must be estimated from the data itself, but in a robust way. One possibility is to use the Median Absolute Deviation (MAD):

$$\text{MAD}(r_i) = \underset{i}{\text{med}}(|r_i - \underset{j}{\text{med}}(r_j)|) \tag{C.22}$$

where r_i are the residuals. Under the assumptions of normal noise distribution, a robust estimate of the standard deviation is calculated as $\hat{\sigma} = 1.4826\,\text{MAD}$.

The minimisation of (C.21) is obtained by zeroing out the derivative:

$$\sum_i \rho'\left(\frac{r_i(\mathbf{x})}{\sigma}\right)\frac{1}{\sigma}\frac{\mathsf{d}\, r_i(\mathbf{x})}{\mathsf{d}\,\mathbf{x}} = 0 \tag{C.23}$$

where ρ' is the derivative of ρ. A widely used, though not the only, way to proceed is to introduce the weight function w such that $\rho'(x) = x \cdot w(x)$. In this way the previous equation becomes:

$$\sum_i w\left(\frac{r_i(\mathbf{x})}{\sigma}\right)\frac{r_i}{\sigma^2}\frac{\mathsf{d}\, r_i(\mathbf{x})}{\mathsf{d}\,\mathbf{x}} = 0. \tag{C.24}$$

The solution of the latter equation is equivalent to the solution of the weighted least-squares problem:

$$\min_{\mathbf{x}} \sum_i w\left(\frac{r_i(\mathbf{x})}{\sigma}\right)\left(\frac{r_i(\mathbf{x})}{\sigma}\right)^2. \tag{C.25}$$

The weights, however, depend on the residuals, the residuals depend on the estimate $\mathbf{x}$, and the latter depends on the weights. Thus, an iterative solution is needed, called Iteratively Reweighted Least-Squares (IRLS) and implemented in Listing C.2. If ρ is not convex, convergence to local minima may occur. The initial estimate can be obtained, for example, with ordinary least-squares; however, Stewart (1999) notes that even in the presence of low percentages of outliers (15%), IRLS may not converge to the correct solution from this estimate. Thus, a robust initial estimate is needed, obtained for example, by one of the methods we will see below. The same author also suggests freezing the $\hat{\sigma}$ estimate after a number of iterations to allow IRLS to converge better.

Listing C.2 IRLS

```
    %SIMPLEIRLS - Robust fit with the IRLS algorithm

    MaxIterations = 100;
    FunctionTol = 1e-6;
    StepTol = 1e-6;

    % start with OLS fit
    weights =  ones(length(x),1);
    model = modelfit(x, weights);
    obj = Inf;

    for  i = 1:MaxIterations

        % Evaluate distances between points and model
        res = modeldist(model, x);
        % Estimate scale
        scale = robstd(res);
        % Compute weights
        weights =  weightfun(res,scale);
        % Fit model with weights
        model = modelfit(x, weights);

        prevobj= obj;
        obj = sum(weights.*res.^2);

        if norm(obj-prevobj)<StepTol || obj<FunctionTol
            break; end
    end
    inliers = abs(res) < (2.5*scale);

end
```

C.3.3 Least Median of Squares

The Least Median of Squares (LMS) method, introduced by Rousseeuw and Leroy (1987), was the most widely used robust estimation technique in computer vision before the advent of RANSAC. In LMS, the parameters are estimated by solving the non-linear minimisation problem:

$$\min_{\mathbf{x}} \left(\text{med}_i \, r_i(\mathbf{x})^2 \right), \tag{C.26}$$

This problem does not have closed-form solutions, and direct numerical minimisation of the objective function immediately becomes impractical as the number of variables increases. One possible approach is the Monte Carlo method based on random sampling.

Given a model that requires a minimum number of p observations to instantiate its independent variables, we take p points at random and compute a test estimate $\hat{\mathbf{x}}$. We compute the residuals of the test estimate and take the median of the squares. After repeating these steps for a number of times (ideally for all p-uples), we choose the test estimate that realised the least median of the squared residuals (Listing C.3).

How many samples of size p should be considered? If m is the cardinality of the set of all data points, there are $\binom{m}{p}$ different subsets of p elements. If only a number equal to k are considered, the question arises of how large k must be for the procedure to compute the correct estimate with good probability. Since the

Listing C.3 Least median of squares

```
    %SIMPLELMS - Robust fit with the LMS algorithm

    n = size(x,2);
    alpha = 0.99; % Desired probability of success
    f = 0.5 ; % Pessimistic estimate of inliers fraction

    MaxIterations  = max( ceil(log(1-alpha)/log(1-f^p)), 100);
    mincost =  Inf;

    for  i = 1:MaxIterations

        % Generate s random indicies in the range 1..npts
        mss = randsample(n, p);
        % Fit model to this minimal sample set.
        model = modelfit(x(:,mss),[]);
        % Evaluate distances between points and model
        sqres = modeldist(model, x).^2;
        % Compute LMS score
        cost = median(sqres);

        scale = 1.4826*sqrt(cost)*(1+5/(length(sqres)-p));
        inliers = sqres < (2.5*scale)^2 ;

        if cost < mincost
            mincost = cost;
            bestmodel = model;  bestinliers = inliers;
        end
    end
end

function [model, inliers] = simpleIRLS(x, modelfit, modeldist)
```

minimum of (C.26) is realised by the outlier-free samples, the capability to compute the correct estimate is related to the presence of at least one of these subsets among all p-tuples. Given a percentage ϵ of outlier samples, the following formula relates k to the probability α of computing the correct estimate:

$$\alpha = 1 - (1 - (1 - \epsilon)^p)^k, \tag{C.27}$$

from which, by fixing a value of α close to 1, one can determine the number k, given p and ϵ:

$$k = \frac{\log(1 - \alpha)}{\log(1 - (1 - \epsilon)^p)}. \tag{C.28}$$

LMS has a breakpoint of 50%, but its the relative efficiency is poor in the presence of Gaussian noise. To overcome this disadvantage, after termination of LMS, a weighted least-squares problem is solved:

$$\min \sum_i [\![|\frac{r_i}{\hat{\sigma}}| \leq 2.5]\!] \left(\frac{r_i}{\hat{\sigma}}\right)^2 \tag{C.29}$$

where the robust estimate of the standard deviation can be obtained with MAD or with

$$\hat{\sigma} = 1.4826\,(1 + 5/(n - p))\sqrt{\underset{i}{\text{med}}\, r_i^2}. \tag{C.30}$$

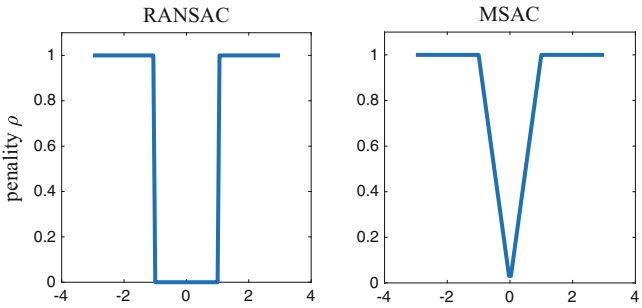


Fig. C.3 Loss function ρ of RANSAC and MSAC viewed as M-estimators. In the graphs $T = 1$

C.3.4 RANSAC

The Random Sample Consensus (RANSAC) algorithm by Fischler and Bolles (1981) uses the same random sampling scheme as LMS, but with a different objective function. In fact, RANSAC seeks to maximise the "consensus" of the test estimate, that is, the number of points whose residuals r_i are smaller (in absolute value) than a given threshold T. This is equivalent to minimising the number of points with residuals *greater* than a threshold T, so RANSAC can be seen as an M-estimator with a step loss function $\rho(r_i) = [\![|r_i| \geq T]\!]$, which is zero for residuals contained in $[-T, T]$ and one otherwise, as shown in Fig. C.3.

The algorithm operates under the implicit assumption that there is a high probability that the consensus for the test estimate computed on a set of p samples free of outliers is higher than the support obtained from the "wrong" test estimates, that is, computed on a sample contaminated with outliers.

The theoretical breakpoint of RANSAC is 50%;[2] however, in practice, if the threshold T is carefully set, RANSAC can withstand even higher levels of contamination. The price to pay for this higher robustness compared to LMS is the presence of a parameter, the threshold T.

Torr and Zisserman (2000) propose to use instead the following loss function:

$$\rho(r_i) = T[\![|r_i| \geq T]\!] + r_i[\![|r_i| < T]\!] \tag{C.31}$$

and call MSAC (M-estimator SAmple Consensus) the resulting variant of RANSAC. They observe that the performance of MSAC is always better than that of RANSAC, so there is no reason to prefer the latter to MSAC. The implementation of MSAC is shown in Listing C.4

[2] The theoretical breakpoint cannot exceed 50%, because it cannot be excluded that a wrong test estimate will gather more consensus than the right ones, if the outliers are arranged appropriately.

Listing C.4 MSAC

```
function [bestmodel, bestinliers] = simpleMSAC(x, modelfit, modeldist, p, t)
    %SIMPLEMSAC - Robust fit with the MSAC algorithm

    n = size(x,2);
    alpha = 0.995; % Desired probability of success
    f = 0.4 ;     % Pessimistic estimate of inliers fraction

    MaxIterations = 5000; % Max number of iterations
    MinIterations = 500;  % Min number of iterations
    mincost =  Inf;

    i = 0;
    while  i < max(ceil(log(1-alpha)/log(1-f^p)), MinIterations)

        % Generate p random indicies in the range 1..n
        mss = randsample(n, p);

        % Fit model to this minimal sample set.
        model = modelfit(x(:,mss),[]);

        % Evaluate distances between points and model
        sqres = modeldist(model, x).^2;
        inliers = sqres < (t^2);

        % Compute MSAc score
        cost = (sum(sqres(inliers)) + (n -sum(inliers)) * t^2);

        if cost < mincost
            mincost = cost;
            bestinliers = inliers; bestmodel = model;
            % Update the estimate of inliers fraction
            f = sum(bestinliers)/n;
        end
        i = i + 1;
        if i > MaxIterations, break, end
    end
end
```

More on RANSAC developments can be found in Raguram et al. (2013); Baráth et al. (2020).

C.4 Propagation of Uncertainty

The errors that inevitably occur in the measurement operations make the estimation of the input variables uncertain, and this uncertainty necessarily propagates to the output variables, which constitute the results of the regression. The latter should therefore be accompanied by an indication of the error, usually provided by the standard deviation.

An uncertain variable is represented by a probability distribution, that is, a probability density that assigns each value of the variable a probability. Detailed knowledge of the distribution is not necessary in most applications, and moreover, it is often not accessible due to the lack of such an accurate model of the measures. For these reasons we choose to identify it by its first two moments –the mean and the covariance– defined as, respectively:

$$\bar{\mathbf{x}} = \mathcal{E}[\mathbf{x}], \quad \Sigma_x = \mathcal{E}\left[(\mathbf{x} - \bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}})^{\mathsf{T}}\right] \tag{C.32}$$

where $\mathcal{E}[\cdot]$ represents the expected value (or expectation).

When two variables are related by a transformation, the problem of propagating uncertainty through the transformation arises. Precisely if two variables $\mathbf{x}$ and $\mathbf{y}$ are related by a linear relation $\mathbf{y} = A\mathbf{x} + \mathbf{b}$ and the former is characterised by $(\bar{\mathbf{x}}, \Sigma_x)$, one wants to obtain mean and covariance of the latter.

Using the definitions and linearity of the expectation operator, it is easily verified that

$$\bar{\mathbf{y}} = \mathbf{A}\bar{\mathbf{x}} + \mathbf{b} \tag{C.33}$$

and also

$$\Sigma_y = \mathbf{A}\Sigma_x\mathbf{A}^\top. \tag{C.34}$$

If the relation $\mathbf{y} = f(\mathbf{x})$ that links $\mathbf{x}$ and $\mathbf{y}$ is not linear, it is necessary to approximate it by its Taylor series expansion around the point $\bar{\mathbf{x}}$ obtaining

$$f(\mathbf{x}) = f(\bar{\mathbf{x}}) + \mathsf{D}\,\mathbf{f}(\bar{\mathbf{x}})(\mathbf{x} - \bar{\mathbf{x}}) + \dots\ , \tag{C.35}$$

where

$$\mathsf{D}\,\mathbf{f}(\bar{\mathbf{x}}) = \left.\frac{\partial f}{\partial \mathbf{x}^\mathsf{T}}\right|_{\mathbf{x}=\bar{\mathbf{x}}} \tag{C.36}$$

is the Jacobian matrix of f evaluated in $\bar{\mathbf{x}}$.

Truncating the expansion to the linear term and computing the expectation yields the linear estimate of the mean:

$$\bar{\mathbf{y}} = f(\bar{\mathbf{x}}). \tag{C.37}$$

Similarly, the first-order estimate of the covariance is

$$\Sigma_y = J\Sigma_x J^\mathsf{T} \text{ with } \quad J = \mathsf{D}\,\mathbf{f}(\bar{\mathbf{x}}). \tag{C.38}$$

C.4.1 Covariance Propagation in Least-Squares

For example, with reference to the notation of Sect. C.2.1, let us compute the covariance of the linear least-squares regression coefficients $\mathbf{x}$, which are a linear function of the $\mathbf{y}$ measures:

$$\mathbf{x} = (A^\mathsf{T}\Sigma_y^{-1}A)^{-1}A^\mathsf{T}\Sigma_y^{-1}\mathbf{y}. \tag{C.39}$$

Then, applying (C.34), we obtain:

$$\Sigma_x = \left(A^\top \Sigma_y^{-1} A\right)^{-1}. \tag{C.40}$$

In the non-linear case, we concentrate on the normal equations of the Gauss-Newton method, and we get:

$$\Sigma_x = \left(J^\top \Sigma_y^{-1} J\right)^{-1} \quad \text{with} \quad J = \mathsf{D}\,\mathbf{f}(\bar{\mathbf{x}}), \tag{C.41}$$

assuming that the Jacobian matrix has full rank. If not, the pesudoinverse replaces the inverse (Hartley and Zisserman 2003), p. 144.

References

Dániel Baráth, Jana Noskova, Maksym Ivashechkin, and Jiri Matas. Magsac++, a fast, reliable and accurate robust estimator. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020*, pages 1301–1309. Computer Vision Foundation / IEEE, 2020. doi: https://doi.org/10.1109/CVPR42600.2020.00138.

M. A. Fischler and R. C. Bolles. Random Sample Consensus: A paradigm model fitting with applications to image analysis and automated cartography. *Communications of the ACM*, 24 381–395, June 1981.

R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*. Cambridge University Press, Cambridge, 2nd edition, 2003.

P. Meer, D. Mintz, D. Y. Kim, and A. Rosenfeld. Robust regression methods in computer vision: a review. *International Journal of Computer Vision*, 6: 59–70, 1991.

Rahul Raguram, Ondrej Chum, Marc Pollefeys, Jiri Matas, and Jan-Michael Frahm. Usac: A universal framework for random sample consensus. *IEEE Trans. Pattern Anal. Mach. Intell.*, 35(8): 2022–2038, 2013. URL http://dblp.uni-trier.de/db/journals/pami/pami35.html#RaguramCPMF13.

P. J. Rousseeuw and A. M. Leroy. *Robust Regression & Outlier Detection*. John Wiley & Sons, New York, 1987.

C. V. Stewart. Robust parameter estimation in computer vision. *SIAM Review*, 41 (3): 513–537, 1999.

P. H. S. Torr and A. Zisserman. MLESAC: A new robust estimator with application to estimating image geometry. *Computer Vision and Image Understanding*, 78: 2000, 2000.

Appendix D
Notions of Projective Geometry

«A colonnade [. . .] when its whole length is seen from the top end, little by little it contracts to the pointed head of a narrow cone, joining roof with floor, and all the right hand with the left, until it has brought all together into the point of a cone that passes out of sight.» [Lucrezio]

D.1 Introduction

In order to introduce the use of projective geometry and therefore of homogeneous coordinates, we follow Leon Battista Alberti and study first the perspective projection of a plane. For a formal treatment of analytic projective geometry, see, e.g. Semple and Kneebone (1952).

D.2 Perspective Projection

Perspective projection is the type of projection that in the vertebrate eye and in cameras governs image formation.

Definition D.1 (Perspective Projection) Perspective projection carries points M of 3D space on the plane $\mathcal{Q}$ (image plane) intersecting the line passing through M and C (centre of projection) with $\mathcal{Q}$.

The basic elements are the image plane $\mathcal{Q}$ and the centre of projection C. The lines of space are transformed into lines in $\mathcal{Q}$.

If we now consider the projection of a plane $\mathcal{G}$ orthogonal to $\mathcal{Q}$ (the *ground plane*), we find that:

- the line f determined by the intersection of the plane $\mathcal{G}$ with the plane parallel to $\mathcal{Q}$ and containing C does not project onto $\mathcal{Q}$;
- the line h intersection of $\mathcal{Q}$ with the plane parallel to $\mathcal{G}$ and passing through C is not the projection of any line on the plane $\mathcal{G}$.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

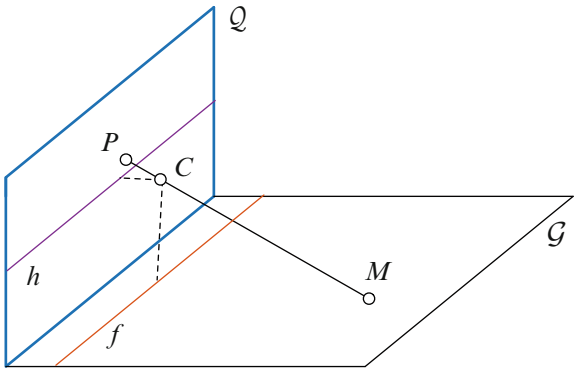


Fig. D.1 Perspective projection of a ground plane $\mathcal{G}$ orthogonal to the image plane $\mathcal{Q}$. The lines h and f are special cases

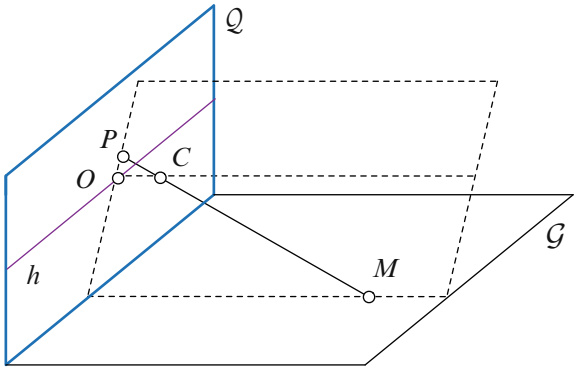


Fig. D.2 The depth line containing M projects onto a line containing P and passing through the principal point O

This situation is not satisfactory, as we would like to have a bijection between $\mathcal{Q}$ and $\mathcal{G}$. To remedy this we add to the usual Euclidean plane $\mathbb{R}^2$ (of which $\mathcal{Q}$ and $\mathcal{G}$ are instances) an "ideal" line, called the line *at infinity*.

We can then say that f projects onto the line at infinity of $\mathcal{Q}$ and that h is the projection of the line at infinity of $\mathcal{G}$. We can think of the line at infinity of a plane as the place where the parallel lines of that plane (metaphorically) meet. Indeed:

Remark D.1 With reference to Fig. D.1, the images of parallel lines in $\mathcal{G}$ intersect at a point h.

Let us consider (Fig. D.2), for example, the special case of parallel lines contained in $\mathcal{G}$ and orthogonal to $\mathcal{Q}$ (called *depth lines* in the Albertian construction). Their perspective image converges to the *principal point* $O \in h$, that is, the orthogonal projection of C onto $\mathcal{Q}$. In fact, the planes containing C and any depth line all contain the half-line exiting C and parallel to $\mathcal{G}$, called the *principal ray*, and thus intersect the image $\mathcal{Q}$ in a line passing through the principal point O.

Definition D.2 (Projective Plane) The projective plane $\mathbb{P}^2$ is defined as the union of the Euclidean plane $\mathbb{R}^2$ with the line at infinity l_∞:

$$\mathbb{P}^2 = \mathbb{R}^2 \cup l_\infty. \tag{D.1}$$

Similarly, one can define the projective space $\mathbb{P}^n$, where the points at infinity constitute a hyperplane.

In the projective plane, two lines have a point of intersection even when they are parallel, but the point is *at infinity*. Every line, therefore, possesses a point at infinity that is approached by travelling along the line in a certain direction, and all lines parallel to that line meet it at the same point at infinity. For now, the term "point at infinity" is just a metaphor, but we can give it a precise mathematical meaning. One way to do this is to identify points at infinity with the absolute direction of a line. Consider the following three axioms of Euclidean geometry:

(i) there exists a single line through two distinct points;
(ii) there exists a single line passing through a point and having a given direction;
(iii) two distinct lines have a single point in common or have the same direction.

If we agree to replace "direction" with "point at infinity" and then group all points at infinity into a line at infinity, we get that the previous three axioms are equivalent to:

(i) there exists a single line passing through two distinct points;
(ii) two distinct lines have a single point in common.

These are the two axioms of incidence in the projective plane.

D.3 Homogeneous Coordinates

Points at infinity are ideal (or improper) points that are added to the usual Euclidean plane. Like complex points, which are introduced by means of their algebraic representation, the same can be done for points at infinity, using *homogeneous coordinates*.

Fixed two reference axes, all points of $\mathbb{R}^2$ admit a unique representation as a pair of real numbers (x, y). Two lines of equations $ax+by+c = 0$ and $a'x+b'y+c' = 0$ meet, if they are not parallel, at a point of $\mathbb{R}^2$ whose coordinates are (by Cramer's rule):

$$x = -\frac{\det\left(\begin{bmatrix} c & b \\ c' & b' \end{bmatrix}\right)}{\det\left(\begin{bmatrix} a & b \\ a' & b' \end{bmatrix}\right)} \qquad y = -\frac{\det\left(\begin{bmatrix} a & c \\ a' & c' \end{bmatrix}\right)}{\det\left(\begin{bmatrix} a & b \\ a' & b' \end{bmatrix}\right)}. \tag{D.2}$$

When the lines are parallel, the denominator cancels out, so the ratio is meaningless (it becomes "infinite"). We note, however, that of the three quantities that appear in

(D.2), at least one is nonzero (if the lines are distinct). The proposal is then to hold these three quantities as representations of the points of $\mathbb{P}^2$.

Essentially, instead of representing the points in the plane by pairs of coordinates (x, y), we agree to represent them by triplets of *homogeneous coordinates* (u, v, w), connected to the Cartesian coordinates of $\mathbb{R}^2$ by the relation $x = u/w$ and $y = v/w$.

We note that proportional triplets represent the same point – this is why they are called homogeneous – and that the triplet $(0, 0, 0)$ is excluded. When $w \neq 0$ the triplet represents a *proper* point (finite), while every triplet for which $w = 0$ represents a *improper* point (at infinity).

Definition D.3 (Projective Plane) The projective plane $\mathbb{P}^2$ consists of the triplets of real numbers $(u, v, w) \neq (0, 0, 0)$ modulo the following equivalence relation: $(u, v, w) \sim \lambda(u, v, w) \ \forall \lambda \neq 0$. The triplets (u, v, w) are the *homogeneous coordinates* of the point of $\mathbb{P}^2$.

Similarly, one can define the projective space $\mathbb{P}^n$.

D.4 Equation of the Line

In the familiar Euclidean plane with Carttesian coordinates, each pair of numbers identifies a point, and, conversely, each point is represented by only one pair. This is not true for lines: the quation of the line is $ax + by + c = 0$ (with a and b not simultaneously null), so the three coefficients of the equation represent a line, but the same line is represented by all the linear equations $\lambda ax + \lambda by + \lambda c = 0$, with $\lambda \neq 0$. In the projective plane, this asymmetry disappears, as points and lines are dual elements, as can be inferred from the axioms.

A line in $\mathbb{P}^2$ is represented by a triplet (a, b, c) in the equation $au + bv + cw = 0$, and proportional triplets represent the same line. This line coincides with the proper line $ax + by + c = 0$ if $a \neq 0 \vee b \neq 0$. Otherwise, the equation $w = 0$ represents the line to infinity l_∞.

By identifying triplets with vectors of $\mathbb{R}^3$, the equation of the line $au + bv + cw = 0$ can be written

$$\mathbf{x}^\mathsf{T}\mathbf{y} = 0 \tag{D.3}$$

where $\mathbf{x} = [u, v, w]^\mathsf{T}$ and $\mathbf{y} = [a, b, c]^\mathsf{T}$. Note the symmetry between the role of the vector representing the line and the one representing the point.

Proposition D.1 *The line through two distinct points* $\mathbf{p}_1$ *and* $\mathbf{p}_2$ *is represented by the triplet* $\mathbf{p}_1 \times \mathbf{p}_2$.

Proof The line passes through both points, in fact: $\mathbf{p}_1^\mathsf{T}(\mathbf{p}_1 \times \mathbf{p}_2) = 0$ and $\mathbf{p}_2^\mathsf{T}(\mathbf{p}_1 \times \mathbf{p}_2) = 0$.

As for the point determined by two lines, the dual proposition applies, which can also be deduced directly from (D.2).

Proposition D.2 *The line l through two distinct points $\mathbf{p}_1$ and $\mathbf{p}_2$ has parametric equation:*

$$l = \{\mathbf{x} \in \mathbb{P}^2 \mid \mathbf{x} = \alpha\mathbf{p}_1 + \beta\mathbf{p}_2, \quad \alpha, \beta \in \mathbb{R}\}. \tag{D.4}$$

Proof $\mathbf{x} = \alpha\mathbf{p}_1 + \beta\mathbf{p}_2 \iff \det([\mathbf{x}, \mathbf{p}_1, \mathbf{p}_2]) = 0 \iff \mathbf{x}^\mathsf{T}(\mathbf{p}_1 \times \mathbf{p}_2) = 0.$

Letting $\lambda = \beta/\alpha$ and accepting the convention that $\mathbf{p}_1 + \lambda\mathbf{p}_2 = \mathbf{p}_2$ when $\lambda = \infty$, the line l has parametric equation:

$$l = \{\mathbf{x} \in \mathbb{P}^2 \mid \mathbf{x} = \mathbf{p}_1 + \lambda\mathbf{p}_2, \quad \lambda \in \mathbb{P}^1\}. \tag{D.5}$$

D.5 Transformations

Definition D.4 (Projectivity) A projectivity or projective transformation $f : \mathbb{P}^n \to \mathbb{P}^n$ is a linear application in homogeneous coordinates:

$$f : \mathbf{x} \to H\mathbf{x} \tag{D.6}$$

where H is a non-singular $(n+1) \times (n+1)$ matrix.

Projectivities also take the name of *homographies* or *collinearities* (since they preserve the collinearity of the points); they form a group.

The perspective projection of the plane $\mathcal{G}$ defined before is a projectivity of $\mathbb{P}^2$ (we introduced $\mathbb{P}^2$ exactly to make this application bijective).

Because of the homogeneous representation of points, the two matrices H and λH with $\lambda \in \mathbb{R}$, $\lambda \neq 0$ represent the same projectivity.

Definition D.5 (Affinity) Projectivities that transform the Euclidean space into itself and points at infinity into points at infinity are called *affinities*.

One can alternatively say that an affinity *preserves* the points of the hyperplane at infinity. Affinities are all and only the projectivities in which the last row of the matrix is $0 \ldots 0, 1$:

$$H = \begin{bmatrix} A_{n\times n} & \mathbf{b} \\ \mathbf{0} & 1 \end{bmatrix}. \tag{D.7}$$

They constitute a subgroup of the projectivities.

In the hyperplane at infinity lies a special conic, the *absolute conic* which has equation $x_1^2 + x_2^2 + \ldots + x_n^2 = 0 \wedge x_{n+1} = 0$.

Definition D.6 (Similitude) Affinities that preserve the absolute conic are called similitudes (or similarities).

Similitudes are all and only affinities in which the matrix has this form:

$$H = \begin{bmatrix} s R_{n\times n} & \mathbf{t} \\ \mathbf{0} & 1 \end{bmatrix} \tag{D.8}$$

where $R_{n\times n} \in O(n)$ and s is a scalar. The similitudes form a subgroup of the projectivities.

The action of the similitude in $\mathbb{R}^n$ can be written as $\widetilde{\mathbf{x}} \rightarrow s R_{n\times n}\widetilde{\mathbf{x}} + \mathbf{t}$ where $\mathbf{x} = [\widetilde{\mathbf{x}}, 1]^\mathsf{T}$ and $\widetilde{\mathbf{x}}$ are the cartesian coordinates.

Definition D.7 (Isometry) Similitudes that preserve distances between points are called *isometries* (or rigid transform).

Isometries are all and only the similitudes in which $s = 1$. They thus form a subgroup of the similitudes which is called *Euclidean group* and is denoted by $E(3)$. In the case where $\det R_{n\times n} = 1$, the matrix $R_{n\times n}$ represents a rotation ($R_{n\times n} \in SO(3)$) and the isometry is said to be "direct". Directed isometries constitute the special Euclidean group denoted by $SE(3)$.

Reference

J. G. Semple and G. T. Kneebone. *Algebraic Projective Geometry*. Oxford University Press, Oxford, 1952.

Appendix E
MATLAB Code

The MATLAB code listed in the text, along with other support functions not listed, can be downloaded from https://github.com/fusiello/computer_vision_Toolkit. Keep in mind that this is code written with educational intent and therefore:

- whenever more efficient code could have been written at the expense of readability, we have preferred the latter;
- to avoid cluttering up the listings, there are no checks on whether the parameters passed to the functions match what is assumed within the functions themselves;
- for the same reason, there is no documentation at the top of the function and comments are used sparingly;
- for reasons of compactness and simplicity, special cases and pathological situations that might occur are not dealt with;
- the input parameters to the functions are limited to the indispensable arguments; other control parameters are wired into the function itself;
- the use of a variable number of input and output parameters, and assignments by default are used less than would be expected in a well-built library.

Conventions related to the interface of functions:

- the points are in Cartesian coordinates;
- both 2D and 3D points are packed into a matrix by columns;
- items (points, PPM, etc.) referred to multiple images are represented with *cell arrays* indexed by the image;
- in cases where the function calculates a transformation, it is such that the second argument is transformed to the first.

The list of the main implemented functions follows (auxiliary ones are omitted). All were tested with MATLAB R2021a and OCTAVE 4.2.1 under macOS 12.3.1.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

```
            autocalMC - Autocalibration from F matrices with Mendonca-Cipolla
            bundleadj - Bundle adjustment
             calibLVH - Camera calibration from infnity homographies
             calibSMZ - Camera calibration from object-image homographies
         calib_radial - Calibrate radial distortion with resection
               camera - Camera exterior orientation given COP,LOOK and UP
    cop_from_bearings - Camera locations from bearings
                  dlt - Direct Linear Transform
        eight_points - 8-points algorithm for F/E
       essential_lin - Essential matrix with 8-points algorithm
        eucl_upgrade - Upgrade projective reconstruction given K
       exterior_iter - Exterior orientation with anisotropic Procrustes analysis
        exterior_lin - Exterior orientation with Fiore's algorithm
      exterior_nonlin - Non-linear refinement of exterior orientation
                  fdh - Compute focal distance from homography
                 fund - Build fundamental matrix from camera matrices
             fund_lin - Fundamental matrix with 8-points algorithm
          fund_nonlin - Non-linear refinement of fundamental matrix
             fund_rob - Robust fundamental matrix
                  gpa - Generalized Procrustes analysis
                  gsd - Ground Sampling Distance (pixel size on the ground)
              hom_lin - Homography with DLT algorithm
           hom_nonlin - Non-linear refinement of H
              hom_rob - Robust homography fit
            hom_synch - Homography syncronisation
                  htx - Apply homogeneous transform
                  icp - Iterative Closet Point algorithm
                 imhs - Harris-Stephens corner strength
           imrectify - Apply rectification to images
        imstereo_ncc - Stereo block-matching with NCC
        imstereo_sch - Stereo block-matching with SCH (Census)
        imstereo_ssd - Stereo block-matching with normalized SSD
               imwarp - Image warp
                 irdx - Undo radial distortion
                  krt - Intrinsic and extrinsic parameters from P
          lsq_nonlin - Levenberg-Marquardt method for non-linear LS
                  opa - Orthogonal Procrustes analysis
         photostereo - Photometric stereo
                 prec - Sturm-Triggs' projective reconstruction
              precond - Normalize the coordinates of kD points
               radial - Regress radial distortion parameters
                  rdx - Apply radial distortion
            rectifyF - Epipolar rectification (uncalibrated)
            rectifyP - Epipolar rectification (calibrated)
        relative_lin - Relative orientation
     relative_nonlin - Non-linear refinement of relative orientation
          reproj_res - Reprojection residual and jacobian for 1 point - 1 camera
    reproj_res_batch - Reprojection residual for all points all cameras
          resect_lin - Camera matrix (resection)
       resect_nonlin - Non-linear refinement of resection
          robust_fit - robust regression
      rotation_synch - Rotation syncronisation
        sampson_fund - Sampson residual for F (and its Jacobian)
         sampson_hom - Sampson residual for H (and its Jacobian)
shape_from_silhouettes - Shape from silhouettes
            simpleGN - Gauss-Newton method for non-linear LS
         space_carve - Space carving algorithm
    translation_synch - Translation synchronisation
        triang_disp - Triangulation from disparity
         triang_lin - Triangulation for one point in multiple images
     triang_lin_batch - Triangulation for n points in multiple images
       triang_nonlin - Non-linear refinement of triangulation
  triang_nonlin_batch - Non-linear refinement of triangulation for n points
```

Index

A
Adjacency matrix, 302
Adjoint matrix, 80
Adjustment of independent models, 207
Affinity, 331
Algebraic distance, 100
Alignment, 225
Autocalibration, 236, 242

B
Baseline, 57, 126, 127
Bearing, 216
Bilinear interpolation, 280
Block matching, 168, 173
Breakpoint, 318, 319
Bundle adjustment, 217–224

C
Calibration, 35–43
Camera
  affine, 244
  pinhole, 5, 10, 15–28
  telecentric, 10
Camera matrix, *see* Perspective Projection Matrix (PPM), 334
Camera obscura, 5, 21
Camera reference frame, 16
Census transform, 171
Centre of projection, 15, 26
Chirality, 78
Circle of confusion, 10
Coded-light, 191
Collinearity equations, 28
Collineation, 64
Commutation matrix, 308
Compatible matrix, 92
Conjugate points, 9
Control points, 35, 49, 50
Convolution, 140
Corresponding points, *see* Homologous Points, 334
Cross-correlation, 140

D
Degree matrix, 303
Depth of field, 10
Depth-speed ambiguity, 73, 74, 236
Diffuse scattering, 13
Direct Linear Transform (DLT), 35, 37, 50, 64, 67, 103
Disparity, 125, 127, 137, 165–182, 274, 275
Disparity map, 166

E
Effective pixel size, 8, 19
Eight-point algorithm, 61, 63
Epipolar constraint, 165
Epipolar geometry, 57–64, 73
Epipolar graph, 203
Epipolar line, 57–60, 70, 168, 258, 260
Epipolar plane, 57
Epipolar rectification, 126, 129–136
Epipole, 57, 59, 61, 68, 70, 71
Essential matrix, 73–80, 92, 202, 241
  computation, 76
  factorisation, 77

© Edizioni Franco Angeli under exclusive license to Springer Nature Switzerland AG 2024

A. Fusiello, *Computer Vision: Three-dimensional Reconstruction Techniques*,
https://doi.org/10.1007/978-3-031-34507-4

Euler, 310
Extrinsic parameters, 23, 49, 90, 240, 333

F
Features
  detection, 139–152, 157–160
  matching, 153, 161
  tracking, 153
Field of view, 6, 21
Filtering, 140
Focal length, 6, 8, 10, 15, 20, 130
Focal plane, 15
Foreshortening effect, 6
Fundamental matrix, 60–64, 75, 202
  computation, 63
  factorisation, 92

G
Generalised Procrustean analysis (GPA), 208, 226
Geometric distance, 100
Gimbal lock, 302

H
Hierarchical reconstruction, 209
Homogeneous coordinates, 330
Homography, 40, 64–68, 96, 132, 266, 270, 272
  calibrated, 81
Homologous points, 57, 74, 125, 333

I
Image plane, 5, 6, 9, 15
Image warping, 279
Incidence matrix, 215, 302
Intersection, *see* Triangulation, 334
Intrinsic parameters, 20, 24, 45, 49, 73, 92, 130, 241, 245, 247, 274, 333
Isometry, 22, 332
Iterative closest point (ICP), 228, 267
Iverson brackets, 13

J
Jacobian matrix, 30, 103, 108, 153, 154, 219–222, 305–309, 315, 318
Joint image space, 110

K
Kernel, 140
  derivative, 144
  Gaussian, 141
  smoothing, 141
  Sobel, 146
Kronecker product, 36, 61, 67, 215, 244, 299
Kruppa constraints, 240

L
Lambert's law, 13
Light Detection And Ranging (LiDAR), 192
Linear regression, 314
Longuet-Higgins equation, 60, 74, 75
Loss function, 319

M
Mapping
  backward, 280
  forward, 279
Marching cubes, 250, 262
Match space, 177
M-estimators, 319
Model, 201
Mosaic
  panoramic, 266
  planar, 266
Mosaicing, 266–270
Motion, 246

N
Nodal points, 9, 29
Non-linear
  calibration, 105
  exterior orientation, 106
  fundamental matrix, 115
  homography, 111
  relative orientation, 118
  triangulation, 107, 108
Normal configuration, 59, 125, 126
Normal equation, 296, 315
Normalised cross-correlation, NCC, 169, 259
Normalised image coordinates, 20, 50, 74–76

O
Optical axis, 8–10, 29, 131, 247
Optical ray, 27, 33

Orientation, 333
absolute, 45–48, 50, 210
exterior, 23, 24, 45, 49–55, 208
interior, 24
relative, 45, 73–80, 212
Orthogonal Procrustes Analysis (OPA), 46, 51, 52, 295
Orthographic projection, 7, 245
Orthophoto, 277, 278
Ortho-projection, 277
Orthoscopy, 29
Outliers, 121, 318–324

P
Parallax, 69–70, 265, 270, 273, 276, 278
Parameters
extrinsic (*see* Extrinsic parameters), 334
intrinsic (*see* Intrinsic parameters), 334
Perspective pose, 49
Perspective Projection Matrix (PPM), 18–28, 333
photo-consistency, 249, 251, 258, 259, 261
Photogrammetry, 23, 28, 32, 49, 91, 207
Photographic scale, 32
Photoplane, 272
Pinhole camera, 5
Pinhole camera, *see* camera
pinhole, 334
Planar mosaics, 266
Pose, *see* Orientation, 334
Preconditioning, 37, 63
Principal axis, 15, 16
Principal point, 15
Projection
orthographic, 10
perspective, 6, 17, 18, 26, 327
Projective bundle, 15
Projectivity, 64, 92, 233, 331

R
Radial distortion, 29, 38
Radiance, 11–13, 194
radiance, 249
Range sensors, 185
Reconstruction, 89, 201, 208
Euclidean, 89, 90, 201, 202
identical, 89
incremental, 208
metric, 90
projective, 90, 92–93, 201, 233–235
from silhouettes, 252
volumetric, 249
Reflectance, 11, 194, 252
Reflectance equation, 12
Registration, 225
Regression, 313
Relief displacement, 273
Reprojection error, 102, 107
Resection, 35, 39, 49, 87
Resection-intersection, 208
Residual, 102, 103, 106, 107, 111, 115, 313
Robust regression, 121, 318

S
Salient points, 139
Sampson distance, 100
Scale Invariant Feature Transform (SIFT), 155–162
Scale-space, 155–156
Scanner, 186, 193
Seven-point algorithm, 62
shape from silhouette, 252
Shape from X, 3
Singular value decomposition (SVD), 36, 51, 62, 63, 76, 77, 81, 234, 241, 244, 247, 292–295, 334
Singular values, 292
Singular vectors, 292
Spanning tree, 303
Stereo, 125, 168, 180, 187, 189, 333
active, 187
correlation based, 168
photometric, 194–197
Stereo matching, 165
Stereopsis, *see* Stereo, 334
Stratification, 94
Structure, 246
Structured lighting, 186–192
Sum of squares of differences (SSD), 149, 150, 169, 173, 175
Synchronisation, 210, 231
of direct isometries, 231
of homographies, 267–268
of rotations, 212–214
of translations, 214–215
Synthesis, 274–281

T
Template matching, 141
Thin lens, 8, 29
Tie points, 73, 87, 203

Triangulation, 75, 87–89, 92, 125–129, 205, 208, 333
active, 186–188
normal case, 126
Triple product, 75, 298

V
Vanishing point, 33, 66, 244
visual hull, 253
volumetric stereo, 250
Voronoi tessellation, 269
voxel, 251–258